Bank Management System Documentation

Adriano Demiri

Andrea Ogreni

Emanuele Berbiu

Erind Kotobelli

Erind Shabani

Joandri Domi

Joel Lluri

Kleo Miraka

Project Overview

This project involves the development of a comprehensive bank management software designed to enhance the efficiency of banking operations and improve customer experience. The product serves a wide range of users, including customers, bank staff, and administrators.

The software is intended to support critical banking processes such as account management, transaction handling and loan approvals. By integrating with existing infrastructure and providing robust security measures, the system seeks to deliver reliable banking services.

Relationships to Other Systems

The BMS integrates with the following external systems and services:

1. **ATM Network**: Supports real-time balance inquiries and cash withdrawals through the bank's ATM fleet. The BMS manages ATM fund allocations, maintenance scheduling, and transaction logging in sync with physical ATM devices.
2. **Inter-Bank Transfer Networks (SWIFT / SEPA / ACH)**: Facilitates secure inter-bank wire transfers and cross-border payments by interfacing with standard financial messaging networks. The BMS initiates, tracks, and reconciles all outgoing and incoming transfers through these protocols.
3. **Payment Processors / POS Systems**: Processes debit and credit card transactions at Points-of-Sale (POS) terminals by communicating with external payment processing networks. Card issuance, spending limits, and fraud flags managed within the BMS are reflected in real time.
4. **SMS / Email Notification Services**: Delivers real-time transaction alerts, low-balance warnings, loan payment reminders, KYC document expiry notifications, and system announcements to customers and staff via third-party SMS gateways and email service providers.
5. **Mobile Banking Application (iOS & Android)**: A customer-facing mobile application that interfaces with the BMS back-end through secure REST APIs. Supports account management, fund transfers, bill payments, QR code payments, biometric authentication, and e-statement access.
6. **Online Banking Web Portal**: A browser-based customer self-service portal that provides the same core banking functionality as the mobile app, optimized for larger screens and standard keyboard interaction.

User Characteristics

Internal Users:

- Administrator
  - Holds the highest system access level; typically an IT professional or system manager.
  - Responsible for user account lifecycle, role and permission management, system health monitoring, database administration, security policy enforcement, and audit log review.
  - High technical expertise; requires full access to all configuration and administrative modules.
- Bank Director
  - Senior executive who uses the system for strategic oversight and high-level decision-making.
  - Accesses executive dashboards, financial reports (P&L, balance sheets), branch KPIs, employee metrics, and regulatory compliance reports.
  - Approves or rejects high-value loan applications and transactions that exceed defined thresholds.
  - Low to moderate technical expertise; requires clear dashboards, exportable reports, and automated alerts.
- Bank Employee — Staff (Account Managers, Loan Officers, Compliance Staff)
  - Back-office personnel handling complex administrative and customer service functions.
  - Creates and manages customer accounts, processes loan applications, updates KYC documentation, handles complaints, and monitors flagged accounts.
  - Moderate technical expertise; requires detailed account management tools, document workflows, and bulk processing capabilities.

- Bank Employee — Teller
  - Front-line staff who interact directly with customers at the branch counter.
  - Processes cash deposits and withdrawals, fund transfers, foreign currency exchanges, mini statements, and utility bill payments.
  - Low technical expertise; requires an intuitive, fast-access transaction interface optimized for high-volume daily use.
- Facilities Manager
  - Oversees all physical operations: premises, security personnel, cleaning staff, IT equipment, vehicles, and vendor contracts.
  - Manages duty rosters, maintenance schedules, supply requisitions, and generates facilities reports for branch managers and the Bank Director.
  - No access to customer financial data or core banking modules; system access strictly limited to facilities management modules.

<u>External Users:</u>

- Customer
  - End-user of banking services who interacts with the system via the online banking portal or mobile application.
  - Views balances and transaction history, performs fund transfers, applies for loans and cards, manages beneficiaries, and submits complaints or feedback.
  - Basic computer literacy assumed; requires an intuitive, accessible interface compliant with WCAG 2.1 AA standards.
- Facilities & Support Staff (Security Guards, Cleaners, IT Technicians, Drivers, Maintenance Workers)
  - Operational personnel responsible for maintaining the bank's physical environment.
  - Access is strictly limited to role-specific tools: guards log shifts and incidents; cleaners track schedules and supply requests; IT technicians manage support tickets and hardware records; drivers log trips and fuel usage; maintenance workers update repair task statuses.

  - Minimal technical expertise; requires simple, task-focused screens with no exposure to financial or customer data.

Assumptions

- All internal and external users have stable internet access sufficient for real-time interaction with the BMS web portal or mobile application.
- Every user has a minimum level of digital literacy appropriate to their role (e.g., basic computer skills for customers and tellers; IT proficiency for the Administrator).
- The bank will provide and maintain secure hosting infrastructure, including firewalls, load balancers, and intrusion-detection systems, as a prerequisite for system deployment.
- Third-party integrations — including SMS gateways, email service providers, credit scoring engines, and inter-bank transfer networks — expose stable, documented APIs that remain available throughout the system's operational lifetime.

Constraints and Dependencies

- **Module Implementation Order (User Management First):** The User Management module must be fully implemented and operational before any other banking functionality can be accessed. Role-based access control, authentication, and permission assignment are foundational dependencies for every other module.
- **Core Module Prerequisites:** The Funds Management, Audit & Reporting, and Fraud Detection modules must be implemented and active before account-related operations (deposits, withdrawals, transfers, loan processing) can be processed, as these modules enforce financial integrity and regulatory controls.
- **Performance and Availability Constraints:** The system must maintain a minimum uptime of 99.9% on a rolling monthly basis. At least 95% of financial transactions must complete within 2 seconds under normal load. The system must support 500 concurrent users under normal conditions and up to 1,000 during peak periods.
- **Facilities Module Isolation:** The Facilities Management and Facilities & Support Staff modules must be architecturally isolated from core banking modules. Personnel in these roles must have zero visibility into customer financial data, account information, or banking transaction records.

Functional Requirements

Req #	Requirement	Priority	Business Process
BS_UM_01	The Administrator shall be able to create, modify, and delete system profiles for all user roles.	1	User Management
BS_UM_02	All users shall be able to log in to the system using their registered credentials.	1	User Management
BS_UM_03	The Administrator shall be able to define and manage role-based access permissions for each user level.	1	User Management
BS_UM_04	The Administrator shall be able to monitor system health, schedule maintenance, and apply software updates and patches.	1	User Management
BS_FM_01	The Bank Director shall be able to view fund allocations and balances across all assigned branches and ATMs.	1	Fund Management
BS_FM_01	The Bank Director shall be able to view fund allocations and balances across all assigned branches and ATMs.	1	Fund Management
BS_FM_03	The Bank Director shall be able to set and update interest rates, fee structures, and bank-wide financial policies.	1	Fund Management
BS_AM_01	The Bank Teller shall be able to open a new bank account on behalf of a customer, subject to supervisor approval.	1	Account Management
BS_AM_02	The Bank Teller shall be able to close an existing bank account upon a verified customer request.	1	Account Management
BS_AM_03	The Bank Teller shall be able to modify customer account details as required.	1	Account Management
BS_AM_04	The Customer shall be able to view their account details and full transaction history through the online portal.	2	Account Management

Req #	Requirement	Priority	Business Process
BS_AM_05	Bank Staff shall be able to create and manage savings, current, fixed deposit, and loan accounts on behalf of customers.	1	Account Management
BS_AM_06	Bank Staff shall be able to update customer KYC documentation and verify identity information.	1	Account Management
BS_AM_07	The Bank Teller shall be able to verify customer identity before processing any in-branch transaction.	2	Account Management
BS_TM_01	The Customer shall be able to deposit money into their bank account.	1	Transaction Management
BS_TM_02	The Customer shall be able to withdraw money from their account, either at a bank branch via a Bank Teller or at an ATM using their bank card.	1	Transaction Management
BS_TM_03	The Customer shall be able to transfer funds to accounts within the same bank, initiate wire transfers to external bank accounts, and configure automated recurring transfers.	1	Transaction Management
BS_TM_04	The Customer shall be able to make payments using their issued debit or credit card.	1	Transaction Management
BS_TM_05	Bank Staff shall be able to process bulk payment disbursements such as salary payments and utility bill settlements.	2	Transaction Management
BS_TM_06	The Customer shall be able to manage and save beneficiaries for repeat fund transfers through the online banking portal or mobile app.	2	Transaction Management
BS_LM_01	The Customer shall be able to submit a loan application online; Bank Staff shall review, process, and approve or reject the application.	1	Loan Management
BS_LM_02	The Customer shall be able to make loan repayments through the online banking portal or at a branch.	1	Loan Management

Req #	Requirement	Priority	Business Process
BS_LM_03	The Bank Director shall be able to approve or reject high-value loan applications that exceed a defined threshold.	1	Loan Management
BS_LM_04	A customer with an approved loan views the amortization schedule, including monthly installments, interest, and remaining balance. The system calculates and displays the repayment plan.	1	Loan Management
BS_CM_01	The Customer shall be able to apply online for a debit or credit card linked to their bank account.	1	Card Management
BS_CM_02	The Customer shall be able to request a chequebook for their eligible checking account.	1	Card Management
BS_CM_03	The Customer shall be able to set and modify spending limits on their issued cards.	2	Card Management
BS_FD_01	Bank Staff shall be able to review flagged accounts, investigate suspicious transactions, and take appropriate corrective action.	1	Fraud Detection
BS_FD_02	The Customer shall be able to report suspected fraudulent activity on their account through the online portal.	1	Fraud Detection
BS_FD_03	The Bank Director shall be able to receive automated alerts for unusual account activity, fraud flags, or system anomalies.	1	Fraud Detection
BS_FCM_01	The Facilities Manager shall be able to manage and approve duty rosters and shift schedules for all facilities and support staff across branches.	2	Facility Management
BS_FCM_02	The Facilities Manager shall be able to assign, track, and monitor IT support tickets submitted by bank staff, including resolution times.	2	Facility Management

Req #	Requirement	Priority	Business Process
BS_FCM_03	Facilities & Support Staff shall be able to log shift check-ins/check-outs, submit incident or task completion reports, and view their assigned schedules.	2	Facility Management
BS_FCM_04	The Facilities Manager shall be able to schedule and track preventive maintenance for buildings, ATMs, vehicles, and equipment.	3	Facility Management
BS_CS_01	The Customer shall be able to view real-time information on nearby ATMs and branches, including location, hours, and availability, through the online banking portal or mobile app.	3	Costumer Service
BS_CS_02	The Customer shall be able to schedule and manage in-branch appointments through the online portal.	3	Costumer Service
BS_CS_03	The Customer shall be able to submit feedback and formal complaints regarding their banking experience through the online portal.	3	Costumer Service
BS_CS_04	The Customer shall be able to request official account-related documents such as statements, certificates, and reference letters.	1	Costumer Service
BS_CS_05	Bank Staff shall be able to handle customer complaints and manage dispute resolution cases within the system.	2	Costumer Service
BS_CS_06	Bank Staff shall be able to generate and send official account statements to customers on request or on a scheduled basis.	2	Costumer Service
BS_CS_07	The Customer shall be able to receive real-time notifications and alerts for account activity, including transactions, login attempts, and payment due dates.	2	Costumer Service
BS_AR_01	The Bank Director shall be able to access complete audit trails to verify compliance with banking regulations and internal policies.	1	Audit and Reporting

Req #	Requirement	Priority	Business Process
BS_AR_02	The Bank Director shall be able to access customer analytics reports, including account growth, activity trends, and segmentation data.	2	Audit and Reporting
BS_AR_03	The Bank Director and Bank Staff shall be able to access fraud detection analytics, including flagged transaction summaries and trend reports.	1	Audit and Reporting
BS_AR_04	The Facilities Manager shall be able to generate facilities reports covering maintenance status, incidents, asset conditions, and staff attendance.	2	Audit and Reporting

Non-Functional Requirements

1. Usability
    a. The system shall provide an intuitive interface that all user levels can navigate without formal training within 30 minutes of first use.
    b. The system shall display clear, descriptive error messages that guide users toward corrective action without exposing technical details
    c. The system shall comply with WCAG 2.1 AA accessibility standards to ensure usability for users with disabilities

2. Performance
    a. At least 95% of all financial transactions shall complete processing within 2 seconds under normal operating load conditions.
    b. The system shall support a minimum of 500 concurrent active users without measurable performance degradation.
    c. During peak load periods the system shall sustain up to 1,000 simultaneous users without service interruption or data errors.
    d. Standard account lookups and report generation queries shall return results within 1 second for datasets up to 5 million records.

3. Availability
    a. The system shall maintain a minimum uptime of 99.9% measured on a rolling monthly basis to ensure continuous banking operations.
    b. Scheduled maintenance windows shall not exceed 4 hours per month and must be communicated to all users at least 48 hours in advance.
    c. The system shall implement automatic failover to a secondary server so that service is restored within 5 minutes of a primary server failure.
    d. The mean time to recovery from any unplanned outage shall not exceed 30 minutes from the point of detection.

4. Security
    a. All data transmitted between client devices and the server shall be encrypted using TLS 1.3 to prevent interception during communication.

b. All data stored in the database including customer records and transaction history shall be encrypted at rest using AES-256.
c. Multi-factor authentication shall be enforced for all user roles combining password credentials with a one-time password via SMS or authenticator app.
d. User sessions shall automatically expire after 10 minutes of inactivity and tokens shall be invalidated on logout to prevent session hijacking.
e. The system shall maintain a tamper-proof audit log of all user actions, financial transactions, and administrative changes including timestamps and IP addresses.

5. Reliability

a. The system shall perform automated daily backups of all data to at least two geographically separated storage locations.
b. Data recovery procedures shall restore the system to a consistent operational state within 1 hour following a catastrophic failure.
c. The system shall enforce ACID transaction properties to ensure that no financial transaction is ever lost, duplicated, or left in a partial state.

6. Scalability & Maintainability

a. The system architecture shall support horizontal scaling to accommodate a 300% increase in user volume without requiring a full system redesign.
b. The system shall follow a modular design so that individual components such as the loan module or reporting engine can be updated or replaced independently.
c. All system documentation including API references, deployment guides, and data schemas shall be kept current with every release.

7. Compliance & Interoperability

a. The system shall comply with GDPR for customer data privacy and PCI-DSS for payment card data security across all modules handling personal or financial information.
b. Transaction records and customer data shall be retained for a minimum of 7 years in accordance with applicable banking and financial regulations.
c. The system shall expose standardized REST API endpoints to enable secure integration with third-party financial institutions and government regulatory systems

Use Cases

User Management – Andrea Ogreni

UC Name	BS_UM_01 – Create, Modify, and Delete System User Profiles
Summary	The Administrator creates, modifies, and deletes system profiles for all user roles within the Bank Management System.
Dependency	None.
Actors	Primary Actor: Administrator.
Preconditions	The Administrator is logged in to the system with full administrative privileges. The target user profile exists (for modify/delete) or the required information for the new profile is available (for create).
Description of the Main Sequence	● Step 1: The Administrator navigates to the User Management module. ● Step 2: The Administrator selects the action: Create, Modify, or Delete. ● Step 3 (Create): The Administrator fills in the required fields – full name, role, contact details, and branch assignment – and submits the form. ● Step 4: The system validates the input, generates a unique user ID and temporary password, and creates the profile. ● Step 5: The system sends login credentials to the new user via email or SMS. ● Step 6: The system logs the action in the audit trail with the Administrator's ID and timestamp.
Description of the Alternative Sequence	● Step 1: The Administrator selects an existing user profile. ● Step 2 (Modify): The Administrator updates the required fields and confirms

	the change. The system saves changes and logs the modification. ● Step 3 (Delete): The Administrator selects Delete and confirms the action. The system deactivates the profile, revokes access tokens, and logs the deletion. ● Step 4: If any required field is missing or invalid during creation/modification, the system displays an error and prompts the Administrator to correct the input.
Non functional requirements	All profile changes must be reflected in the system within 2 seconds. Deleted/deactivated profiles must have their sessions immediately invalidated. All actions must be logged in a tamper-proof audit trail.
Postconditions	The user profile has been successfully created, modified, or deleted. The audit trail contains a permanent record of the action. If created, the user receives their access credentials.
UC Name	BS_UM_01 – Create, Modify, and Delete System User Profiles

UC Name	BS_UM_02 – User Login to the System
Summary	All registered users log in to the Bank Management System using their credentials and complete multi-factor authentication to gain access.
Dependency	BS_UM_01 (User profile must exist before login is possible).
Actors	Primary Actor: Any registered user (Administrator, Bank Director, Bank Employee – Staff / Teller, Facilities Manager, Facilities & Support Staff, Customer).
Preconditions	The user has an active system profile with registered credentials. The system is online and accessible. The MFA service (SMS or authenticator app) is operational.
Description of the Main Sequence	● Step 1: The user navigates to the system login page. ● Step 2: The user enters their registered username and password. ● Step 3: The system validates the credentials against the database. ● Step 4: The system triggers the MFA step – sending a One-Time Password (OTP) via SMS or authenticator app. ● Step 5: The user enters the OTP within the allowed time window. ● Step 6: The system verifies the OTP and issues a secure, time-limited session token. ● Step 7: The system redirects the user to their role-specific dashboard. ● Step 8: The successful login event is recorded in the audit log.

Description of the Alternative Sequence	● Step 1: If the username or password is incorrect, the system displays an error message without specifying which field is wrong. ● Step 2: After a configurable number of consecutive failed login attempts (e.g., 5), the system temporarily locks the account and notifies the user. ● Step 3: If the OTP expires before entry, the system offers the option to resend a new OTP. ● Step 4: If the account is locked or deactivated, the system displays an appropriate message and prompts the user to contact the Administrator.
Non functional requirements	Login processing (credential validation and session token issuance) must complete within 2 seconds. The system must enforce brute-force protection with account lockout. Session tokens must expire after 10 minutes of inactivity. All login attempts (successful and failed) must be logged with user ID, timestamp, and IP address.
Postconditions	The user is authenticated and has an active session with access restricted to their assigned role and permissions. The login event is recorded in the audit trail.

UC Name	BS_UM_03 – Define and Manage Role-Based Access Permissions
Summary	The Administrator defines and manages the specific access permissions assigned to each user role, controlling which modules, data, and features each role can access.
Dependency	BS_UM_01 (User profiles must exist before permissions can be assigned).
Actors	*Primary Actor: Administrator.*
Preconditions	The Administrator is logged in with full administrative privileges. At least one user role exists within the system.
Description of the Main Sequence	● Step 1: The Administrator navigates to the Role & Permission Management module. ● Step 2: The Administrator selects an existing role or creates a new custom role. ● Step 3: The Administrator reviews the list of available system modules and features. ● Step 4: The Administrator enables or disables specific permissions for the selected role (e.g., view, create, edit, delete, approve). ● Step 5: The Administrator saves the updated permission set. ● Step 6: The system applies the new permissions immediately to all active users assigned to that role. ● Step 7: The system records the permission change in the audit log with the Administrator's ID and timestamp.

Description of the Alternative Sequence	● Step 1: If the Administrator attempts to assign permissions that conflict with regulatory compliance rules (e.g., a Teller gaining access to audit logs), the system displays a warning. ● Step 2: The Administrator can proceed with override justification or cancel the change. ● Step 3: If no roles exist yet, the system prompts the Administrator to create a role before assigning permissions.
Non functional requirements	Permission changes must take effect within 2 seconds for all active sessions. The system must enforce the least privilege principle – no role should receive access beyond what is explicitly defined. All permission changes must be logged in the tamper-proof audit trail.
Postconditions	The role's permissions are updated and active. All users assigned to the affected role immediately reflect the new access rights. The change is permanently recorded in the audit trail.

UC Name	BS_UM_04 – Monitor System Health, Schedule Maintenance, and Apply Updates
Summary	The Administrator monitors the health and performance of the Bank Management System, schedules maintenance windows, and applies software updates and security patches.
Dependency	None.
Actors	Primary Actor: Administrator.
Preconditions	The Administrator is logged in with full administrative privileges. The system health dashboard and monitoring tools are operational.
Description of the Main Sequence	● Step 1: The Administrator accesses the System Health Dashboard. ● Step 2: The Administrator reviews real-time metrics including server CPU/memory usage, database performance, network status, and application response times. ● Step 3: The Administrator reviews any automated alerts flagged by the monitoring service. ● Step 4: The Administrator schedules a maintenance window by specifying the date, time, and estimated duration. ● Step 5: The system automatically sends advance notifications to all users at least 48 hours before the scheduled maintenance. ● Step 6: During the maintenance window, the Administrator applies software updates or security patches via the patch management interface. ● Step 7: After updates are applied, the Administrator verifies system stability through the health dashboard. ● Step 8: The system logs all maintenance activities and patch deployments in the audit trail.

Description of the Alternative Sequence	● Step 1: If a critical system alert is triggered, the Administrator receives an immediate notification. ● Step 2: The Administrator investigates the issue via the health dashboard and may initiate an emergency maintenance action. ● Step 3: If an applied patch causes a system error, the Administrator initiates a rollback procedure to restore the previous stable version. ● Step 4: If scheduled maintenance extends beyond the planned window, additional notifications are sent to all users.
Non functional requirements	System health dashboards must refresh in real time with a maximum delay of 5 seconds. Maintenance notifications must be delivered to all users at least 48 hours in advance. Patch deployment must not exceed 4 hours of downtime per month. All maintenance actions and patch history must be logged in the audit trail.
Postconditions	System health metrics have been reviewed. Any scheduled maintenance has been communicated to users and executed. Software and security updates have been successfully applied and verified. All actions are recorded in the audit trail.

Fund Management – Andrea Ogreni

UC Name	BS_FM_01 – View Fund Allocations and Balances Across Branches and ATMs
Summary	The Bank Director views current fund allocations and cash balances across all assigned bank branches and ATMs to support financial oversight and decision-making.
Dependency	BS_UM_02 (Bank Director must be logged in). Relevant branch and ATM data must be populated in the system.
Actors	Primary Actor: Bank Director.
Preconditions	The Bank Director is authenticated and has an active session. Branch and ATM financial data is available and up to date in the system.
Description of the Main Sequence	● Step 1: The Bank Director navigates to the Financial Overview or Branch Management module. ● Step 2: The Bank Director selects the Fund Allocations & Balances view. ● Step 3: The system displays a summary dashboard showing total funds allocated to each branch and ATM, current cash balances, and fund distribution breakdowns. ● Step 4: The Bank Director can filter the view by branch, region, date, or ATM unit. ● Step 5: The system logs the access event in the audit trail.
Description of the Alternative Sequence	● Step 1: If data for a specific branch or ATM is unavailable or not yet updated, the system displays a 'Data Pending' indicator for that unit. ● Step 2: If the Bank Director attempts to access a branch outside their assigned scope, the system denies access and displays an authorization message.

	● Step 3: If real-time data fails to load within the expected timeframe, the system displays the last available cached data with a timestamp and a notification.
Non functional requirements	Fund allocation data must refresh in real time or within a maximum delay of 5 seconds. The dashboard must support filtering across all branches and ATMs without performance degradation. Access to financial data must be restricted to authorized roles only, enforced by Role-Based Access Control. All data access events must be logged.
Postconditions	The Bank Director has successfully reviewed the current fund allocations and balances across all assigned branches and ATMs. The access event is recorded in the audit trail.

UC Name	BS_FM_02 – Allocate and Redistribute Funds to Financial Units
Summary	The Bank Director allocates and redistributes funds across assigned financial units (branches and ATMs) to ensure optimal cash availability and operational balance.
Dependency	BS_FM_01 (The Bank Director should review current balances before making allocation decisions). BS_UM_02 (Bank Director must be logged in).
Actors	Primary Actor: Bank Director.
Preconditions	The Bank Director is authenticated and has an active session. Current fund allocation and balance data is visible. Sufficient overall funds are available for redistribution.
Description of the Main Sequence	● Step 1: The Bank Director navigates to the Fund Allocation module. ● Step 2: The Bank Director reviews the current balances and identifies units requiring fund allocation or redistribution. ● Step 3: The Bank Director selects the source unit (or central reserve) and the destination financial unit. ● Step 4: The Bank Director enters the amount to be allocated or transferred. ● Step 5: The system validates that the source has sufficient funds and that the amount does not exceed any defined limits. ● Step 6: The Bank Director confirms the allocation/redistribution action. ● Step 7: The system processes the fund movement, updates balances in real time, and generates a transaction record.

	● Step 8: The system logs the action in the audit trail with the Bank Director's ID, timestamp, and details of the fund movement.
Description of the Alternative Sequence	● Step 1: If the source unit has insufficient funds or exceeds defined regulatory or policy limits, the system displays an error message and prevents the transaction from proceeding.
Non functional requirements	Fund allocation transactions must complete processing within 2 seconds under normal operating conditions. All fund movements must be logged as permanent, tamper-proof records. The system must enforce balance validation before processing any allocation. RBAC must ensure only the Bank Director and authorized roles can initiate fund movements.
Postconditions	Funds have been successfully allocated or redistributed to the designated financial unit. Account balances for affected units are updated in real time. A permanent transaction record and audit log entry have been created.

UC Name	BS_FM_03 – Set and Update Interest Rates, Fee Structures, and Bank-Wide Financial Policies
Summary	The Bank Director sets, reviews, and updates the bank's interest rates, fee structures, and financial policies that apply across all branches and account types.
Dependency	BS_UM_02 (Bank Director must be logged in). Existing policy and rate configurations must be present in the system.
Actors	Primary Actor: Bank Director.
Preconditions	The Bank Director is authenticated and has an active session. The Financial Policy Management module is accessible. Current rate and fee configurations are visible.
Description of the Main Sequence	• Step 1: The Bank Director navigates to the Financial Policy Management module. • Step 2: The Bank Director selects the category to update: interest rates, fee structures, or bank-wide policies. • Step 3: The Bank Director reviews the current configuration and enters the new values. • Step 4: The Bank Director sets an effective date for the changes. • Step 5: The system validates the inputs against compliance rules. • Step 6: The Bank Director confirms and submits the changes. • Step 7: The system applies the changes and logs the policy update in the audit trail.

Description of the Alternative Sequence	● Step 1: If the proposed rate, fee, or policy violates compliance limits, the system blocks the update and displays a compliance warning.
Non functional requirements	Policy changes must be propagated across all affected modules within 5 seconds of confirmation. The system must enforce regulatory compliance rules for all interest rate and fee updates, blocking non-compliant entries automatically. All policy changes must be permanently logged with before and after values in the tamper-proof audit trail.
Postconditions	The updated interest rates, fee structures, and/or financial policies are active in the system from the specified effective date. All affected modules and accounts reflect the new configuration. A permanent audit record of the change including previous and new values has been created.

Account Management – Joel Lluri

UC NAME	BS_AM_01
SUMMARY	This use case describes how a Bank Teller opens a new bank account for a customer in the system, subject to supervisor approval.
DEPENDENCY	**BS_UM_02** – All users shall be able to log in to the system using their registered credentials. **BS_AM_06** – Bank Staff shall be able to update customer KYC documentation and verify identity information. **BS_AM_07** – The Bank Teller shall be able to verify customer identity before processing any in-branch transaction.
ACTORS	· **Primary Actor:** Bank Teller · **Secondary Actor:** Customer · **Supporting Actor:** Supervisor
PRECONDITIONS	· The Bank Teller is logged into the system. · The customer is present and requests account creation. · The customer provides the required identification and KYC documents. · The system is available and operational.

DESCRIPTION OF MAIN SEQUENCE	· Step 1: The Bank Teller logs into the system. · Step 2: The Bank Teller selects the option to open a new bank account. · Step 3: The Bank Teller enters the customer's personal and contact details. · Step 4: The Bank Teller verifies the customer's identity. · Step 5: The Bank Teller uploads or records the customer's KYC documents. · Step 6: The Bank Teller selects the required account type. · Step 7: The system validates the entered information. · Step 8: The request is sent for supervisor approval. · Step 9: The supervisor approves the request. · Step 10: The system creates the account and generates a unique account number. · Step 11: The system confirms successful account creation
DESCRIPTION OF ALTERNATIVE SEQUENCE	· Step 1: If the Bank Teller enters invalid login credentials, the system denies access. · Step 2: If required customer data is missing, the system prompts the teller to complete the missing fields. · Step 3: If identity verification fails, the account opening process is stopped. · Step 4: If KYC documents are invalid or incomplete, the system rejects the request until corrected documents are provided. · Step 5: If the supervisor rejects the request, the account is not created. · Step 6: If the system validation fails, the system displays an error and the process is halted.

NON FUNCTIONAL REQ	· The system shall provide secure access through authenticated login. · The system shall enforce role-based access control so only authorized users can open accounts. · The system shall protect customer data using encryption and secure storage. · The system shall maintain an audit trail of all actions performed during account creation. · The system should process the account creation request efficiently under normal system load.
POSTCONDITION	· A new bank account is created for the customer if the main sequence is completed successfully. · The customer's account information and KYC data are securely stored in the system. · A unique account number is assigned to the new account. · The account becomes available for future banking operations.

UC NAME	BS_AM_02
SUMMARY	This use case describes how a Bank Teller closes an existing bank account upon a verified customer request.
DEPENDENCY	**BS_UM_02** – All users shall be able to log in to the system using their registered credentials. **BS_AM_07** – The Bank Teller shall be able to verify customer identity before processing any in-branch transaction.
ACTORS	· **Primary Actor:** Bank Teller · **Secondary Actor:** Customer
PRECONDITIONS	· The Bank Teller is logged into the system . · The customer requests account closure . · The customer's identity is verified . · The account exists in the system.
DESCRIPTION OF MAIN SEQUENCE	· Step 1: The Bank Teller logs into the system · Step 2: The Bank Teller searches for the customer account · Step 3: The Bank Teller verifies the customer's identity · Step 4: The system displays account details and current balance · Step 5: The system checks for any remaining balance or pending transactions · Step 6: The Bank Teller confirms the account closure request · Step 7: The system processes the account closure · Step 8: The system updates the account status to "closed" · Step 9: The system confirms successful closure

DESCRIPTION OF ALTERNATIVE SEQUENCE	· Step 1: If login fails, access is denied · Step 2: If the account is not found, the system displays an error · Step 3: If identity verification fails, the process is stopped · Step 4: If the account has a remaining balance, the system requires withdrawal or transfer before closure · Step 5: If there are pending transactions, the system prevents closure · Step 6: If system validation fails, the process is halted and an error is displayed
NON FUNCTIONAL REQ	· The system shall enforce secure login and authentication · The system shall implement role-based access control · The system shall log all account closure activities for audit purposes · The system shall maintain high availability and performance during operations
POSTCONDITION	· The account is marked as closed in the system · The account can no longer be used for transactions · All actions are recorded in audit logs

UC NAME	BS_AM_03
SUMMARY	This use case describes how a Bank Teller modifies an existing customer account's details when changes are required.
DEPENDENCY	**BS_UM_02** – All users shall be able to log in to the system using their registered credentials. **BS_AM_07** – The Bank Teller shall be able to verify customer identity before processing any in-branch transaction.
ACTORS	· **Primary Actor:** Bank Teller · **Secondary Actor:** Customer
PRECONDITIONS	· The Bank Teller is logged into the system. · The customer requests modification of account details. · The customer's identity is verified. · The customer account exists in the system.
DESCRIPTION OF MAIN SEQUENCE	· Step 1: The Bank Teller logs into the system. · Step 2: The Bank Teller searches for the customer account. · Step 3: The Bank Teller verifies the customer's identity. · Step 4: The system displays the current account details. · Step 5: The Bank Teller updates the required account information. · Step 6: The system validates the modified data. · Step 7: The system saves the updated account details. · Step 8: The system confirms that the modification was successful.

DESCRIPTION OF ALTERNATIVE SEQUENCE	· Step 1: If login fails, the system denies access. · Step 2: If the account is not found, the system displays an error. · Step 3: If identity verification fails, the modification process is stopped. · Step 4: If the entered data is incomplete or invalid, the system rejects the changes and prompts correction. · Step 5: If the system fails to save the changes, an error message is displayed and the account details remain unchanged.
NON FUNCTIONAL REQ	· The system shall enforce secure login and authentication. · The system shall apply role-based access control so only authorized users can modify account details. · The system shall protect customer information using encryption. · The system shall maintain an audit log of all account modifications. · The system shall process account detail updates efficiently under normal load conditions.
POSTCONDITION	· The customer account details are updated in the system. · The updated information is securely stored. · The modification activity is recorded in the audit logs.

UC NAME	BS_AM_04
SUMMARY	This use case describes how a Customer views their account details and full transaction history through the online portal.
DEPENDENCY	**BS_UM_02** – All users shall be able to log in to the system using their registered credentials.
ACTORS	· **Primary Actor:** Customer
PRECONDITIONS	· The Customer is logged into the system. · The Customer has an existing bank account. · The online portal is available and operational.
DESCRIPTION OF MAIN SEQUENCE	· Step 1: The Customer logs into the online banking portal. · Step 2: The Customer selects the account to view. · Step 3: The system displays the account details. · Step 4: The Customer selects the option to view transaction history. · Step 5: The system retrieves the full transaction history for the selected account. · Step 6: The system displays the transaction history to the Customer.
DESCRIPTION OF ALTERNATIVE SEQUENCE	· Step 1: If login fails, the system denies access. · Step 2: If transaction history cannot be retrieved, the system displays an error message. · Step 3: If the portal is unavailable, the Customer cannot access account information.

NON FUNCTIONAL REQ	· The system shall enforce secure login and authentication. · The system shall ensure that Customers can view only their own account information. · The system shall protect account and transaction data using encryption. · The system shall display account and transaction information efficiently under normal load conditions. · The system shall maintain availability of the online portal for customer access.
POSTCONDITION	The Customer has viewed their account details and transaction history.

UC NAME	BS_AM_05
SUMMARY	This use case describes how Bank Staff create and manage different types of customer accounts, including savings, current, fixed deposit, and loan accounts
DEPENDENCY	**BS_UM_02** – All users shall be able to log in to the system using their registered credentials. **BS_AM_06** – Bank Staff shall be able to update customer KYC documentation and verify identity information.
ACTORS	· **Primary Actor:** Bank Staff
PRECONDITIONS	· The Bank Staff is logged into the system. · The customer exists in the system. · The customer's identity and KYC information are verified. · The system is operational.
DESCRIPTION OF MAIN SEQUENCE	· Step 1: The Bank Staff logs into the system · Step 2: The Bank Staff selects the account management module · Step 3: The Bank Staff searches for an existing customer or creates a new customer profile · Step 4: The Bank Staff verifies customer identity and KYC information · Step 5: The Bank Staff selects the type of account to create or manage · Step 6: The Bank Staff enters or updates account details · Step 7: The system validates the entered or modified data · Step 8: The system saves the account information · Step 9: The system confirms successful account creation or update

DESCRIPTION OF ALTERNATIVE SEQUENCE	· Step 1: If login fails, access is denied · Step 2: If customer is not found, the system prompts to create a new customer profile · Step 3: If KYC verification fails, the process is stopped · Step 4: If required data is missing or invalid, the system rejects the input and requests correction · Step 5: If system validation fails, the process is halted and an error is displayed
NON FUNCTIONAL REQ	· The system shall enforce secure authentication and role-based access control · The system shall ensure data integrity and validation during account operations · The system shall protect customer data using encryption · The system shall log all account management actions for audit purposes · The system shall process requests efficiently under normal load conditions
POSTCONDITION	· Customer accounts are created or updated successfully · Account data is securely stored in the system · All actions are recorded in audit logs

UC NAME	BS_AM_06
SUMMARY	This use case describes how Bank Staff update customer KYC documentation and verify customer identity information in the system.
DEPENDENCY	**BS_UM_02** – All users shall be able to log in to the system using their registered credentials
ACTORS	· **Primary Actor:** Bank Staff · **Secondary Actor:** Customer
PRECONDITIONS	· The Bank Staff is logged into the system. · The customer exists in the system. · The customer provides the required KYC or identity documents. · The system is available and operational.
DESCRIPTION OF MAIN SEQUENCE	· Step 1: The Bank Staff logs into the system. · Step 2: The Bank Staff searches for the customer profile. · Step 3: The system displays the customer's current KYC and identity information. · Step 4: The Bank Staff collects the updated KYC or identity documents from the customer. · Step 5: The Bank Staff enters or uploads the updated customer information into the system. · Step 6: The system validates the entered data and document details. · Step 7: The system saves the updated KYC and identity information. · Step 8: The system confirms that the update was successful.

DESCRIPTION OF ALTERNATIVE SEQUENCE	· Step 1: If login fails, the system denies access. · Step 2: If the customer profile is not found, the system displays an error. · Step 3: If the required KYC documents are missing or incomplete, the system rejects the update. · Step 4: If the entered identity information is invalid, the system prompts correction. · Step 5: If the system fails to save the updated information, an error message is displayed and the previous data remains unchanged.
NON FUNCTIONAL REQ	· The system shall enforce secure login and authentication. · The system shall apply role-based access control so only authorized staff can update KYC information. · The system shall protect customer identity and document data using encryption. · The system shall maintain an audit log of all KYC updates and identity verification actions. · The system shall validate and save updated information efficiently under normal load conditions.
POSTCONDITION	· The customer's KYC documentation and identity information are updated in the system. · The updated data is securely stored. · The update activity is recorded in the audit logs.

UC NAME	BS_AM_07
SUMMARY	This use case describes how a Bank Teller verifies a customer's identity before processing any in-branch transaction.
DEPENDENCY	**BS_UM_02** – All users shall be able to log in to the system using their registered credentials
ACTORS	· **Primary Actor:** Bank Teller · **Secondary Actor:** Customer
PRECONDITIONS	· The Bank Teller is logged into the system. · The customer is present and requests an in-branch transaction. · The customer provides valid identification documents. · The system is available and operational.
DESCRIPTION OF MAIN SEQUENCE	· Step 1: The Bank Teller logs into the system. · Step 2: The customer presents an identification document. · Step 3: The Bank Teller searches for the customer record in the system. · Step 4: The Bank Teller compares the customer's identification document with the information stored in the system. · Step 5: The system displays the customer's identity details for verification. · Step 6: The Bank Teller confirms that the identity information matches. · Step 7: The system records that identity verification was completed successfully.

DESCRIPTION OF ALTERNATIVE SEQUENCE	· Step 1: If login fails, the system denies access. · Step 2: If the customer record is not found, the system displays an error. · Step 3: If the identification document is missing, the verification process cannot continue. · Step 4: If the document details do not match the system record, identity verification fails. · Step 5: If the identification document is invalid or expired, the system rejects the verification process.
NON FUNCTIONAL REQ	· The system shall enforce secure login and authentication. · The system shall ensure that only authorized users can access customer identity information. · The system shall protect identity data using encryption and secure storage. · The system shall maintain an audit log of all identity verification activities. · The system shall support efficient retrieval of customer identity records under normal system load.
POSTCONDITION	· The customer's identity is verified successfully if the main sequence is completed. · The verification activity is recorded in the system.

Transaction Management – Adriano Demiri

UC Name	BS_TM_01
Summary	The Customer shall be able to deposit money into their bank account.
Dependency	BS_UM_02 (Customer must be logged in). Bank staff must verify customer identity before processing any in-branch transaction.
Actors	Primary Actor: Customer. Secondary Actor: Bank System or teller (for in-branch deposits).
Preconditions	The Customer has an active bank account. The Customer is authenticated and logged in. For in-branch deposits, a Bank Teller is available.
Description of the Main Sequence	Step 1: The Customer selects the Deposit option from the banking portal, mobile app, or requests service at the branch counter. Step 2: The Customer specifies the deposit amount and type (cash, or electronic transfer). Step 3: The system (or Bank Teller) validates the deposit details, including source and amount. Step 4: The system credits the account with the deposit amount. Step 5: The system generates a transaction reference and sends a confirmation notification to the Customer. Step 6: The transaction is recorded in the audit trail with timestamp, amount, and actor details.

Description of the Alternative Sequence	Step 1: If the deposit amount is invalid or exceeds defined limits, the system displays an error and prompts correction. Step 2: If the electronic transfer source is unverified, the system places the deposit on hold pending review.
Non functional requirements	Deposit transactions must be processed and reflected in the account balance within 5 seconds for cash and electronic deposits. All deposit activity must be logged in a tamper-proof audit trail.
Postconditions	The Customer’s account balance is updated to reflect the deposit. A transaction record is created. The Customer receives a confirmation notification.

UC Name	BS_TM_02
Summary	The Customer shall be able to withdraw money from their account, either at a bank branch via a Bank Teller or at an ATM using their bank card.
Dependency	BS_UM_02 (Customer must be authenticated). Bank Teller must verify customer identity for in-branch withdrawals.
Actors	Primary Actor: Customer. Secondary Actor: Bank Teller,ATM
Preconditions	The Customer has an active account with sufficient funds. The Customer is authenticated. For ATM withdrawals, the Customer's card is valid and active.
Description of the Main Sequence	Step 1: The Customer requests a withdrawal via the ATM, banking portal, or branch counter. Step 2: The Customer specifies the withdrawal amount. Step 3: The system validates the requested amount against the available balance and daily withdrawal limits. Step 4: If validation passes, the system debits the account and authorises the cash release (ATM) or instructs the Teller. Step 5: The system generates a transaction reference and sends a confirmation notification to the Customer. Step 6: The transaction is recorded in the audit trail.

Description of the Alternative Sequence	Step 1: If the account balance is insufficient, the system declines the request and notifies the Customer. Step 2: If the withdrawal amount exceeds the daily limit, the system displays the limit and prompts the Customer to enter a lower amount. Step 3: If the ATM card is expired or blocked, the ATM retains the card and notifies the Customer to contact the bank.
Non functional requirements	Withdrawal authorisation must complete within 3 seconds. The system must enforce configurable daily and per-transaction withdrawal limits. Failed withdrawal attempts must be logged with reason codes.
Postconditions	The Customer's account is debited by the withdrawal amount. A transaction record is created. The Customer receives a confirmation notification.

UC Name	BS_TM_03
Summary	The Customer shall be able to transfer funds to accounts within the same bank, initiate wire transfers to external bank accounts, and configure automated recurring transfers.
Dependency	BS_UM_02 (Customer must be authenticated and logged in).
Actors	Primary Actor: Customer. Secondary Actor: Banking System (for inter-bank transfers).
Preconditions	The Customer has an active account with sufficient funds. The destination account details are valid. For inter-bank transfers, the external banking system (SWIFT/SEPA/ACH) is available.
Description of the Main Sequence	Step 1: The Customer navigates to the Fund Transfer module. Step 2: The Customer selects the transfer type: internal, inter-bank, or scheduled/recurring. Step 3: The Customer enters the destination account details, amount, and optional transfer date. Step 4: The system validates the source balance, destination account, and transfer limits. Step 5: The Customer confirms the transfer details. Step 6: The system processes the transfer, debits the source account, and initiates the credit to the destination account. Step 7: The system sends a confirmation notification and records the transaction in the audit trail.

Description of the Alternative Sequence	Step 1: If the destination account is invalid or unrecognised, the system displays an error and halts the transaction. Step 2: If the source balance is insufficient, the system declines the transfer and notifies the Customer. Step 3: For inter-bank transfers, if the external system is unavailable, the transaction is queued and the Customer is notified of the expected processing time. Step 4: For scheduled transfers, if the account balance is insufficient on the due date, the system notifies the Customer and may retry based on configuration.
Non functional requirements	Internal transfers must be completed within 5 seconds. Inter-bank transfers must be initiated within 10 seconds of confirmation. Transfer limits must be enforced per account type and customer category. All transfer activity must be logged with full traceability.
Postconditions	The source account is debited and the destination account is credited (for internal transfers). Inter-bank transfer instructions are dispatched to the external system. Recurring transfer schedules are saved and activated. All transactions are recorded in the audit trail.

UC Name	BS_TM_04
Summary	The Customer shall be able to make payments using their issued debit or credit card.
Dependency	BS_UM_02. The Customer must have an active debit or credit card linked to their account.
Actors	Primary Actor: Customer. Secondary Actor: Payment/Bank System.
Preconditions	The Customer has an active debit or credit card. For debit payments, the linked account has sufficient funds. For credit payments, the Customer has available credit. The payment terminal or gateway is operational.
Description of the Main Sequence	Step 1: The Customer initiates a payment at a POS terminal or online checkout. Step 2: The Customer provides card details (physical card swipe/tap, or online card number entry). Step 3: The payment system sends an authorisation request to the bank. Step 4: The system validates the card status, available balance or credit limit, and transaction limits. Step 5: The system approves the authorisation and reserves the funds. Step 6: The transaction is finalised, the account is debited (debit) or the credit balance is updated (credit). Step 7: The Customer and merchant receive confirmation. The transaction is logged in the audit trail.

Description of the Alternative Sequence	Step 1: If the card is blocked or expired, the system declines the transaction and notifies the Customer. Step 2: If funds or credit are insufficient, the transaction is declined. Step 3: If a transaction is flagged as potentially fraudulent, the system may require additional authentication (e.g., OTP) before approving.
Non functional requirements	Payment authorisation responses must be returned within 3 seconds. The system must enforce configurable daily spending limits per card. All payment attempts (approved and declined) must be logged with merchant details, amount, timestamp, and outcome.
Postconditions	The payment is processed and the Customer's account or credit balance is updated. A transaction record is created. The Customer and merchant receive confirmation.

UC Name	BS_TM_05
Summary	Bank Staff shall be able to process bulk payment disbursements such as salary payments and utility bill settlements.
Dependency	BS_UM_02 The customer or corporate client must have authorised the bulk payment batch.
Actors	Primary Actor: Bank Staff.
Preconditions	Bank Staff are logged in with appropriate privileges. A validated and authorised bulk payment file or instruction set is available. The source account has sufficient funds to cover all disbursements.
Description of the Main Sequence	Step 1: Bank Staff navigates to the Bulk Payment Processing module. Step 2: Bank Staff uploads or selects the bulk payment batch (e.g., payroll file, bill payment list). Step 3: The system validates each payment record in the batch for completeness and correctness. Step 4: The system calculates the total disbursement amount and verifies it against the source account balance. Step 5: Bank Staff confirms and submits the batch for processing. Step 6: The system processes each payment in the batch, debiting the source account and crediting each recipient. Step 7: The system generates a batch processing report and notifies Bank Staff of the outcome. Step 8: All disbursements are recorded in the audit trail.

Description of the Alternative Sequence	Step 1: If individual payment records contain errors (invalid account numbers, missing fields), the system flags them and allows Bank Staff to correct or exclude them before resubmitting. Step 2: If the source account balance is insufficient to cover the full batch, the system halts processing and notifies Bank Staff. Step 3: If the external payment system is unavailable, the batch is queued and Bank Staff are notified of the delay.
Non functional requirements	Bulk payment batches must be processed within a defined service window. The system must support concurrent batch submissions without data corruption. All batch transactions must be fully traceable with individual and aggregate audit records.
Postconditions	All valid payments in the batch are disbursed. A batch processing report is available for Bank Staff review. All transactions are recorded in the audit trail. Failed or rejected records are flagged for remediation.

UC Name	BS_TM_06
Summary	The Customer shall be able to manage and save beneficiaries for repeat fund transfers through the online banking portal or mobile app.
Dependency	BS_UM_02. BS_TM_03 (Beneficiaries are used within the Fund Transfer process).
Actors	Primary Actor: Customer. Secondary Actor: None
Preconditions	The Customer has an active account and is authenticated. The Customer wishes to add, edit, or remove a saved beneficiary.
Description of the Main Sequence	Step 1: The Customer navigates to the Beneficiary Management section in the online portal or mobile app. Step 2: The Customer selects an action: Add, Edit, or Delete a beneficiary. Step 3 (Add): The Customer enters the beneficiary's name, account number, bank details, and an optional nickname. Step 4: The system validates the beneficiary account details. Step 5: The system saves the beneficiary to the Customer's profile. Step 6: The Customer receives a confirmation notification. The action is recorded in the audit trail.

Description of the Alternative Sequence	Step 1: If the beneficiary account details are invalid or unrecognised, the system displays an error and prompts the Customer to correct the information. Step 2 (Edit): The Customer selects an existing beneficiary, updates the required fields, and confirms. The system saves the changes. Step 3 (Delete): The Customer selects a beneficiary and confirms deletion. The system removes the beneficiary from the saved list. This does not affect past transactions. Step 4: If the Customer attempts to add a duplicate beneficiary, the system warns the Customer and requests confirmation to proceed.
Non functional requirements	Beneficiary data must be saved and immediately available for use within the Fund Transfer module. All beneficiary changes must be logged in the audit trail. Beneficiary data must be encrypted at rest.
Postconditions	The beneficiary list is updated (added, edited, or deleted). The Customer can immediately use saved beneficiaries in fund transfer transactions. All changes are recorded in the audit trail.

Loan Management – Emanuele Berbiu

UC Name	BS_LM_01
Summary	A registered customer submits a loan application; the system validates it and forwards it to bank staff for review and approval or rejection.
Dependency	None
Actors	Customer (Primary), Bank Staff, Bank System
Preconditions	Customer is registered and logged in.
Main Sequence	**Step 1:** Customer logs into system **Step 2:** Fills loan application form **Step 3:** Submits application **Step 4:** System validates application **Step 5:** Application is sent to Bank Staff **Step 6:** Staff reviews application **Step 7:** Staff approves or rejects application **Step 8:** System notifies customer
Alt. Sequence	• Invalid data → System shows error • Missing documents → Request additional info
Non-Functional Req.	Secure data handling · Fast processing time
Postconditions	Loan application is approved or rejected.

UC Name	BS_LM_02
Summary	A customer with an active loan selects a repayment method (online or branch), makes a payment, and the system updates the loan balance accordingly.
Dependency	BS_LM_01 (Depends on approved loan)
Actors	Customer (Primary), Bank Staff, Bank System
Preconditions	Customer has an active loan.
Main Sequence	Step 1: Customer selects repayment option Step 2: Chooses online or branch Step 3: Makes payment Step 4: System processes payment Step 5: Loan balance is updated Step 6: Confirmation sent
Alt. Sequence	• Payment failure → Retry • Incorrect amount → Show error
Non-Functional Req.	Transaction security · High availability
Postconditions	Loan balance is updated.

UC Name	BS_LM_03
Summary	When a loan application exceeds a defined threshold, it is escalated to the Bank Director for a second-level review and final approval or rejection.
Dependency	Depends on BS_LM_01
Actors	Customer (Primary), Bank Staff, Bank System
Preconditions	Loan exceeds defined threshold.
Main Sequence	Step 1: Staff reviews application Step 2: System detects high-value loan Step 3: Application sent to Director Step 4: Director reviews Step 5: Director approves or rejects Step 6: System notifies customer
Alt. Sequence	• Missing info → Request clarification
Non-Functional Req.	Authorization control · Audit logging
Postconditions	Loan is approved or rejected by Director.

UC Name	BS_LM_04
Summary	A customer with an approved loan views the amortization schedule, including monthly installments, interest, and remaining balance. The system calculates and displays the repayment plan.
Dependency	BS_LM_01 (Depends on approved loan)
Actors	Customer (Primary), Bank System
Preconditions	Customer has an active approved loan.
Main Sequence	Step 1: Customer opens loan details Step 2: Selects amortization schedule Step 3: System calculates repayment plan Step 4: System displays installments and remaining balance
Alt. Sequence	• Invalid action → Show error
Non-Functional Req.	Accurate calculations · Fast response time
Postconditions	Amortization schedule displayed.

Card Management – Emanuele Berbiu

UC Name	BS_CM_01
Summary	A customer with a valid bank account applies for a new card; bank staff reviews the request and either issues the card or rejects the application.
Dependency	BS_UM_01 (Customer account must exist)
Actors	Customer (Primary), Bank Staff, Bank System
Preconditions	Customer has a valid account.
Main Sequence	Step 1: Customer fills card application Step 2: Submits request Step 3: System validates Step 4: Staff reviews Step 5: Approve or reject Step 6: Notify customer
Alt. Sequence	• Invalid data → Show error
Non-Functional Req.	Data security · Fast processing
Postconditions	Card issued or request rejected.

UC Name	BS_CM_02
Summary	A customer modifies the spending limit on an existing card through card settings; the system validates and saves the new limit in real time.
Dependency	BS_CM_01 (Card must exist)
Actors	Customer (Primary), Bank System
Preconditions	Customer owns a valid card.
Main Sequence	Step 1: Customer opens card settings Step 2: Enters new limit Step 3: System validates Step 4: Saves changes Step 5: Notifies customer
Alt. Sequence	• Invalid limit → Show error
Non-Functional Req.	Real-time update · Security
Postconditions	Spending limit updated.

UC Name	BS_CM_03
Summary	A customer temporarily blocks or unblocks their card through card settings; the system validates the action and updates the card status in real time.
Dependency	BS_CM_01 (Card must exist)
Actors	Customer (Primary), Bank System
Preconditions	Customer owns a valid card.
Main Sequence	Step 1: Customer opens card settings Step 2: Selects block or unblock option Step 3: System validates request Step 4: Saves changes Step 5: Notifies customer
Alt. Sequence	• Invalid action → Show error
Non-Functional Req.	Real-time update · Security
Postconditions	Card status updated (blocked or unblocked).

Fraud Detection – Joandri Domi

UC Name	BS_FD_01
Summary	Bank Staff reviews an account flagged by the fraud detection system, investigates the suspicious transactions, and applies an appropriate corrective action.
Dependency	None. Flagged accounts are generated automatically by the fraud detection engine. A fraud case submitted via BS_FD_02 may also appear in this queue, but BS_FD_01 can execute independently.
Actors	Primary: Bank Staff (Compliance / Back-Office). Secondary: System; Bank Director (if the case is escalated).
Preconditions	Bank Staff is logged in with fraud investigation access. At least one account has been flagged by the fraud detection engine or submitted via BS_FD_02.
Main Sequence	Step 1: System notifies Bank Staff of a flagged account and displays it on the Fraud Management Dashboard. Step 2: Bank Staff selects the account and reviews its transaction history, risk score, and flag reason. Step 3: Bank Staff determines the activity is genuinely suspicious and selects a corrective action (e.g. freeze account, block card, file SAR). Step 4: System applies the action, records it in the audit log, and notifies the customer. Step 5: Bank Staff records investigation notes and closes the case.
Alternative Sequence	Step 1: Bank Staff determines the flagged activity is legitimate. Step 2: Bank Staff marks the alert as a false positive and enters a justification note. Step 3: System clears the flag and logs the decision. No corrective action is applied.

Non-Functional Requirements	Security: All investigation actions are recorded in a tamper-proof audit log (user ID, IP address, timestamp). Access is restricted by role. Retention: Investigation records must be retained for a minimum of 7 years per BS_AR_03 reporting requirements.
Postconditions	The flagged account has been reviewed. A corrective action has been applied and logged, or the alert has been cleared as a false positive.

UC Name	BS_FD_02
Summary	A Customer reports suspected fraudulent activity on their account through the online banking portal, creating a fraud case for internal review by Bank Staff.
Dependency	None. This is a customer-initiated use case. A successful submission creates a fraud case that is subsequently handled under BS_FD_01.
Actors	Primary: Customer. Secondary: System; Bank Staff (notified to follow up via BS_FD_01).
Preconditions	Customer is logged in to the online banking portal. Customer has an active account and has identified activity they believe to be unauthorised.
Main Sequence	Step 1: Customer navigates to the Report Fraud section and selects the suspicious transaction(s) from their history. Step 2: Customer completes the fraud report form with a description of the issue and date of discovery. Step 3: Customer submits the report. Step 4: System validates the form, creates a fraud case with a unique reference number, and flags the associated transactions for internal review. Step 5: System sends a confirmation to the Customer (email/SMS) with the reference number and notifies Bank Staff via BS_FD_01.
Alternative Sequence	Step 1: Customer cannot identify a specific transaction to select. Step 2: Customer describes the suspected fraud in the free-text description field without attaching a transaction. Step 3: System creates the fraud case without a linked transaction and flags it for manual review by Bank Staff.

Non-Functional Requirements	Availability: The fraud reporting feature must maintain 99.9% uptime as a critical customer-facing function. Accessibility: Must comply with WCAG 2.1 AA standards. Performance: Form submission and confirmation must complete within 2 seconds.
Postconditions	A fraud case has been created with a unique reference number. The Customer has received confirmation and the case is visible to Bank Staff in the BS_FD_01 queue.

UC Name	BS_FD_03
Summary	The Bank Director receives automated real-time alerts when the system detects unusual account activity, fraud flags, or system anomalies.
Dependency	May be triggered by an escalation action within BS_FD_01 when Bank Staff escalates a high-risk case to the Bank Director. Also fires independently when the fraud detection engine crosses an alert threshold.
Actors	Primary: Bank Director. Secondary: System (Fraud Detection Engine); Bank Staff (if investigation is delegated back via BS_FD_01); Administrator (for system anomaly alerts).
Preconditions	Bank Director has a system profile with notification channels configured. The Fraud Detection Engine and Anomaly Monitor are active. An event meeting the defined alert threshold has occurred, or Bank Staff has escalated a case via BS_FD_01.
Main Sequence	Step 1: System detects an unusual pattern or threshold breach and generates an alert record (event type, severity, affected account, timestamp). Step 2: System delivers the alert to the Bank Director via the executive dashboard and configured external channels (SMS / email). Step 3: Bank Director reviews the alert and selects a response: mark as reviewed, delegate investigation to Bank Staff (returning to BS_FD_01), or escalate to compliance or law enforcement. Step 4: System records the Bank Director's response in the audit trail.
Alternative Sequence	Step 1: The alert is not acknowledged by the Bank Director within the defined SLA (Service Level Agreement) window. Step 2: System automatically escalates the alert to a designated backup executive contact. Step 3: Backup contact receives the alert and follows the same response steps. System logs the escalation event.

Non-Functional Requirements	Performance: Alerts must be delivered within 30 seconds of event detection. Security: Dashboard access is restricted to the Bank Director and Administrator, enforced by mandatory MFA and IP whitelisting. Audit: All alert events and director responses feed into the fraud analytics accessible via BS_AR_03.
Postconditions	The Bank Director (or backup contact) has been notified and a response has been recorded in the audit trail. If delegated, a case is returned to the BS_FD_01 queue.

Facilities Management – Joandri Domi

UC Name	BS_FCM_01
Summary	The Facilities Manager creates and publishes shift schedules for all facilities and support staff across branches, ensuring full post coverage and notifying staff of their assignments.
Dependency	Depends on BS_UM_01. Staff members must have active system profiles before they can be assigned to a roster. Published rosters are a prerequisite for BS_FCM_03.
Actors	Primary: Facilities Manager. Secondary: System; Facilities & Support Staff (receive and view their assigned schedules via BS_FCM_03).
Preconditions	Facilities Manager is logged in with roster management access. Active system profiles exist for all staff members to be scheduled (created via BS_UM_01).
Main Sequence	Step 1: Facilities Manager opens the Roster Management module and selects a branch and scheduling period. Step 2: Facilities Manager assigns staff members to shifts, specifying role, post, location, and shift times. Step 3: Facilities Manager submits the draft roster for system validation. Step 4: System confirms the roster is valid (no duplicate assignments, uncovered posts, or working-hours violations). Step 5: Facilities Manager approves and publishes the roster. System notifies all assigned staff of their shifts.

Alternative Sequence	Step 1: System detects a conflict during validation (e.g. a mandatory post is uncovered or a staff member is double-booked). Step 2: System highlights the conflicting entries and blocks publication. Step 3: Facilities Manager resolves the conflicts and re-submits the roster for validation. Step 4: System confirms the roster is now valid and Facilities Manager proceeds to publish.
Non-Functional Requirements	Availability: The roster module must be accessible 24/7 as shift changes can occur at any time. Performance: Saves and publications must complete within 2 seconds; staff notifications must be delivered within 2 minutes of publication.
Postconditions	The roster has been published for the selected branch and period. All assigned staff have been notified. The schedule is recorded and available to staff via BS_FCM_03.

UC Name	BS_FCM_02
Summary	The Facilities Manager reviews IT support tickets, assigns them to IT Technicians with a priority level and deadline, and tracks progress through to resolution.
Dependency	Depends on BS_FCM_03. IT support tickets are created when Facilities & Support Staff submit issue reports through their portal. The Facilities Manager cannot assign tickets that do not yet exist.
Actors	Primary: Facilities Manager. Secondary: Bank Staff (submits the ticket); IT Technician (resolves the ticket via BS_FCM_03); System.
Preconditions	Facilities Manager is logged in with IT support management access. At least one open IT support ticket exists in the system, submitted via BS_FCM_03 or by bank staff directly.
Main Sequence	Step 1: Facilities Manager opens the IT Support Ticket module and selects an unassigned ticket. Step 2: Facilities Manager assigns the ticket to an available IT Technician and sets a resolution deadline based on priority. Step 3: System notifies the IT Technician of the assignment. Step 4: IT Technician resolves the issue and marks the ticket as completed with resolution notes. Step 5: System checks if ticket was overdue. Alerts the Facilities Manager. Step 6: Ticket is closed by system.
Alternative Sequence	Step 1: Facilities Manager determines the submitted resolution is incomplete or incorrect. Step 2: Facilities Manager rejects the closure and adds a rejection note explaining what remains unresolved. Step 3: System re-opens the ticket, notifies the IT Technician, and resets the resolution deadline.

	Step 4: IT Technician performs the additional work and re-submits for review.
Non-Functional Requirements	Performance: Ticket status changes must be reflected on the dashboard within 5 seconds. Reporting: Resolution times per technician feed into the facilities performance reports generated via BS_AR_04. Access: IT Technicians may only view and update their own assigned tickets.
Postconditions	The ticket has been assigned, resolved, and closed. Resolution time and all activity are logged. SLA compliance data is available for BS_AR_04 reporting.

UC Name	BS_FCM_03
Summary	Facilities & Support Staff log their shift attendance and submit task completion or incident reports through the Facilities Staff Portal.
Dependency	Depends on BS_FCM_01. A shift schedule must have been published by the Facilities Manager before a staff member can check in to an assigned shift.
Actors	Primary: Facilities & Support Staff (security guard, cleaner, IT technician, driver, or maintenance worker). Secondary: Facilities Manager (reviews attendance and submitted reports); System.
Preconditions	The staff member is logged in to the Facilities Staff Portal. A shift has been assigned and published for the staff member via BS_FCM_01.
Main Sequence	Step 1: Staff member views their assigned schedule and selects Check In at the start of their shift. Step 2: System records the check-in timestamp and marks the shift as active. Step 3: Staff member performs their duties and submits a task completion or incident report. Step 4: System saves the report and notifies the Facilities Manager. Step 5: At shift end, staff member selects Check Out. System records the timestamp, marks the shift as completed, and calculates the total duration.
Alternative Sequence	Step 1: The staff member does not check out within a defined window after the scheduled shift end. Step 2: System flags the shift as having a missing check-out and notifies the Facilities Manager. Step 3: Facilities Manager verifies the shift end time and manually closes the shift.

	Step 4: System logs the manual closure and records the discrepancy for audit purposes.
Non-Functional Requirements	Security: Staff access is strictly limited to facilities modules; no financial or customer data is accessible. Accessibility: The portal must be usable on mobile devices to support on-site logging. Performance: Check-in submissions and report saves must process within 2 seconds.
Postconditions	The shift is logged with verified check-in and check-out times. All submitted reports are saved and visible to the Facilities Manager. Attendance data is available for payroll and BS_AR_04 reporting.

Costumer Service – Kleo Miraka

UC Name	*UC_BS_CS_01 – View Nearby ATMs and Branches*
Summary	*Customer views real-time information about nearby ATMs and bank branches.*
Dependency	*Requires login*
Actors	***Primary Actor:** Customer* *Secondary Actor: Location/Mapping Service*
Preconditions	*Customer has internet access.* *Location services enabled (if using mobile app).*
Description of the Main Sequence	• *Step 1: Customer opens banking app* • *Step 2: Navigates to "Locations" section* • *Step 3: System retrieves nearby ATMs/branches* • *Step 4: Displays locations, hours, and availability* • *Step 5: Customer views details*
Description of the Alternative Sequence	• *Step 1: Location disabled → Customer enters location manually* • *Step 2: No nearby results → System displays message*
Non functional requirements	*Real-time data updates* *Fast response time*
Postconditions	*Customer views accurate ATM/branch information*

UC Name	*UC_BS_CS_02 – Manage Branch Appointments*
Summary	*Customer schedules and manages in-branch appointments.*
Dependency	*Requires login*
Actors	***Primary Actor:*** *Customer* *Secondary Actor: Banking System*
Preconditions	*Customer is logged in*
Description of the Main Sequence	• *Step 1: Customer selects "Appointments"* • *Step 2: Chooses branch, date, and time* • *Step 3: Confirms appointment* • *Step 4: System saves appointment* • *Step 5: Customer can view/edit/cancel appointment*
Description of the Alternative Sequence	• *Step 1: Selected time unavailable → Prompt new selection* • *Step 2: Cancellation → System updates schedule*
Non functional requirements	*High availability* *Confirmation notifications*
Postconditions	*Appointment scheduled or updated*

UC Name	*UC_BS_CS_03 – Submit Feedback and Complaints*
Summary	*Customer submits feedback or formal complaints.*
Dependency	*Requires login*
Actors	***Primary Actor**: Customer* *Secondary Actor: Banking System*
Preconditions	*Customer is logged in*
Description of the Main Sequence	• *Step 1: Customer selects "Feedback/Complaints"* • *Step 2: Enters details of feedback/complaint* • *Step 3: Submits form* • *Step 4: System records and acknowledges submission*
Description of the Alternative Sequence	• *Step 1: Missing details → Prompt completion* • *Step 2: System error → Show failure message*
Non functional requirements	*Data confidentiality* *Reliable storage*
Postconditions	*Feedback/complaint recorded in system*

UC Name	*UC_BS_CS_04 – Request Account Documents*
Summary	*Customer requests official account-related documents.*
Dependency	*Requires login*
Actors	***Primary Actor:*** *Customer* *Secondary Actor: Banking System*
Preconditions	*Customer is logged in*
Description of the Main Sequence	• *Step 1: Customer selects "Request Documents"* • *Step 2: Chooses document type (statement, certificate, etc.)* • *Step 3: Submits request* • *Step 4: System processes request* • *Step 5: Document is generated and delivered*
Description of the Alternative Sequence	• *Step 1: Invalid request → System shows error* • *Step 2: Processing delay → Notification sent later*
Non functional requirements	*Secure document handling* *Timely processing*
Postconditions	*Requested document delivered to customer*

UC Name	*UC_BS_CS_05 – Manage Complaints and Disputes*
Summary	*Bank staff handle customer complaints and resolve disputes.*
Dependency	*Depends on UC_BS_CS_03*
Actors	***Primary Actor:**Bank Staff* *Secondary Actor: Banking System*
Preconditions	*Complaint exists in system.* *Staff is authenticated.*
Description of the Main Sequence	• *Step 1: Staff logs into system* • *Step 2: Views complaint list* • *Step 3: Selects a complaint* • *Step 4: Investigates issue* • *Step 5: Updates status and resolution* • *Step 6: System notifies customer*
Description of the Alternative Sequence	*Step 1: Insufficient information → Request more details* *Step 2: Escalation required → Forward to higher authority*
Non functional requirements	*Audit trail for actions.* *Timely resolution.*
Postconditions	*Complaint resolved or escalated*

UC Name	*UC_BS_CS_06 – Generate Account Statements*
Summary	*Bank Teller generate and send account statements.*
Dependency	*Depends on UC_BS_CS_03(when requested)* *None(when is scheduled)*
Actors	***Primary Actor:*** *Bank Teller* *Secondary Actor**:** Banking System*
Preconditions	*Staff is logged in* *Customer account exists*
Description of the Main Sequence	• *Step 1: Teller selects customer account* • *Step 2: Chooses statement period* • *Step 3: Generates statement* • *Step 4: Sends to customer (email/portal)*
Description of the Alternative Sequence	*Step 1: Invalid date range → Prompt correction* *Step 2: Delivery failure → Retry or notify staff*
Non functional requirements	*Accurate data generation* *Secure delivery*
Postconditions	*Statement delivered to customer*

UC Name	*UC_BS_CS_07 – Receive Notifications and Alerts*
Summary	*Customer receives real-time alerts for account activities.*
Dependency	*Requires login*
Actors	***Primary Actor:*** *Customer* *Secondary Actor: Banking System*
Preconditions	*Customer account is active* *Notification preferences configured*
Description of the Main Sequence	● *Step 1: System detects account activity (transaction, login, etc.)* ● *Step 2: Generates notification* ● *Step 3: Sends alert to customer (app/SMS/email)* ● *Step 4: Customer views notification*
Description of the Alternative Sequence	*Step 1: Notification delivery failure → Retry or fallback channel* *Step 2: Disabled notifications → No alert sent*
Non functional requirements	*Real-time delivery.* *High reliability.*
Postconditions	*Customer informed of account activity*

Audit and Reporting – Erind Shabani

UC Name	BS_AR_01
Summary	The Bank Director accesses complete, tamper-proof audit trails within the Bank Management System to verify that all user actions, financial transactions, and administrative changes comply with banking regulations and internal policies.
Dependency	BS_UM_02 (Bank Director must be logged in). Audit trail data is generated by all system modules as a prerequisite.
Actors	Primary Actor: Bank Director. Secondary Actor: System (Audit Log Engine).
Preconditions	The Bank Director is authenticated and has an active session. The Bank Director's role has been granted audit trail access by the Administrator. Audit log records exist in the system.
Description of the Main Sequence	Step 1: The Bank Director navigates to the Audit & Compliance module from the executive dashboard. Step 2: The Bank Director applies search filters – date range, user ID, action type, module, branch, or IP address – to scope the audit query. Step 3: The system retrieves and displays the matching audit records, each showing: user ID, timestamp, IP address, action performed, affected record, and outcome. Step 4: The Bank Director reviews the records and drills down into individual entries to view full action details. Step 5: The Bank Director optionally exports the audit report in PDF or CSV format for regulatory submission or internal documentation. Step 6: The system logs the Bank Director's access to the audit trail, recording the session ID, filters used, timestamp, and IP address.
Description of the Alternative Sequence	Step 1: If the selected filter combination returns no records, the system displays a 'No matching audit records found' message and suggests broadening the search criteria. Step 2: If the Bank Director attempts to access audit records outside their permitted scope (e.g., another director's personal access logs), the system blocks the query and displays an authorisation error. Step 3: If the export fails due to a system error, the system displays an error notification and prompts the Bank Director to retry or contact the Administrator. Step 4: If the audit log service is temporarily unavailable, the system displays a degraded-mode warning and provides the last cached snapshot with a clear timestamp.
Non-Functional Requirements	Security: Audit trail records must be tamper-proof and write-protected; no user — including the Administrator — may delete or silently modify a log entry.

	Performance: Audit queries must return results within 3 seconds for datasets of up to 10 million records. Retention: Audit logs must be retained for a minimum of 7 years in compliance with applicable banking and financial regulations. Access Control: Access is restricted exclusively to the Bank Director and Administrator, enforced by RBAC with mandatory MFA on every login. Availability: The audit log module must maintain 99.9% uptime as a critical compliance function.
Postconditions	The Bank Director has reviewed the requested audit trail records. If exported, an official audit report has been generated and is available for download. The Bank Director's access session and all queries are permanently recorded in the audit log.

UC Name	BS_AR_02
Summary	The Bank Director accesses customer analytics reports covering account growth trends, activity patterns, and customer segmentation data to support strategic decision-making and business planning.
Dependency	BS_UM_02 (Bank Director must be logged in). Customer account and transaction data must be available in the system.
Actors	Primary Actor: Bank Director. Secondary Actor: System (Analytics & Reporting Engine).
Preconditions	The Bank Director is authenticated and has an active session. The Bank Director's role has been granted access to the analytics reporting module. Customer account and transaction data exists in the system and is up to date.
Description of the Main Sequence	Step 1: The Bank Director navigates to the Analytics & Reporting module from the executive dashboard. Step 2: The Bank Director selects the Customer Analytics report category. Step 3: The Bank Director configures the report parameters – date range, branch, account type, customer segment (individual, business, corporate), or customer tier (standard, premium, VIP). Step 4: The system generates and displays the analytics report, including: new account acquisition counts, dormant account figures, account type distribution, activity trend charts, and segmentation breakdowns. Step 5: The Bank Director reviews the report, applies additional drill-down filters where needed, and interprets the visualised data. Step 6: The Bank Director optionally exports the report in PDF or CSV format. Step 7: The system logs the report access event in the audit trail, including parameters used, timestamp, and the Bank Director's user ID.
Description of the Alternative Sequence	Step 1: If insufficient data exists for the selected parameters (e.g., a newly opened branch with no transaction history), the system displays a 'Insufficient data for the selected period' message and suggests alternative date ranges. Step 2: If the analytics engine is processing a large dataset and the report takes longer than expected, the system displays a progress indicator and delivers the report asynchronously, notifying the Bank Director when it is ready. Step 3: If the export fails, the system displays an error and allows the Bank Director to retry. Step 4: If the Bank Director attempts to access analytics for branches outside their assigned scope, the system filters the data to only permitted locations and displays a scope notification.

Non-Functional Requirements	Performance: Standard analytics reports must generate within 5 seconds. Large cross-branch reports may be processed asynchronously with a maximum delivery time of 60 seconds. Data Accuracy: Reports must reflect data that is no more than 5 minutes stale under normal operating conditions. Access Control: Customer analytics data is restricted to the Bank Director role; access is enforced by RBAC and MFA. Data Privacy: Customer analytics must be aggregated and anonymised where required by GDPR; no raw personal identifiers should be exposed in summary reports. Audit: All report generation events, including filters applied and export actions, must be logged in the tamper-proof audit trail.
Postconditions	The Bank Director has reviewed the customer analytics report for the selected parameters. If exported, the report file is available for download. The report access event is permanently recorded in the audit trail.

UC Name	BS_AR_03
Summary	The Bank Director and authorised Bank Staff access fraud detection analytics reports, including summaries of flagged transactions, fraud case outcomes, alert trend data, and false-positive rates, to support oversight, pattern analysis, and strategic fraud prevention.
Dependency	BS_UM_02 (User must be authenticated). BS_FD_01 and BS_FD_03 generate the fraud investigation and alert records that populate this analytics view.
Actors	Primary Actors: Bank Director; Bank Staff (Compliance / Back-Office — with restricted access to aggregated analytics only). Secondary Actor: System (Fraud Detection Analytics Engine).
Preconditions	The Bank Director or authorised Bank Staff is authenticated and has an active session. The user's role has been granted access to the fraud analytics module by the Administrator. Fraud detection records — flagged transactions, case outcomes, and alert logs — exist in the system.
Description of the Main Sequence	Step 1: The user navigates to the Fraud Detection Analytics module from their role-specific dashboard. Step 2: The user selects a report type: Flagged Transaction Summary, Fraud Case Outcome Report, Alert Trend Analysis, or False-Positive Rate Report. Step 3: The user configures report parameters – date range, branch, account type, alert severity, or fraud category. Step 4: The system generates and displays the analytics report, including: total flagged transaction counts, resolved vs. open case figures, trend charts over the selected period, and breakdown by fraud category. Step 5: The Bank Director may drill down into specific alert categories or branches for deeper analysis. Bank Staff view is limited to aggregated summaries without access to individual customer identifiers. Step 6: The user optionally exports the report in PDF or CSV format. Step 7: The system logs the report access event in the audit trail, including user ID, role, filters used, and timestamp.
Description of the Alternative Sequence	Step 1: If no fraud records exist for the selected parameters, the system displays a 'No fraud activity recorded for the selected criteria' message. Step 2: If Bank Staff attempts to access individual customer-level fraud records (beyond their permitted aggregated view), the system blocks the query and displays an authorisation error. Step 3: If the analytics engine is processing a large dataset, the system delivers the report asynchronously and notifies the user when it is ready, with a maximum wait time of 60 seconds. Step 4: If the export fails, the system displays an error and prompts the user to retry.

Non-Functional Requirements	Access Control: The Bank Director has full access to fraud analytics including individual case details. Bank Staff access is restricted to aggregated summaries only, enforced by RBAC. Performance: Standard fraud analytics reports must generate within 5 seconds. Complex trend analyses spanning multiple years may be processed asynchronously. Security: Access to fraud analytics is protected by mandatory MFA. The Bank Director's access is additionally restricted by IP whitelisting per the Privileged Access Management policy. Retention: Fraud analytics records and underlying case data must be retained for a minimum of 7 years. Audit: All fraud analytics access events — including user, role, filters used, and export actions — are permanently logged in the tamper-proof audit trail and feed into BS_AR_01 compliance reporting.
Postconditions	The user has reviewed the fraud detection analytics report for the selected parameters. If exported, the report file is available for download. The report access event is permanently recorded in the audit trail.

UC Name	BS_AR_04
Summary	The Facilities Manager generates comprehensive facilities reports covering maintenance status, security incidents, asset conditions, and staff attendance across all branches, for submission to branch managers and the Bank Director.
Dependency	BS_FCM_01 (shift and attendance data), BS_FCM_02 (IT support ticket resolution data), BS_FCM_03 (check-in/check-out and incident report data), and BS_FCM_04 (maintenance task completion records) must be populated for the report to contain complete data.
Actors	Primary Actor: Facilities Manager. Secondary Actor: System (Facilities Reporting Engine); Bank Director (report recipient).
Preconditions	The Facilities Manager is authenticated and has an active session. The Facilities Manager's role has been granted access to the facilities reporting module. Relevant operational data — attendance logs, maintenance records, incident reports, and asset status records — exists in the system for the selected reporting period.
Description of the Main Sequence	Step 1: The Facilities Manager navigates to the Facilities Reporting module. Step 2: The Facilities Manager selects the report type: Maintenance Status Report, Security Incident Report, Asset Condition Report, Staff Attendance Report, or a combined Facilities Overview Report. Step 3: The Facilities Manager configures the report parameters – branch, date range, staff category, asset type, or incident severity. Step 4: The system compiles data from the relevant modules: maintenance task records (BS_FCM_04), shift attendance logs (BS_FCM_03), IT support ticket outcomes (BS_FCM_02), and incident reports submitted by facilities staff. Step 5: The system generates and displays the facilities report with summaries, status indicators, and any overdue or unresolved items highlighted. Step 6: The Facilities Manager reviews the report, adds any commentary or notes, and optionally exports it in PDF or CSV format. Step 7: The Facilities Manager submits the finalised report to the branch manager and/or Bank Director through the internal messaging module. Step 8: The system logs the report generation event in the audit trail, recording the Facilities Manager's user ID, report type, parameters, and timestamp.
Description of the Alternative Sequence	Step 1: If data for a specific module or branch is incomplete for the selected period (e.g., a branch has no check-out records for a shift), the system highlights the data gap with a warning indicator and generates the report with the available data, noting what is missing.

	Step 2: If the selected report parameters return no records, the system displays a 'No facilities activity recorded for the selected criteria' message and prompts the Facilities Manager to adjust the filters. Step 3: If the report generation takes longer than expected due to a large dataset, the system processes the report asynchronously and notifies the Facilities Manager when it is ready. Step 4: If the export or report submission fails, the system displays an error and allows the Facilities Manager to retry.
Non-Functional Requirements	Performance: Standard facilities reports must generate within 5 seconds. Large multi-branch reports may be processed asynchronously with a maximum delivery time of 60 seconds. Access Control: The Facilities Reporting module is restricted to the Facilities Manager role. The Facilities Manager has no access to customer financial data, banking transactions, or core banking modules — report scope is strictly limited to facilities and operational data. Data Completeness: The system must surface data gaps or missing records clearly within the report rather than silently omitting them. Retention: Facilities reports and the underlying operational records must be retained for a minimum of 5 years for compliance and asset lifecycle tracking. Audit: All report generation events, export actions, and submissions are permanently logged with the Facilities Manager's user ID and timestamp.
Postconditions	The facilities report has been generated and reviewed by the Facilities Manager. If exported, the report file is available for download. If submitted, the report has been delivered to the branch manager and/or Bank Director via the internal messaging module. The report generation event is permanently recorded in the audit trail.

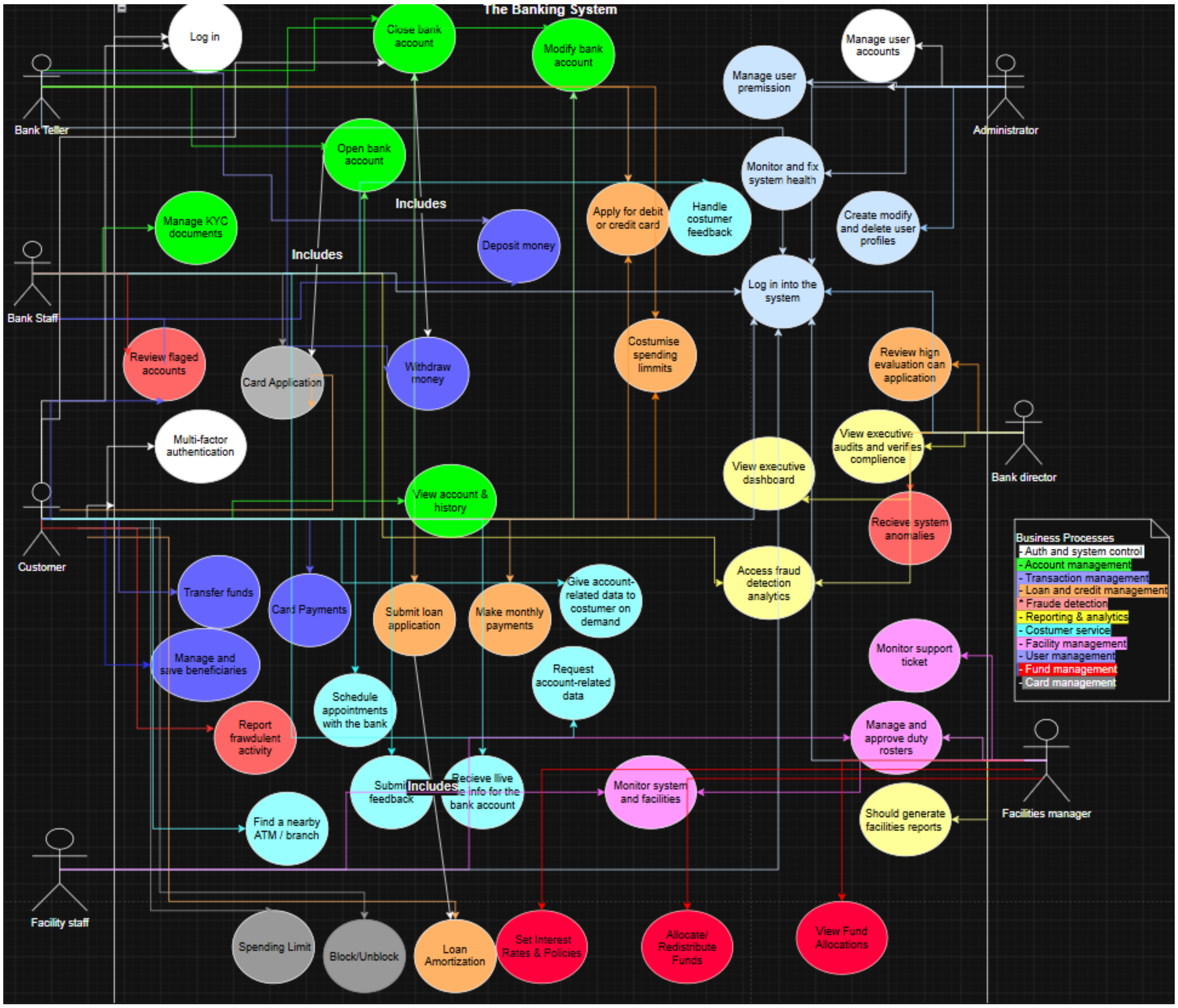
Diagrams

*Every diagram you will be able to find them in the git repository with this link https://github.com/erindkotobelli12/BankManagementSystem

*The Erd Diagram and Class Diagram are too large to be put in the documentation but you can open them in in the repository with the .svg file format. If you want to see them more detailed ,you can open them at this link https://app.diagrams.net/ for the files that are with .drawio format. The files that are with the format .bpmn you can open them at this link https://bpmn.io/ .

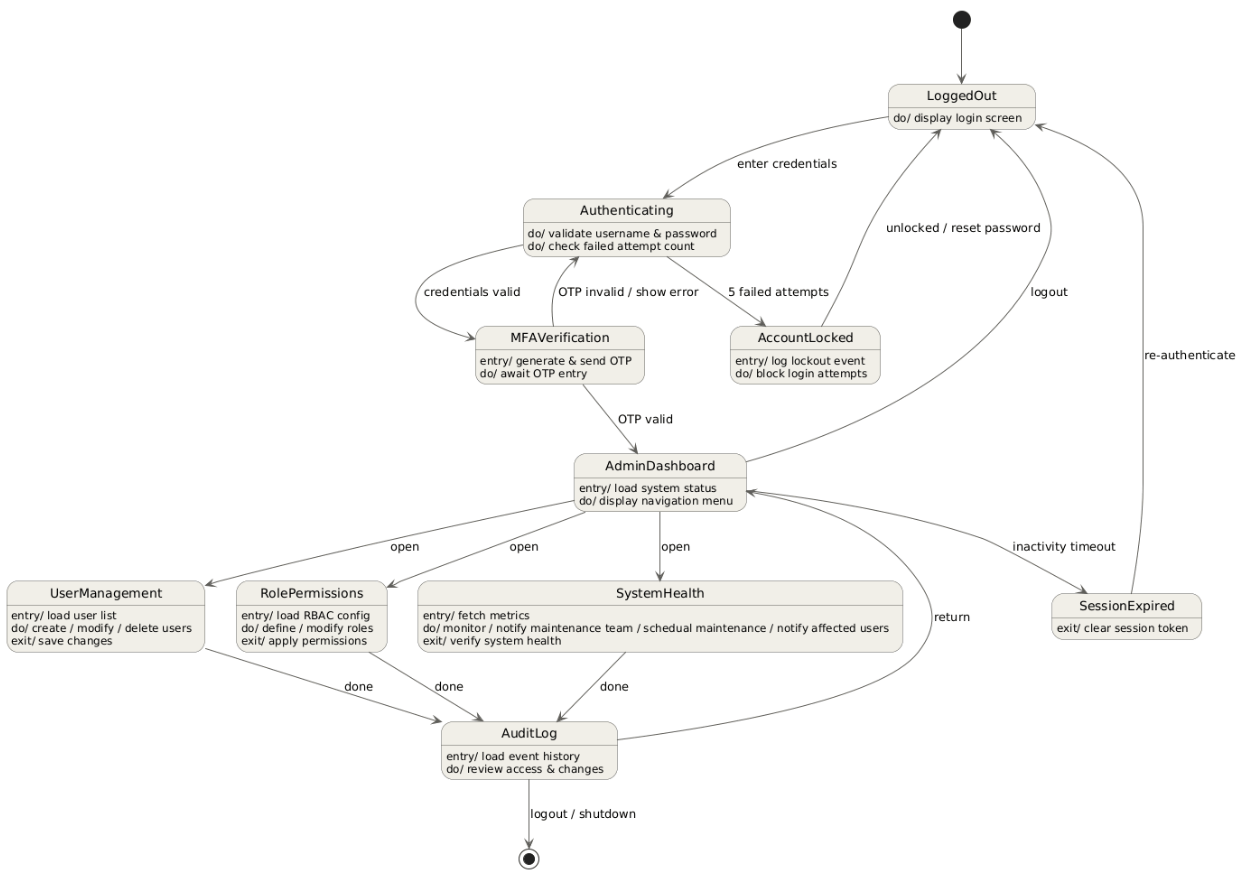
* The Erd Diagram, Class Diagram, Use Case Diagram are made by Erind Kotobelli

Use Case

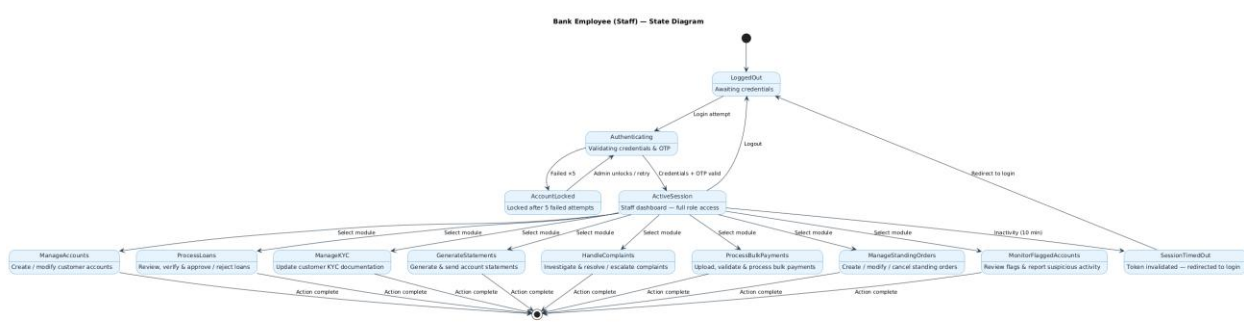

The Banking System
Log in
Close bank account
Modify bank account
Manage user accounts
Manage user premission
Bank Teller
Administrator
Open bank account
Monitor and fix system health
Includes
Manage KYC documents
Deposit money
Apply for debit or credit card
Handle costumer feedback
Create modify and delete user profiles
Includes
Bank Staff
Log in into the system
Review flaged accounts
Card Application
Withdraw money
Costumise spending limmits
Review high evaluation loan application
Multi-factor authentication
View executive audits and verifies complience
Bank director
View executive dashboard
View account & history
Recieve system anomalies
Customer
Access fraud detection analytics
Transfer funds
Card Payments
Submit loan application
Make monthly payments
Give account-related data to costumer on demand
Business Processes
- Auth and system control
- Account management
- Transaction management
- Loan and credit management
* Fraude detection
- Reporting & analytics
- Costumer service
- Facility management
- User management
- Fund management
- Card management
Manage and save beneficiaries
Request account-related data
Monitor support ticket
Schedule appointments with the bank
Report frawdulent activity
Manage and approve duty rosters
feedback
Includes
Recieve live info for the bank account
Monitor system and facilities
Facilities manager
Find a nearby ATM / branch
Should generate facilities reports
Facility staff
Spending Limit
Block/Unblock
Loan Amortization
Set Interest Rates & Policies
Allocate/ Redistribute Funds
View Fund Allocations

State Diagrams

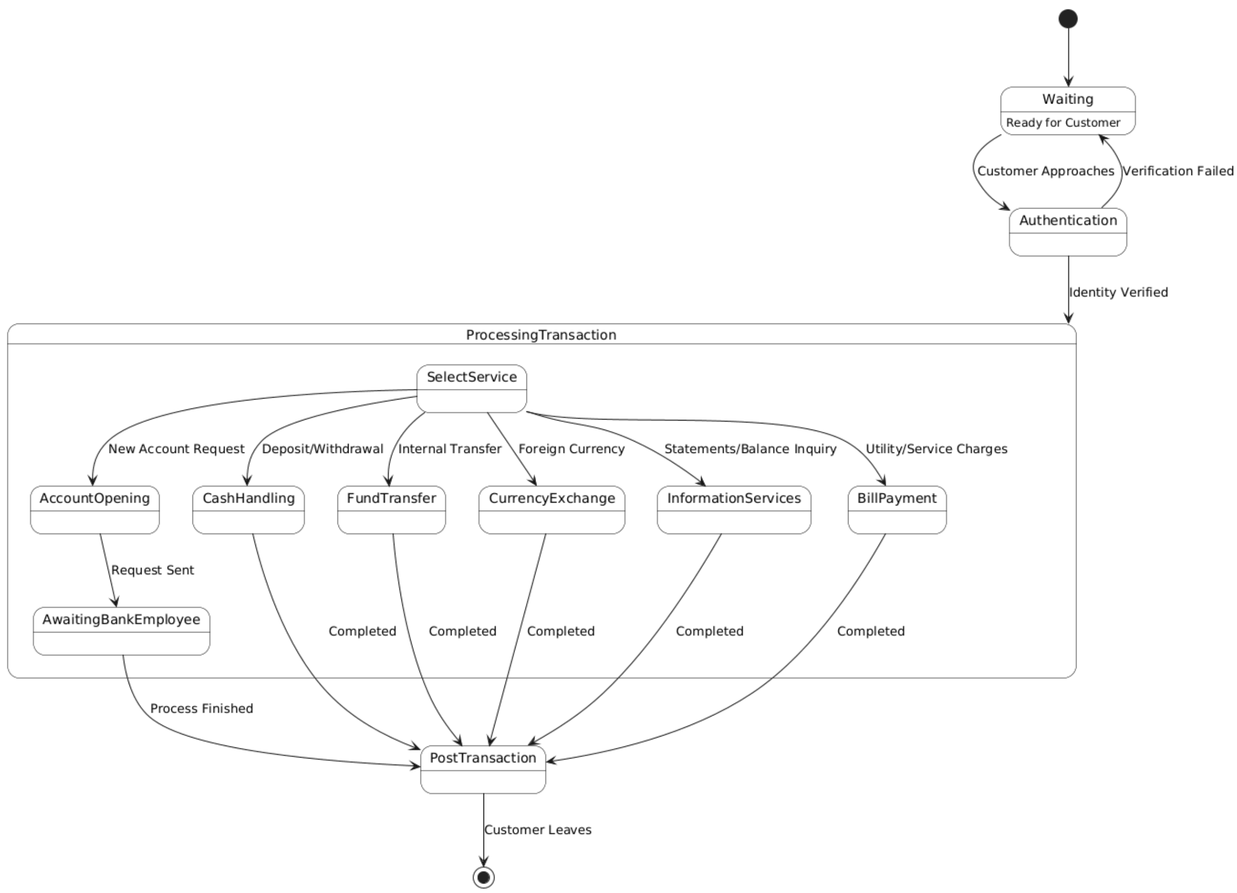
Andrea Ogreni



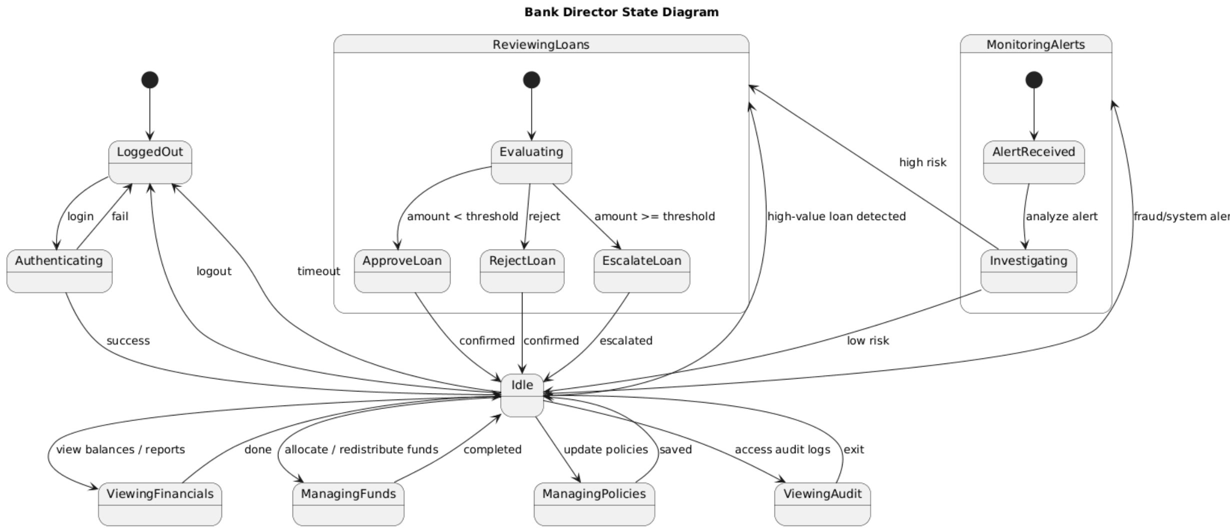
Joel Lluri



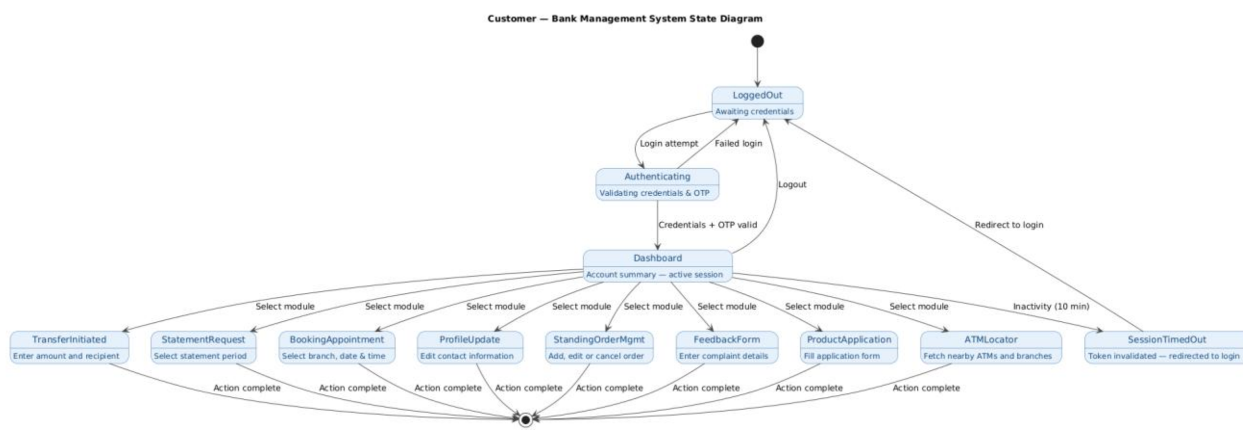
Adriano Demiri



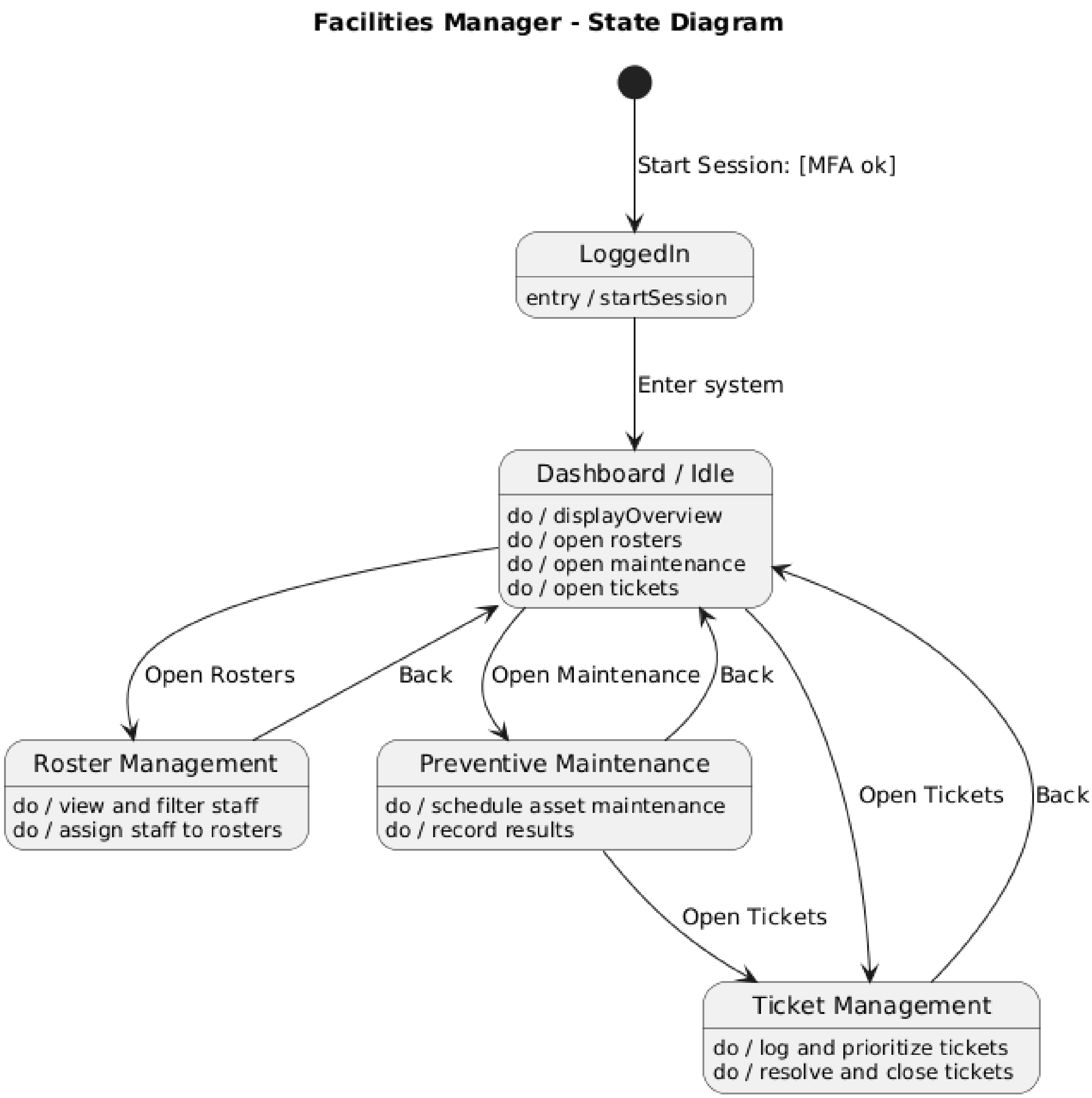
Emanuele Berbiu



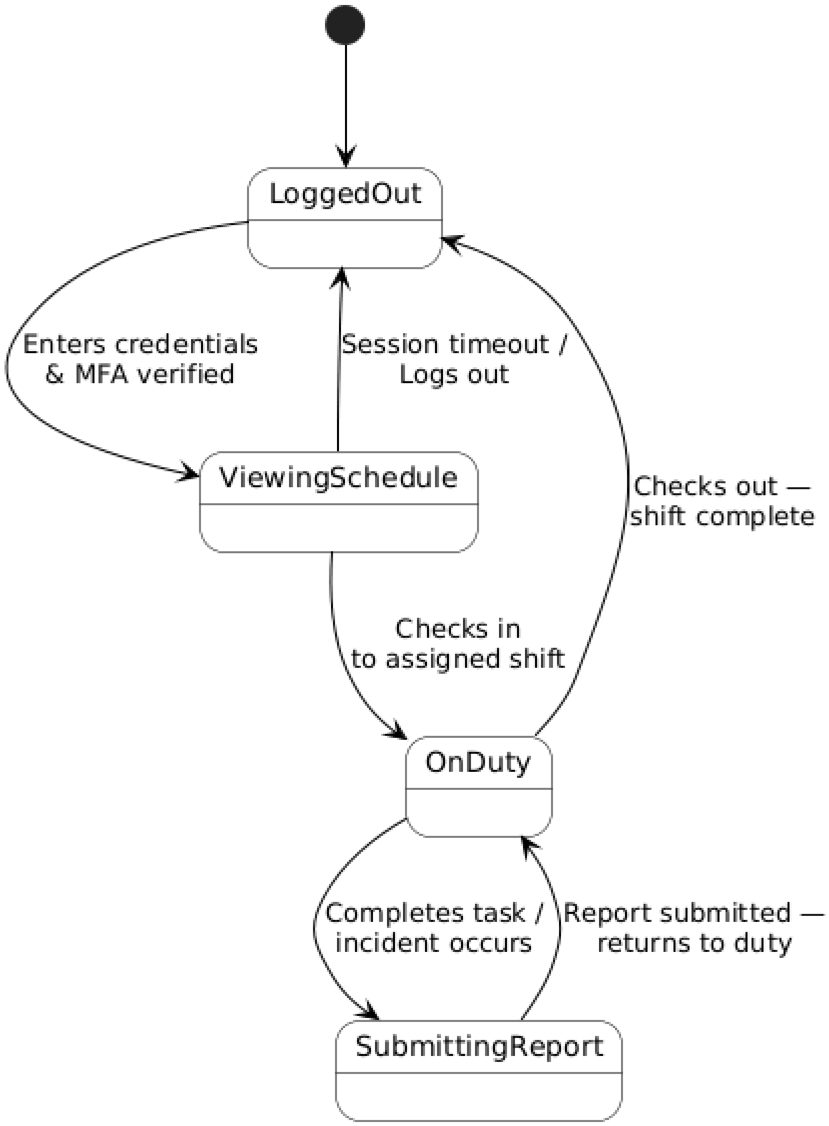
Kleo Miraka



Joandri Domi



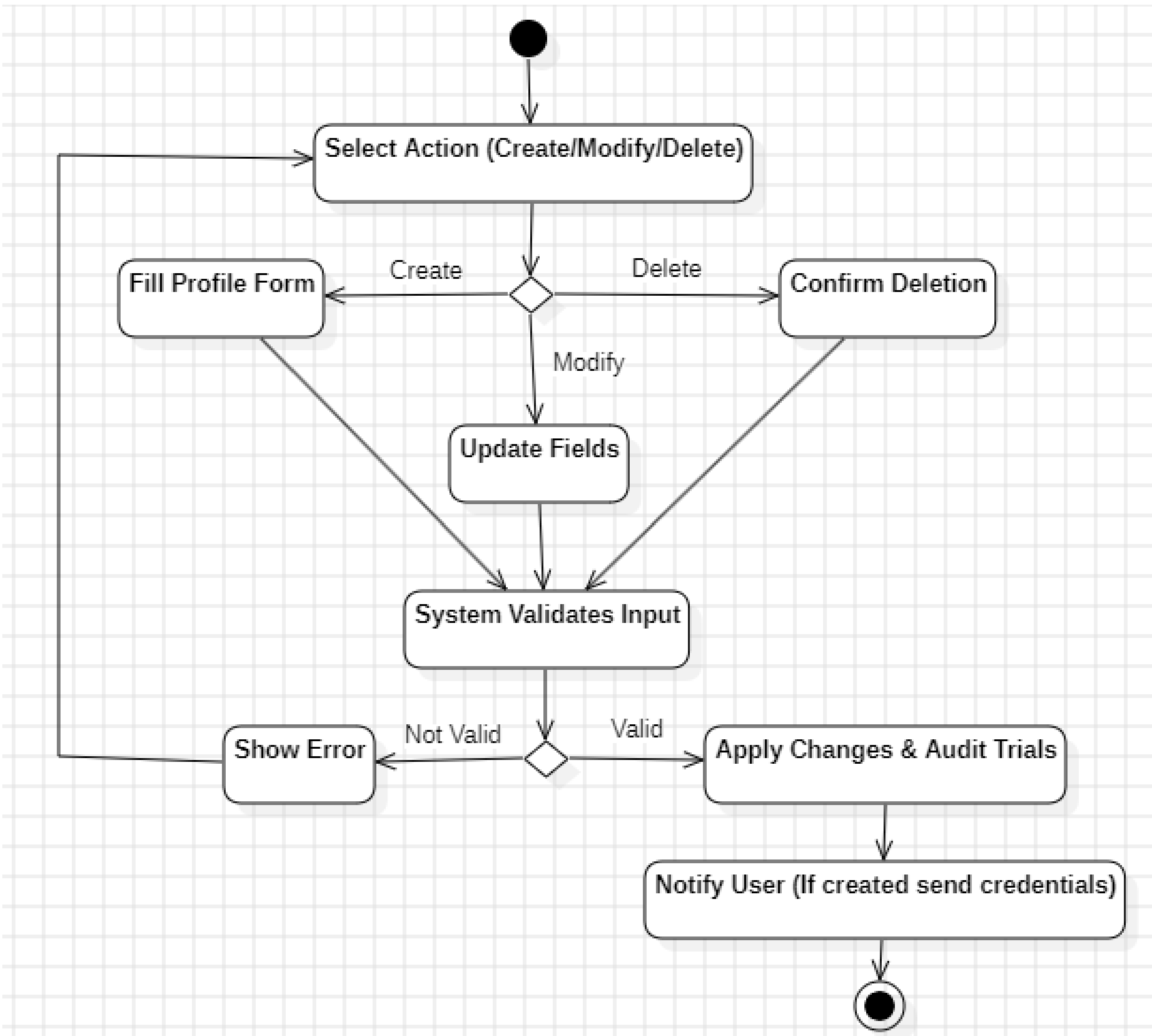
Erind Shabani



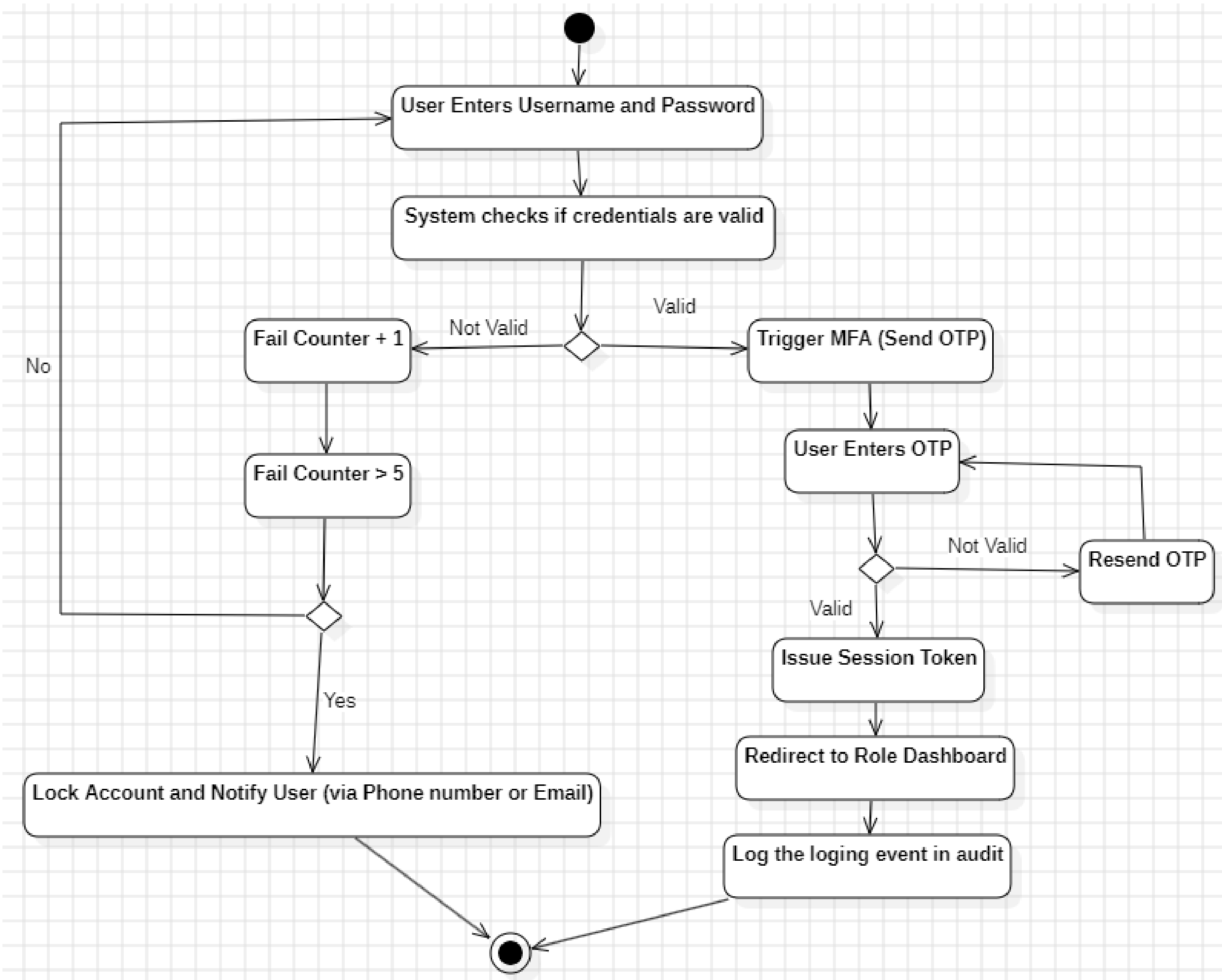
Activity Diagram

Andrea Ogreni

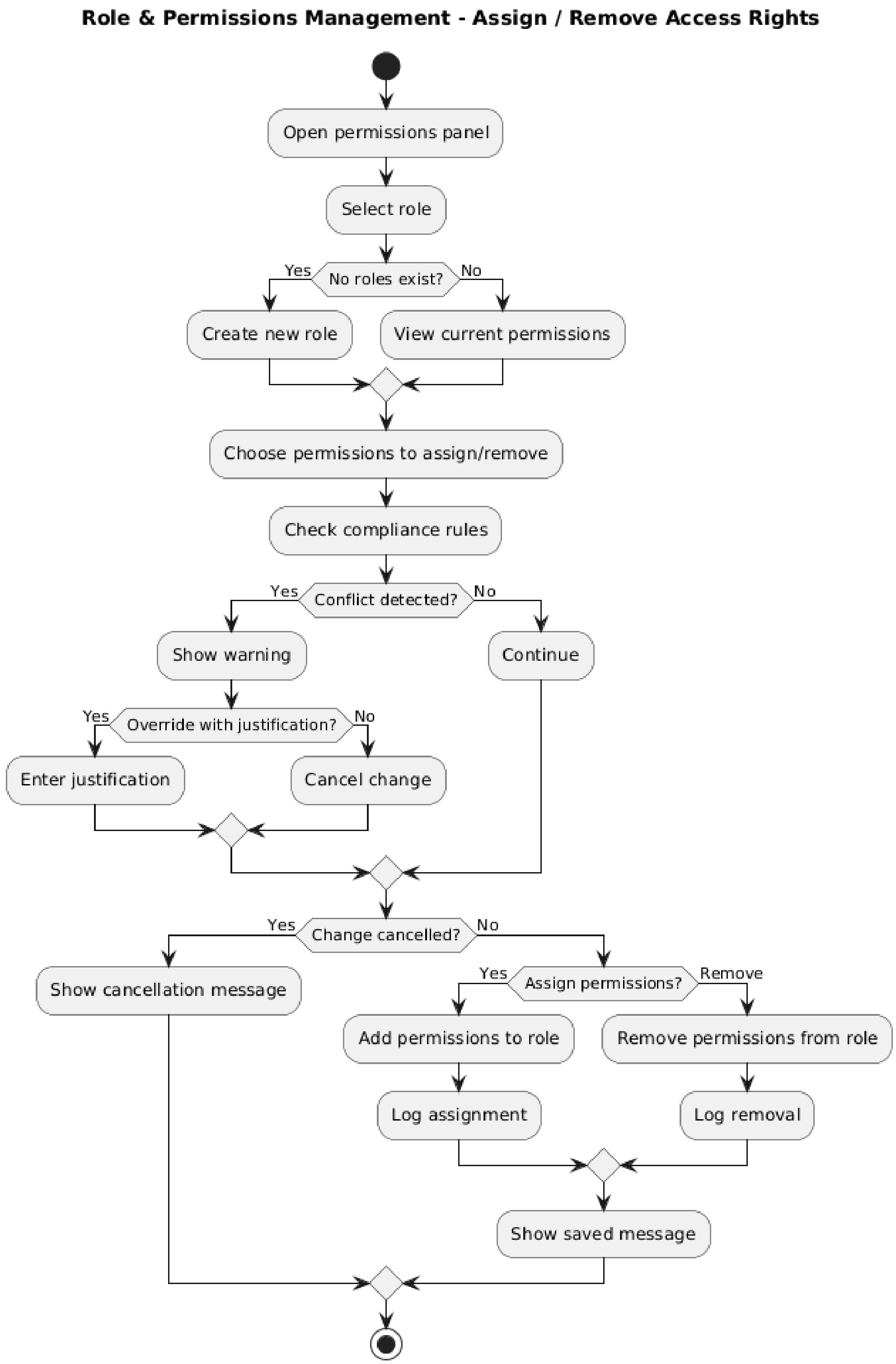
BS_UM_01



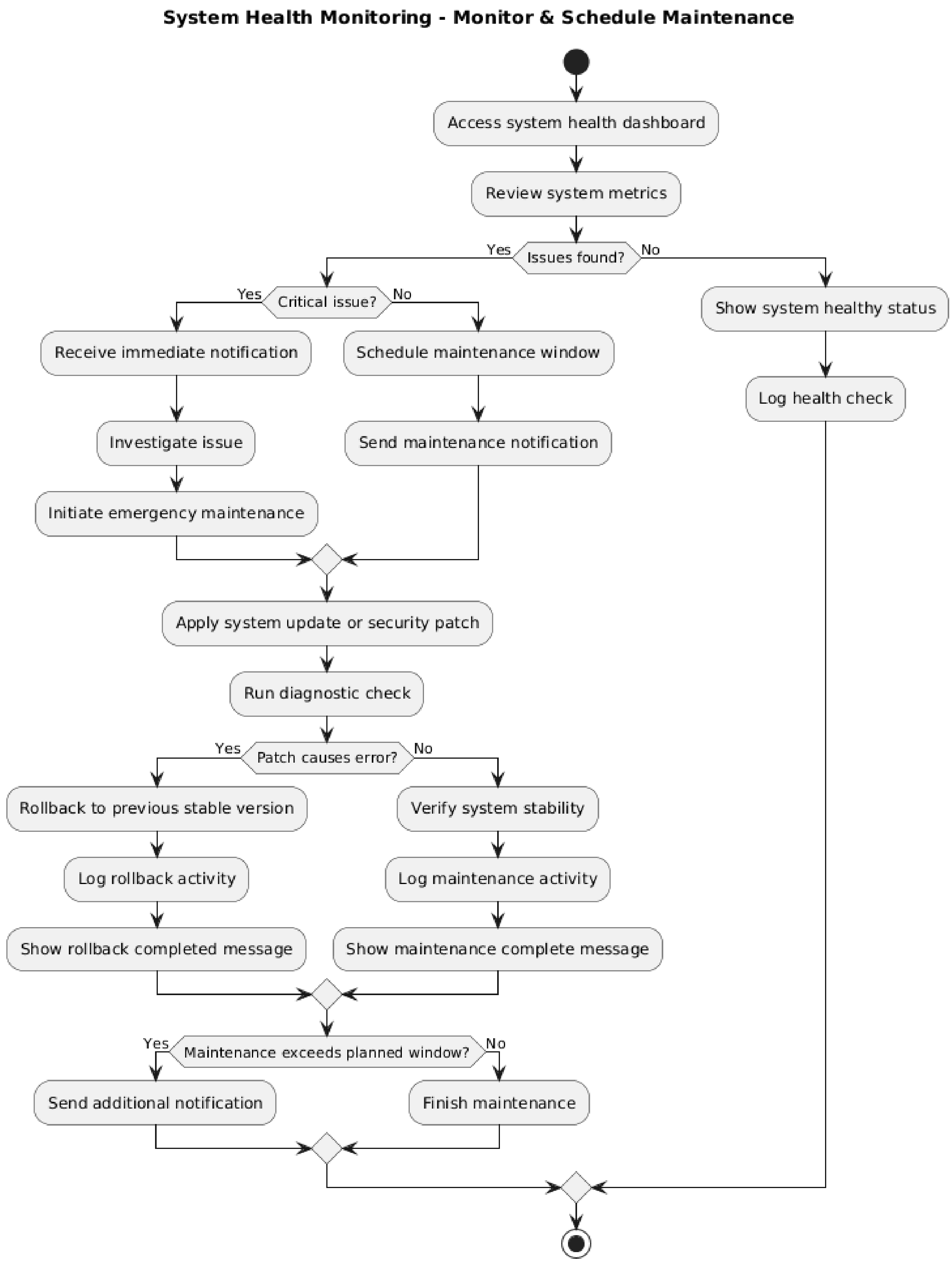
BS_UM_02


User Enters Username and Password
System checks if credentials are valid
Not Valid
Valid
Fail Counter + 1
Trigger MFA (Send OTP)
No
Fail Counter > 5
User Enters OTP
Not Valid
Resend OTP
Valid
Issue Session Token
Yes
Redirect to Role Dashboard
Lock Account and Notify User (via Phone number or Email)
Log the loging event in audit

BS_UM_03

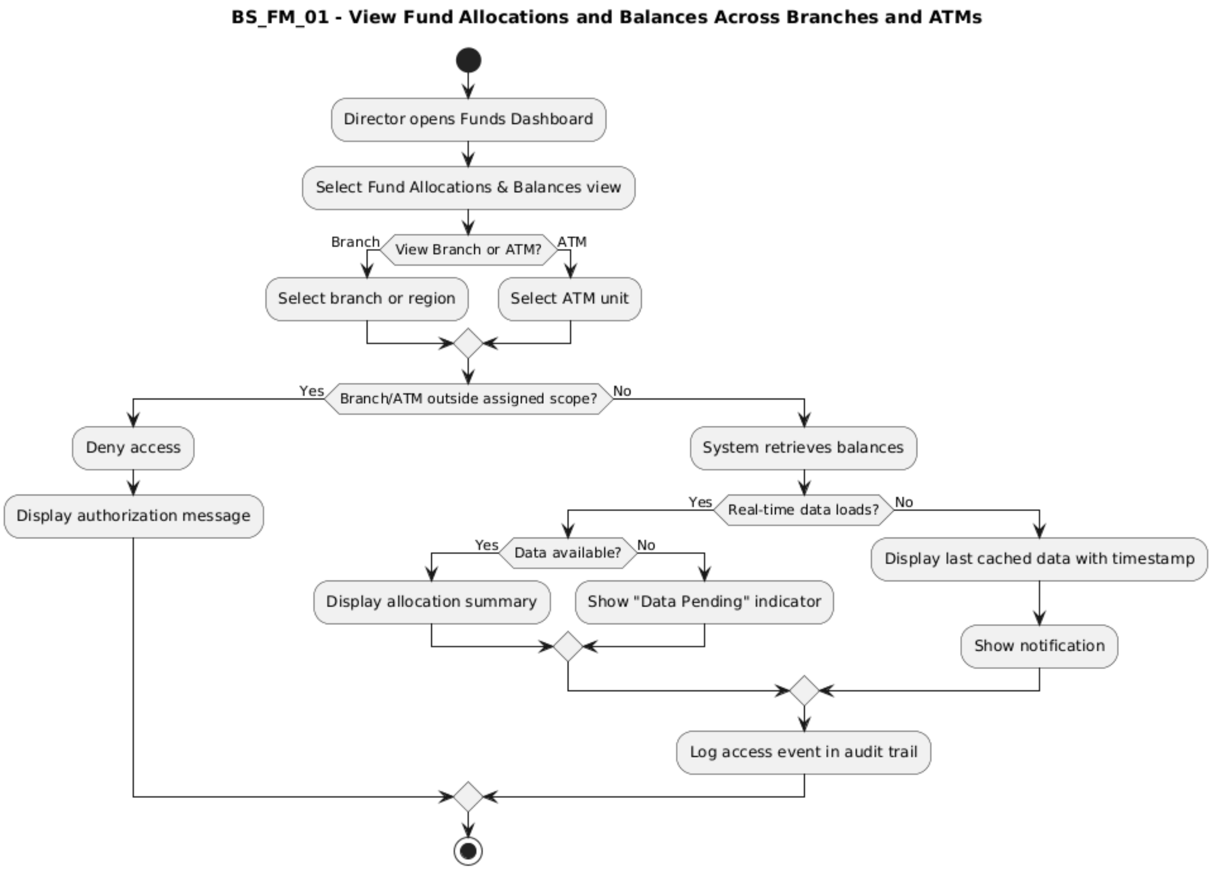

Role & Permissions Management - Assign / Remove Access Rights
Open permissions panel
Select role
No roles exist?
Yes
No
Create new role
View current permissions
Choose permissions to assign/remove
Check compliance rules
Conflict detected?
Yes
No
Show warning
Continue
Override with justification?
Yes
No
Enter justification
Cancel change
Change cancelled?
Yes
No
Show cancellation message
Assign permissions?
Yes
Remove
Add permissions to role
Remove permissions from role
Log assignment
Log removal
Show saved message

BS_UM_04

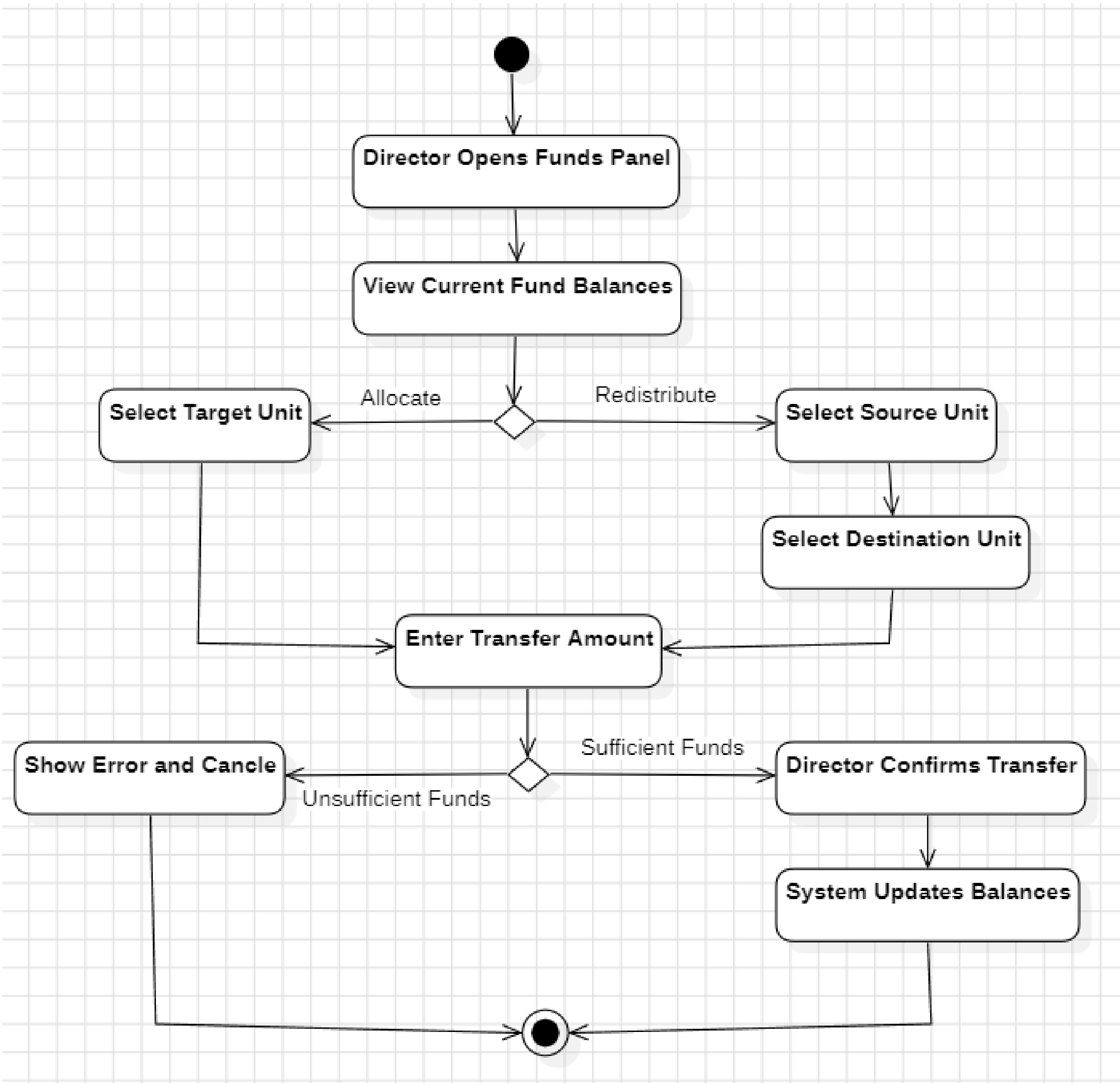

System Health Monitoring - Monitor & Schedule Maintenance
Access system health dashboard
Review system metrics
Issues found?
Yes
No
Critical issue?
Yes
No
Show system healthy status
Receive immediate notification
Schedule maintenance window
Log health check
Investigate issue
Send maintenance notification
Initiate emergency maintenance
Apply system update or security patch
Run diagnostic check
Patch causes error?
Yes
No
Rollback to previous stable version
Verify system stability
Log rollback activity
Log maintenance activity
Show rollback completed message
Show maintenance complete message
Maintenance exceeds planned window?
Yes
No
Send additional notification
Finish maintenance

Andrea Ogreni

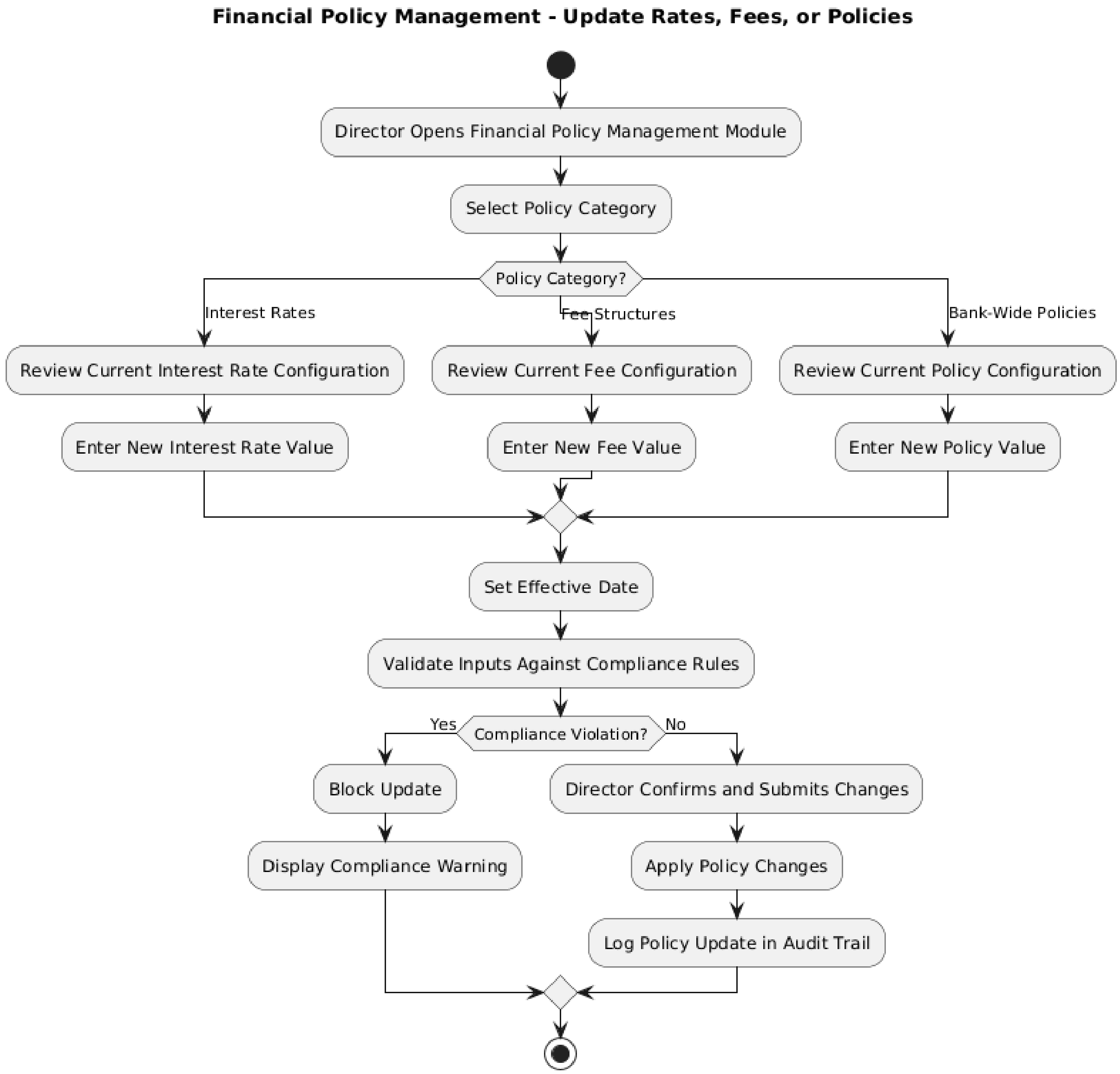
BS_FM_01



BS_FM_02


Director Opens Funds Panel
View Current Fund Balances
Allocate
Redistribute
Select Target Unit
Select Source Unit
Select Destination Unit
Enter Transfer Amount
Sufficient Funds
Show Error and Cancle
Unsufficient Funds
Director Confirms Transfer
System Updates Balances

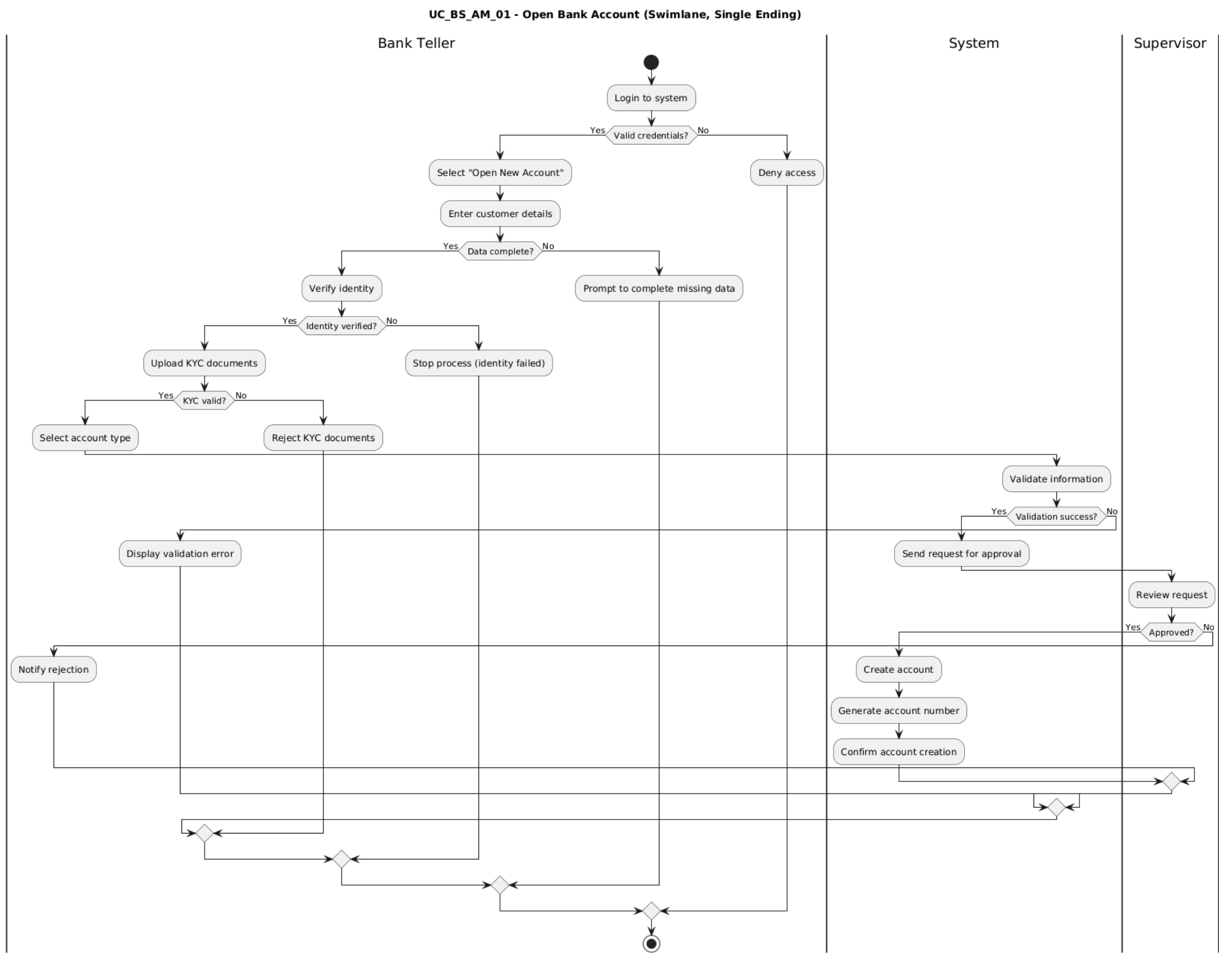
BS_FM_03



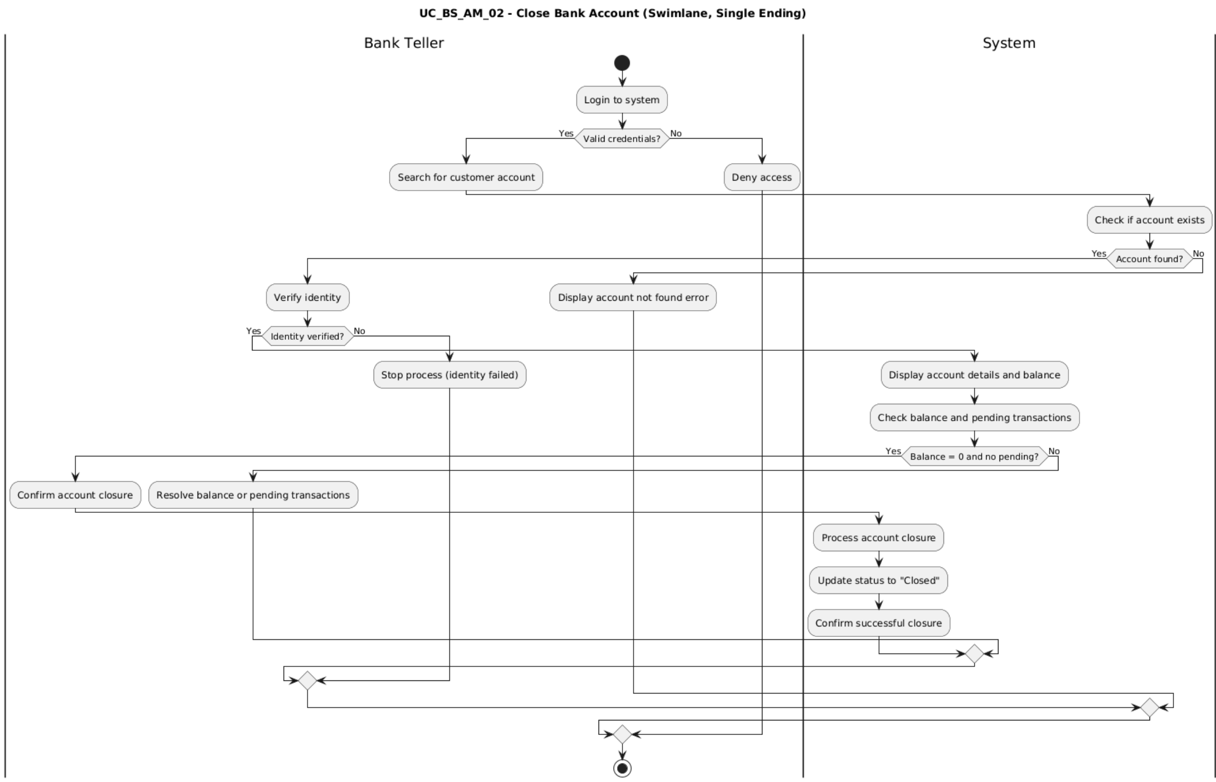
Financial Policy Management - Update Rates, Fees, or Policies
Director Opens Financial Policy Management Module
Select Policy Category
Policy Category?
Interest Rates
Fee Structures
Bank-Wide Policies
Review Current Interest Rate Configuration
Review Current Fee Configuration
Review Current Policy Configuration
Enter New Interest Rate Value
Enter New Fee Value
Enter New Policy Value
Set Effective Date
Validate Inputs Against Compliance Rules
Yes
Compliance Violation?
No
Block Update
Director Confirms and Submits Changes
Display Compliance Warning
Apply Policy Changes
Log Policy Update in Audit Trail

Joel Lluri

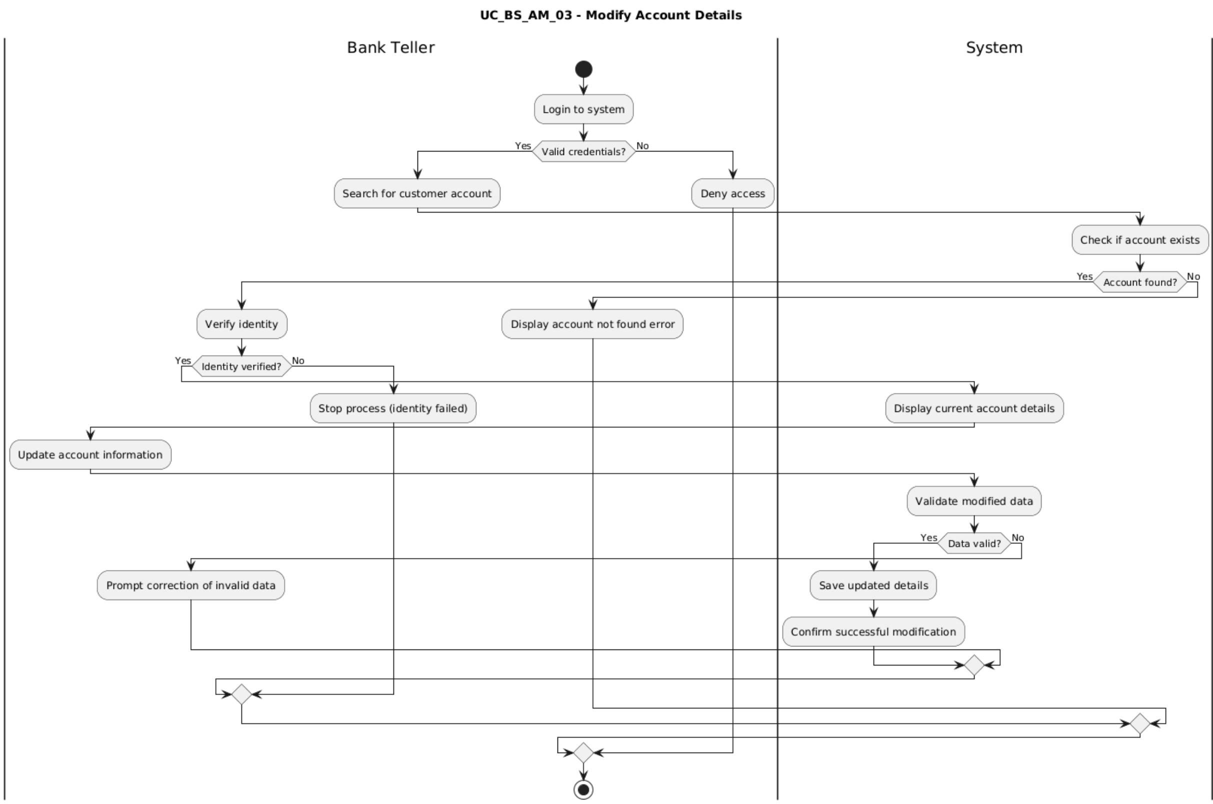
BS_AM_01



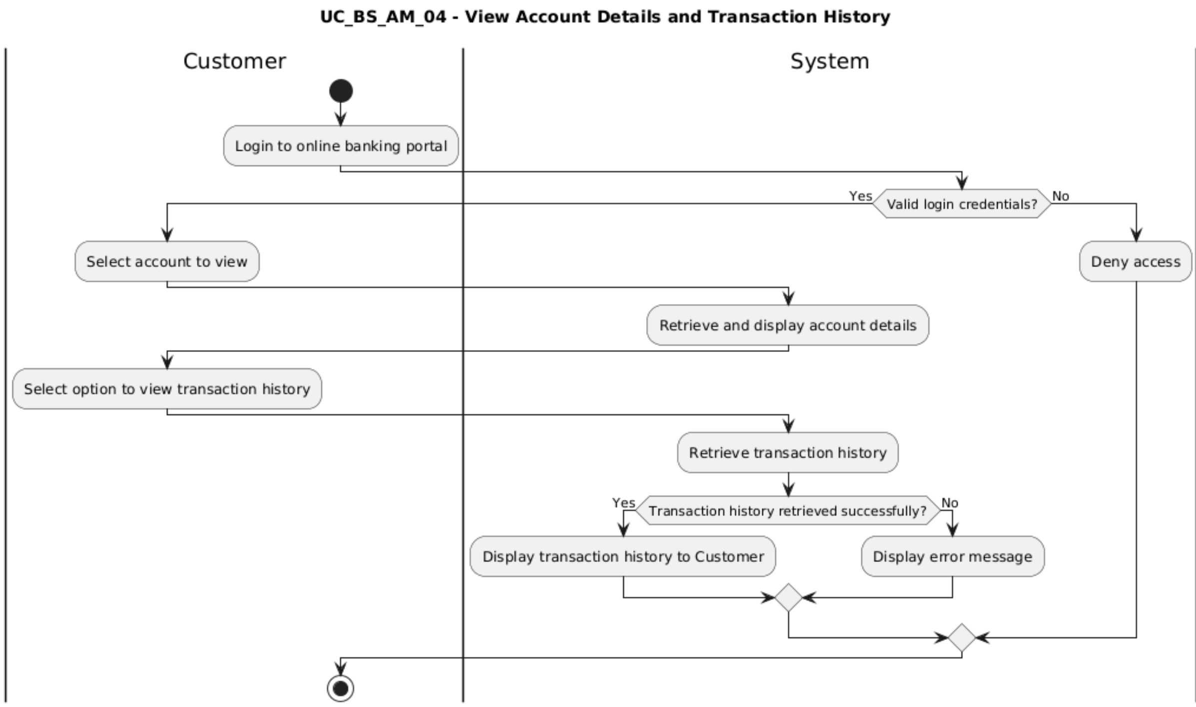
BS_AM_02


UC_BS_AM_02 - Close Bank Account (Swimlane, Single Ending)
Bank Teller
System
Login to system
Valid credentials?
Yes
No
Search for customer account
Deny access
Check if account exists
Account found?
Yes
No
Verify identity
Display account not found error
Identity verified?
Yes
No
Stop process (identity failed)
Display account details and balance
Check balance and pending transactions
Balance = 0 and no pending?
Yes
No
Confirm account closure
Resolve balance or pending transactions
Process account closure
Update status to "Closed"
Confirm successful closure

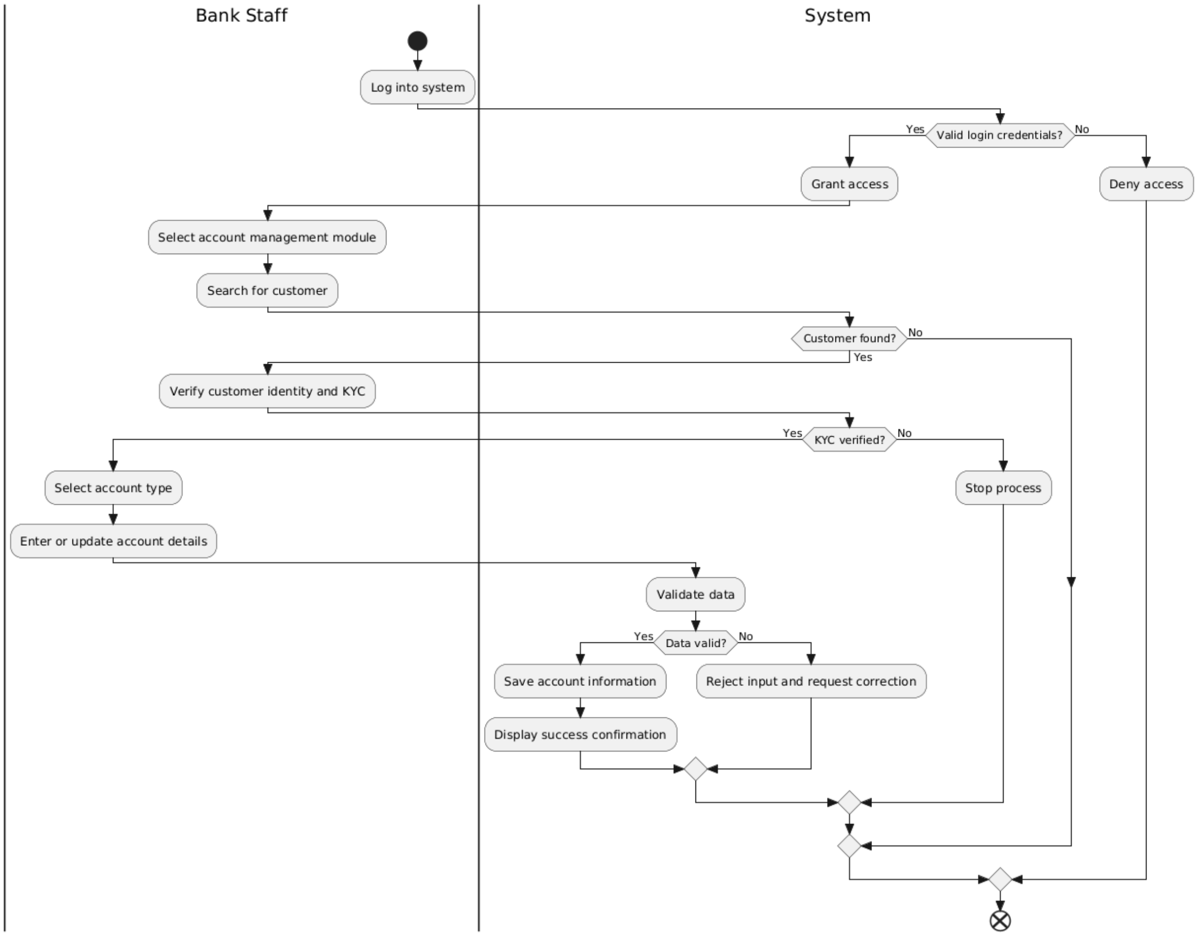
BS_AM_03


UC_BS_AM_03 - Modify Account Details
Bank Teller
System
Login to system
Valid credentials?
Yes
No
Search for customer account
Deny access
Check if account exists
Account found?
Yes
No
Verify identity
Display account not found error
Identity verified?
Yes
No
Stop process (identity failed)
Display current account details
Update account information
Validate modified data
Data valid?
Yes
No
Prompt correction of invalid data
Save updated details
Confirm successful modification

BS_AM_04

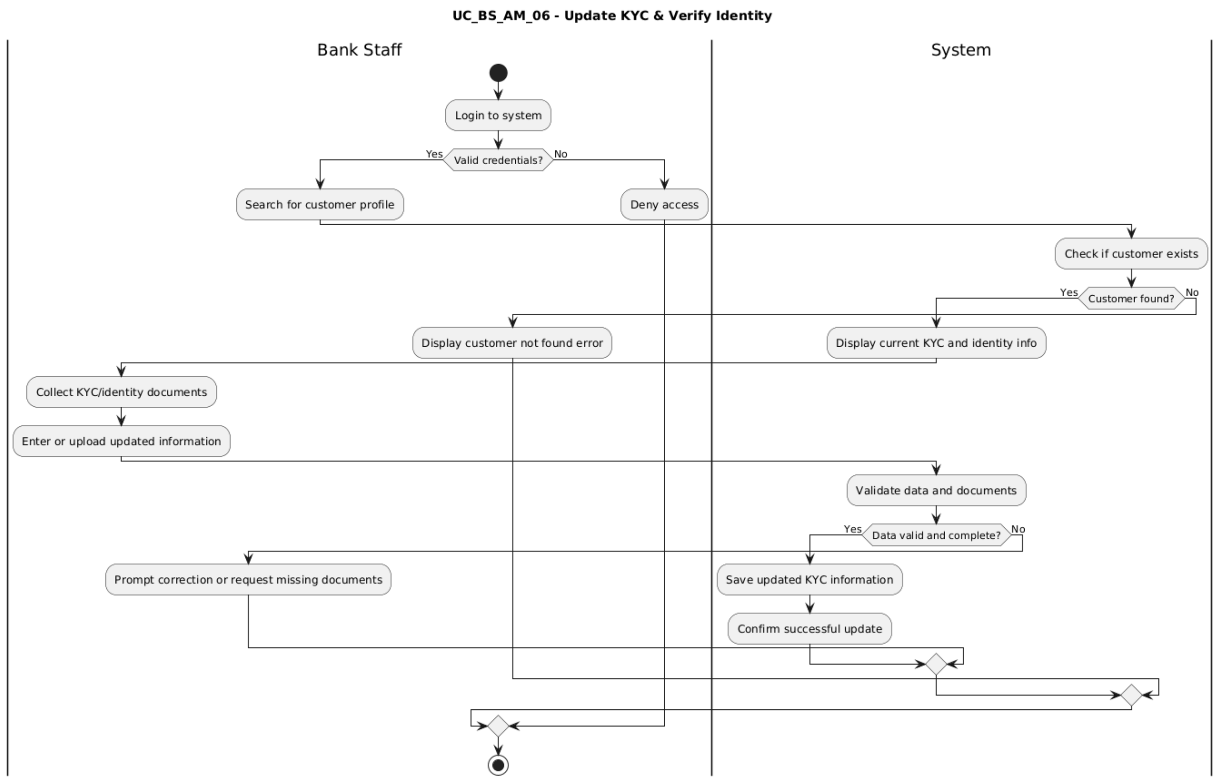

UC_BS_AM_04 - View Account Details and Transaction History
Customer
System
Login to online banking portal
Valid login credentials?
Yes
No
Select account to view
Deny access
Retrieve and display account details
Select option to view transaction history
Retrieve transaction history
Transaction history retrieved successfully?
Yes
No
Display transaction history to Customer
Display error message

BS_AM_05

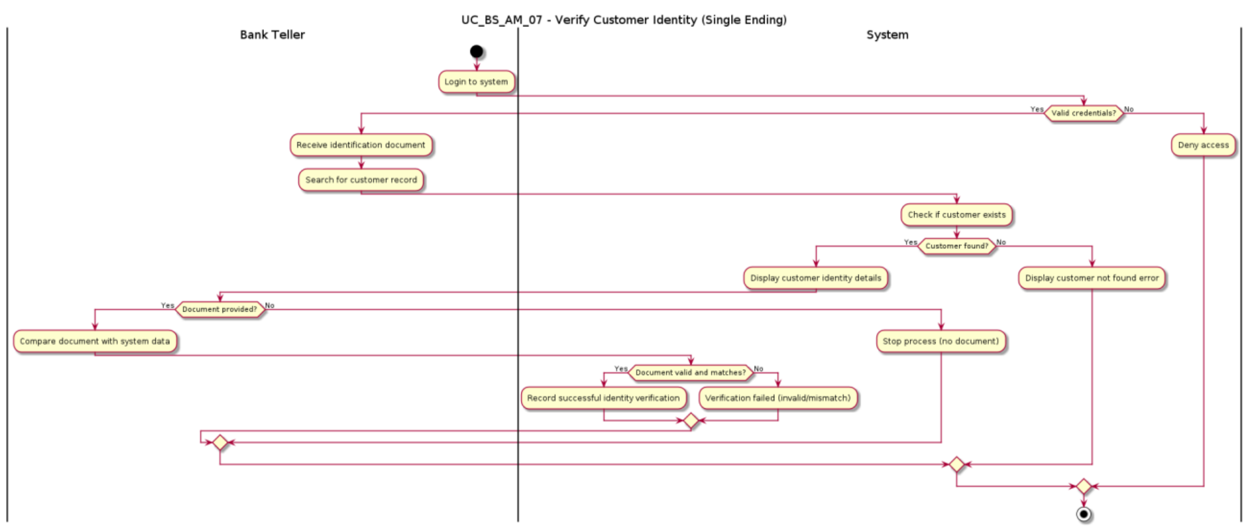


Bank Staff
System
Log into system
Valid login credentials?
Yes
No
Grant access
Deny access
Select account management module
Search for customer
Customer found?
No
Yes
Verify customer identity and KYC
KYC verified?
Yes
No
Select account type
Stop process
Enter or update account details
Validate data
Data valid?
Yes
No
Save account information
Reject input and request correction
Display success confirmation

BS_AM_06

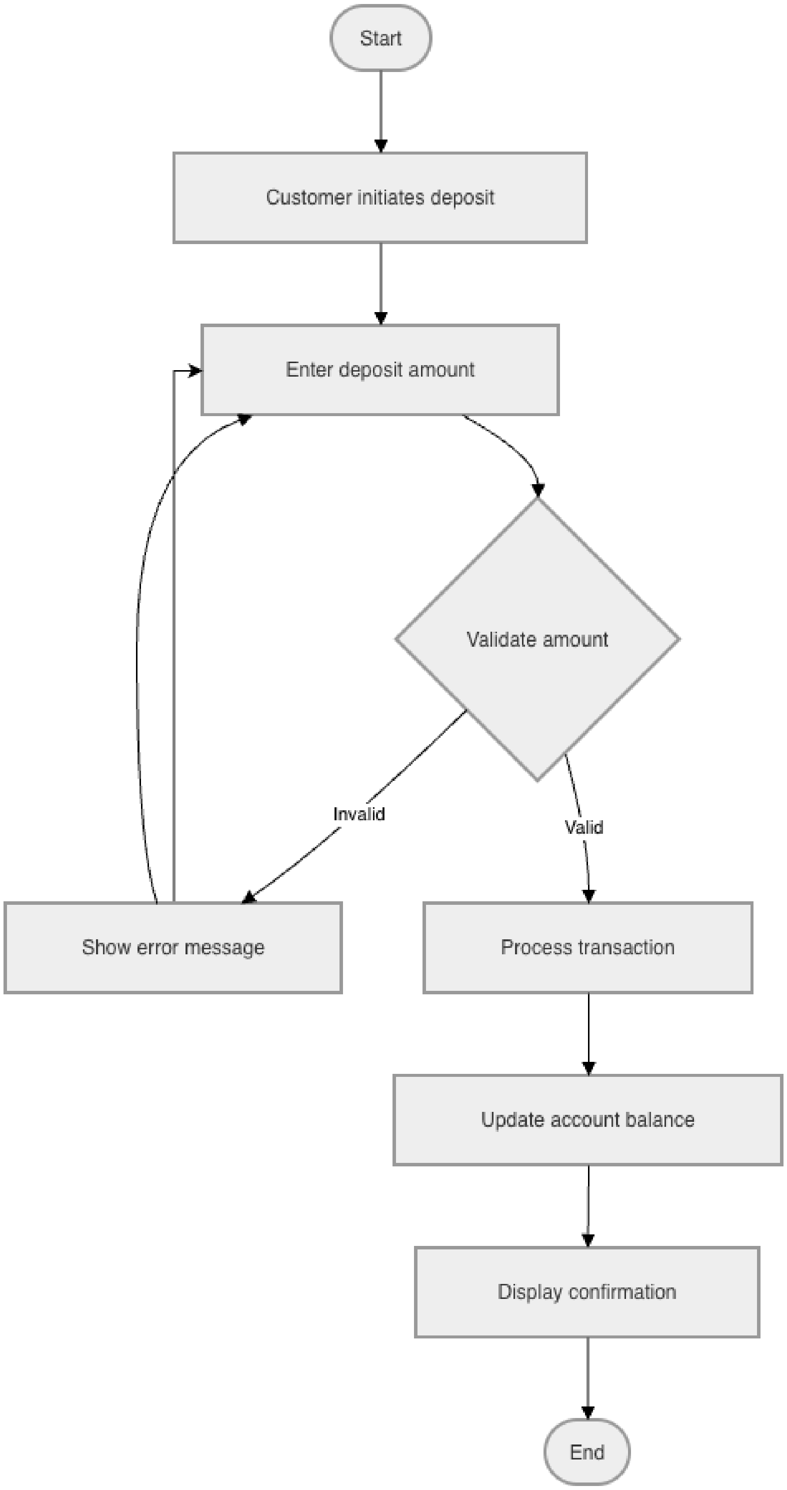


BS_AM_07

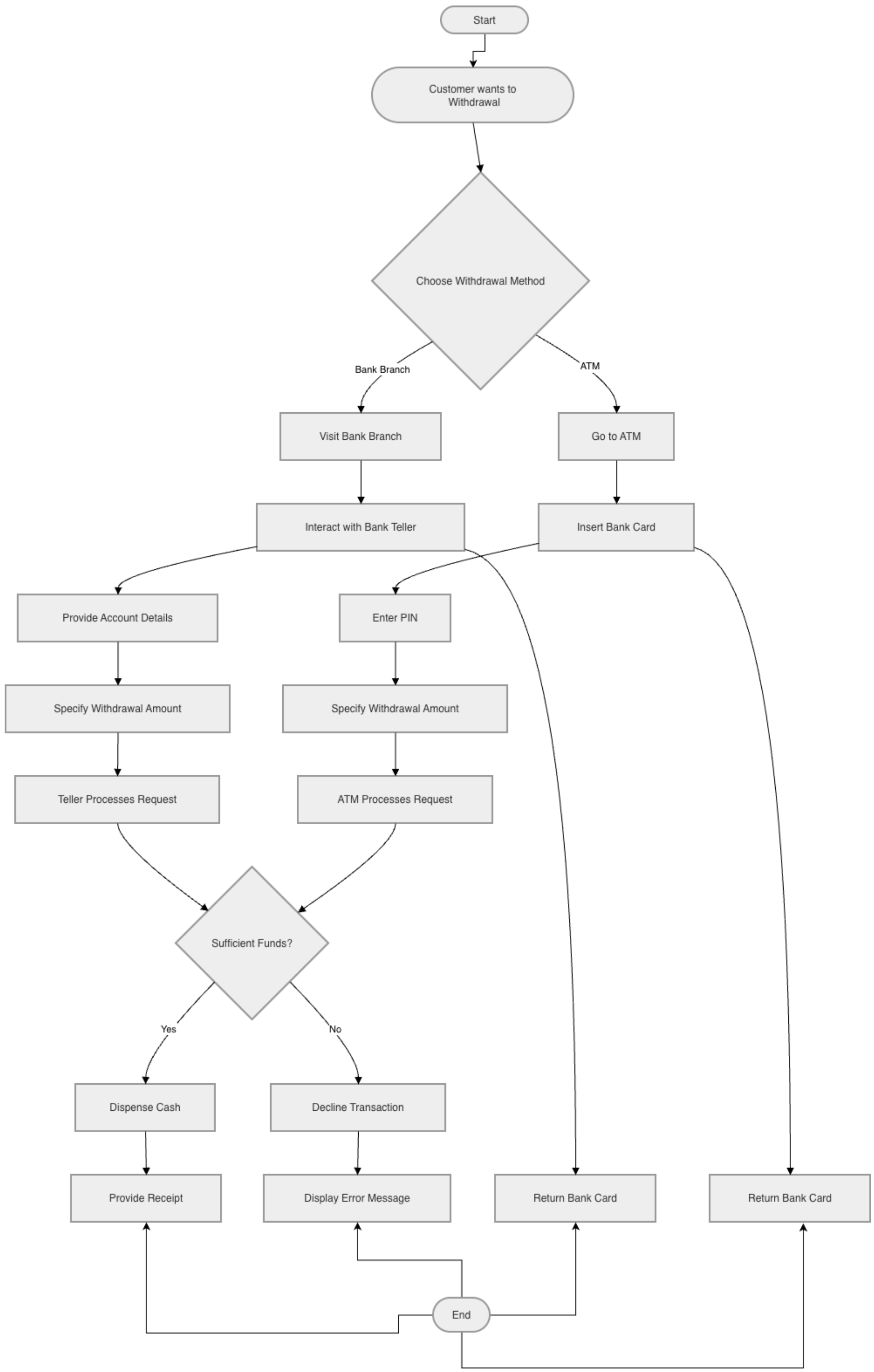


Andriano Demiri

BS_TM_01

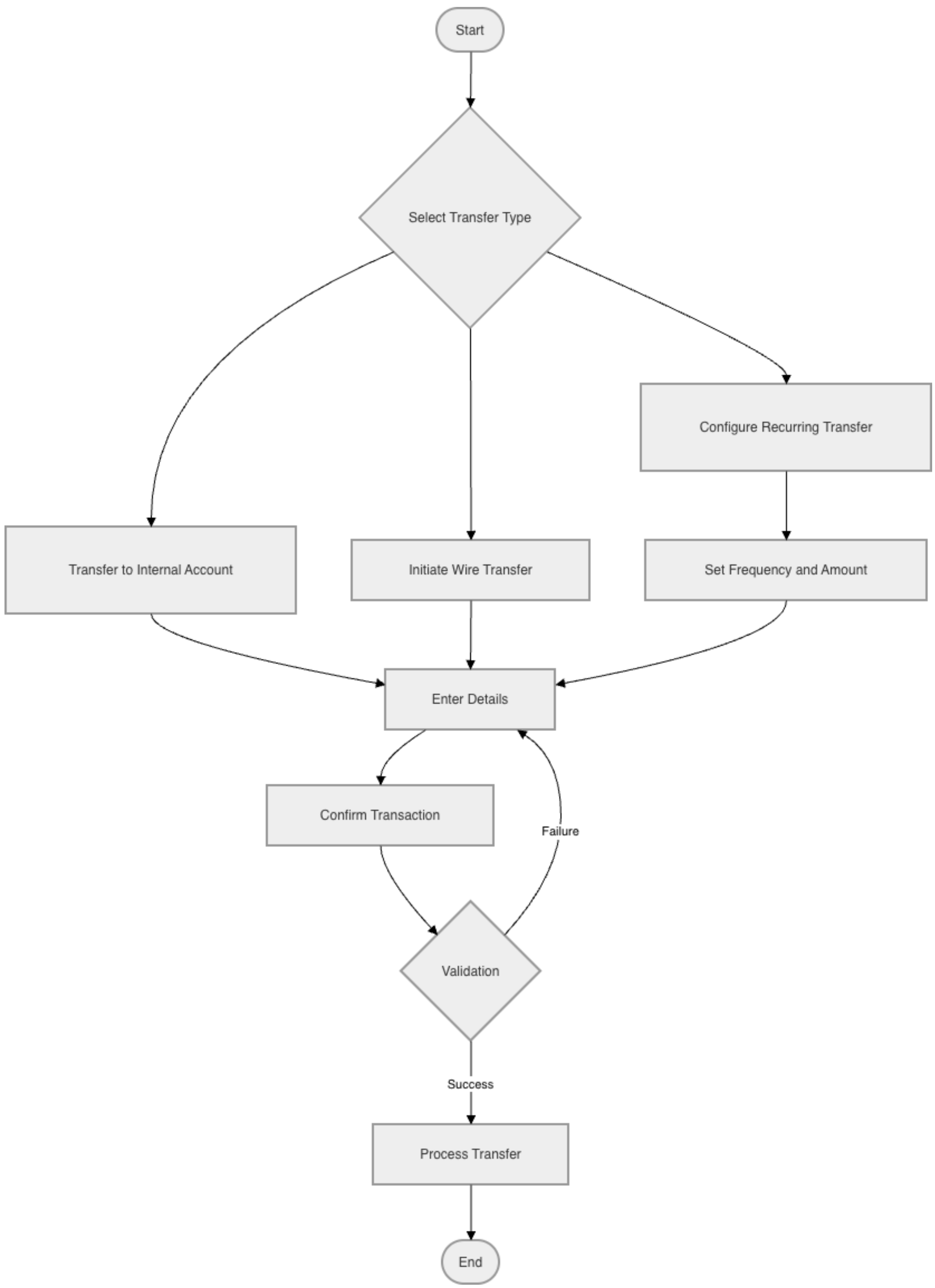


BS_TM_02



Start
Customer wants to Withdrawal
Choose Withdrawal Method
Bank Branch
ATM
Visit Bank Branch
Go to ATM
Interact with Bank Teller
Insert Bank Card
Provide Account Details
Enter PIN
Specify Withdrawal Amount
Specify Withdrawal Amount
Teller Processes Request
ATM Processes Request
Sufficient Funds?
Yes
No
Dispense Cash
Decline Transaction
Provide Receipt
Display Error Message
Return Bank Card
Return Bank Card
End

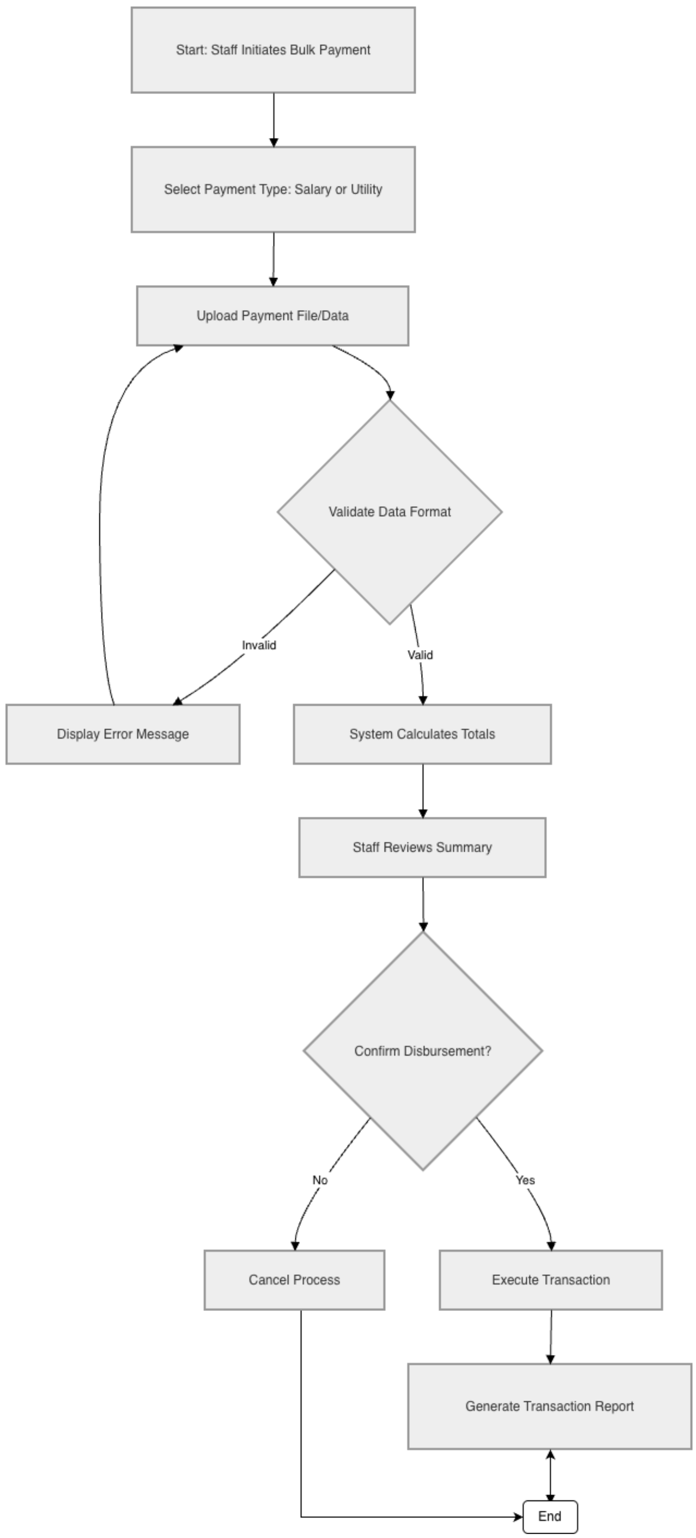
BS_TM_03


Start
Select Transfer Type
Configure Recurring Transfer
Transfer to Internal Account
Initiate Wire Transfer
Set Frequency and Amount
Enter Details
Confirm Transaction
Failure
Validation
Success
Process Transfer
End

BS_TM_04

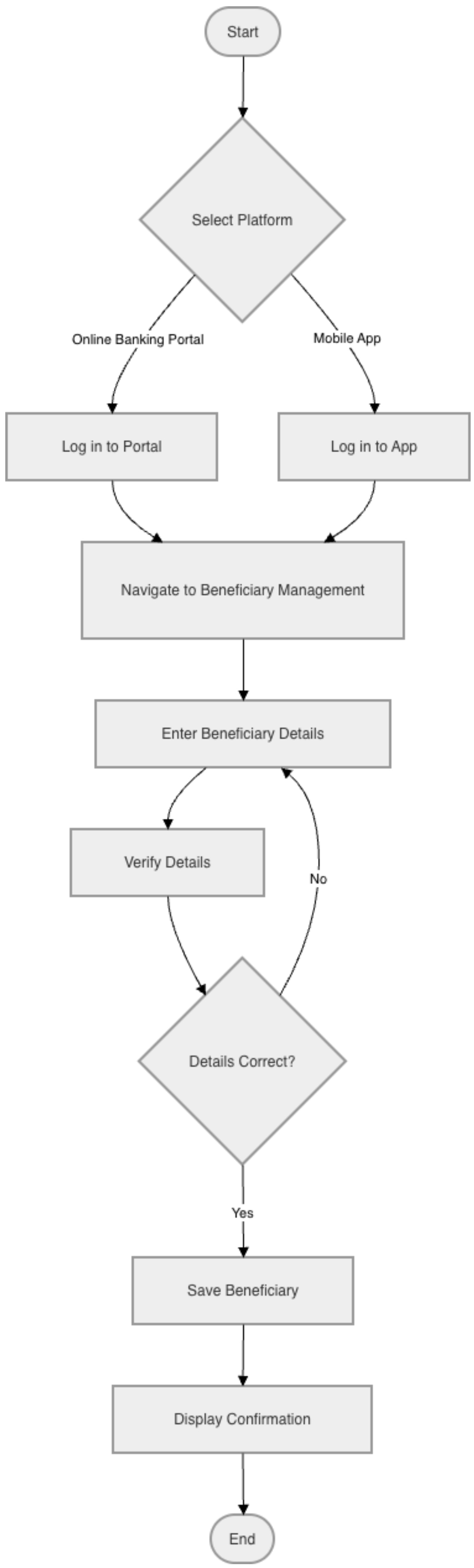

Start
Customer selects payment method
Choose Card Type
Credit Card
Debit Card
Enter Credit Card Details
Enter Debit Card Details
Validate Card Information
Is Card Valid?
No
Yes
Display Error Message
Process Payment
Display Payment Confirmation
End

BS_TM_05



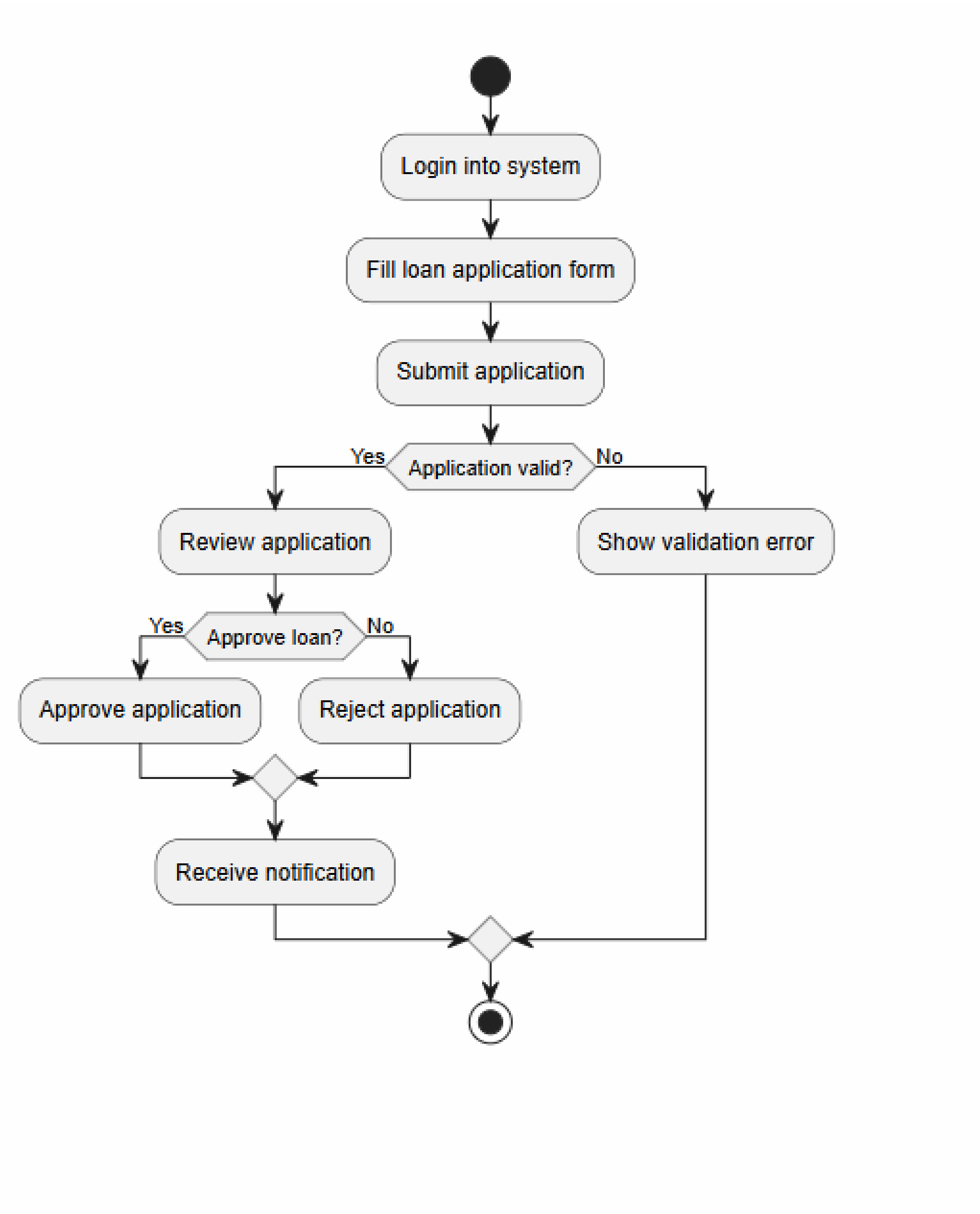
Start: Staff Initiates Bulk Payment
Select Payment Type: Salary or Utility
Upload Payment File/Data
Validate Data Format
Invalid
Valid
Display Error Message
System Calculates Totals
Staff Reviews Summary
Confirm Disbursement?
No
Yes
Cancel Process
Execute Transaction
Generate Transaction Report
End

BS_TM_06

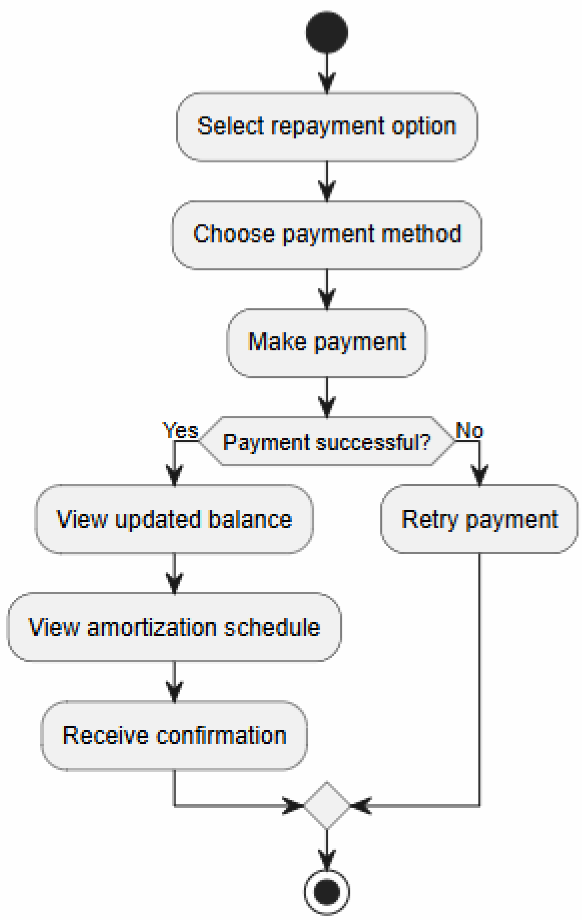

Start
Select Platform
Online Banking Portal
Mobile App
Log in to Portal
Log in to App
Navigate to Beneficiary Management
Enter Beneficiary Details
Verify Details
No
Details Correct?
Yes
Save Beneficiary
Display Confirmation
End

Emanuele Berbiu

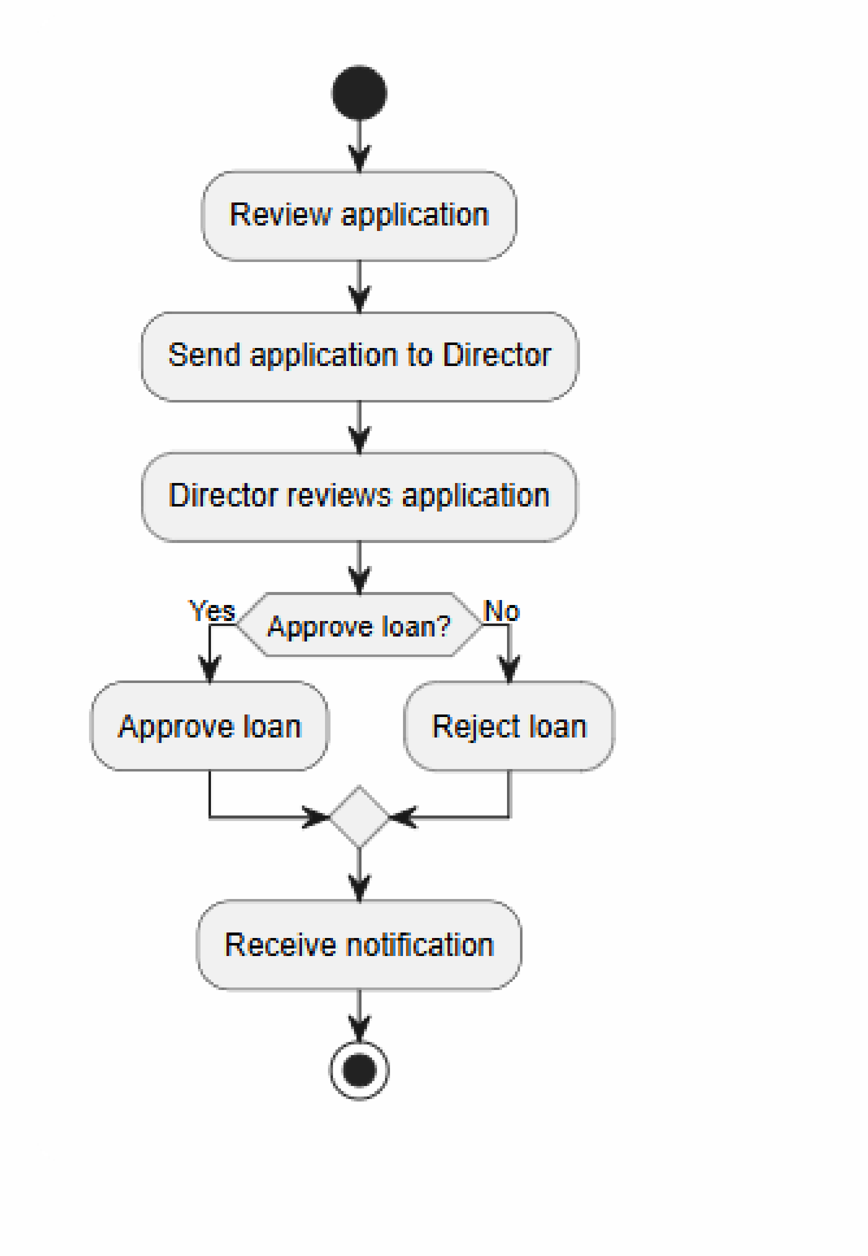
BS_LM_01



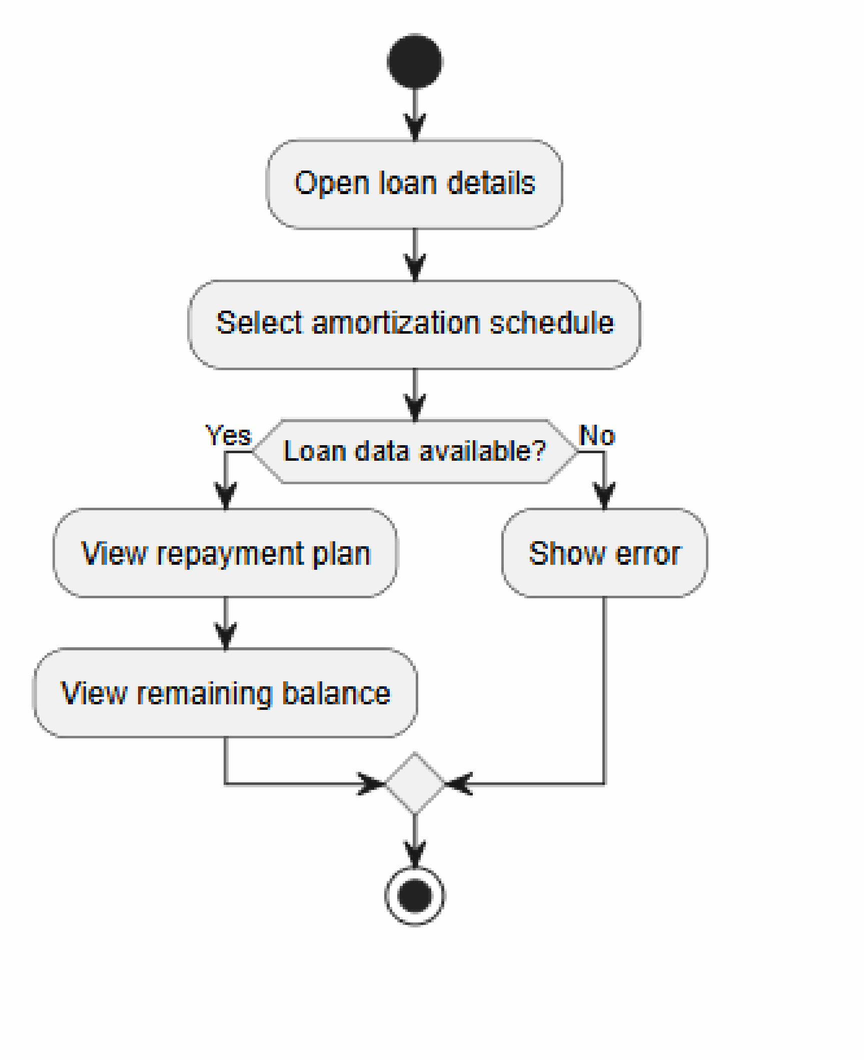
BS_LM_02


Select repayment option
Choose payment method
Make payment
Payment successful?
Yes
No
View updated balance
Retry payment
View amortization schedule
Receive confirmation

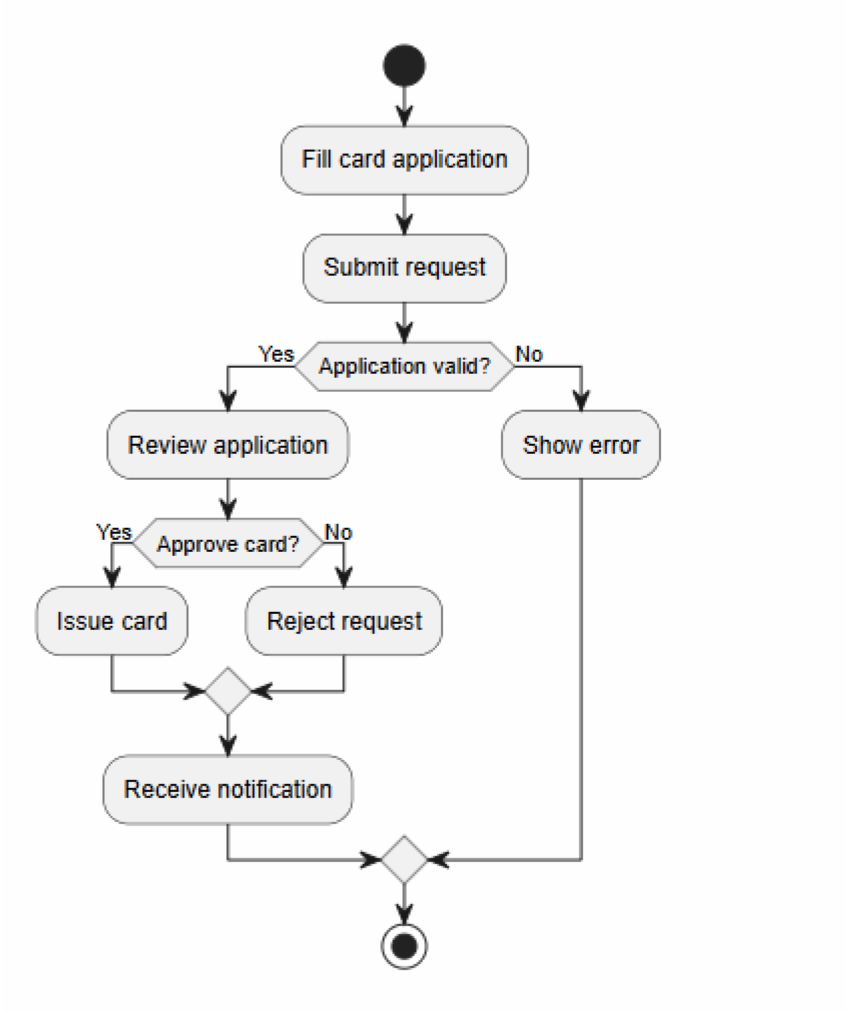
BS_LM_03


Review application
Send application to Director
Director reviews application
Yes
Approve loan?
No
Approve loan
Reject loan
Receive notification

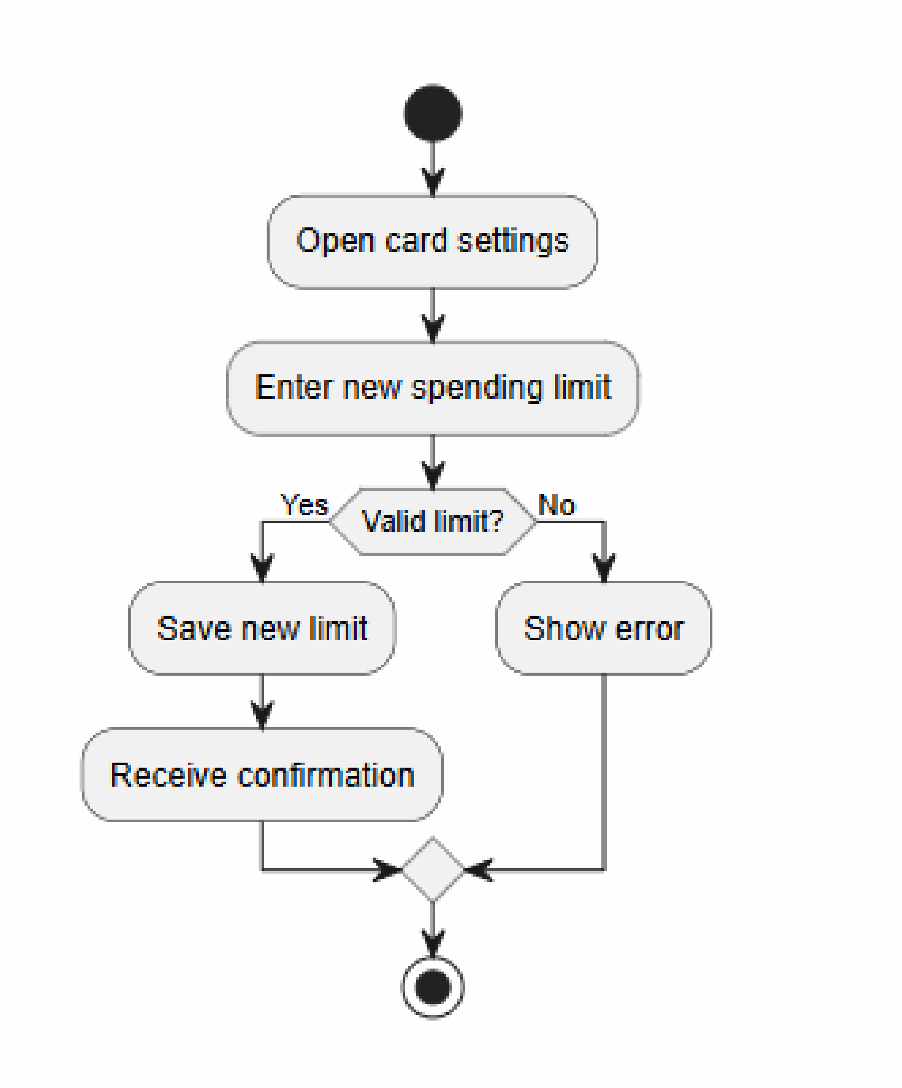
BS_LM_04


Open loan details
Select amortization schedule
Yes
Loan data available?
No
View repayment plan
Show error
View remaining balance

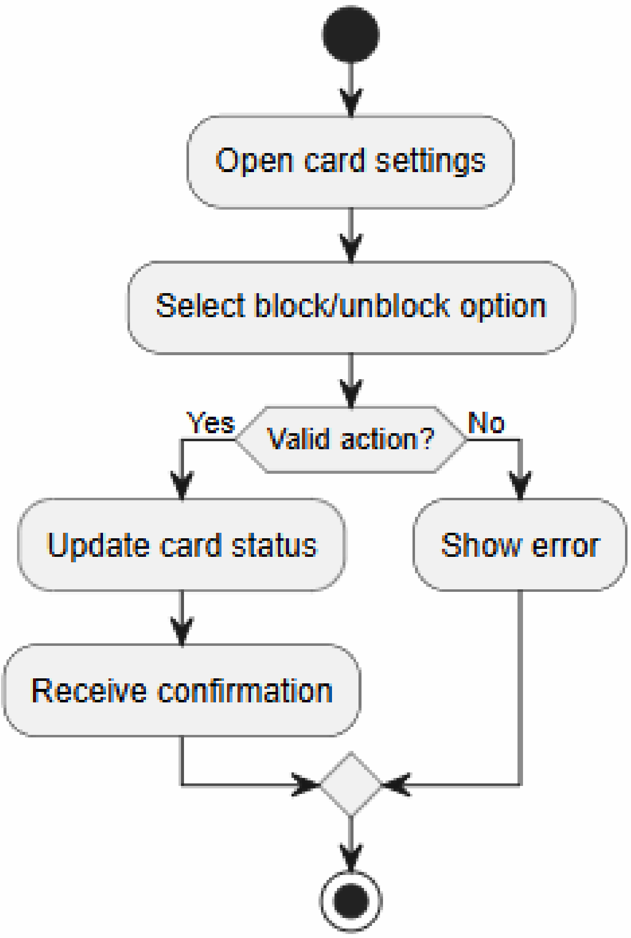
BS_CM_01


Fill card application
Submit request
Yes
Application valid?
No
Review application
Show error
Yes
Approve card?
No
Issue card
Reject request
Receive notification

BS_CM_02

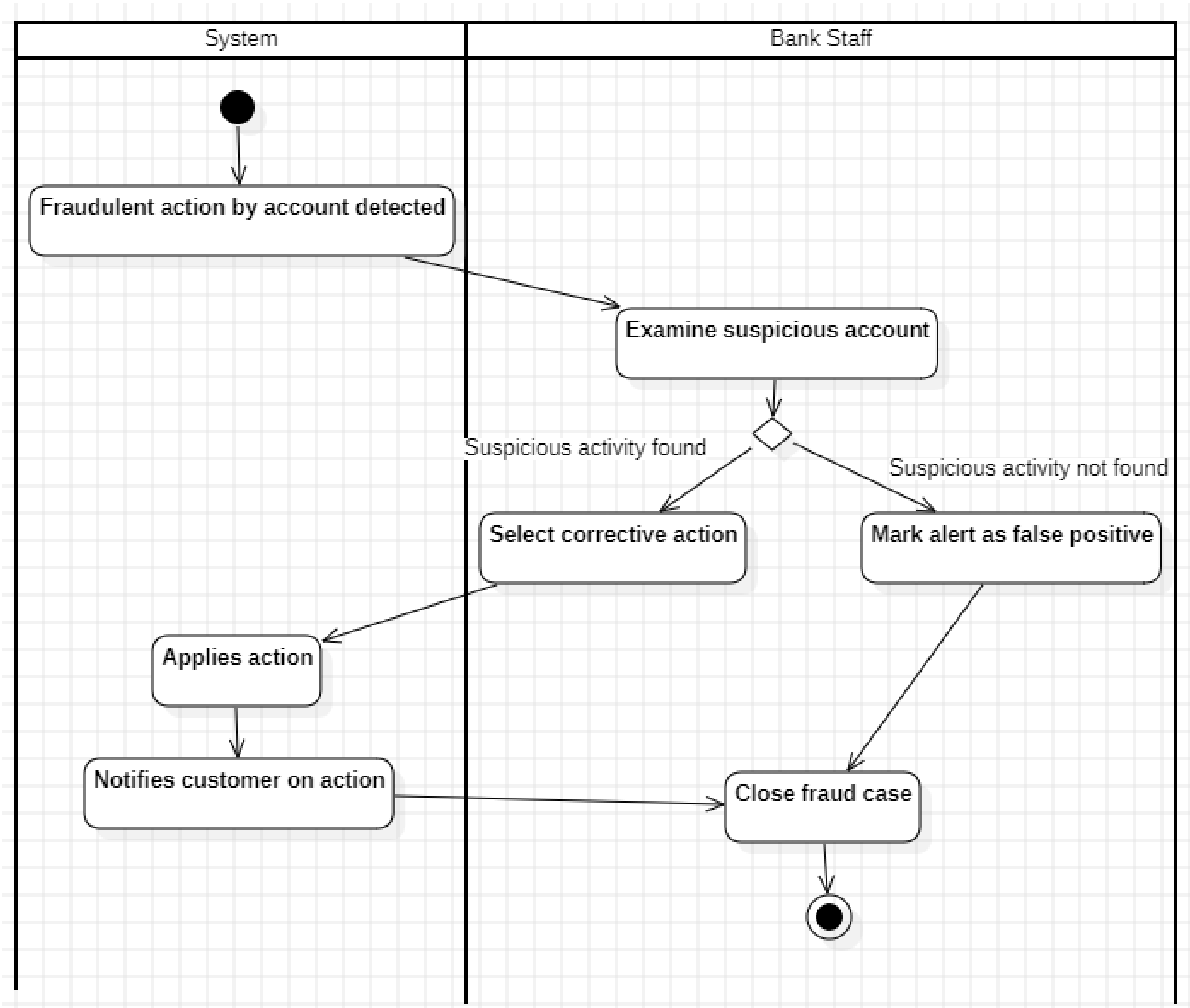

Open card settings
Enter new spending limit
Yes
Valid limit?
No
Save new limit
Show error
Receive confirmation

BS_CM_03

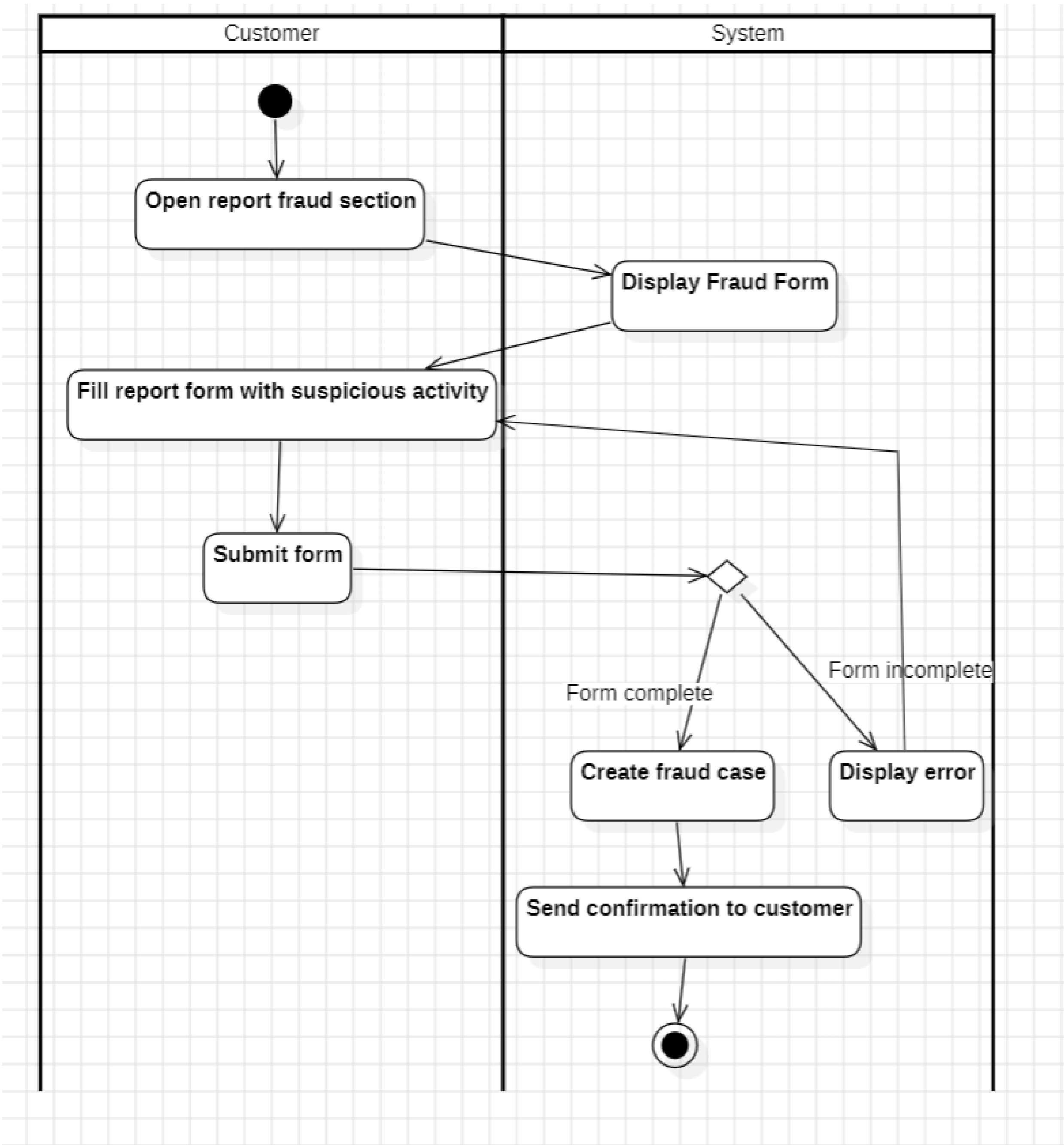

Open card settings
Select block/unblock option
Valid action?
Yes
No
Update card status
Show error
Receive confirmation

Joandri Domi

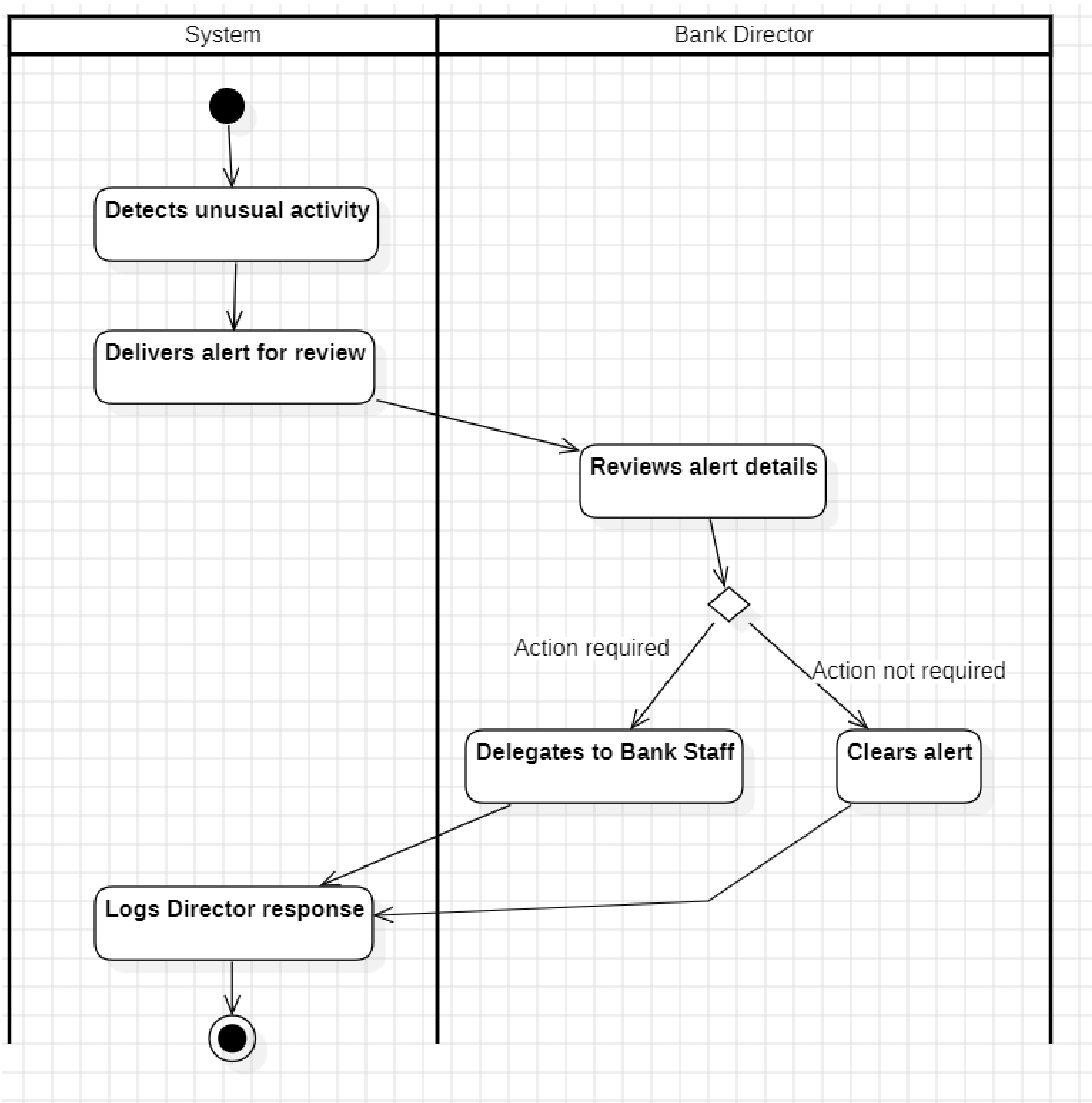
BS_FD_01



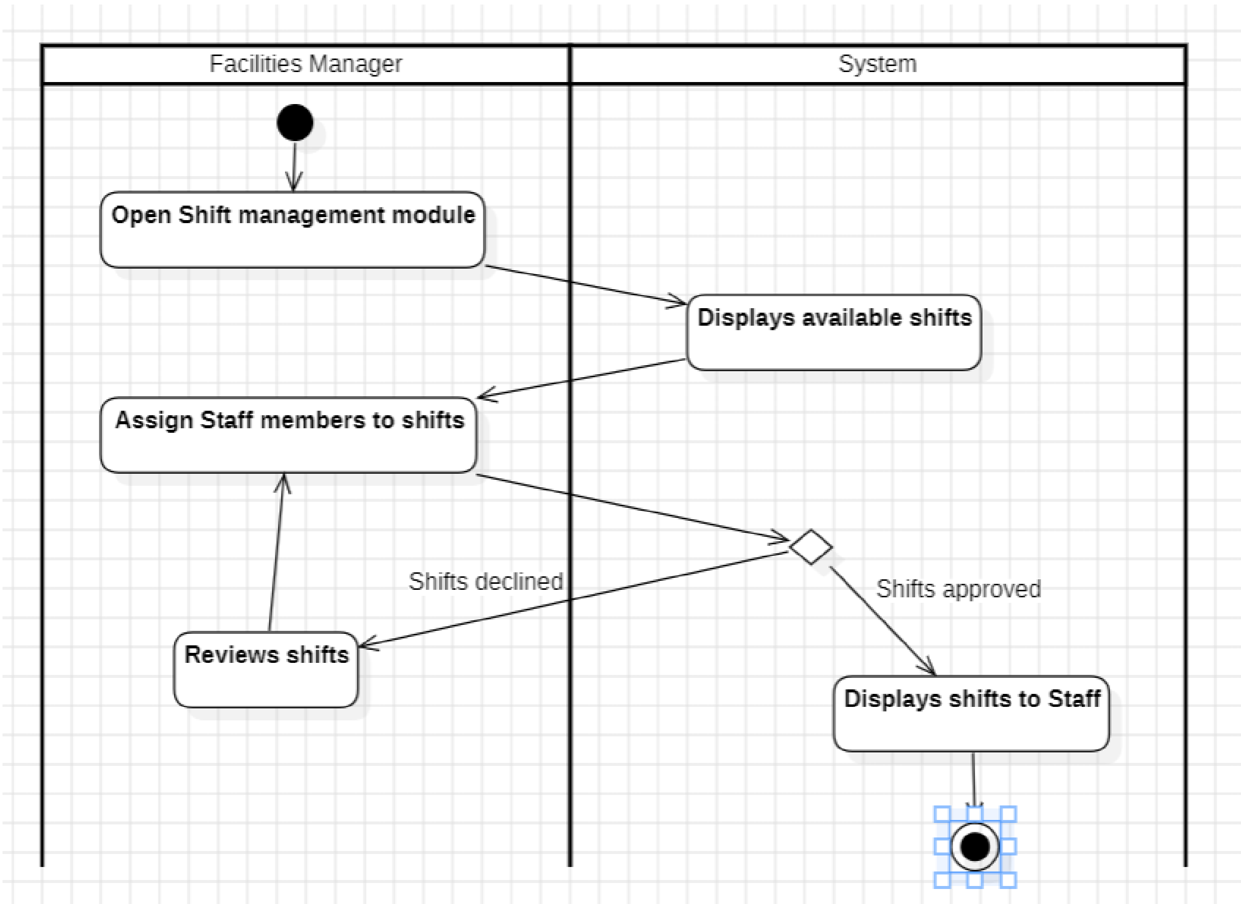
BS_FD_02


Customer
System
Open report fraud section
Display Fraud Form
Fill report form with suspicious activity
Submit form
Form complete
Form incomplete
Create fraud case
Display error
Send confirmation to customer

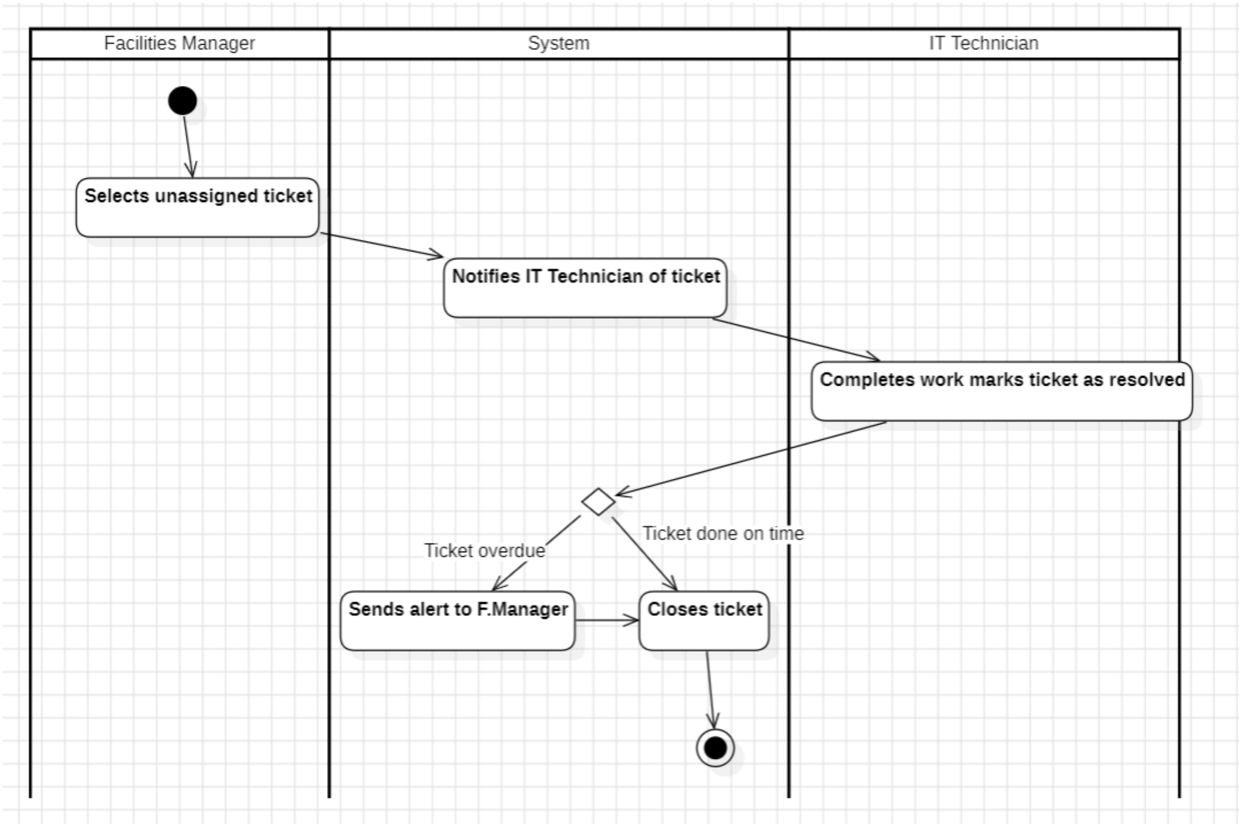
BS_FD_03


System
Bank Director
Detects unusual activity
Delivers alert for review
Reviews alert details
Action required
Action not required
Delegates to Bank Staff
Clears alert
Logs Director response

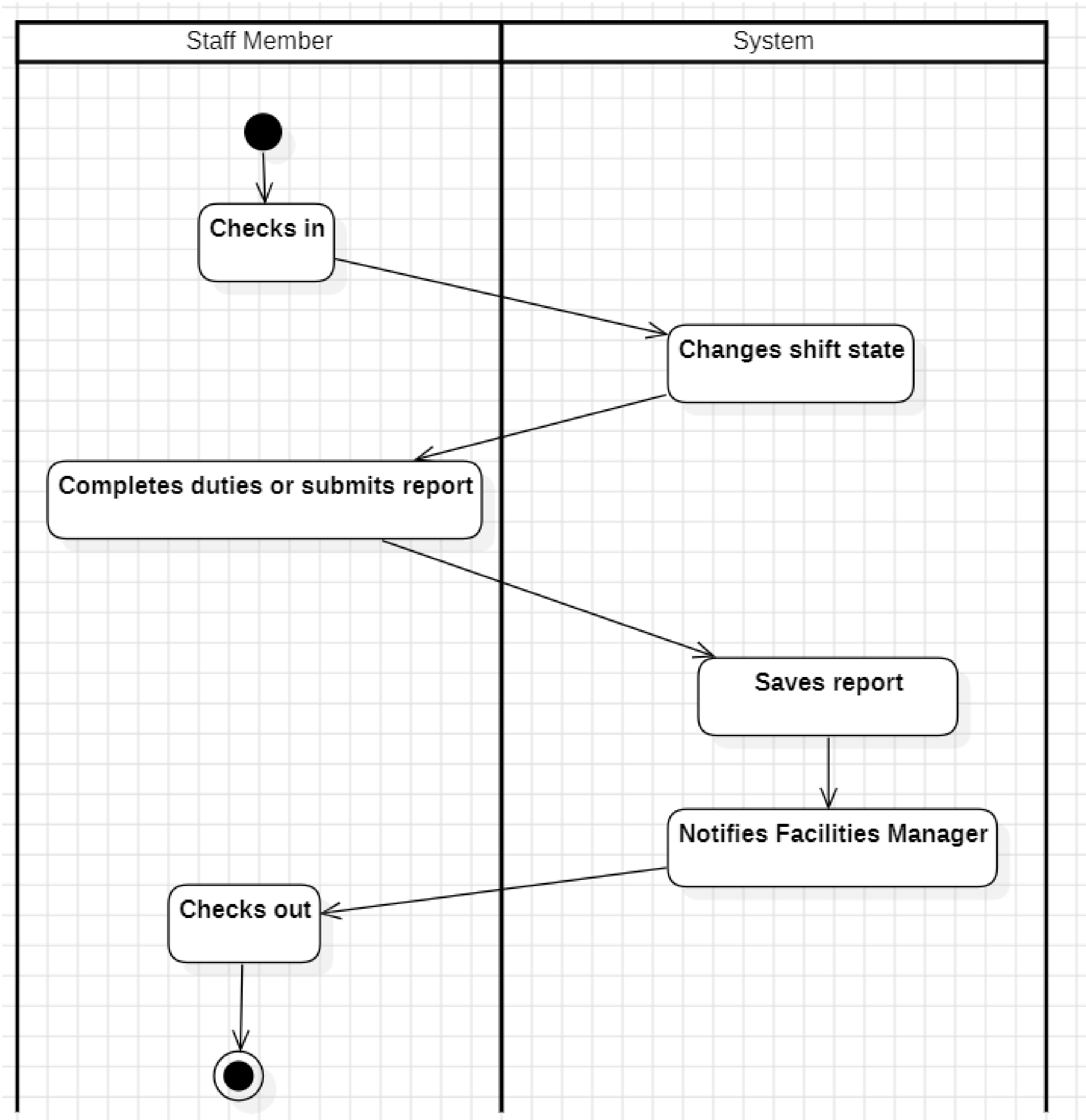
BS_FCM_01


Facilities Manager
System
Open Shift management module
Displays available shifts
Assign Staff members to shifts
Shifts declined
Shifts approved
Reviews shifts
Displays shifts to Staff

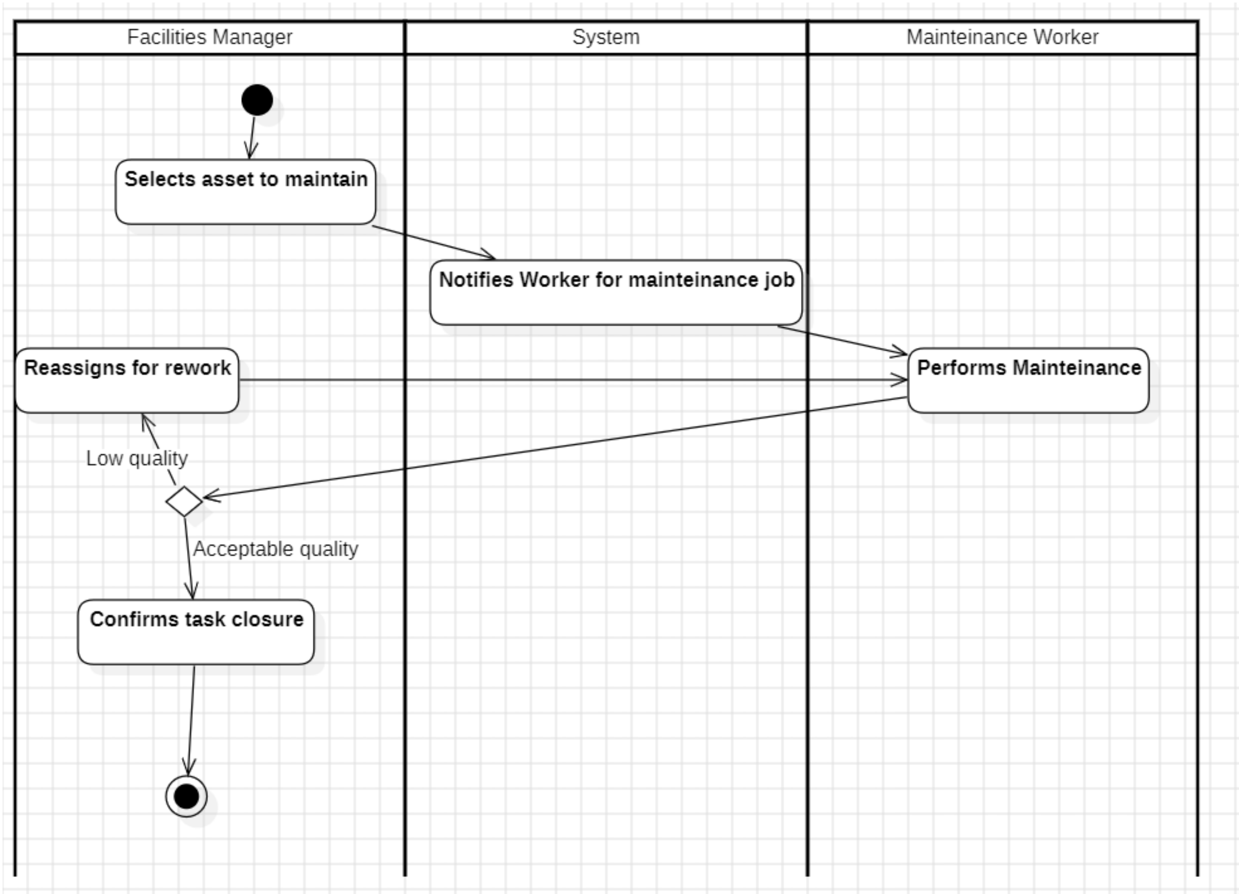
BS_FCM_02


Facilities Manager
System
IT Technician
Selects unassigned ticket
Notifies IT Technician of ticket
Completes work marks ticket as resolved
Ticket overdue
Ticket done on time
Sends alert to F.Manager
Closes ticket

BS_FCM_03

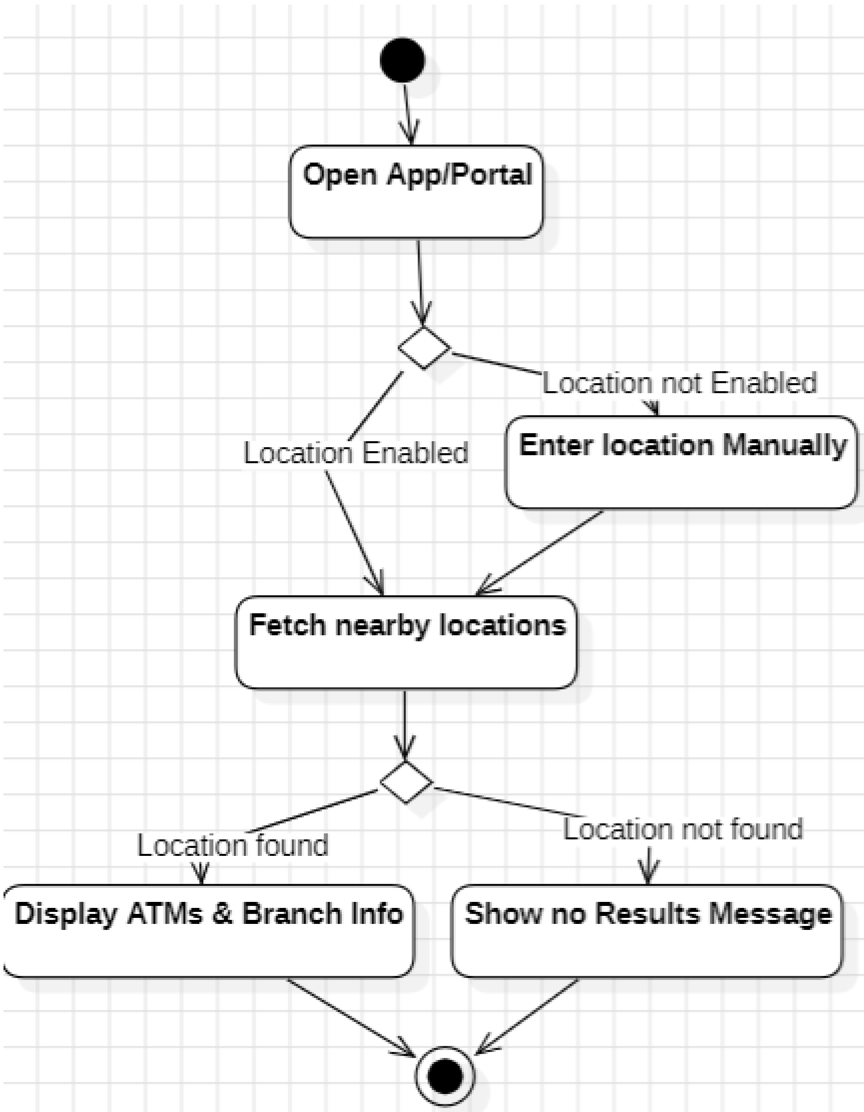

Staff Member
System
Checks in
Changes shift state
Completes duties or submits report
Saves report
Notifies Facilities Manager
Checks out

BS_FCM_04

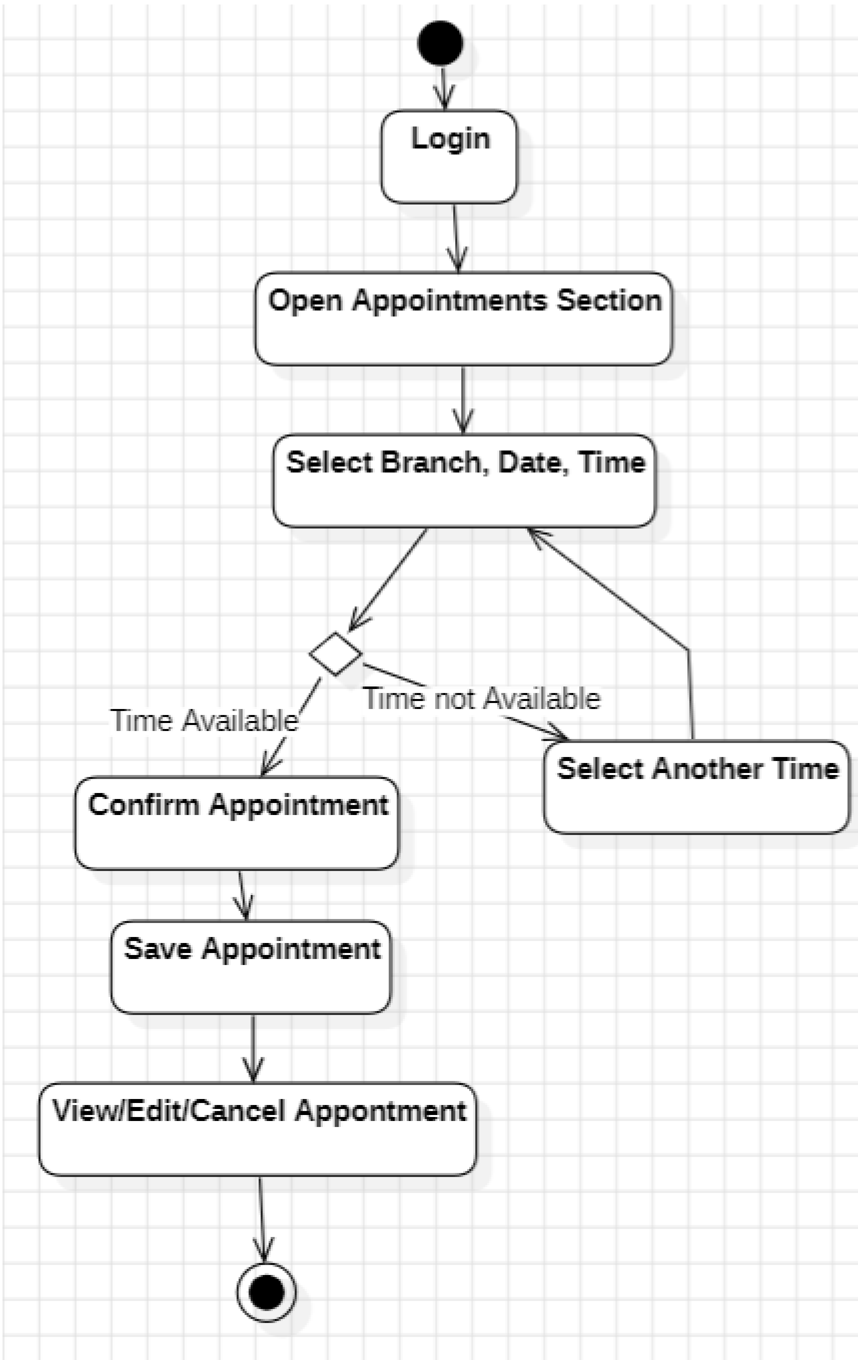

Facilities Manager
System
Mainteinance Worker
Selects asset to maintain
Notifies Worker for mainteinance job
Reassigns for rework
Performs Mainteinance
Low quality
Acceptable quality
Confirms task closure

Kleo Miraka

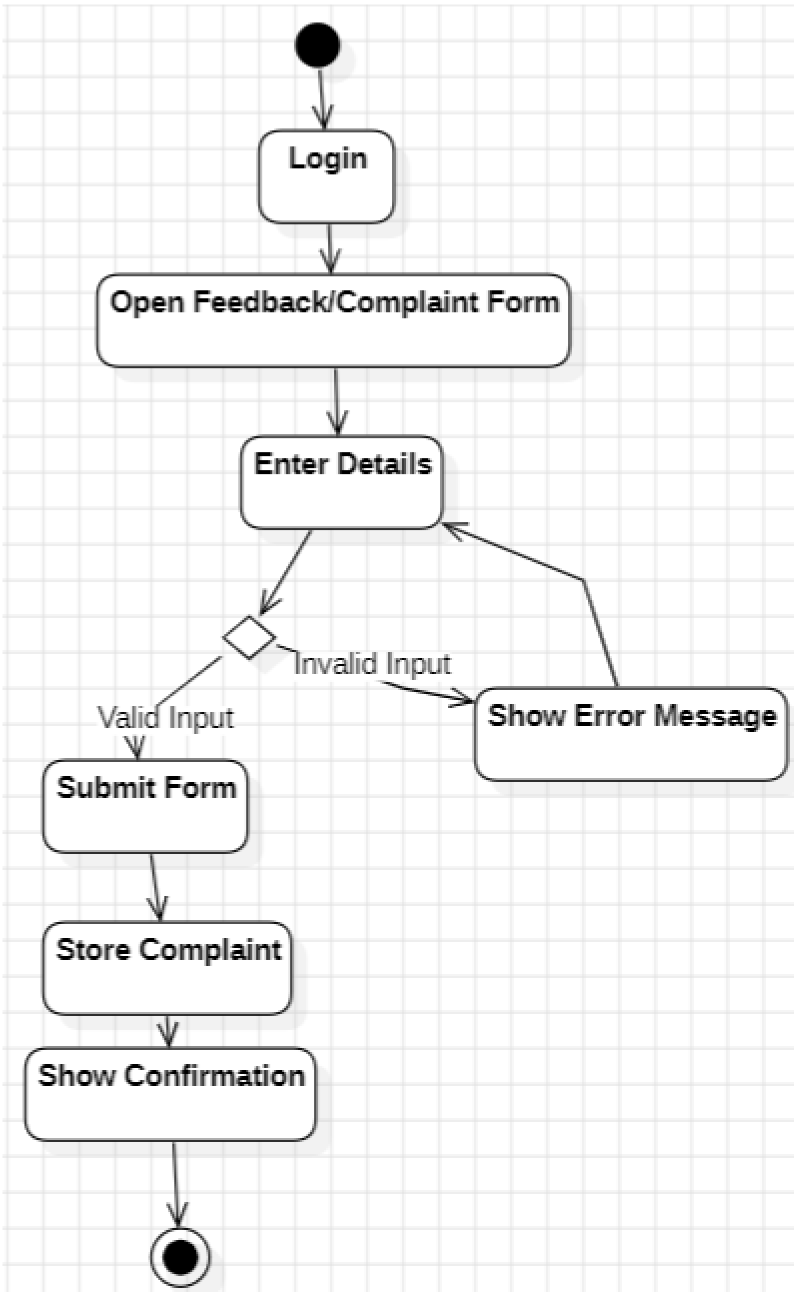
BS_CS_01



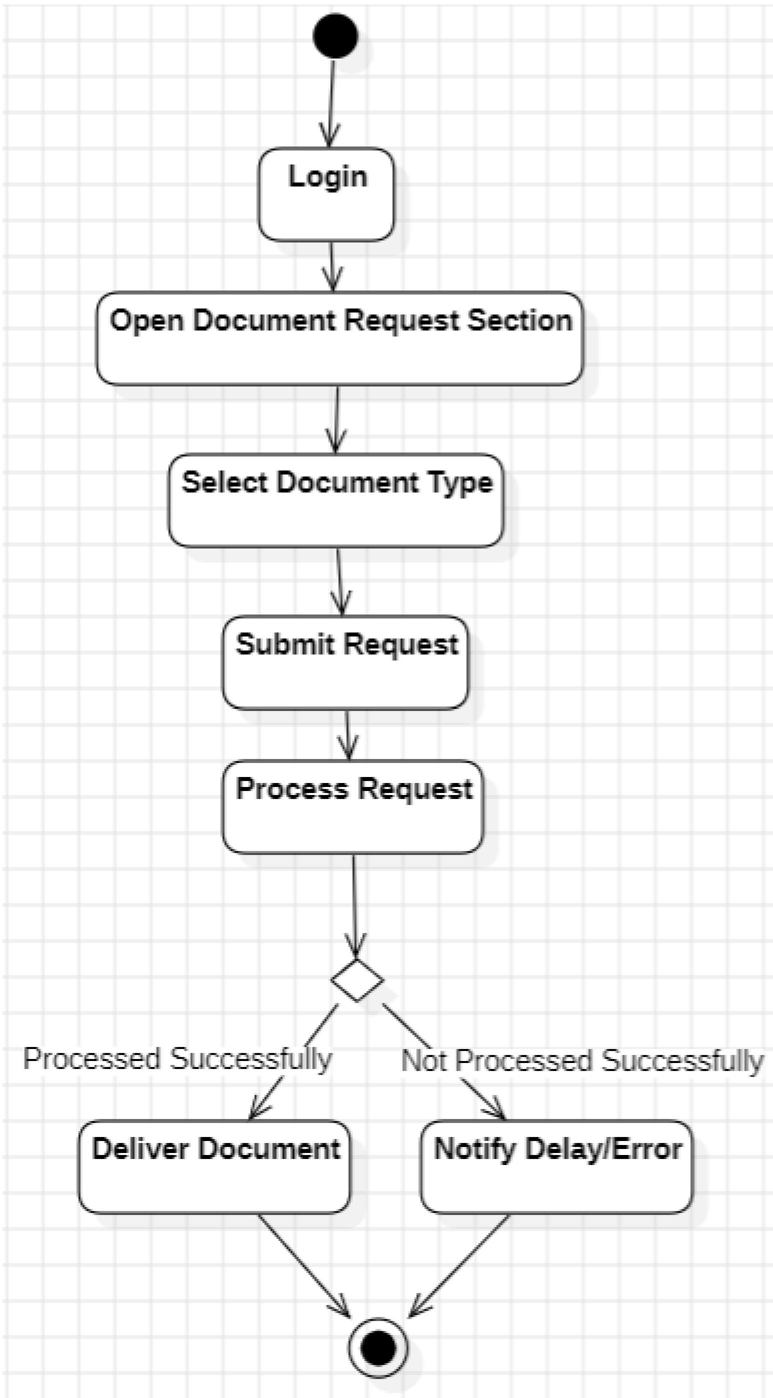
BS_CS_02


Login
Open Appointments Section
Select Branch, Date, Time
Time Available
Time not Available
Select Another Time
Confirm Appointment
Save Appointment
View/Edit/Cancel Appontment

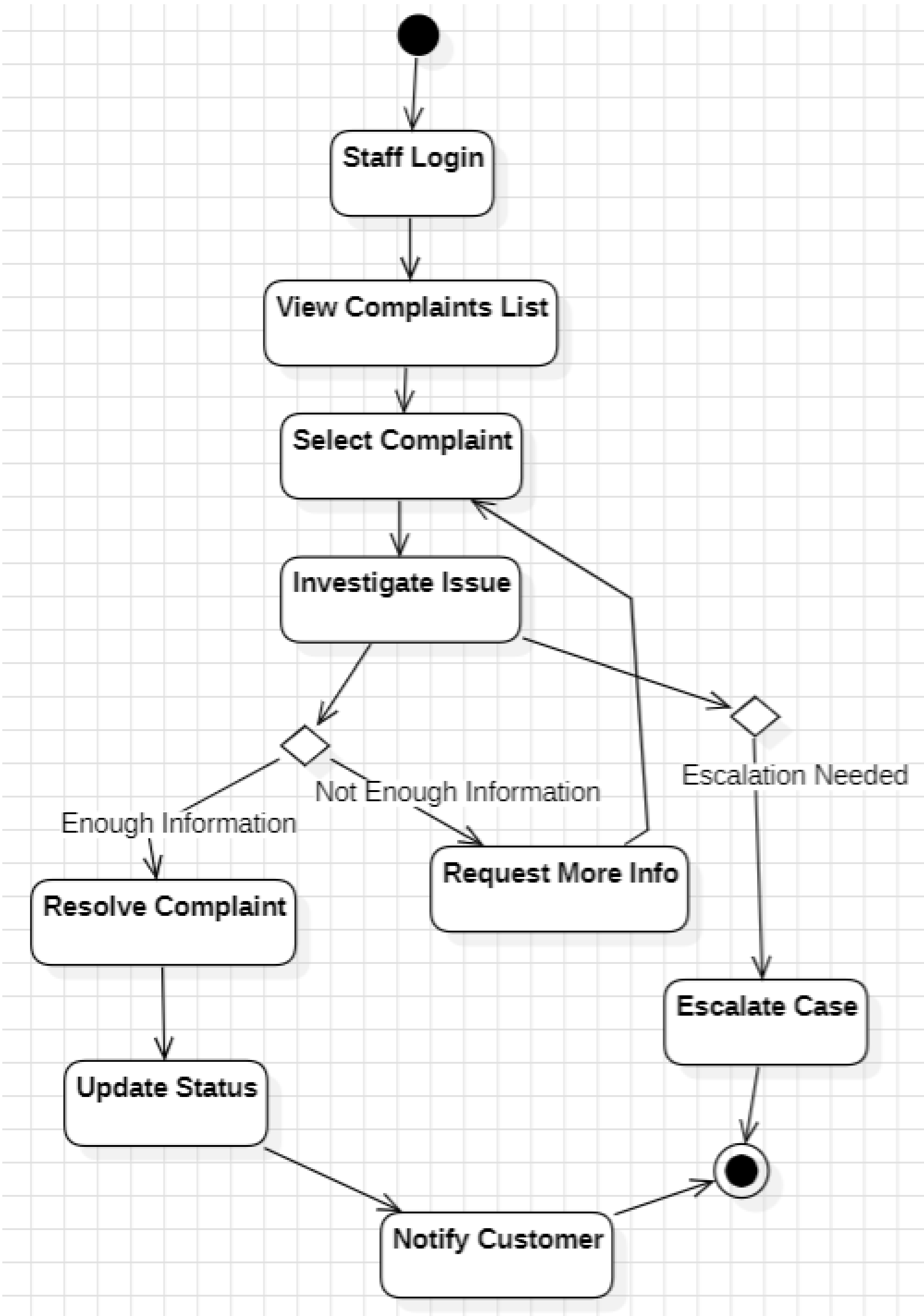
BS_CS_03


Login
Open Feedback/Complaint Form
Enter Details
Invalid Input
Show Error Message
Valid Input
Submit Form
Store Complaint
Show Confirmation

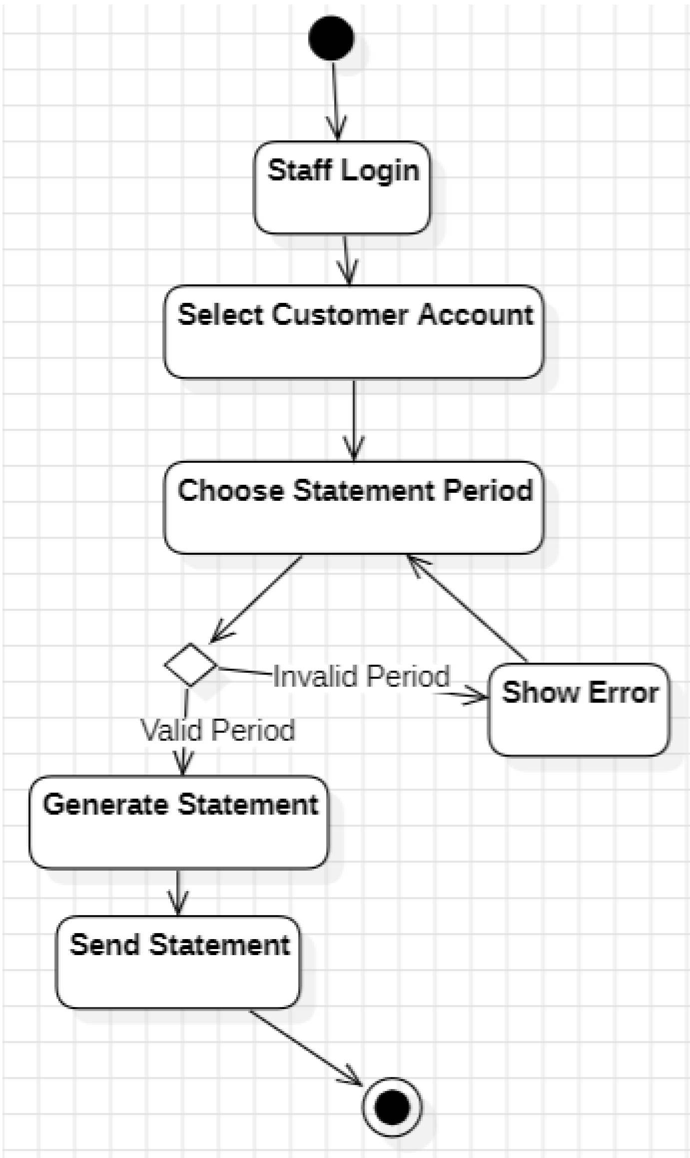
BS_CS_04


Login
Open Document Request Section
Select Document Type
Submit Request
Process Request
Processed Successfully
Not Processed Successfully
Deliver Document
Notify Delay/Error

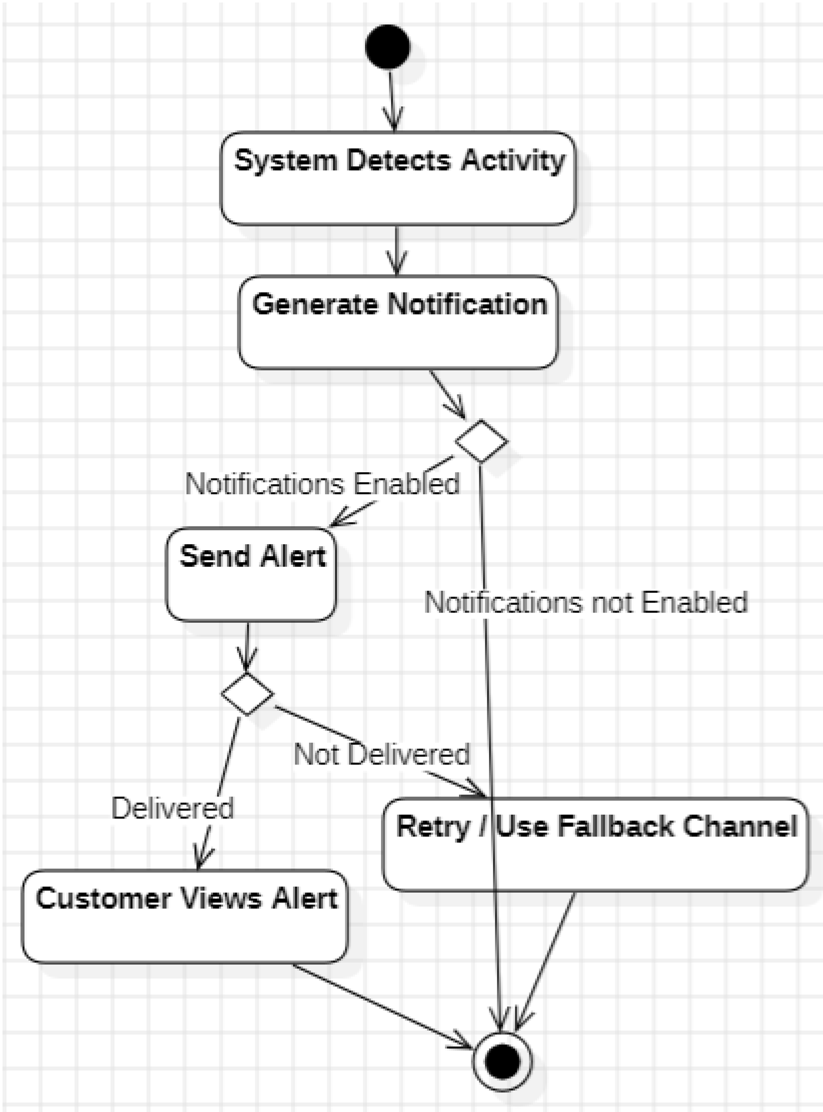
BS_CS_05


Staff Login
View Complaints List
Select Complaint
Investigate Issue
Not Enough Information
Escalation Needed
Enough Information
Request More Info
Resolve Complaint
Escalate Case
Update Status
Notify Customer

BS_CS_06


Staff Login
Select Customer Account
Choose Statement Period
Invalid Period
Show Error
Valid Period
Generate Statement
Send Statement

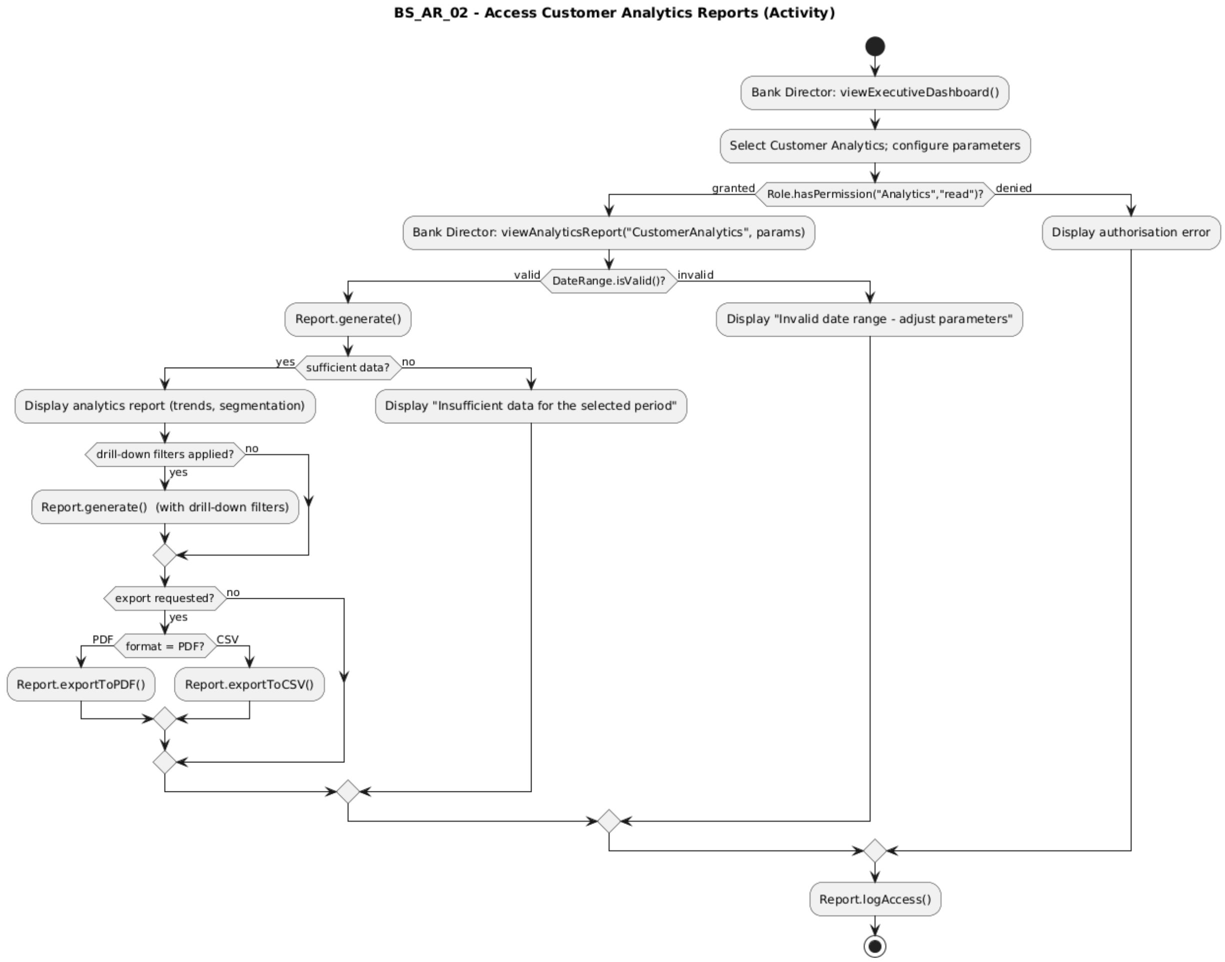
BS_CS_07


System Detects Activity
Generate Notification
Notifications Enabled
Notifications not Enabled
Send Alert
Not Delivered
Delivered
Retry / Use Fallback Channel
Customer Views Alert

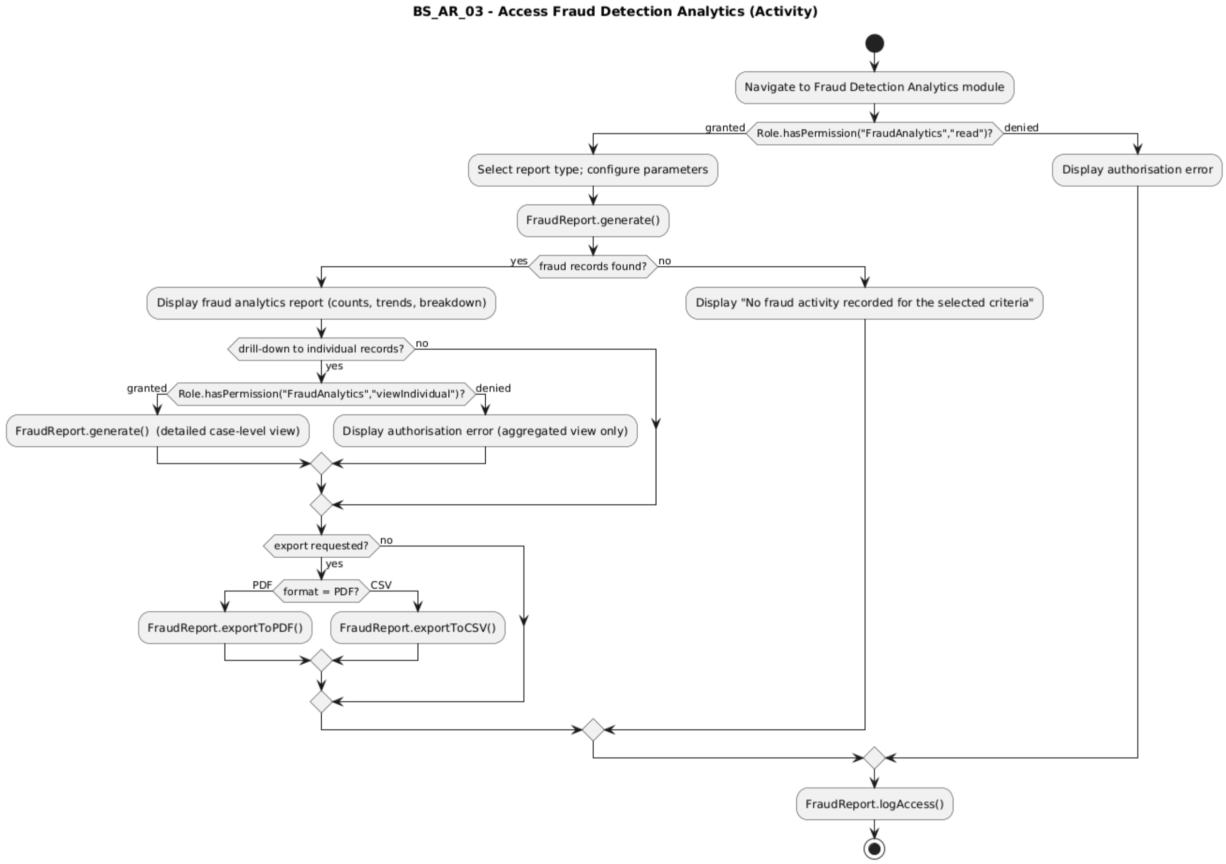
Erind Shabani

BS_AR_01

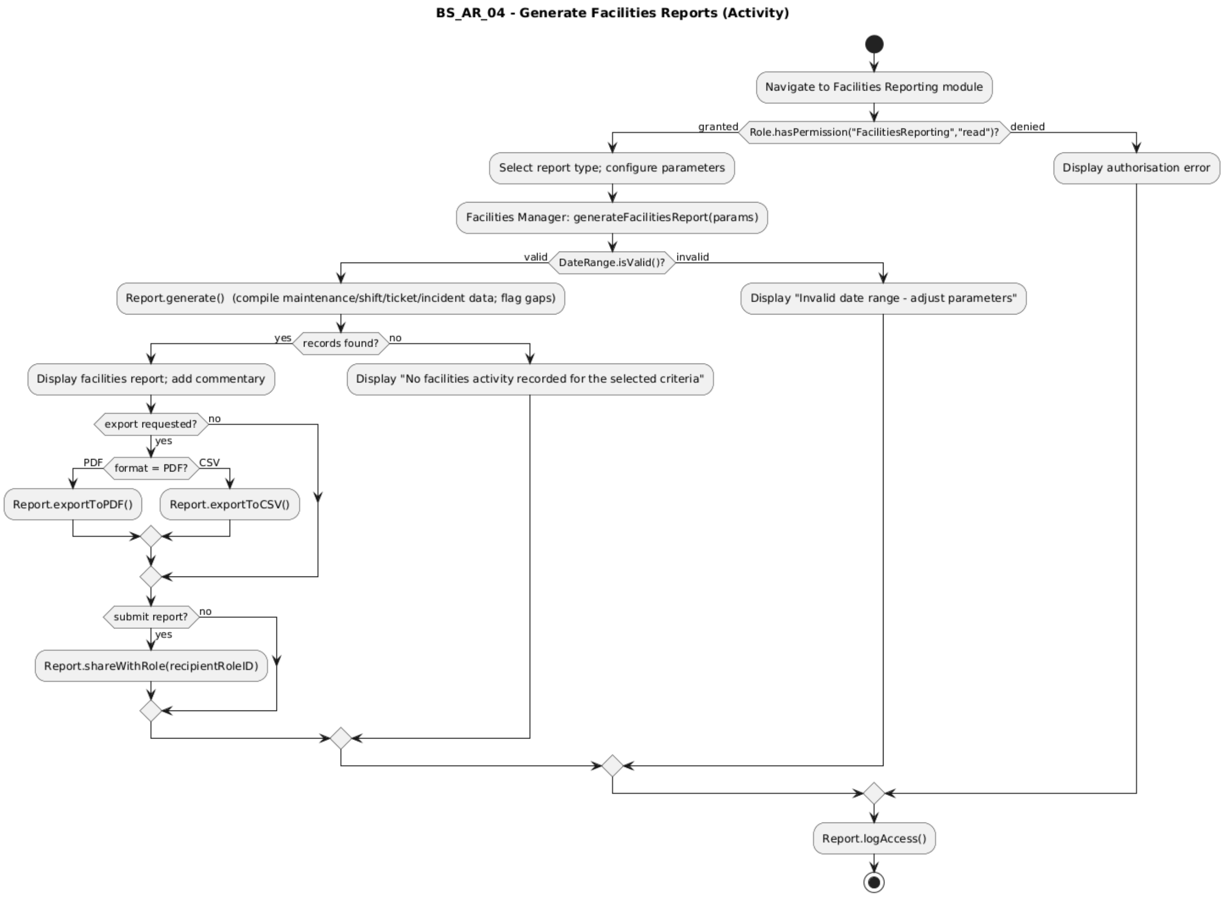
BS_AR_02


BS_AR_02 - Access Customer Analytics Reports (Activity)
Bank Director: viewExecutiveDashboard()
Select Customer Analytics; configure parameters
Role.hasPermission("Analytics","read")?
granted
denied
Bank Director: viewAnalyticsReport("CustomerAnalytics", params)
Display authorisation error
DateRange.isValid()?
valid
invalid
Report.generate()
Display "Invalid date range - adjust parameters"
sufficient data?
yes
no
Display analytics report (trends, segmentation)
Display "Insufficient data for the selected period"
drill-down filters applied?
no
yes
Report.generate() (with drill-down filters)
export requested?
no
yes
format = PDF?
PDF
CSV
Report.exportToPDF()
Report.exportToCSV()
Report.logAccess()

BS_AR_03


BS_AR_03 - Access Fraud Detection Analytics (Activity)
Navigate to Fraud Detection Analytics module
Role.hasPermission("FraudAnalytics","read")?
granted
denied
Select report type; configure parameters
Display authorisation error
FraudReport.generate()
fraud records found?
yes
no
Display fraud analytics report (counts, trends, breakdown)
Display "No fraud activity recorded for the selected criteria"
drill-down to individual records?
yes
no
Role.hasPermission("FraudAnalytics","viewIndividual")?
granted
denied
FraudReport.generate() (detailed case-level view)
Display authorisation error (aggregated view only)
export requested?
yes
no
format = PDF?
PDF
CSV
FraudReport.exportToPDF()
FraudReport.exportToCSV()
FraudReport.logAccess()

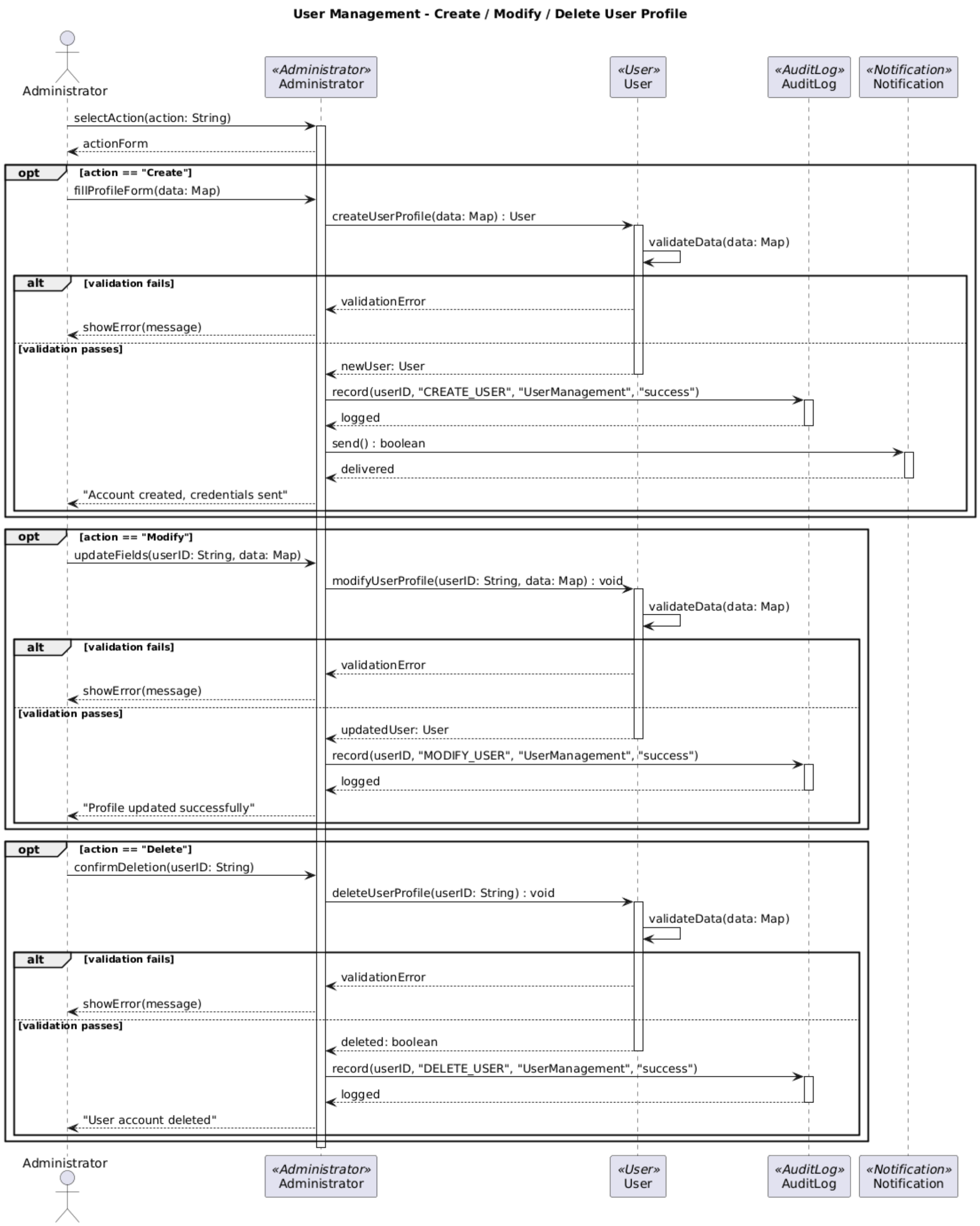
BS_AR_04


BS_AR_04 - Generate Facilities Reports (Activity)
Navigate to Facilities Reporting module
Role.hasPermission("FacilitiesReporting","read")?
granted
denied
Select report type; configure parameters
Display authorisation error
Facilities Manager: generateFacilitiesReport(params)
DateRange.isValid()?
valid
invalid
Report.generate() (compile maintenance/shift/ticket/incident data; flag gaps)
Display "Invalid date range - adjust parameters"
records found?
yes
no
Display facilities report; add commentary
Display "No facilities activity recorded for the selected criteria"
export requested?
no
yes
format = PDF?
PDF
CSV
Report.exportToPDF()
Report.exportToCSV()
submit report?
no
yes
Report.shareWithRole(recipientRoleID)
Report.logAccess()

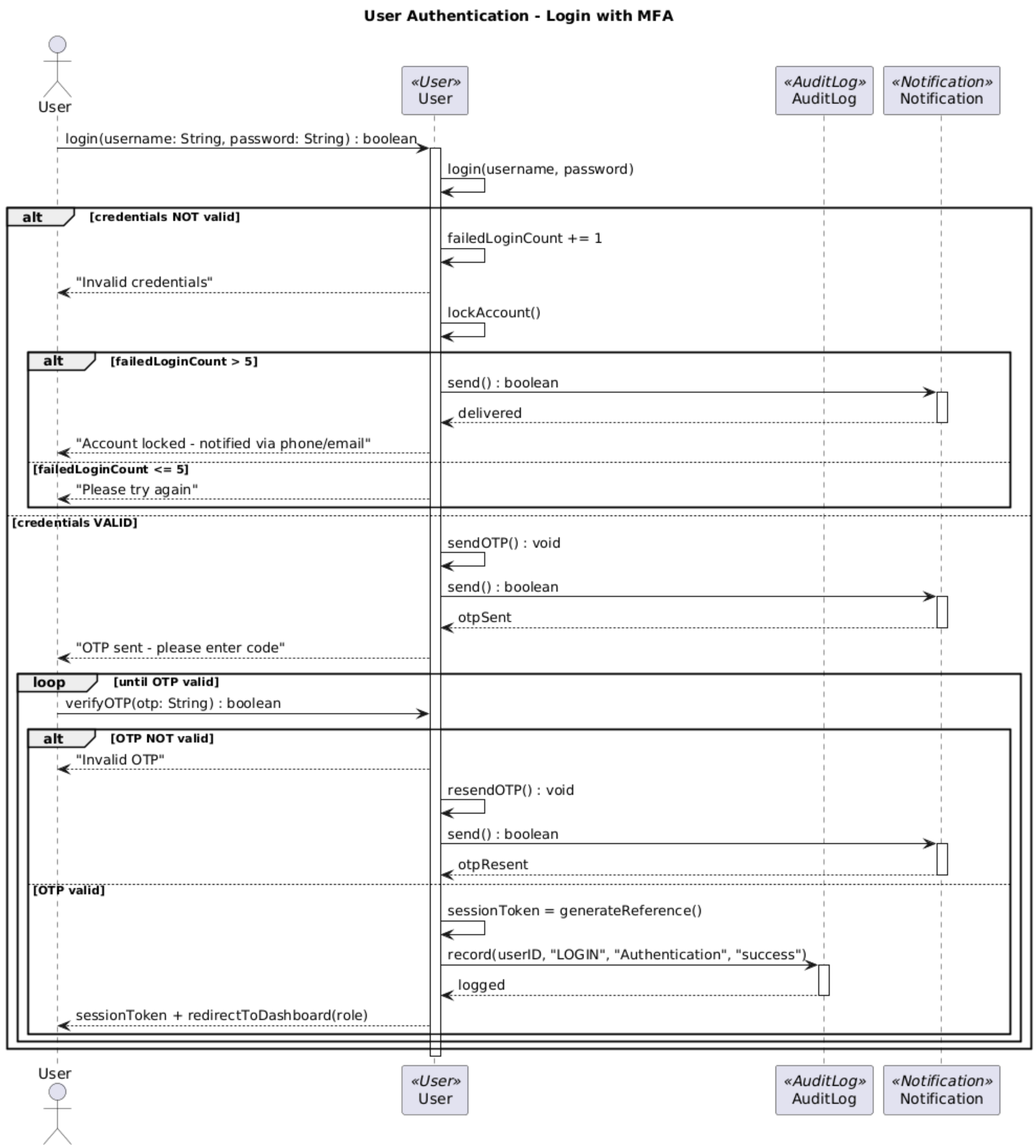
Sequence Diagrams

Andrea Ogreni

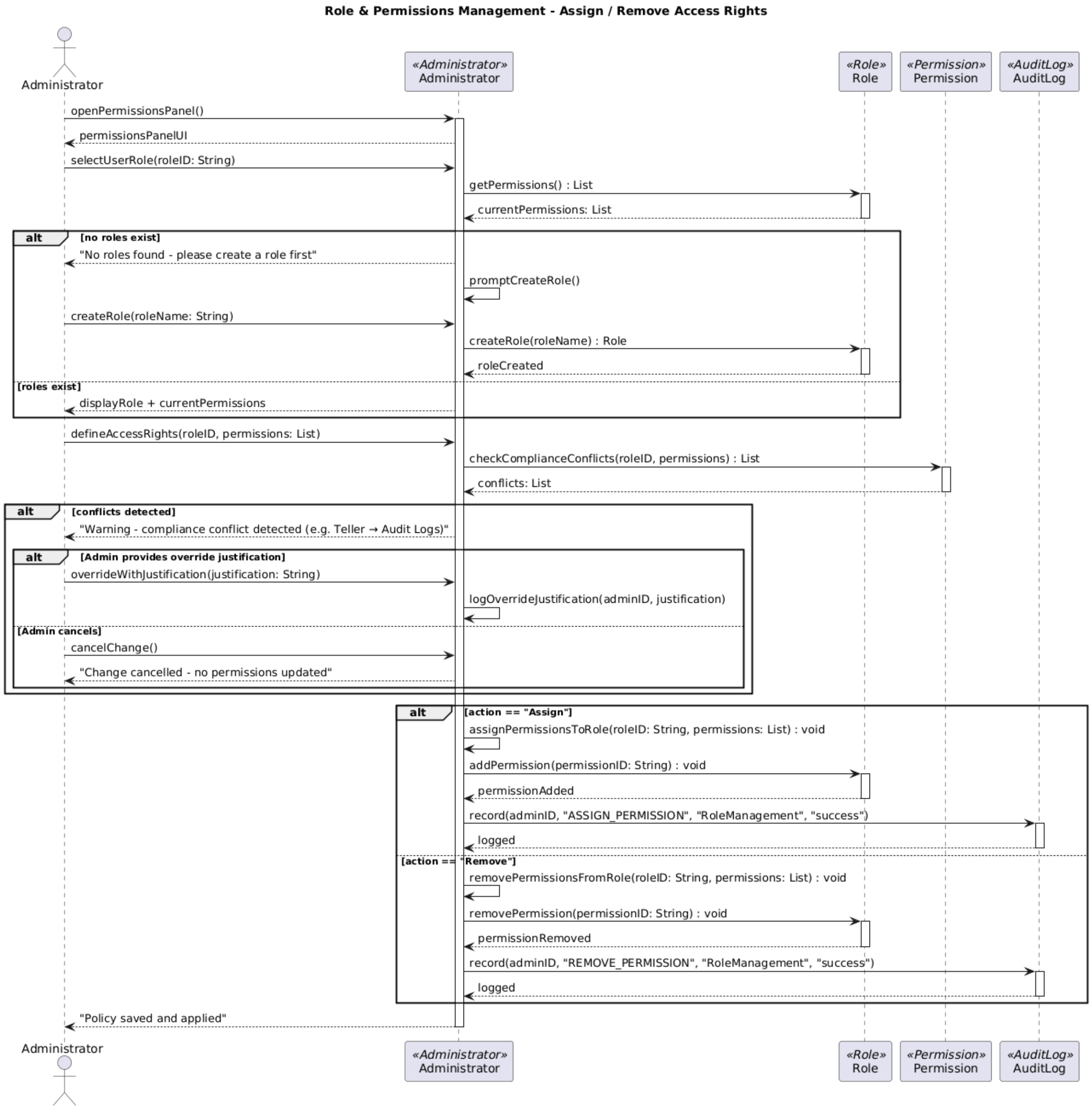
BS_UM_01



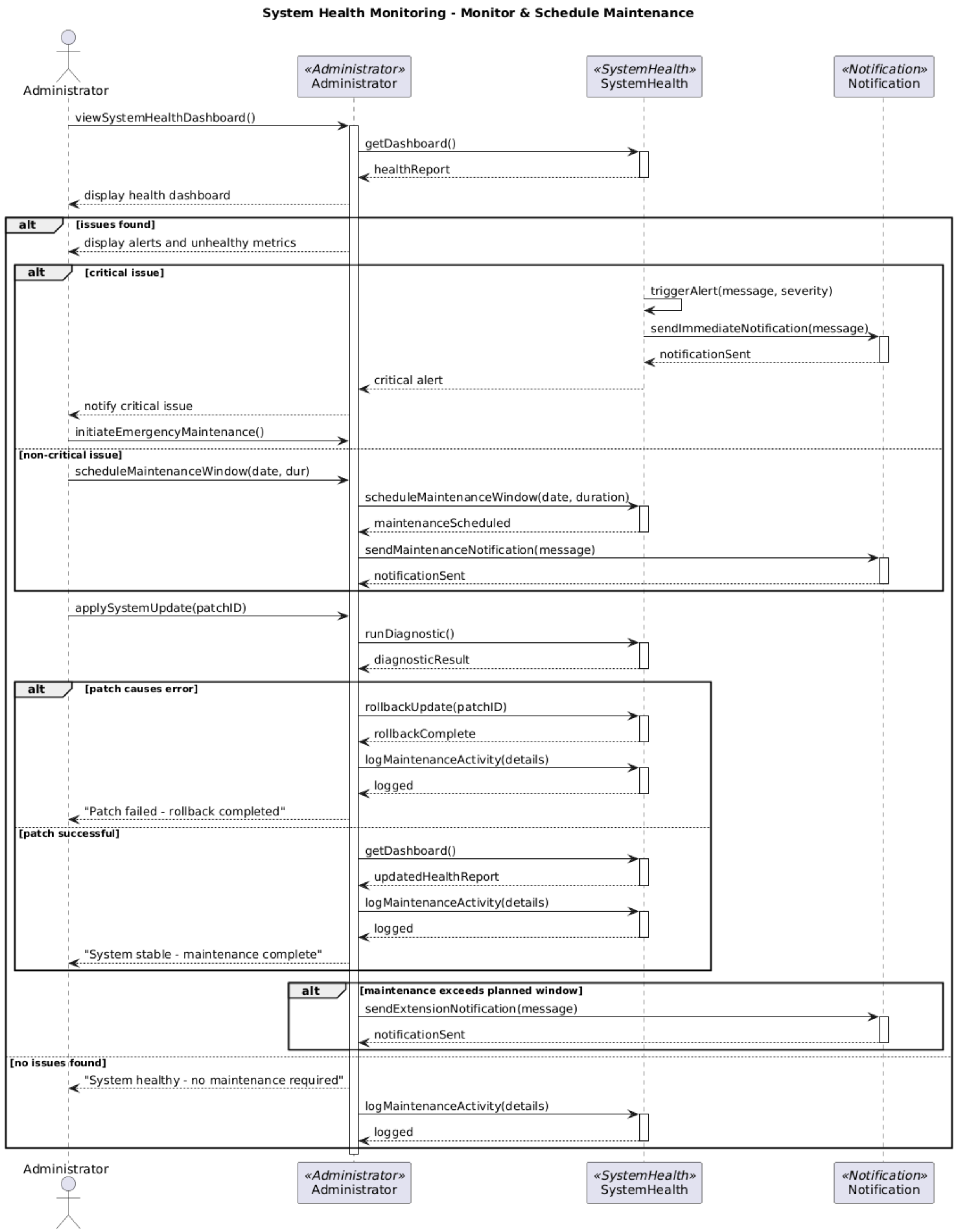
BS_UM_02


User Authentication - Login with MFA
User
«User»
User
«AuditLog»
AuditLog
«Notification»
Notification
login(username: String, password: String) : boolean
login(username, password)
alt
[credentials NOT valid]
failedLoginCount += 1
"Invalid credentials"
lockAccount()
alt
[failedLoginCount > 5]
send() : boolean
delivered
"Account locked - notified via phone/email"
[failedLoginCount <= 5]
"Please try again"
[credentials VALID]
sendOTP() : void
send() : boolean
otpSent
"OTP sent - please enter code"
loop
[until OTP valid]
verifyOTP(otp: String) : boolean
alt
[OTP NOT valid]
"Invalid OTP"
resendOTP() : void
send() : boolean
otpResent
[OTP valid]
sessionToken = generateReference()
record(userID, "LOGIN", "Authentication", "success")
logged
sessionToken + redirectToDashboard(role)

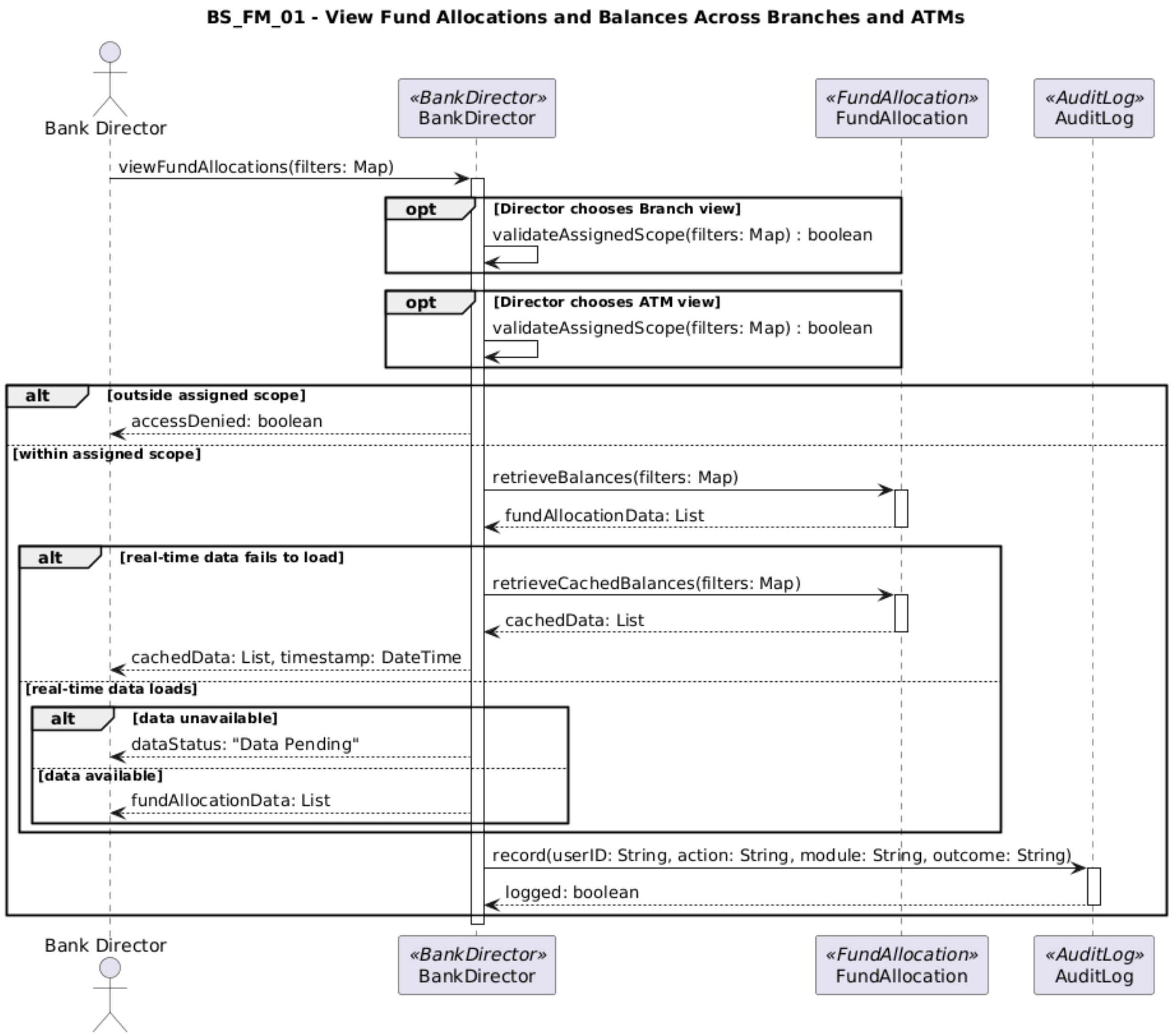
BS_UM_03


Role & Permissions Management - Assign / Remove Access Rights
Administrator
«Administrator» Administrator
«Role» Role
«Permission» Permission
«AuditLog» AuditLog
openPermissionsPanel()
permissionsPanelUI
selectUserRole(roleID: String)
getPermissions() : List
currentPermissions: List
alt [no roles exist]
"No roles found - please create a role first"
promptCreateRole()
createRole(roleName: String)
createRole(roleName) : Role
roleCreated
[roles exist]
displayRole + currentPermissions
defineAccessRights(roleID, permissions: List)
checkComplianceConflicts(roleID, permissions) : List
conflicts: List
alt [conflicts detected]
"Warning - compliance conflict detected (e.g. Teller → Audit Logs)"
alt [Admin provides override justification]
overrideWithJustification(justification: String)
logOverrideJustification(adminID, justification)
[Admin cancels]
cancelChange()
"Change cancelled - no permissions updated"
alt [action == "Assign"]
assignPermissionsToRole(roleID: String, permissions: List) : void
addPermission(permissionID: String) : void
permissionAdded
record(adminID, "ASSIGN_PERMISSION", "RoleManagement", "success")
logged
[action == "Remove"]
removePermissionsFromRole(roleID: String, permissions: List) : void
removePermission(permissionID: String) : void
permissionRemoved
record(adminID, "REMOVE_PERMISSION", "RoleManagement", "success")
logged
"Policy saved and applied"

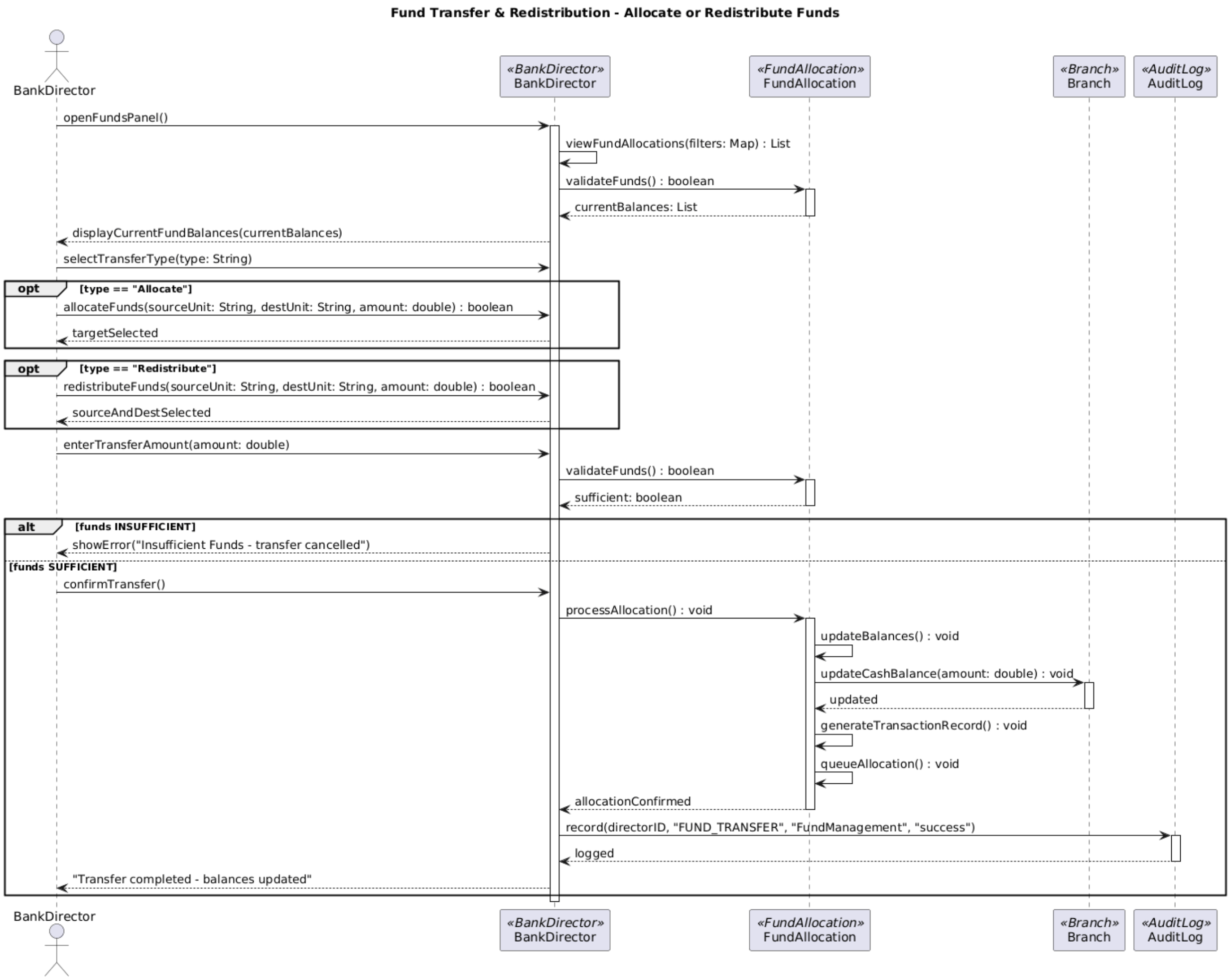
BS_UM_04


System Health Monitoring - Monitor & Schedule Maintenance
Administrator
«Administrator» Administrator
«SystemHealth» SystemHealth
«Notification» Notification
viewSystemHealthDashboard()
getDashboard()
healthReport
display health dashboard
alt
[issues found]
display alerts and unhealthy metrics
alt
[critical issue]
triggerAlert(message, severity)
sendImmediateNotification(message)
notificationSent
critical alert
notify critical issue
initiateEmergencyMaintenance()
[non-critical issue]
scheduleMaintenanceWindow(date, dur)
scheduleMaintenanceWindow(date, duration)
maintenanceScheduled
sendMaintenanceNotification(message)
notificationSent
applySystemUpdate(patchID)
runDiagnostic()
diagnosticResult
alt
[patch causes error]
rollbackUpdate(patchID)
rollbackComplete
logMaintenanceActivity(details)
logged
"Patch failed - rollback completed"
[patch successful]
getDashboard()
updatedHealthReport
logMaintenanceActivity(details)
logged
"System stable - maintenance complete"
alt
[maintenance exceeds planned window]
sendExtensionNotification(message)
notificationSent
[no issues found]
"System healthy - no maintenance required"
logMaintenanceActivity(details)
logged
Administrator
«Administrator» Administrator
«SystemHealth» SystemHealth
«Notification» Notification

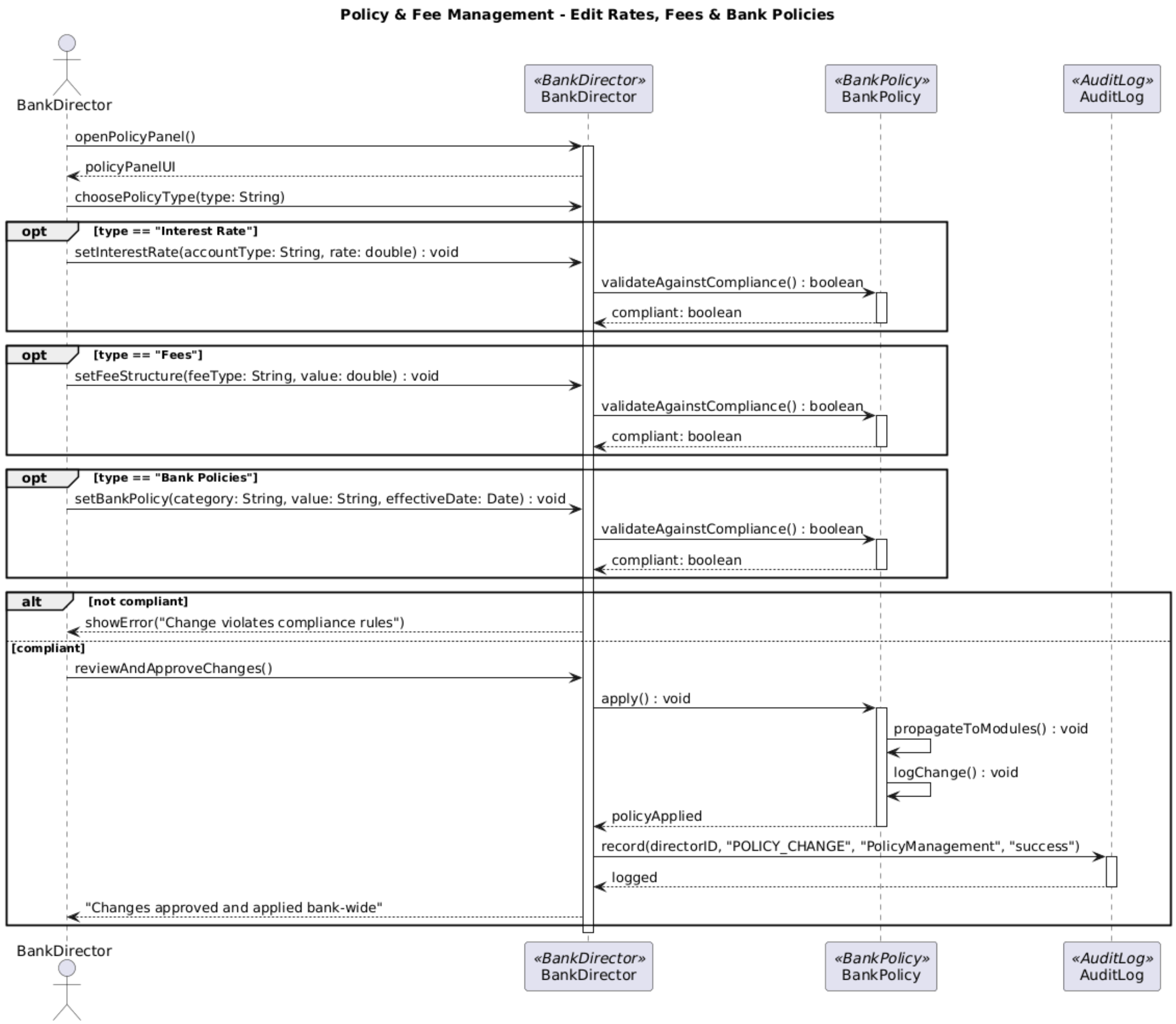
BS_FM_01


BS_FM_01 - View Fund Allocations and Balances Across Branches and ATMs
Bank Director
«BankDirector»
BankDirector
«FundAllocation»
FundAllocation
«AuditLog»
AuditLog
viewFundAllocations(filters: Map)
opt
[Director chooses Branch view]
validateAssignedScope(filters: Map) : boolean
opt
[Director chooses ATM view]
validateAssignedScope(filters: Map) : boolean
alt
[outside assigned scope]
accessDenied: boolean
[within assigned scope]
retrieveBalances(filters: Map)
fundAllocationData: List
alt
[real-time data fails to load]
retrieveCachedBalances(filters: Map)
cachedData: List
cachedData: List, timestamp: DateTime
[real-time data loads]
alt
[data unavailable]
dataStatus: "Data Pending"
[data available]
fundAllocationData: List
record(userID: String, action: String, module: String, outcome: String)
logged: boolean
Bank Director
«BankDirector»
BankDirector
«FundAllocation»
FundAllocation
«AuditLog»
AuditLog

BS_FM_02

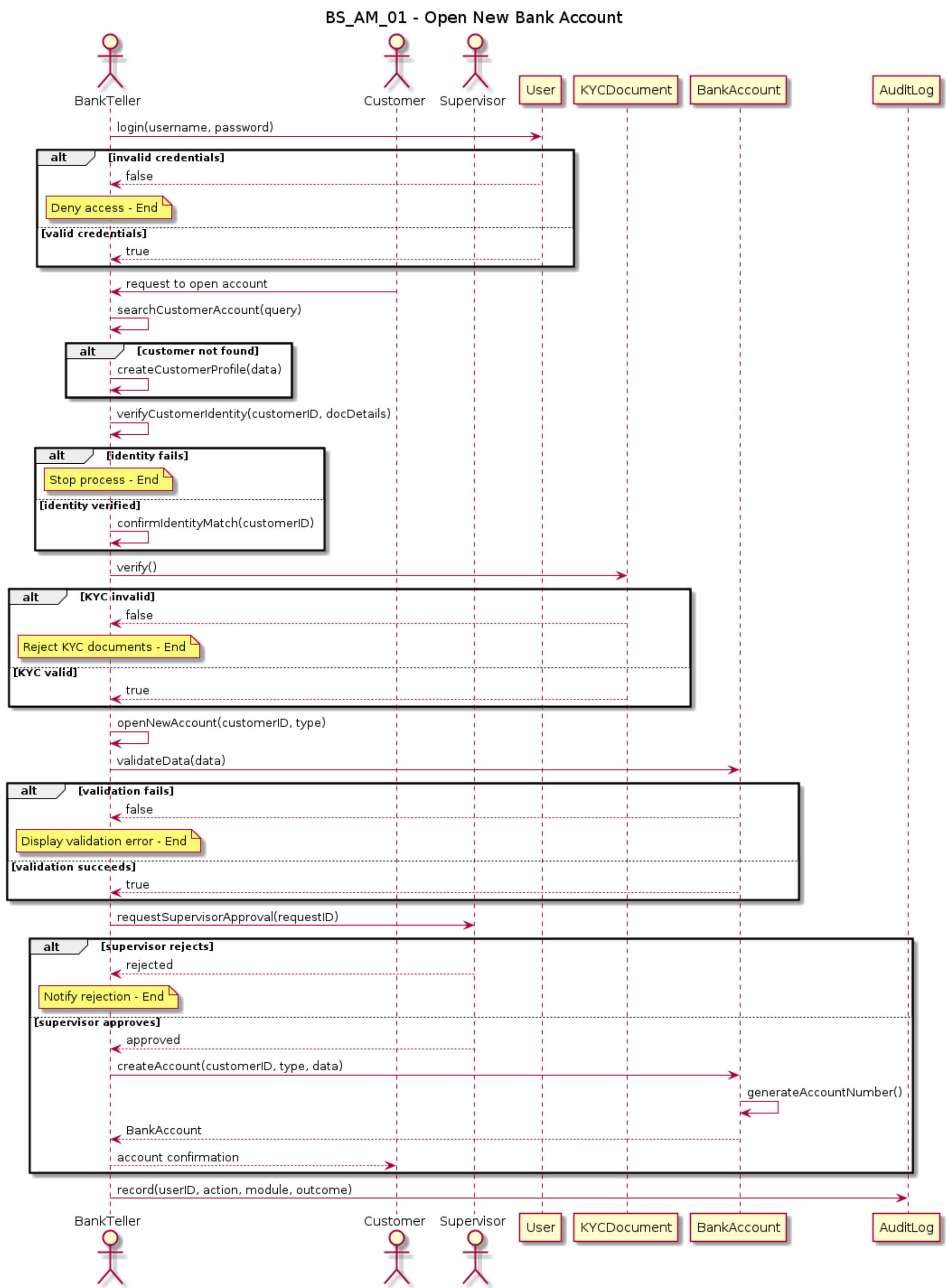

Fund Transfer & Redistribution - Allocate or Redistribute Funds
BankDirector
«BankDirector»
BankDirector
«FundAllocation»
FundAllocation
«Branch»
Branch
«AuditLog»
AuditLog
openFundsPanel()
viewFundAllocations(filters: Map) : List
validateFunds() : boolean
currentBalances: List
displayCurrentFundBalances(currentBalances)
selectTransferType(type: String)
opt
[type == "Allocate"]
allocateFunds(sourceUnit: String, destUnit: String, amount: double) : boolean
targetSelected
opt
[type == "Redistribute"]
redistributeFunds(sourceUnit: String, destUnit: String, amount: double) : boolean
sourceAndDestSelected
enterTransferAmount(amount: double)
validateFunds() : boolean
sufficient: boolean
alt
[funds INSUFFICIENT]
showError("Insufficient Funds - transfer cancelled")
[funds SUFFICIENT]
confirmTransfer()
processAllocation() : void
updateBalances() : void
updateCashBalance(amount: double) : void
updated
generateTransactionRecord() : void
queueAllocation() : void
allocationConfirmed
record(directorID, "FUND_TRANSFER", "FundManagement", "success")
logged
"Transfer completed - balances updated"
BankDirector
«BankDirector»
BankDirector
«FundAllocation»
FundAllocation
«Branch»
Branch
«AuditLog»
AuditLog

BS_FM_03

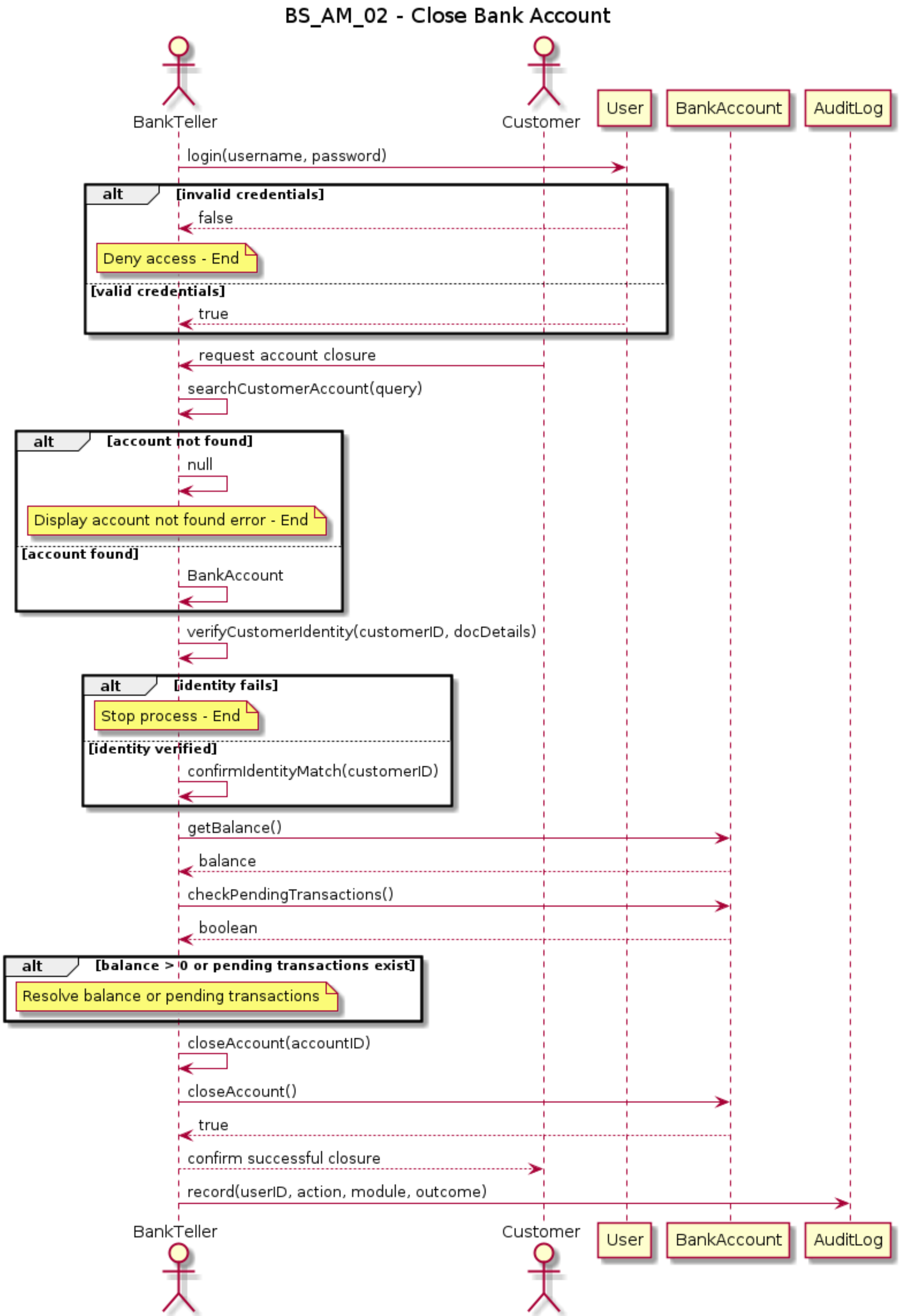

Policy & Fee Management - Edit Rates, Fees & Bank Policies
BankDirector
«BankDirector» BankDirector
«BankPolicy» BankPolicy
«AuditLog» AuditLog
openPolicyPanel()
policyPanelUI
choosePolicyType(type: String)
opt [type == "Interest Rate"]
setInterestRate(accountType: String, rate: double) : void
validateAgainstCompliance() : boolean
compliant: boolean
opt [type == "Fees"]
setFeeStructure(feeType: String, value: double) : void
validateAgainstCompliance() : boolean
compliant: boolean
opt [type == "Bank Policies"]
setBankPolicy(category: String, value: String, effectiveDate: Date) : void
validateAgainstCompliance() : boolean
compliant: boolean
alt [not compliant]
showError("Change violates compliance rules")
[compliant]
reviewAndApproveChanges()
apply() : void
propagateToModules() : void
logChange() : void
policyApplied
record(directorID, "POLICY_CHANGE", "PolicyManagement", "success")
logged
"Changes approved and applied bank-wide"

Joel Lluri

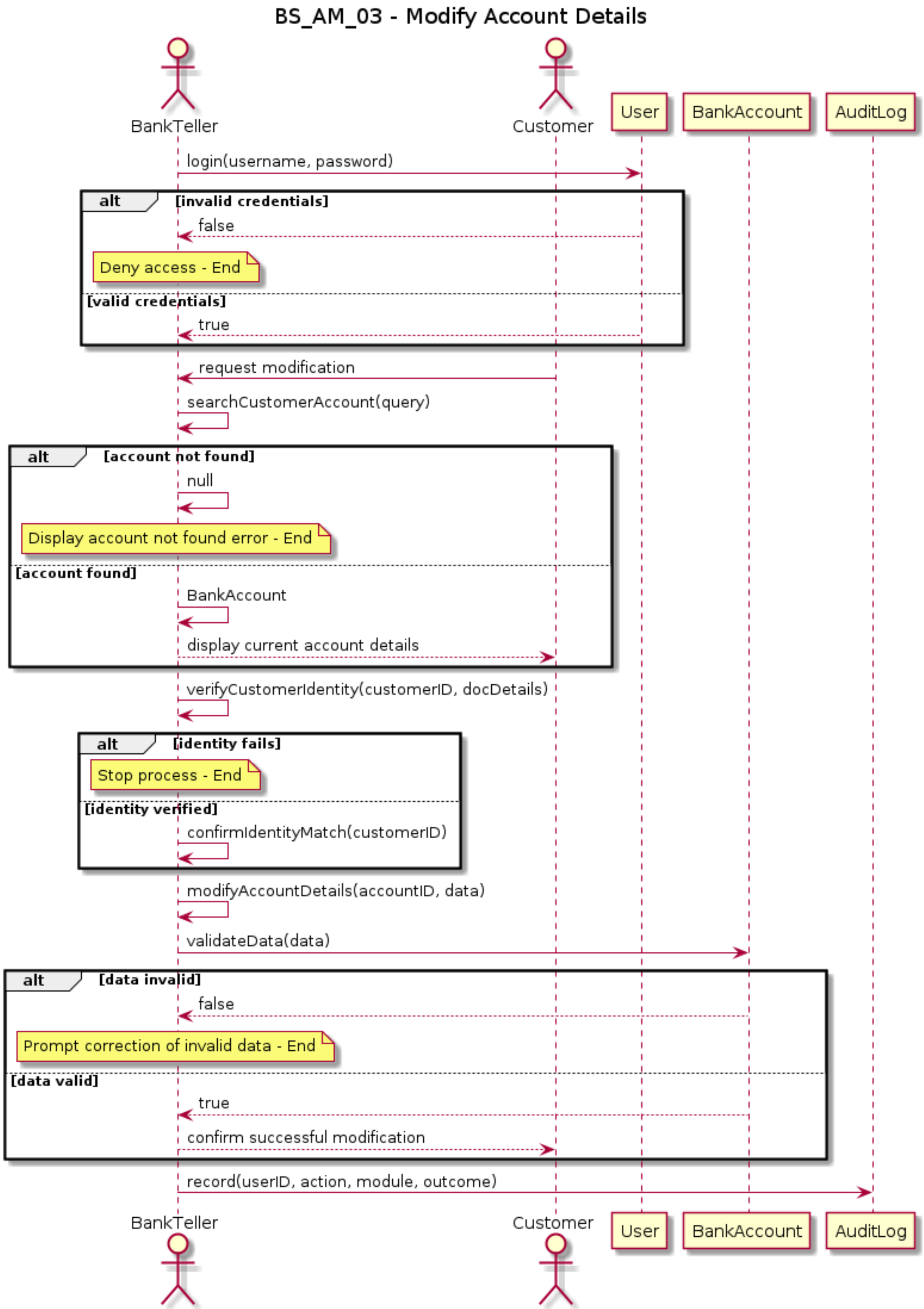
BS_AM_01



BS_AM_02

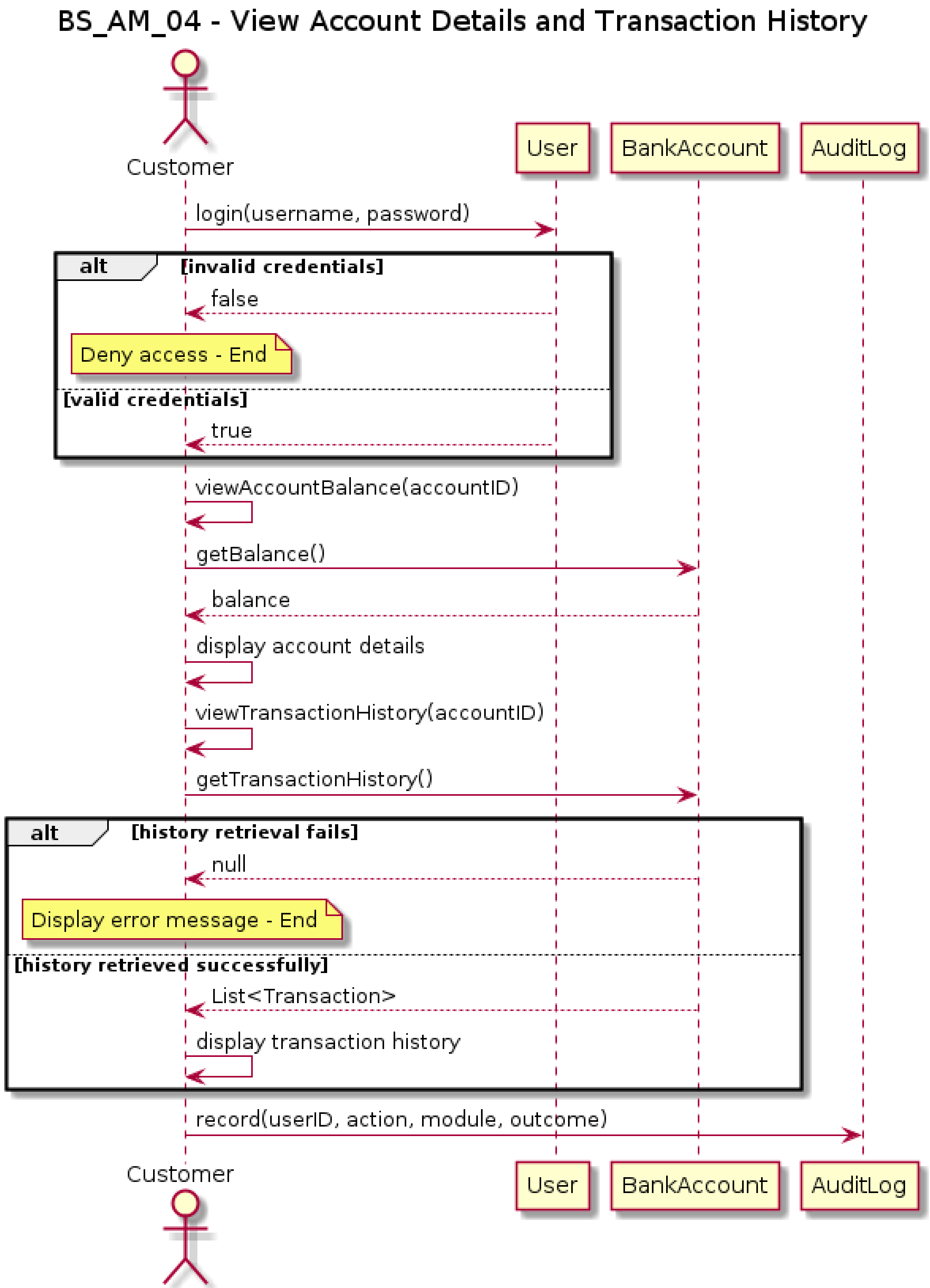

BS_AM_02 - Close Bank Account
BankTeller
Customer
User
BankAccount
AuditLog
login(username, password)
alt
[invalid credentials]
false
Deny access - End
[valid credentials]
true
request account closure
searchCustomerAccount(query)
alt
[account not found]
null
Display account not found error - End
[account found]
BankAccount
verifyCustomerIdentity(customerID, docDetails)
alt
[identity fails]
Stop process - End
[identity verified]
confirmIdentityMatch(customerID)
getBalance()
balance
checkPendingTransactions()
boolean
alt
[balance > 0 or pending transactions exist]
Resolve balance or pending transactions
closeAccount(accountID)
closeAccount()
true
confirm successful closure
record(userID, action, module, outcome)
BankTeller
Customer
User
BankAccount
AuditLog

BS_AM_03

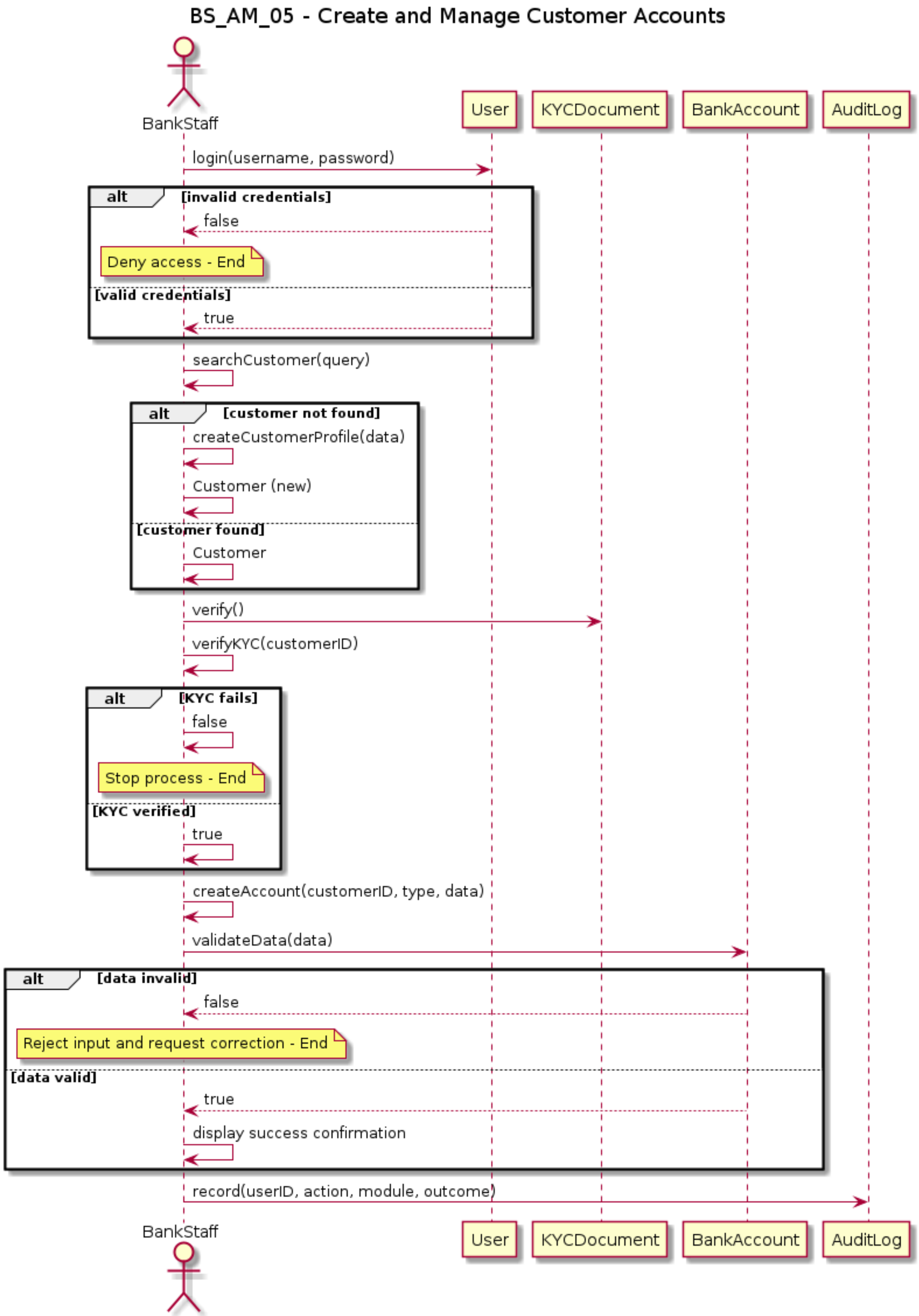


BS_AM_03 - Modify Account Details
BankTeller
Customer
User
BankAccount
AuditLog
login(username, password)
alt
[invalid credentials]
false
Deny access - End
[valid credentials]
true
request modification
searchCustomerAccount(query)
alt
[account not found]
null
Display account not found error - End
[account found]
BankAccount
display current account details
verifyCustomerIdentity(customerID, docDetails)
alt
[identity fails]
Stop process - End
[identity verified]
confirmIdentityMatch(customerID)
modifyAccountDetails(accountID, data)
validateData(data)
alt
[data invalid]
false
Prompt correction of invalid data - End
[data valid]
true
confirm successful modification
record(userID, action, module, outcome)
BankTeller
Customer
User
BankAccount
AuditLog

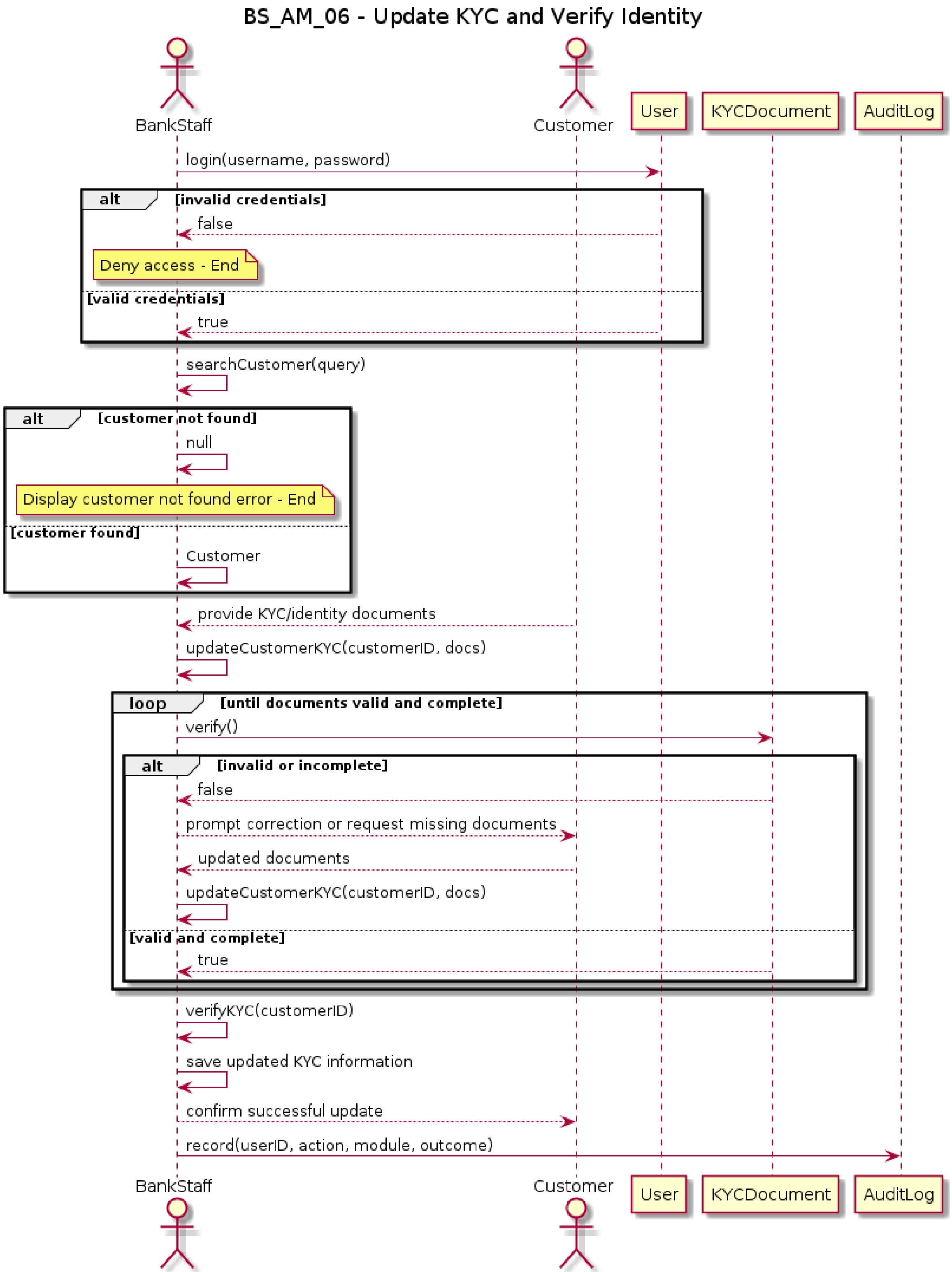
BS_AM_04


BS_AM_04 - View Account Details and Transaction History
Customer
User
BankAccount
AuditLog
login(username, password)
alt
[invalid credentials]
false
Deny access - End
[valid credentials]
true
viewAccountBalance(accountID)
getBalance()
balance
display account details
viewTransactionHistory(accountID)
getTransactionHistory()
alt
[history retrieval fails]
null
Display error message - End
[history retrieved successfully]
List<Transaction>
display transaction history
record(userID, action, module, outcome)
Customer
User
BankAccount
AuditLog

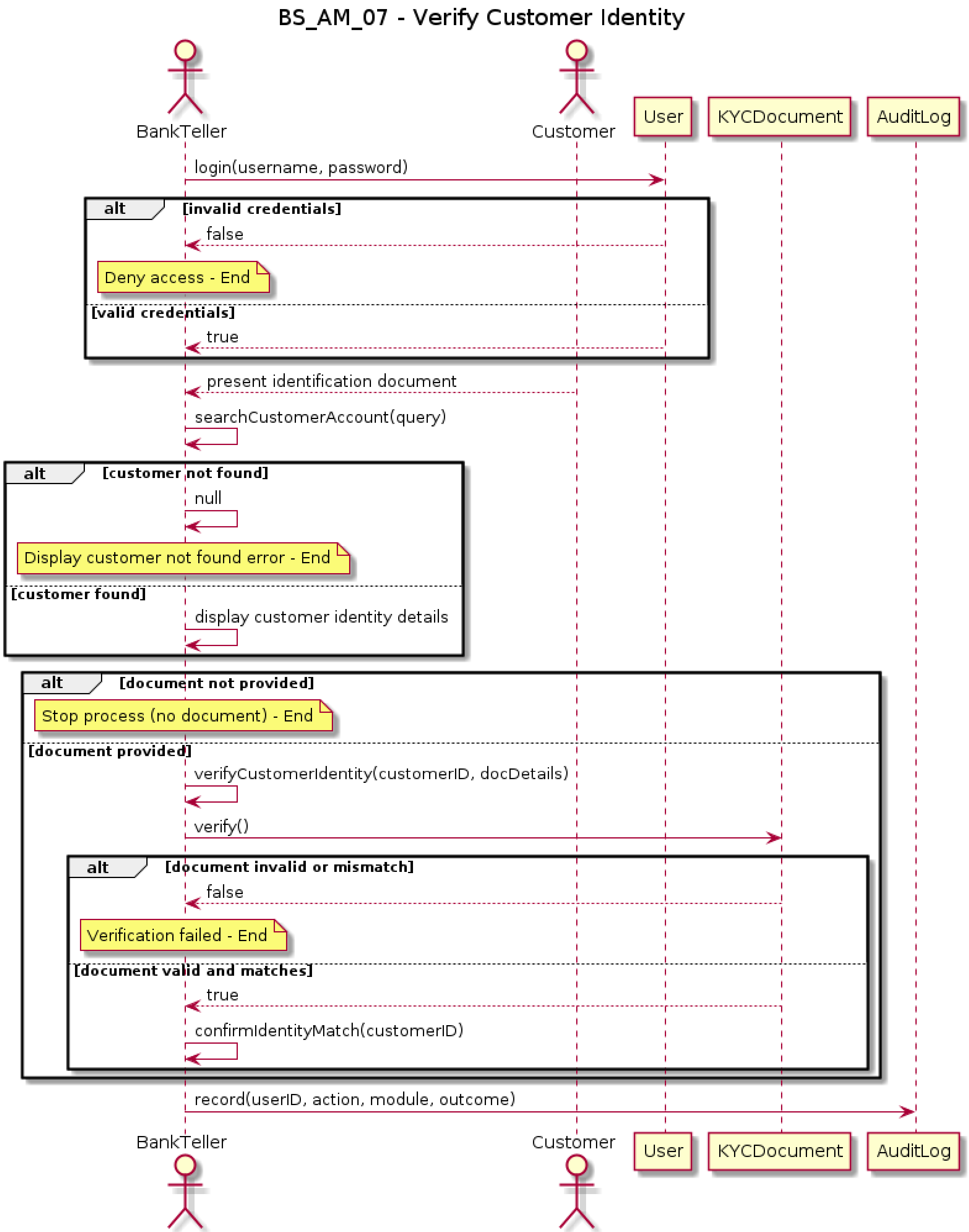
BS_AM_05


BS_AM_05 - Create and Manage Customer Accounts
BankStaff
User
KYCDocument
BankAccount
AuditLog
login(username, password)
alt
[invalid credentials]
false
Deny access - End
[valid credentials]
true
searchCustomer(query)
alt
[customer not found]
createCustomerProfile(data)
Customer (new)
[customer found]
Customer
verify()
verifyKYC(customerID)
alt
[KYC fails]
false
Stop process - End
[KYC verified]
true
createAccount(customerID, type, data)
validateData(data)
alt
[data invalid]
false
Reject input and request correction - End
[data valid]
true
display success confirmation
record(userID, action, module, outcome)

BS_AM_06

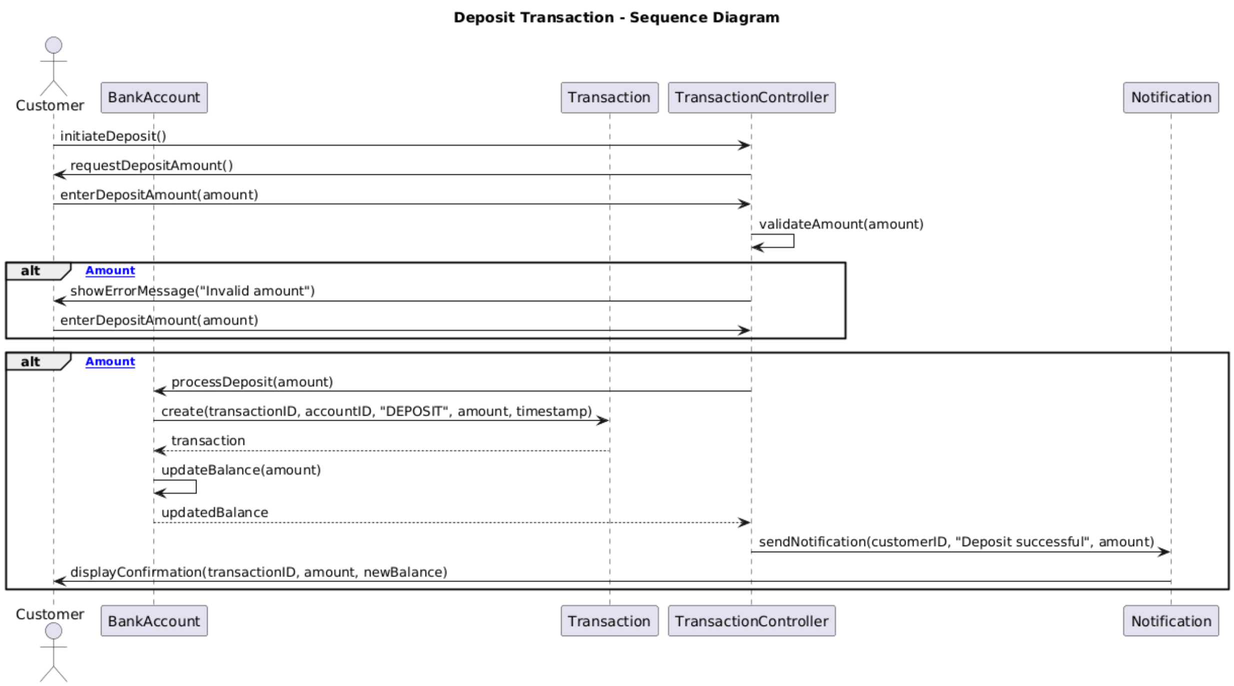

BS_AM_06 - Update KYC and Verify Identity
BankStaff
Customer
User
KYCDocument
AuditLog
login(username, password)
alt
[invalid credentials]
false
Deny access - End
[valid credentials]
true
searchCustomer(query)
alt
[customer not found]
null
Display customer not found error - End
[customer found]
Customer
provide KYC/identity documents
updateCustomerKYC(customerID, docs)
loop
[until documents valid and complete]
verify()
alt
[invalid or incomplete]
false
prompt correction or request missing documents
updated documents
updateCustomerKYC(customerID, docs)
[valid and complete]
true
verifyKYC(customerID)
save updated KYC information
confirm successful update
record(userID, action, module, outcome)
BankStaff
Customer
User
KYCDocument
AuditLog

BS_AM_07

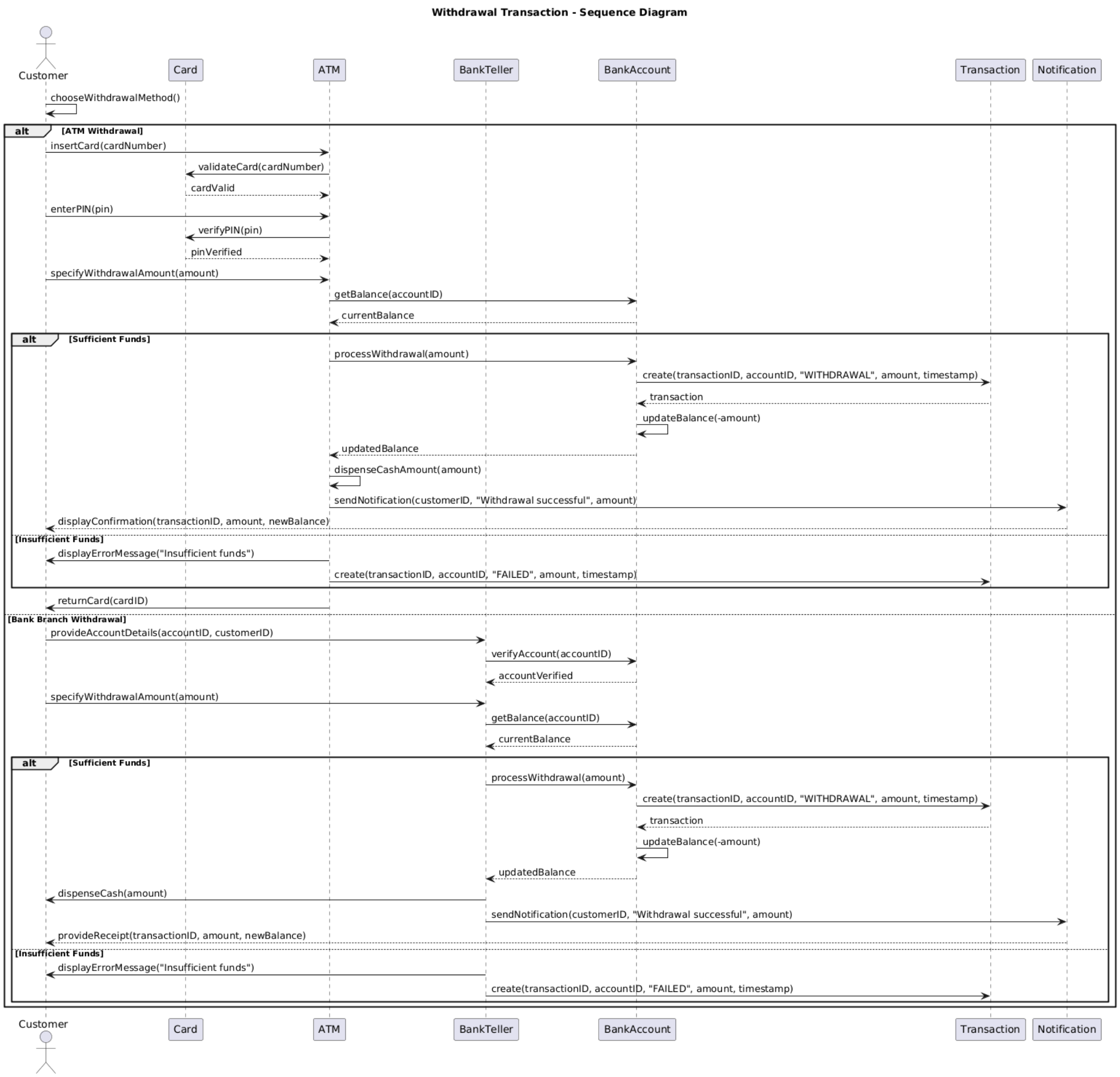

BS_AM_07 - Verify Customer Identity
BankTeller
Customer
User
KYCDocument
AuditLog
login(username, password)
alt
[invalid credentials]
false
Deny access - End
[valid credentials]
true
present identification document
searchCustomerAccount(query)
alt
[customer not found]
null
Display customer not found error - End
[customer found]
display customer identity details
alt
[document not provided]
Stop process (no document) - End
[document provided]
verifyCustomerIdentity(customerID, docDetails)
verify()
alt
[document invalid or mismatch]
false
Verification failed - End
[document valid and matches]
true
confirmIdentityMatch(customerID)
record(userID, action, module, outcome)
BankTeller
Customer
User
KYCDocument
AuditLog

Adriano Demiri

BS_TM_01

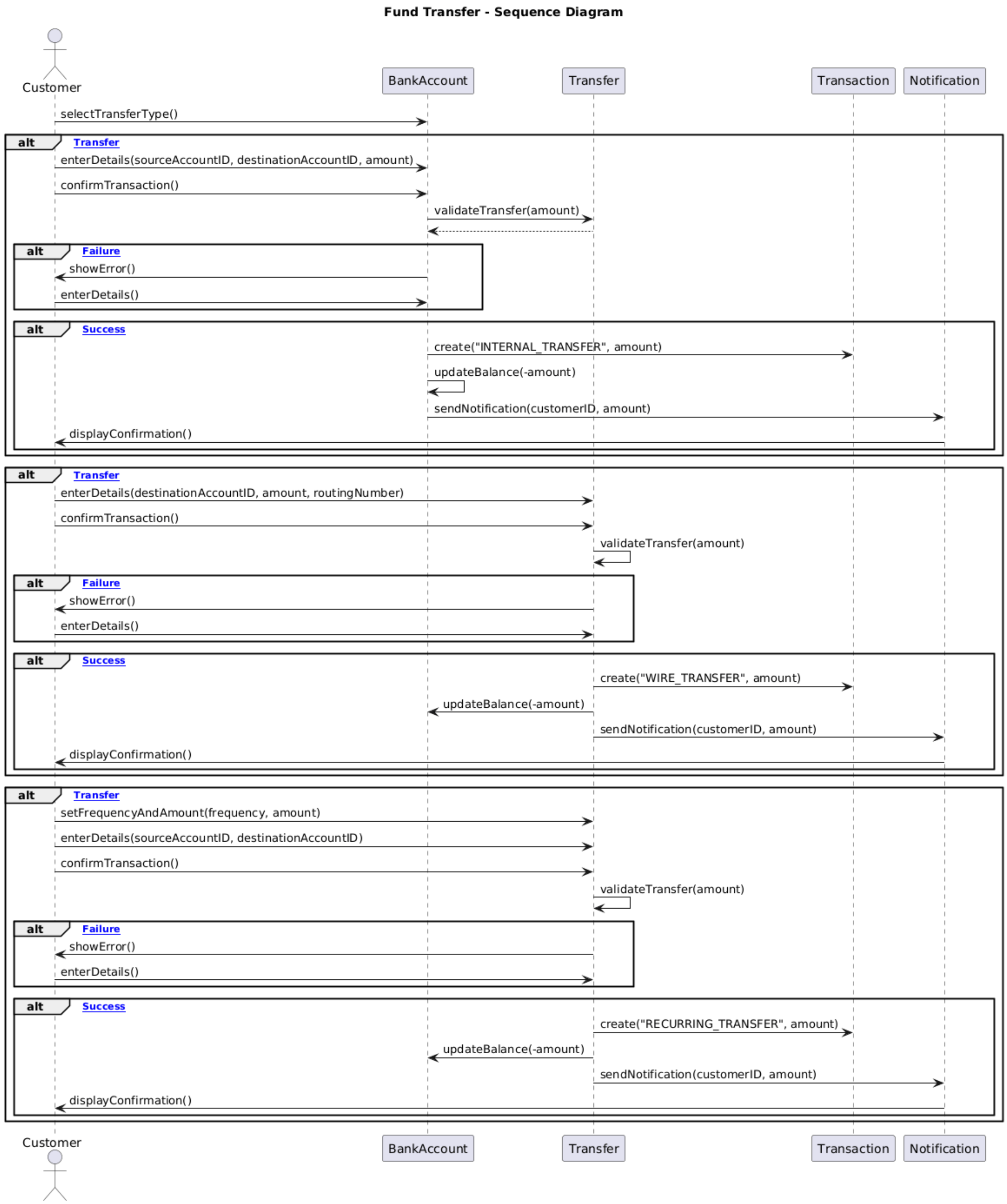


BS_TM_02

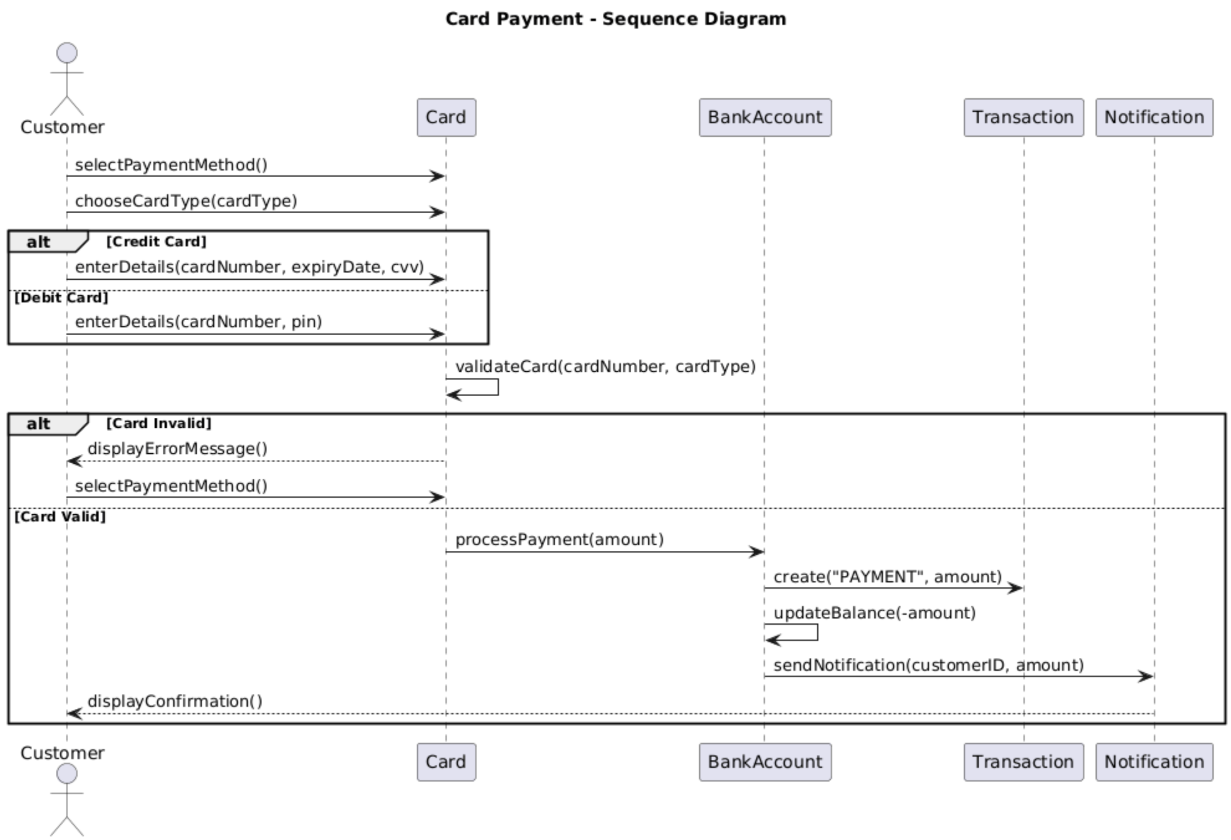


Withdrawal Transaction - Sequence Diagram
Customer
Card
ATM
BankTeller
BankAccount
Transaction
Notification
chooseWithdrawalMethod()
alt [ATM Withdrawal]
insertCard(cardNumber)
validateCard(cardNumber)
cardValid
enterPIN(pin)
verifyPIN(pin)
pinVerified
specifyWithdrawalAmount(amount)
getBalance(accountID)
currentBalance
alt [Sufficient Funds]
processWithdrawal(amount)
create(transactionID, accountID, "WITHDRAWAL", amount, timestamp)
transaction
updateBalance(-amount)
updatedBalance
dispenseCashAmount(amount)
sendNotification(customerID, "Withdrawal successful", amount)
displayConfirmation(transactionID, amount, newBalance)
[Insufficient Funds]
displayErrorMessage("Insufficient funds")
create(transactionID, accountID, "FAILED", amount, timestamp)
returnCard(cardID)
[Bank Branch Withdrawal]
provideAccountDetails(accountID, customerID)
verifyAccount(accountID)
accountVerified
specifyWithdrawalAmount(amount)
getBalance(accountID)
currentBalance
alt [Sufficient Funds]
processWithdrawal(amount)
create(transactionID, accountID, "WITHDRAWAL", amount, timestamp)
transaction
updateBalance(-amount)
updatedBalance
dispenseCash(amount)
sendNotification(customerID, "Withdrawal successful", amount)
provideReceipt(transactionID, amount, newBalance)
[Insufficient Funds]
displayErrorMessage("Insufficient funds")
create(transactionID, accountID, "FAILED", amount, timestamp)

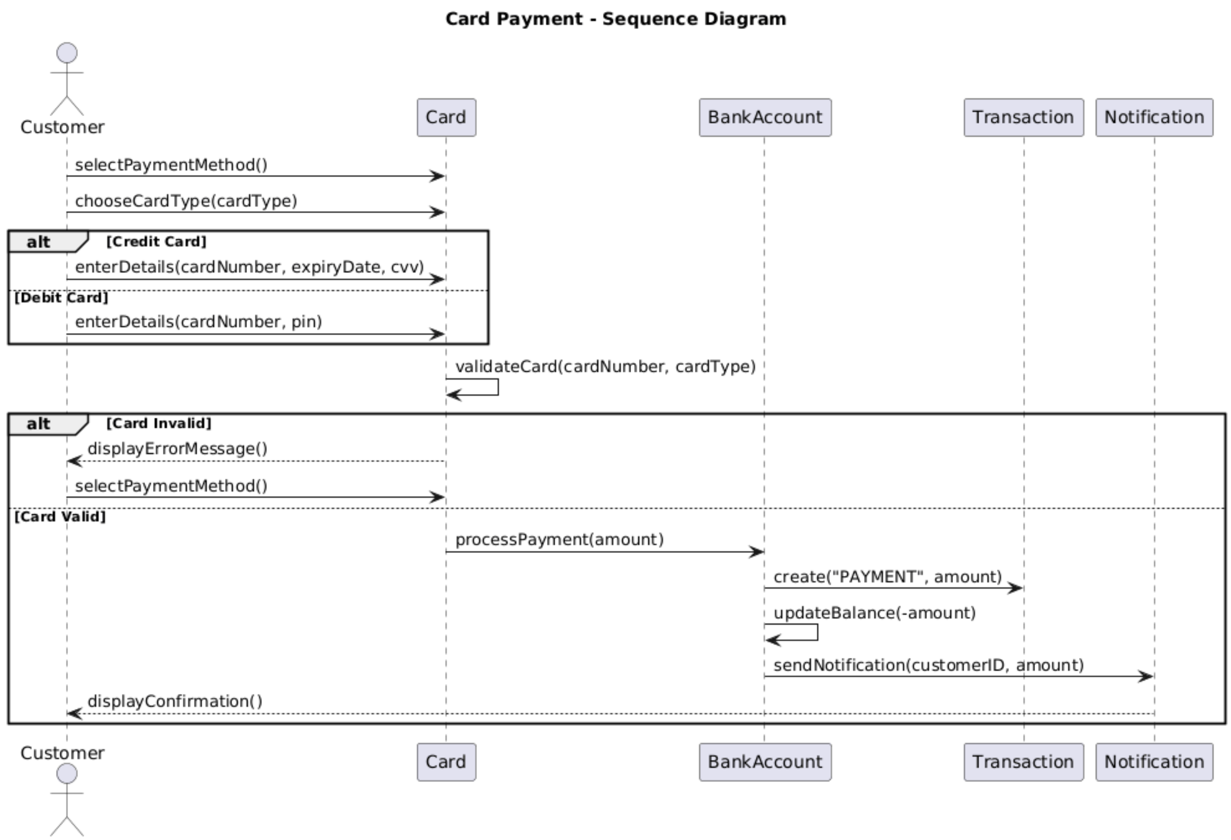
BS_TM_03


Fund Transfer - Sequence Diagram
Customer
BankAccount
Transfer
Transaction
Notification
selectTransferType()
alt
Transfer
enterDetails(sourceAccountID, destinationAccountID, amount)
confirmTransaction()
validateTransfer(amount)
alt
Failure
showError()
enterDetails()
alt
Success
create("INTERNAL_TRANSFER", amount)
updateBalance(-amount)
sendNotification(customerID, amount)
displayConfirmation()
alt
Transfer
enterDetails(destinationAccountID, amount, routingNumber)
confirmTransaction()
validateTransfer(amount)
alt
Failure
showError()
enterDetails()
alt
Success
create("WIRE_TRANSFER", amount)
updateBalance(-amount)
sendNotification(customerID, amount)
displayConfirmation()
alt
Transfer
setFrequencyAndAmount(frequency, amount)
enterDetails(sourceAccountID, destinationAccountID)
confirmTransaction()
validateTransfer(amount)
alt
Failure
showError()
enterDetails()
alt
Success
create("RECURRING_TRANSFER", amount)
updateBalance(-amount)
sendNotification(customerID, amount)
displayConfirmation()
Customer
BankAccount
Transfer
Transaction
Notification

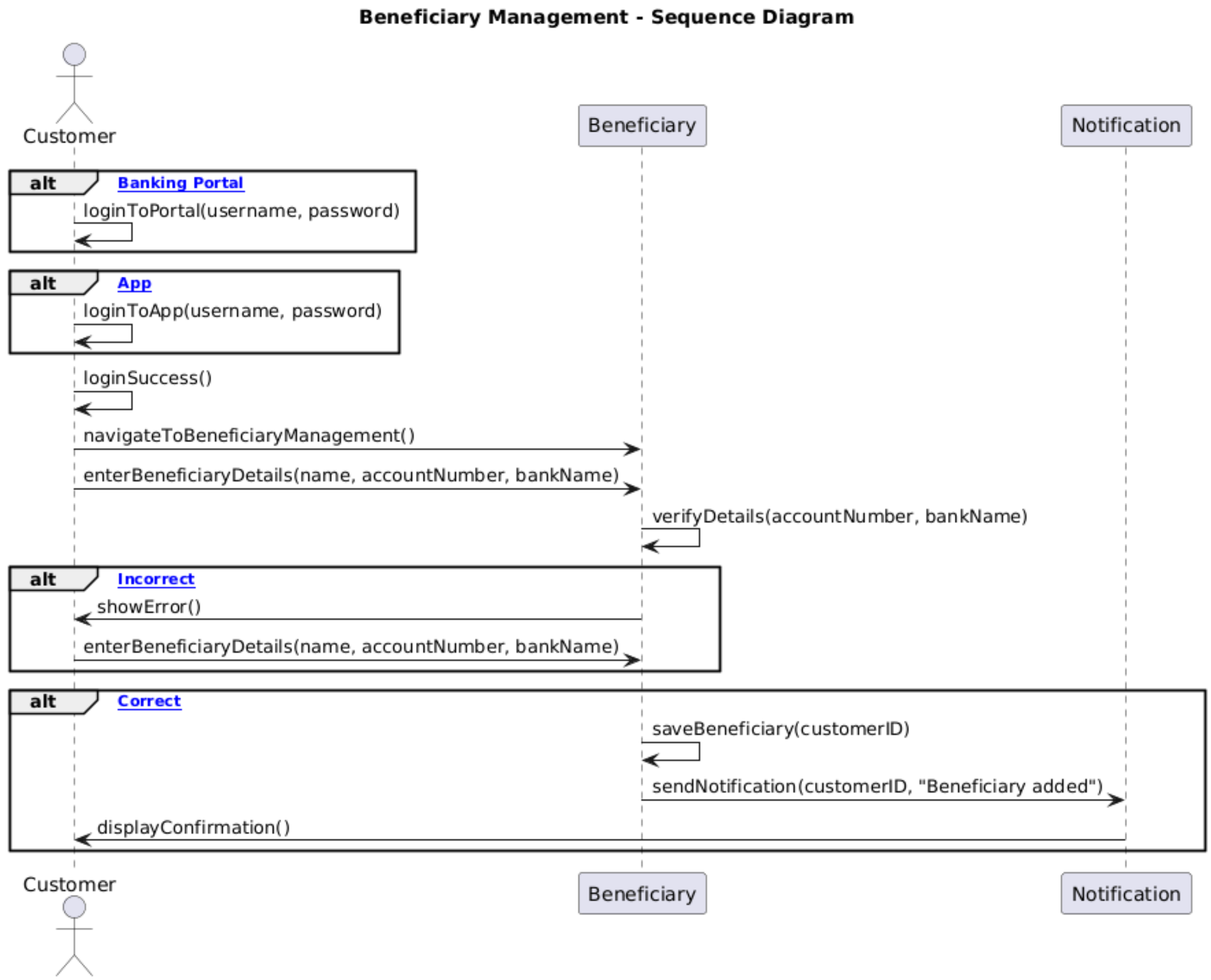
BS_TM_04


Card Payment - Sequence Diagram
Customer
Card
BankAccount
Transaction
Notification
selectPaymentMethod()
chooseCardType(cardType)
alt
[Credit Card]
enterDetails(cardNumber, expiryDate, cvv)
[Debit Card]
enterDetails(cardNumber, pin)
validateCard(cardNumber, cardType)
alt
[Card Invalid]
displayErrorMessage()
selectPaymentMethod()
[Card Valid]
processPayment(amount)
create("PAYMENT", amount)
updateBalance(-amount)
sendNotification(customerID, amount)
displayConfirmation()
Customer
Card
BankAccount
Transaction
Notification

BS_TM_05

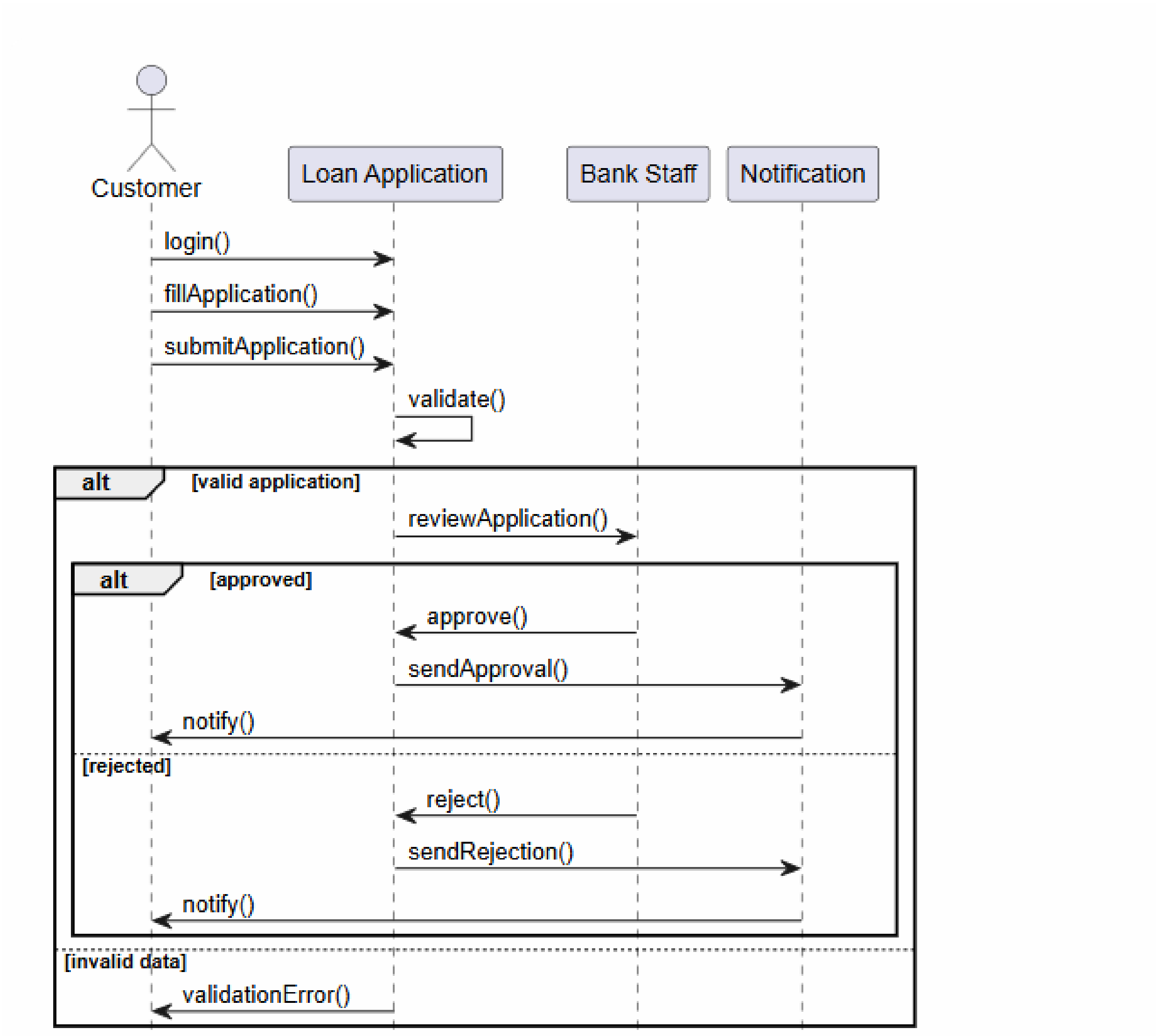

Card Payment - Sequence Diagram
Customer
Card
BankAccount
Transaction
Notification
selectPaymentMethod()
chooseCardType(cardType)
alt
[Credit Card]
enterDetails(cardNumber, expiryDate, cvv)
[Debit Card]
enterDetails(cardNumber, pin)
validateCard(cardNumber, cardType)
alt
[Card Invalid]
displayErrorMessage()
selectPaymentMethod()
[Card Valid]
processPayment(amount)
create("PAYMENT", amount)
updateBalance(-amount)
sendNotification(customerID, amount)
displayConfirmation()
Customer
Card
BankAccount
Transaction
Notification

BS_TM_06

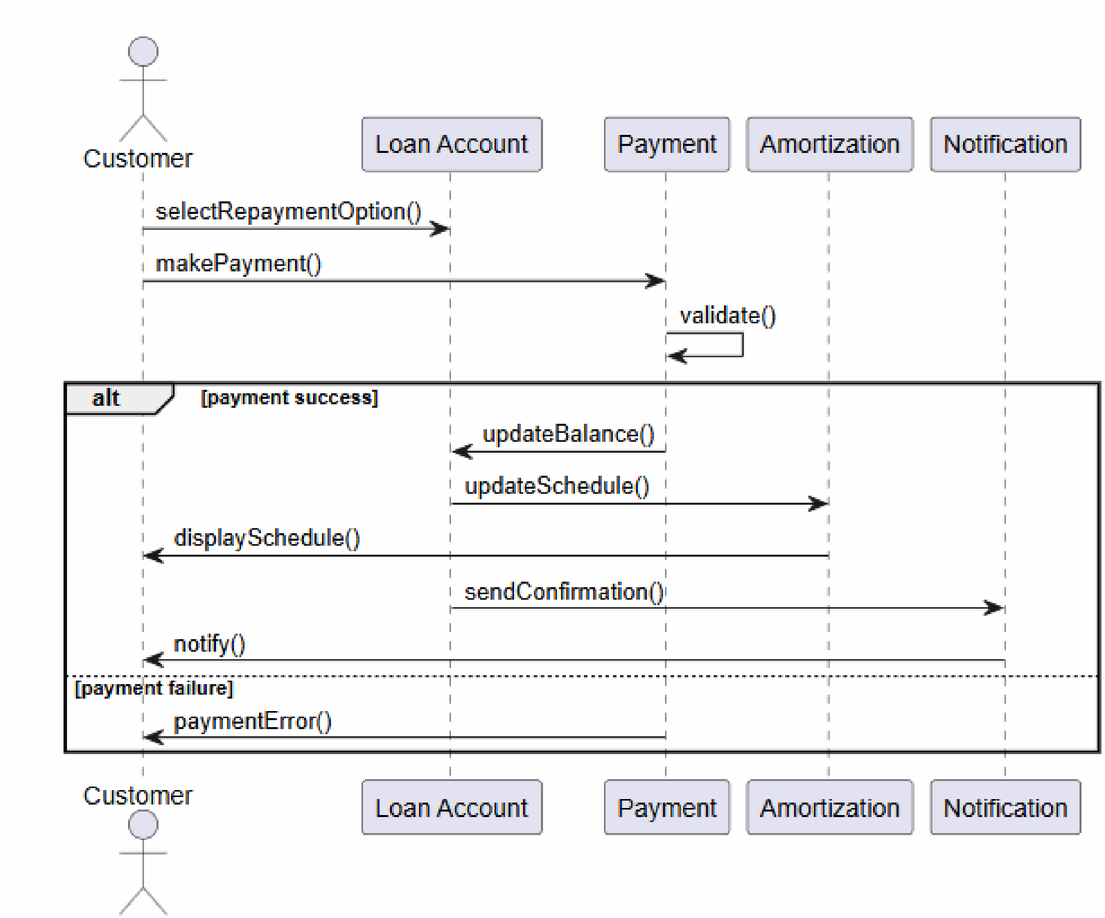

Beneficiary Management - Sequence Diagram
Customer
Beneficiary
Notification
alt
Banking Portal
loginToPortal(username, password)
alt
App
loginToApp(username, password)
loginSuccess()
navigateToBeneficiaryManagement()
enterBeneficiaryDetails(name, accountNumber, bankName)
verifyDetails(accountNumber, bankName)
alt
Incorrect
showError()
enterBeneficiaryDetails(name, accountNumber, bankName)
alt
Correct
saveBeneficiary(customerID)
sendNotification(customerID, "Beneficiary added")
displayConfirmation()
Customer
Beneficiary
Notification

Emauele Berbiu

BS_LM_01

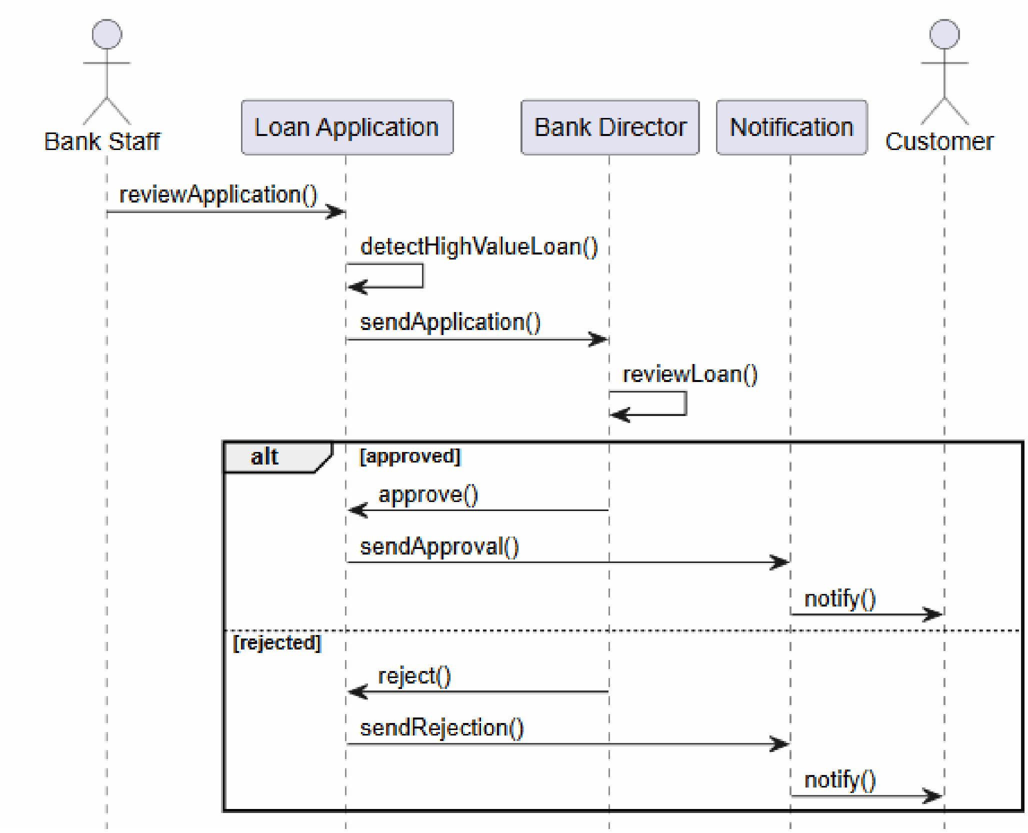


BS_LM_02

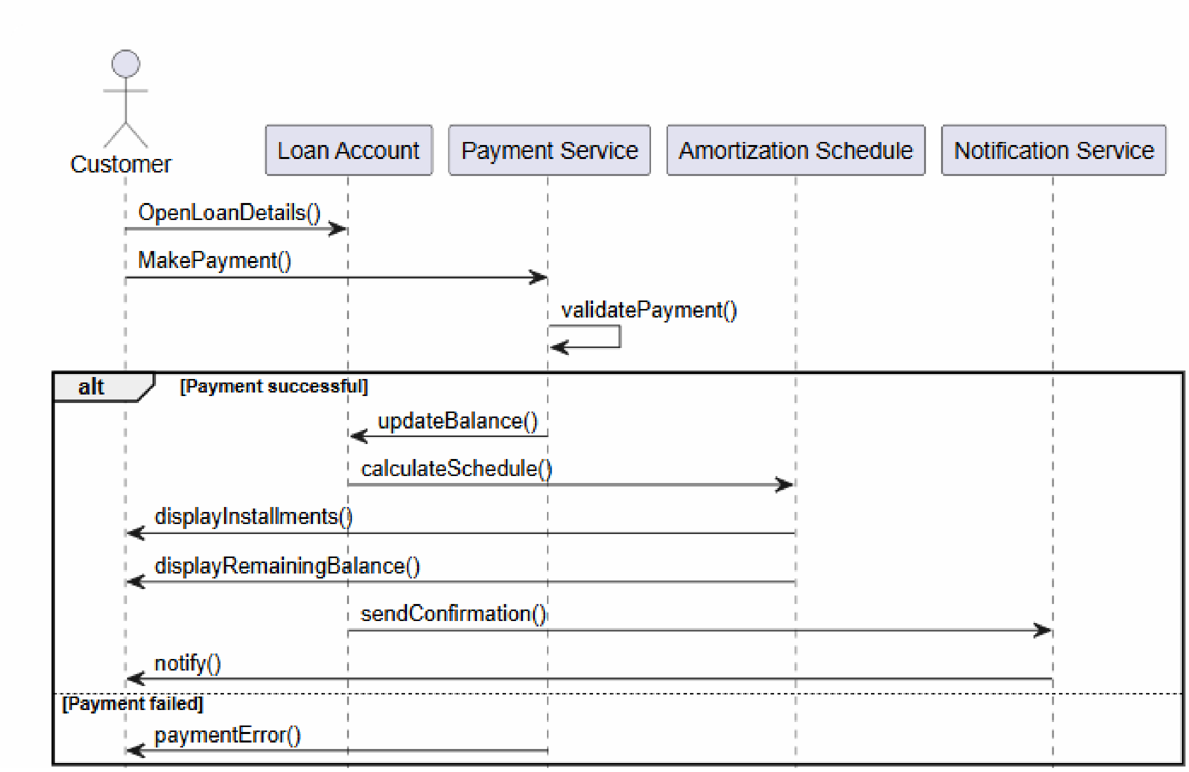


Customer
Loan Account
Payment
Amortization
Notification
selectRepaymentOption()
makePayment()
validate()
alt
[payment success]
updateBalance()
updateSchedule()
displaySchedule()
sendConfirmation()
notify()
[payment failure]
paymentError()
Customer
Loan Account
Payment
Amortization
Notification

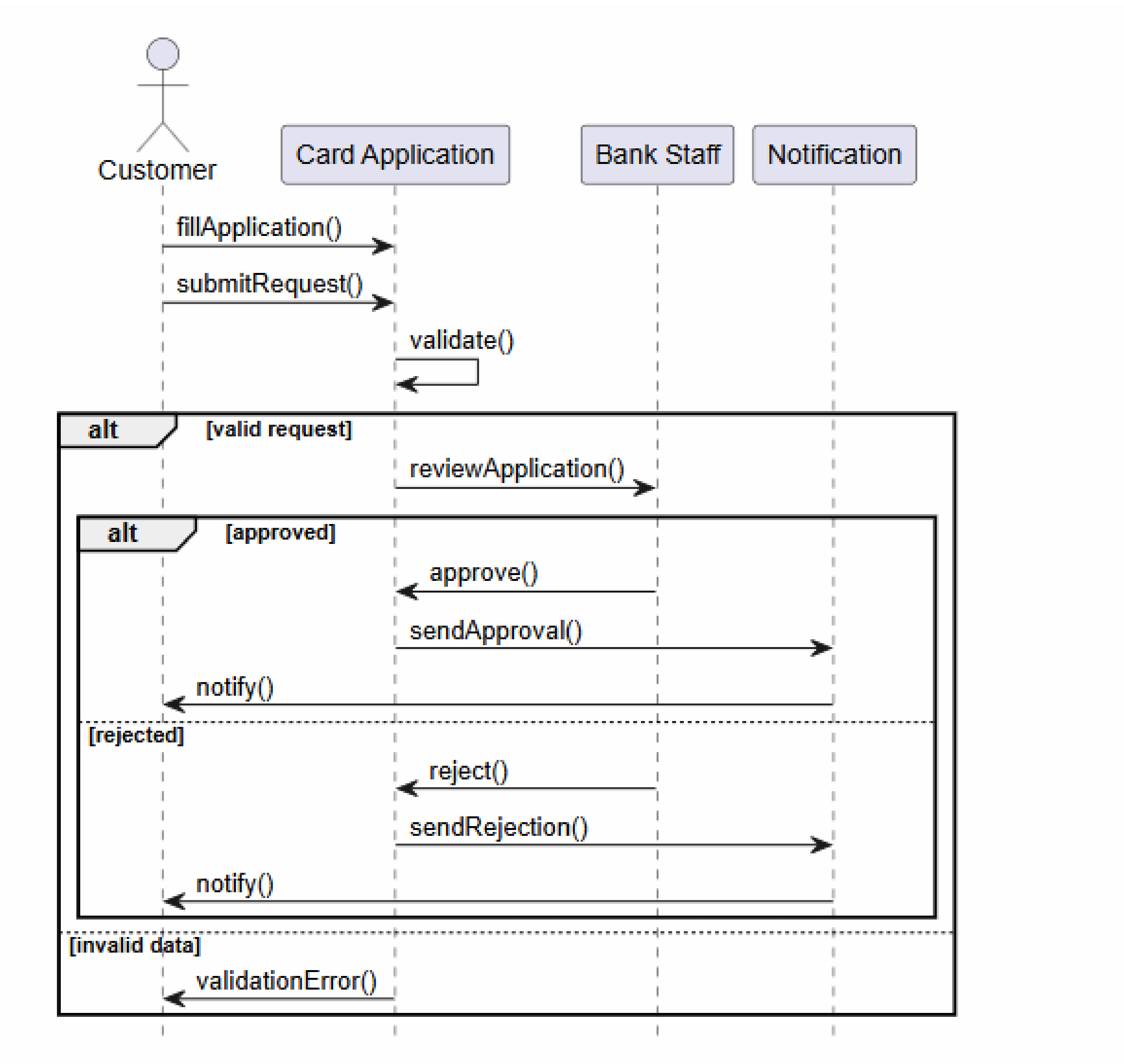
BS_LM_03


Bank Staff
Loan Application
Bank Director
Notification
Customer
reviewApplication()
detectHighValueLoan()
sendApplication()
reviewLoan()
alt
[approved]
approve()
sendApproval()
notify()
[rejected]
reject()
sendRejection()
notify()

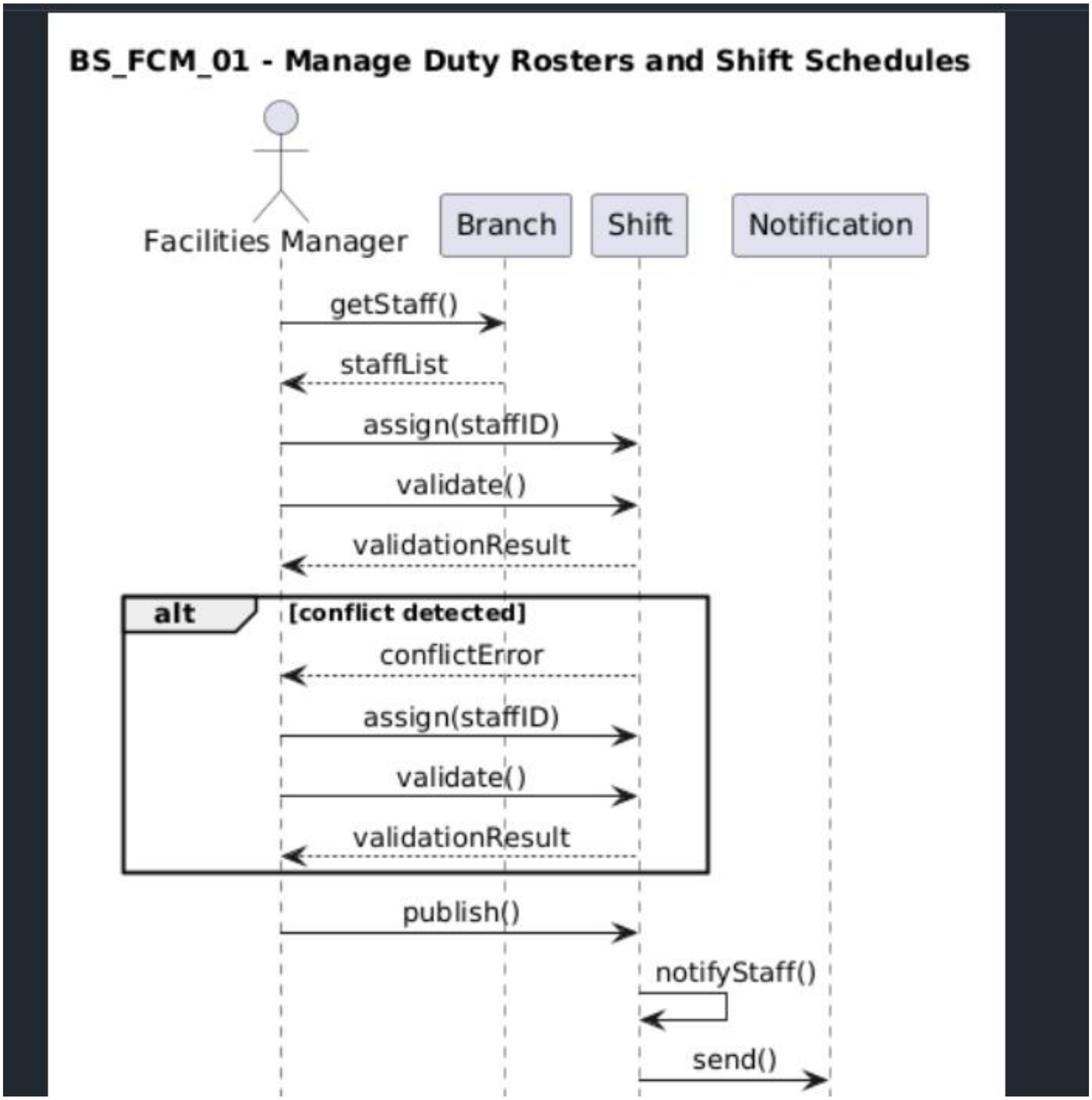
BS_LM_04


Customer
Loan Account
Payment Service
Amortization Schedule
Notification Service
OpenLoanDetails()
MakePayment()
validatePayment()
alt
[Payment successful]
updateBalance()
calculateSchedule()
displayInstallments()
displayRemainingBalance()
sendConfirmation()
notify()
[Payment failed]
paymentError()

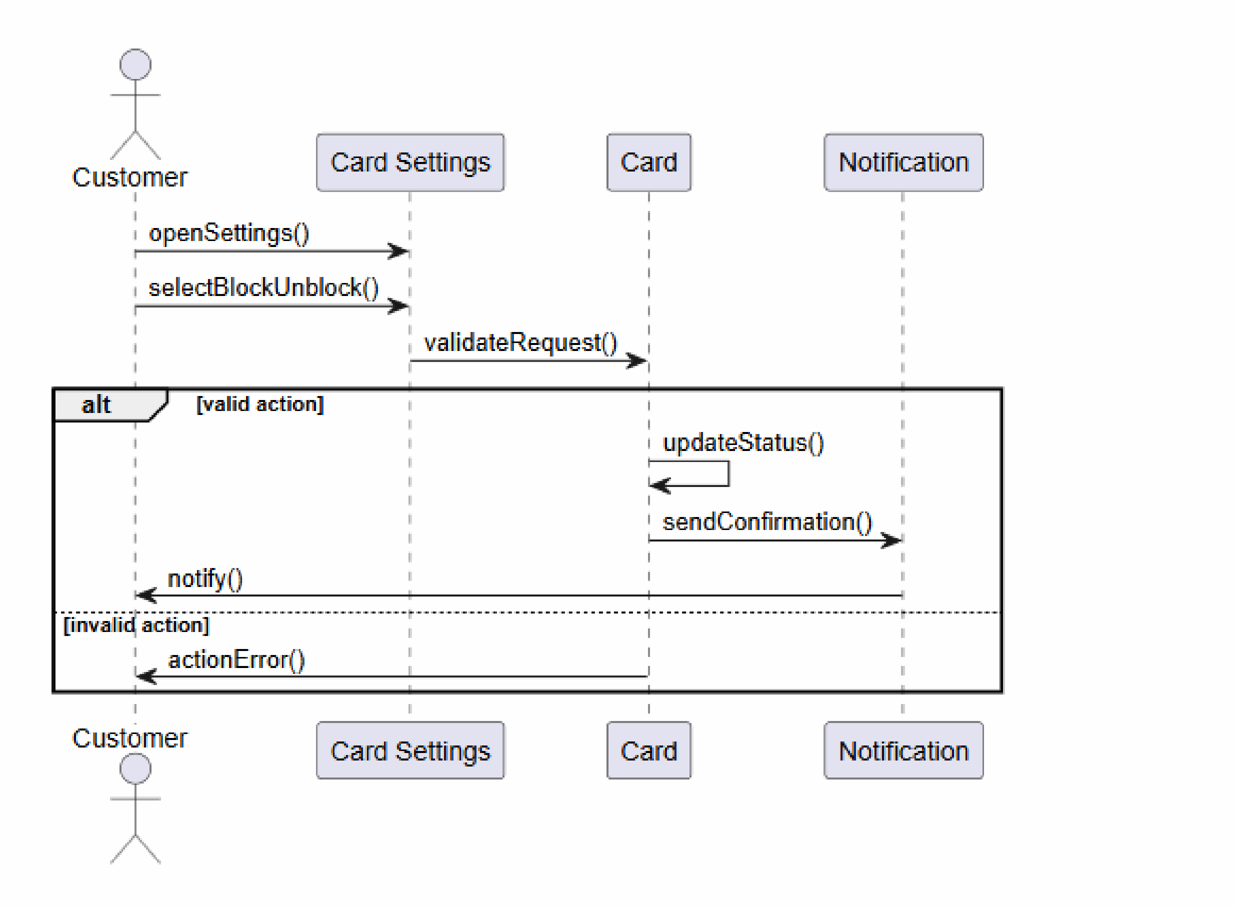
BS_CM_01


Customer
Card Application
Bank Staff
Notification
fillApplication()
submitRequest()
validate()
alt
[valid request]
reviewApplication()
alt
[approved]
approve()
sendApproval()
notify()
[rejected]
reject()
sendRejection()
notify()
[invalid data]
validationError()

BS_CM_02

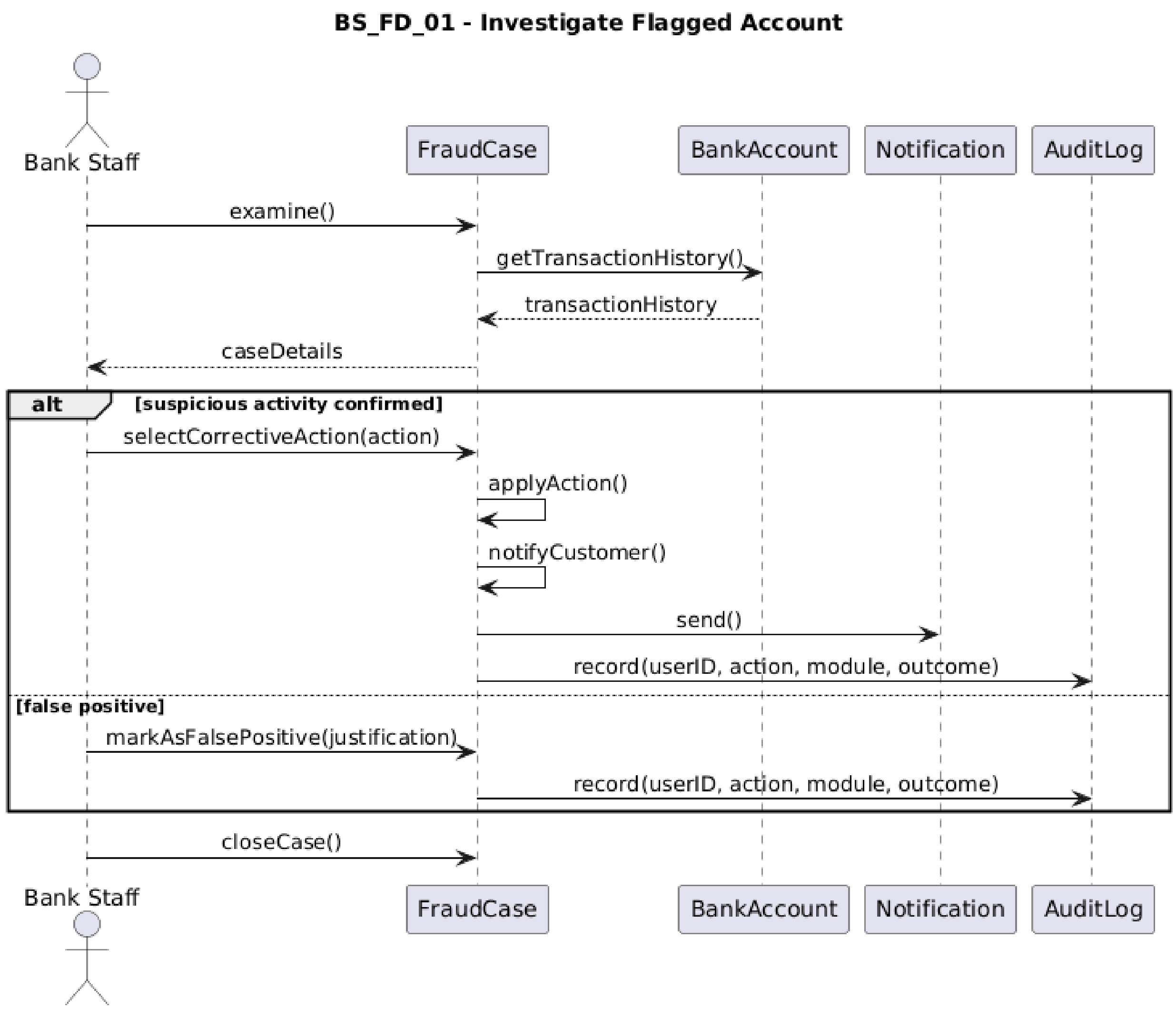

BS_FCM_01 - Manage Duty Rosters and Shift Schedules
Facilities Manager
Branch
Shift
Notification
getStaff()
staffList
assign(staffID)
validate()
validationResult
alt
[conflict detected]
conflictError
assign(staffID)
validate()
validationResult
publish()
notifyStaff()
send()

BS_CM_03

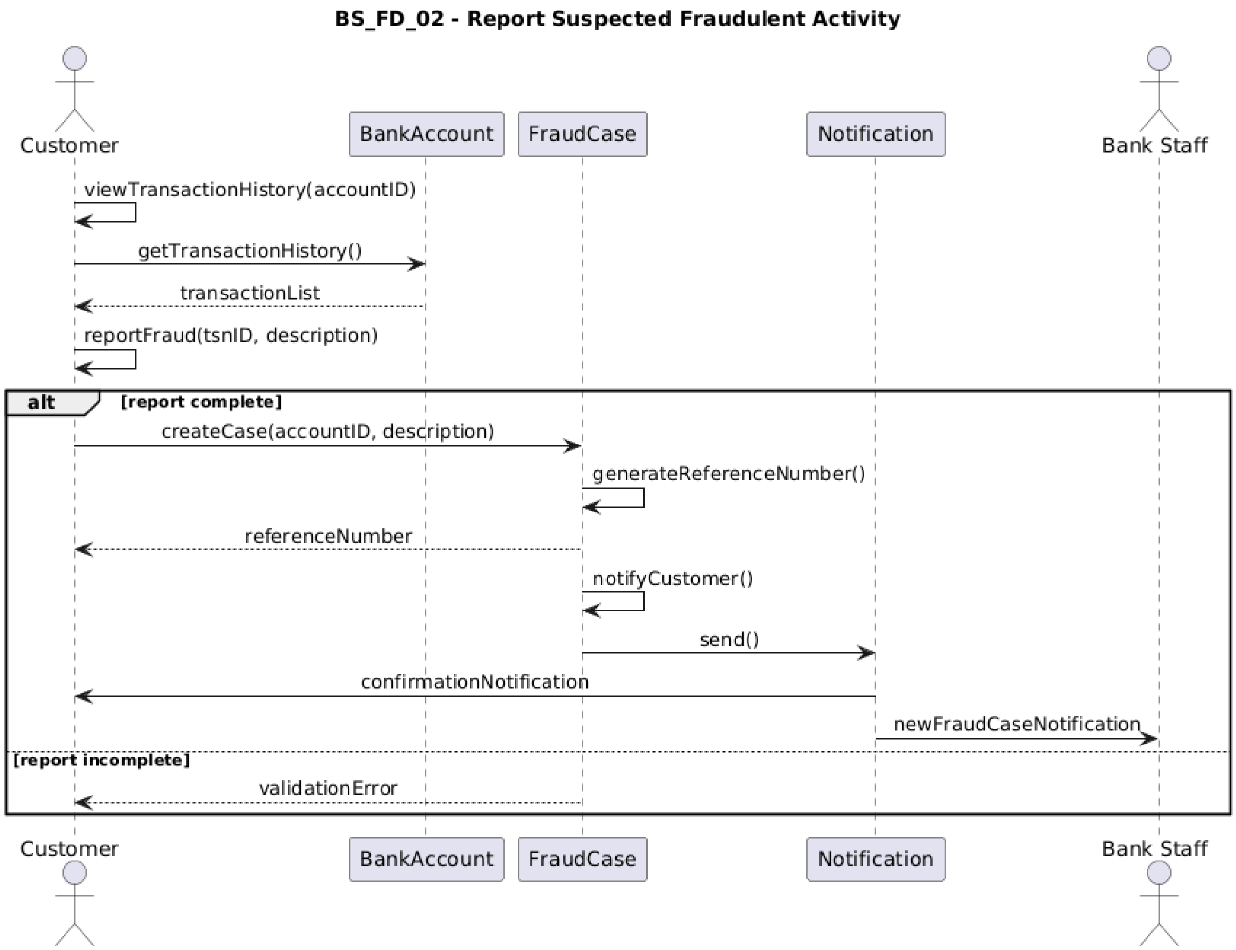

Customer
Card Settings
Card
Notification
openSettings()
selectBlockUnblock()
validateRequest()
alt
[valid action]
updateStatus()
sendConfirmation()
notify()
[invalid action]
actionError()
Customer
Card Settings
Card
Notification

Joandri Domi

BS_FD_01

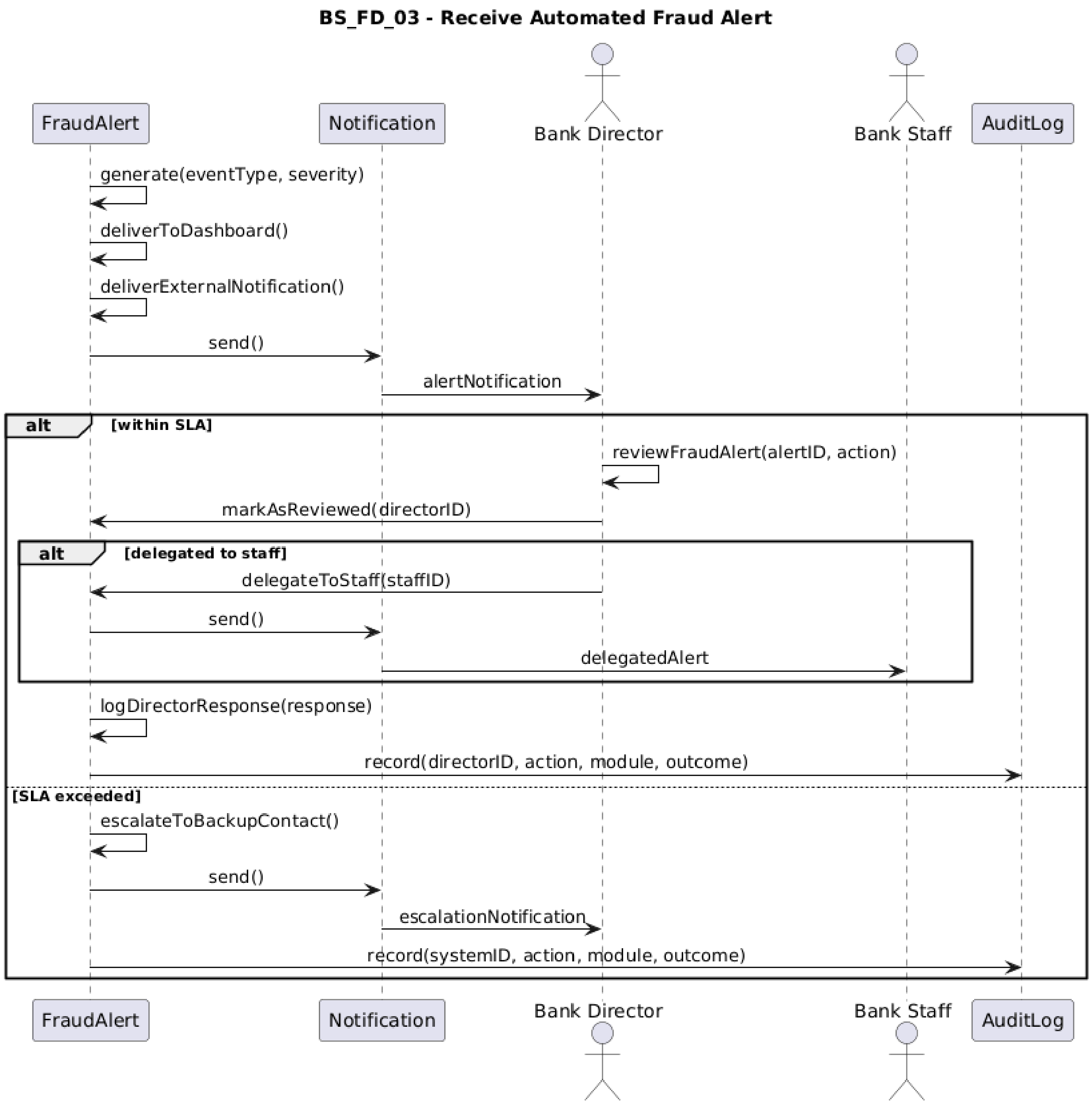


BS_FD_02

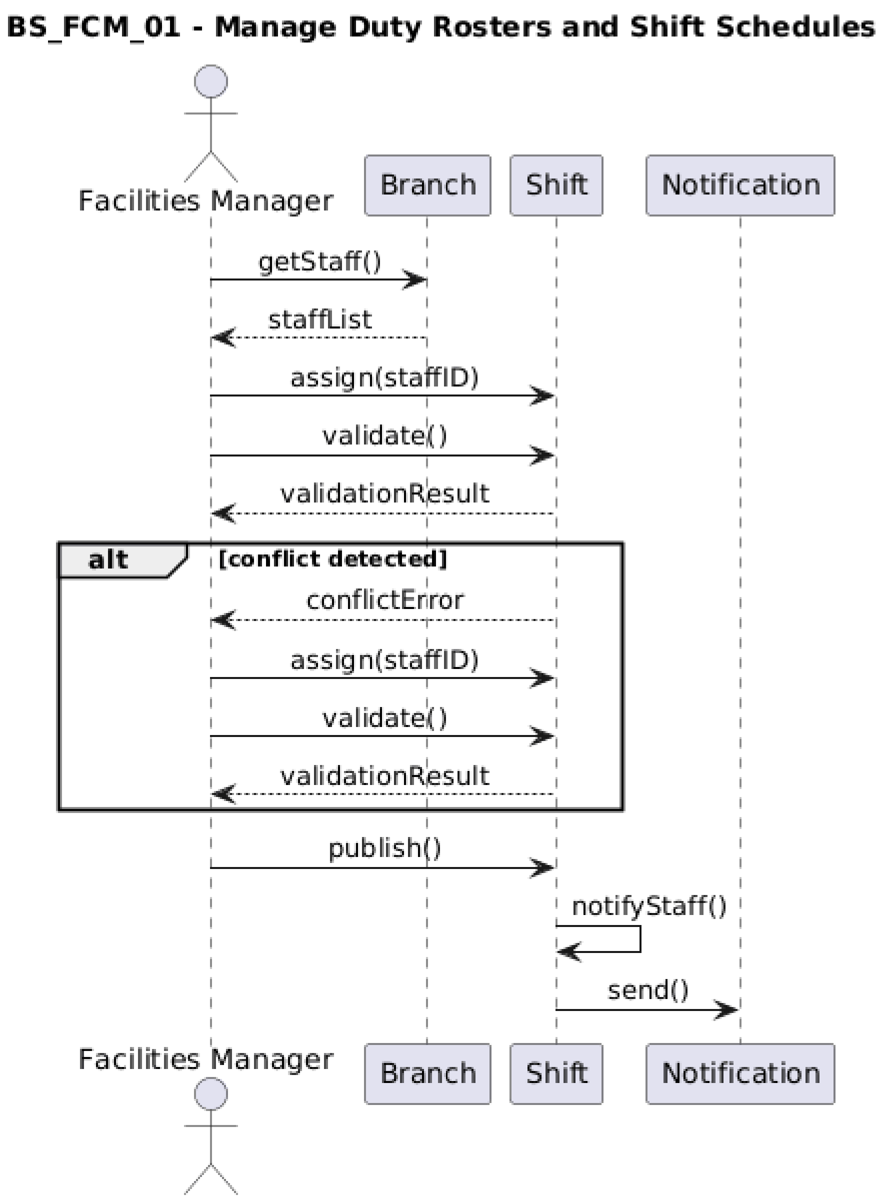


BS_FD_02 - Report Suspected Fraudulent Activity
Customer
BankAccount
FraudCase
Notification
Bank Staff
viewTransactionHistory(accountID)
getTransactionHistory()
transactionList
reportFraud(tsnID, description)
alt
[report complete]
createCase(accountID, description)
generateReferenceNumber()
referenceNumber
notifyCustomer()
send()
confirmationNotification
newFraudCaseNotification
[report incomplete]
validationError
Customer
BankAccount
FraudCase
Notification
Bank Staff

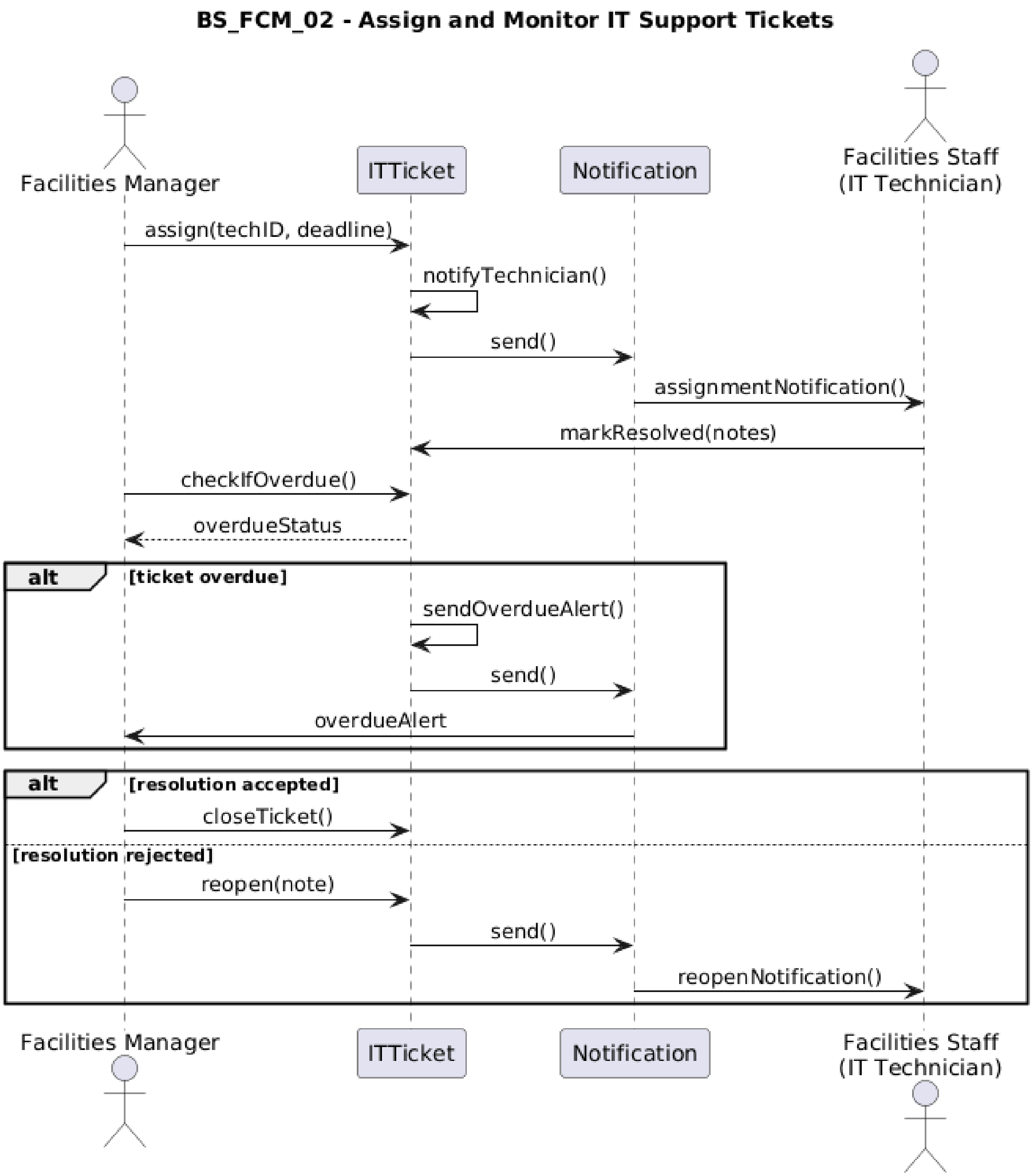
BS_FD_03


BS_FD_03 - Receive Automated Fraud Alert
FraudAlert
Notification
Bank Director
Bank Staff
AuditLog
generate(eventType, severity)
deliverToDashboard()
deliverExternalNotification()
send()
alertNotification
alt
[within SLA]
reviewFraudAlert(alertID, action)
markAsReviewed(directorID)
alt
[delegated to staff]
delegateToStaff(staffID)
send()
delegatedAlert
logDirectorResponse(response)
record(directorID, action, module, outcome)
[SLA exceeded]
escalateToBackupContact()
send()
escalationNotification
record(systemID, action, module, outcome)
FraudAlert
Notification
Bank Director
Bank Staff
AuditLog

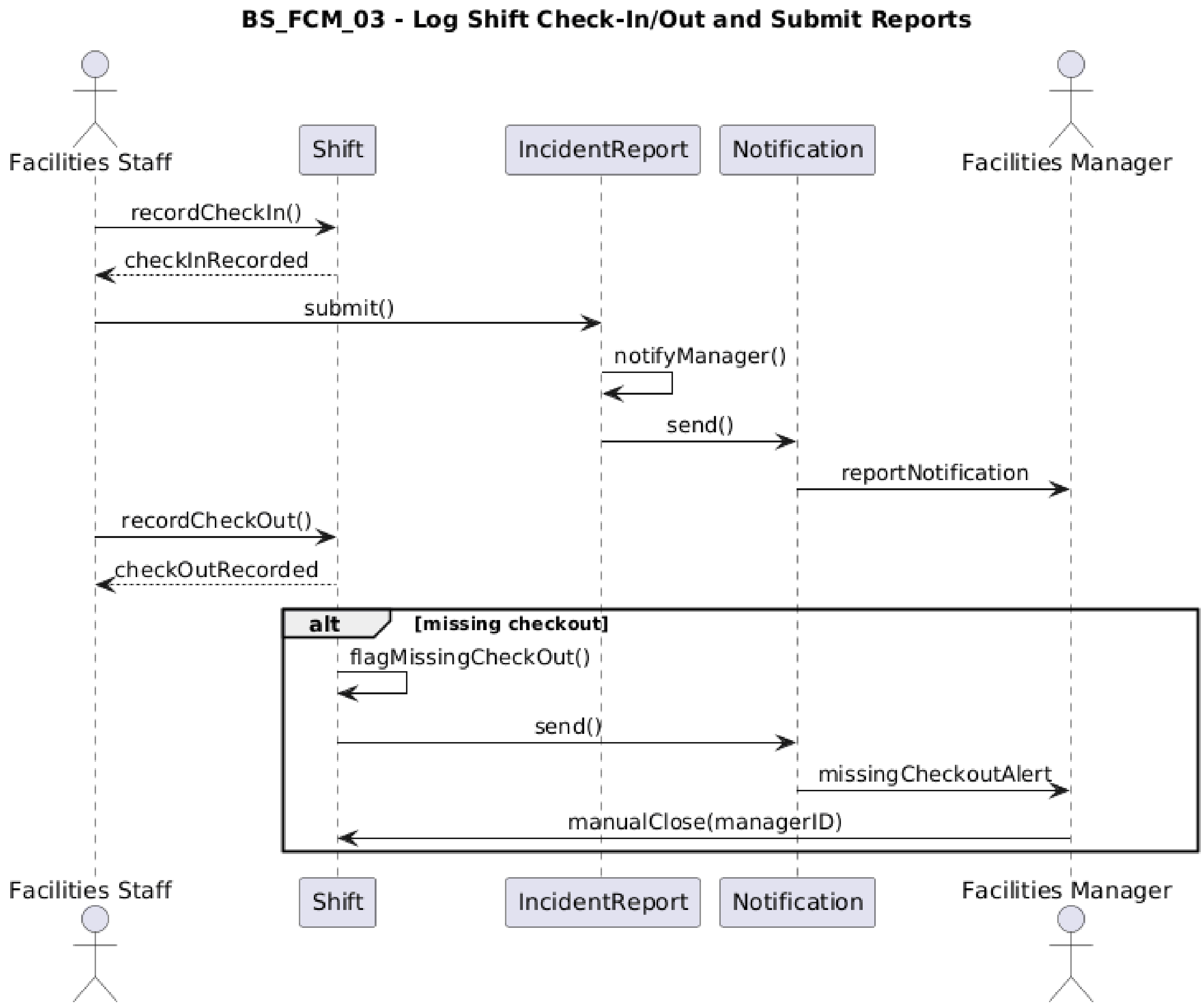
BS_FCM_01


BS_FCM_01 - Manage Duty Rosters and Shift Schedules
Facilities Manager
Branch
Shift
Notification
getStaff()
staffList
assign(staffID)
validate()
validationResult
alt
[conflict detected]
conflictError
assign(staffID)
validate()
validationResult
publish()
notifyStaff()
send()
Facilities Manager
Branch
Shift
Notification

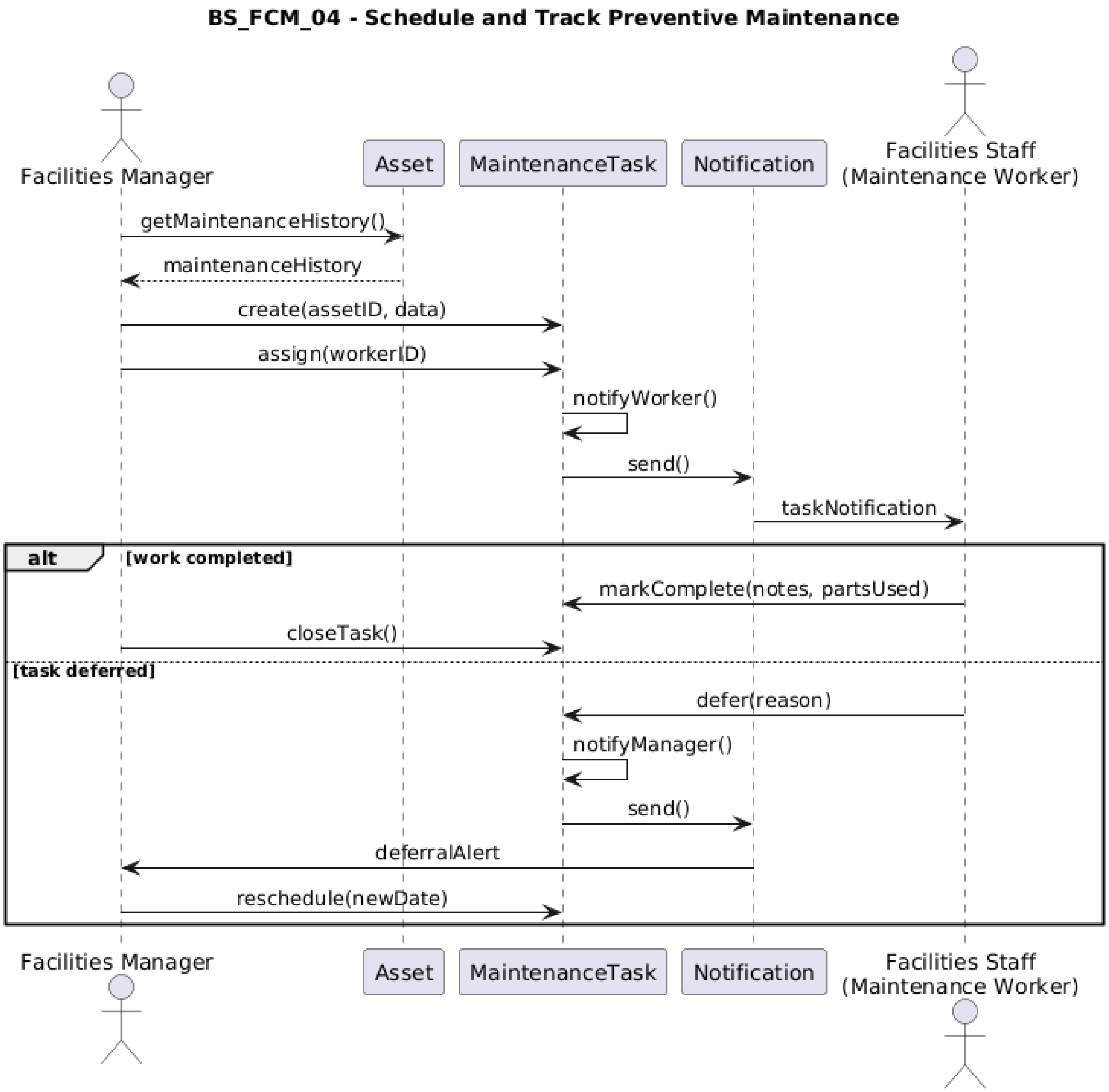
BS_FCM_02


BS_FCM_02 - Assign and Monitor IT Support Tickets
Facilities Manager
ITTicket
Notification
Facilities Staff
(IT Technician)
assign(techID, deadline)
notifyTechnician()
send()
assignmentNotification()
markResolved(notes)
checkIfOverdue()
overdueStatus
alt
[ticket overdue]
sendOverdueAlert()
send()
overdueAlert
alt
[resolution accepted]
closeTicket()
[resolution rejected]
reopen(note)
send()
reopenNotification()

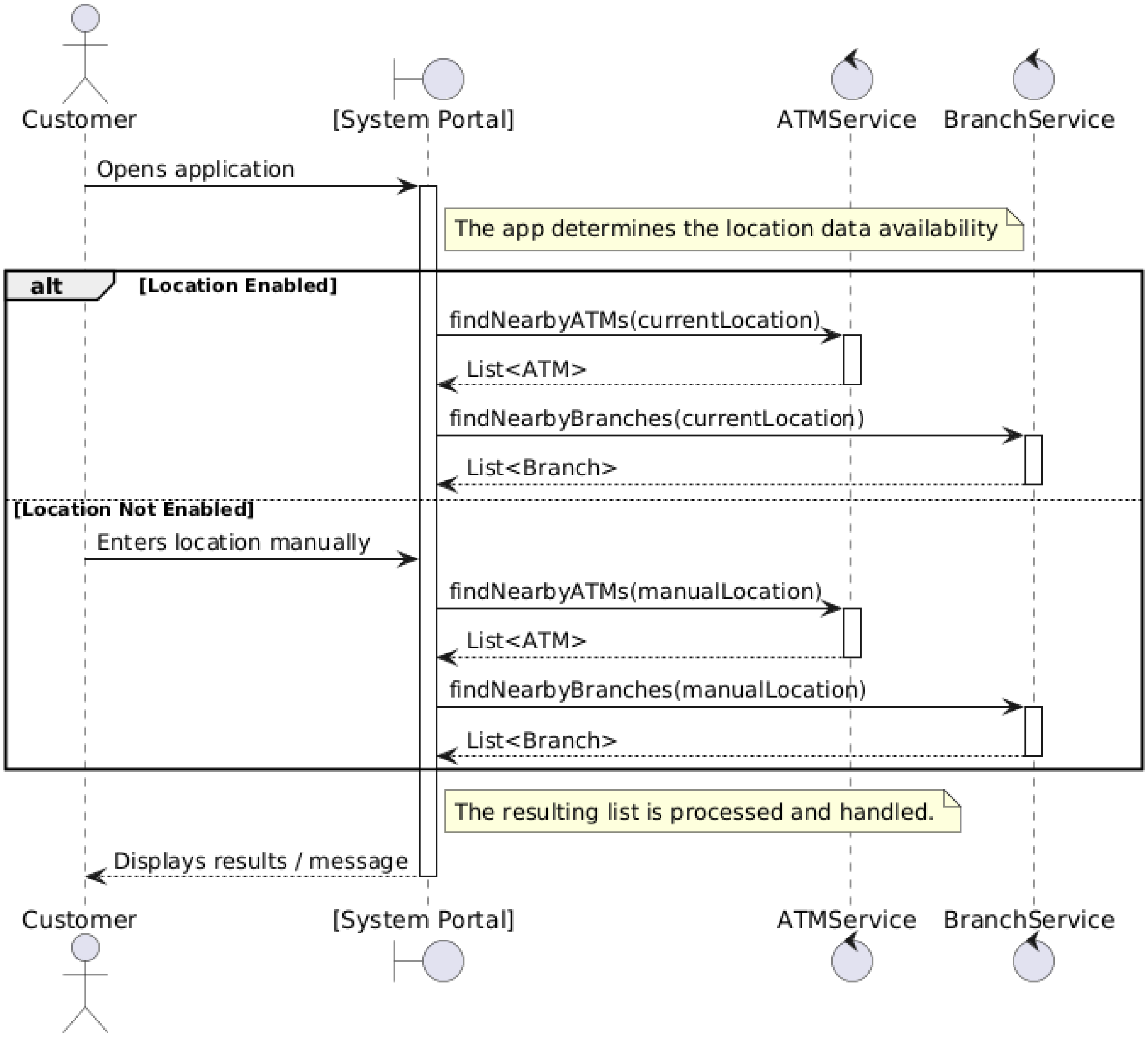
BS_FCM_03


BS_FCM_03 - Log Shift Check-In/Out and Submit Reports
Facilities Staff
Shift
IncidentReport
Notification
Facilities Manager
recordCheckIn()
checkInRecorded
submit()
notifyManager()
send()
reportNotification
recordCheckOut()
checkOutRecorded
alt
[missing checkout]
flagMissingCheckOut()
send()
missingCheckoutAlert
manualClose(managerID)
Facilities Staff
Shift
IncidentReport
Notification
Facilities Manager

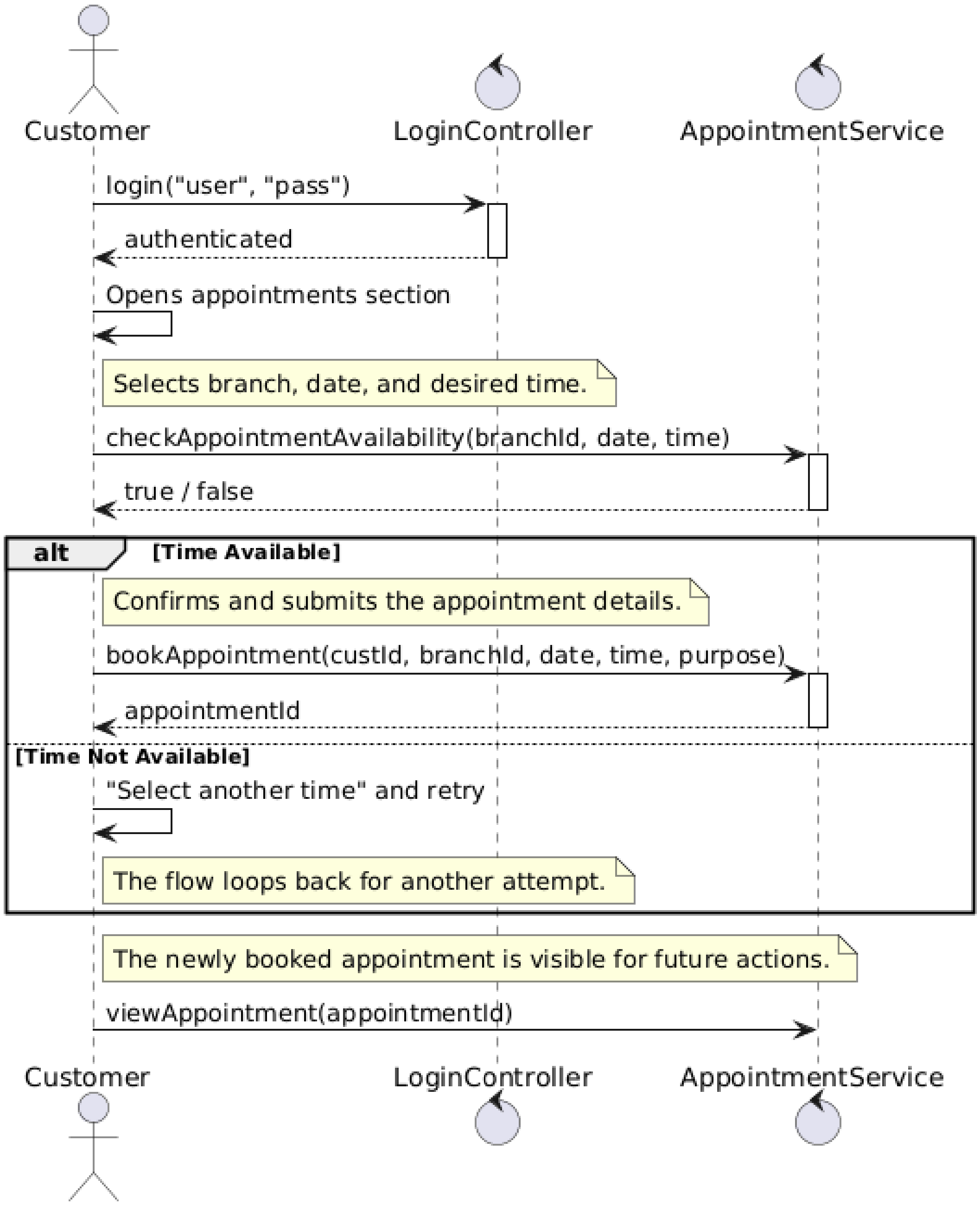
BS_FCM_04


BS_FCM_04 - Schedule and Track Preventive Maintenance
Facilities Manager
Asset
MaintenanceTask
Notification
Facilities Staff
(Maintenance Worker)
getMaintenanceHistory()
maintenanceHistory
create(assetID, data)
assign(workerID)
notifyWorker()
send()
taskNotification
alt
[work completed]
markComplete(notes, partsUsed)
closeTask()
[task deferred]
defer(reason)
notifyManager()
send()
deferralAlert
reschedule(newDate)
Facilities Manager
Asset
MaintenanceTask
Notification
Facilities Staff
(Maintenance Worker)

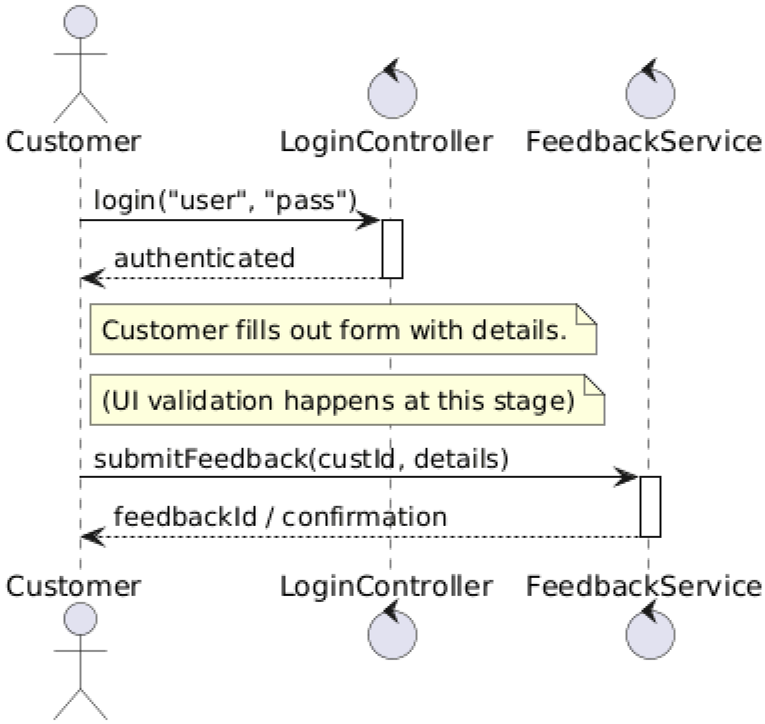
BS_CS_01


Customer
[System Portal]
ATMService
BranchService
Opens application
The app determines the location data availability
alt
[Location Enabled]
findNearbyATMs(currentLocation)
List<ATM>
findNearbyBranches(currentLocation)
List<Branch>
[Location Not Enabled]
Enters location manually
findNearbyATMs(manualLocation)
List<ATM>
findNearbyBranches(manualLocation)
List<Branch>
The resulting list is processed and handled.
Displays results / message
Customer
[System Portal]
ATMService
BranchService

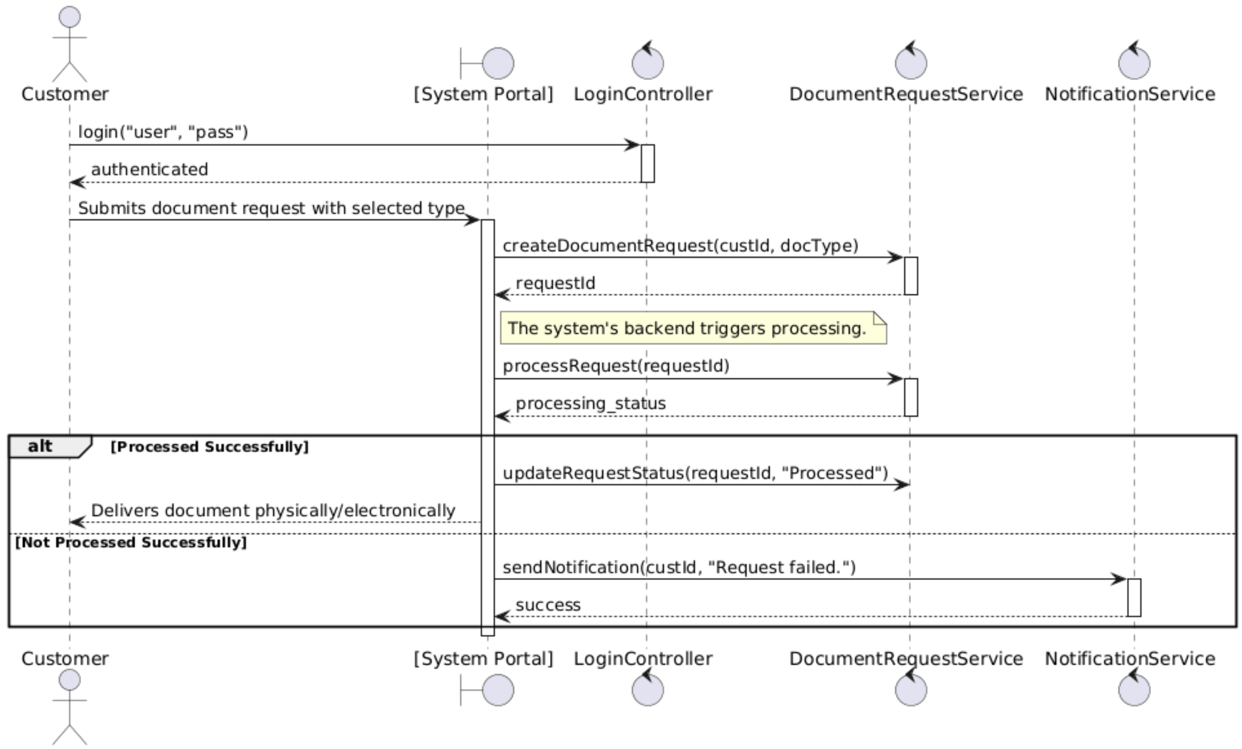
BS_CS_02


Customer
LoginController
AppointmentService
login("user", "pass")
authenticated
Opens appointments section
Selects branch, date, and desired time.
checkAppointmentAvailability(branchId, date, time)
true / false
alt
[Time Available]
Confirms and submits the appointment details.
bookAppointment(custId, branchId, date, time, purpose)
appointmentId
[Time Not Available]
"Select another time" and retry
The flow loops back for another attempt.
The newly booked appointment is visible for future actions.
viewAppointment(appointmentId)
Customer
LoginController
AppointmentService

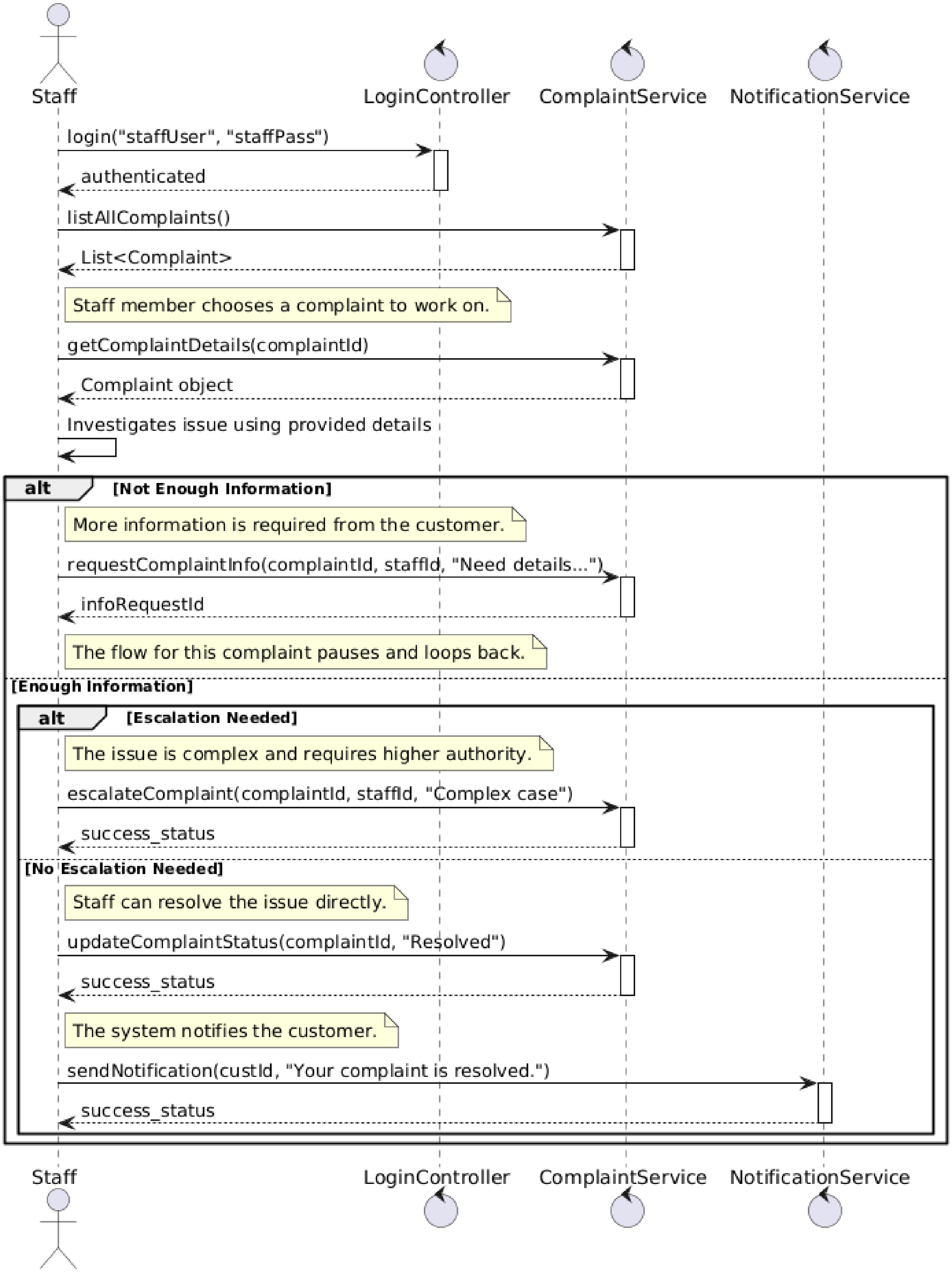
BS_CS_03


Customer
LoginController
FeedbackService
login("user", "pass")
authenticated
Customer fills out form with details.
(UI validation happens at this stage)
submitFeedback(custId, details)
feedbackId / confirmation
Customer
LoginController
FeedbackService

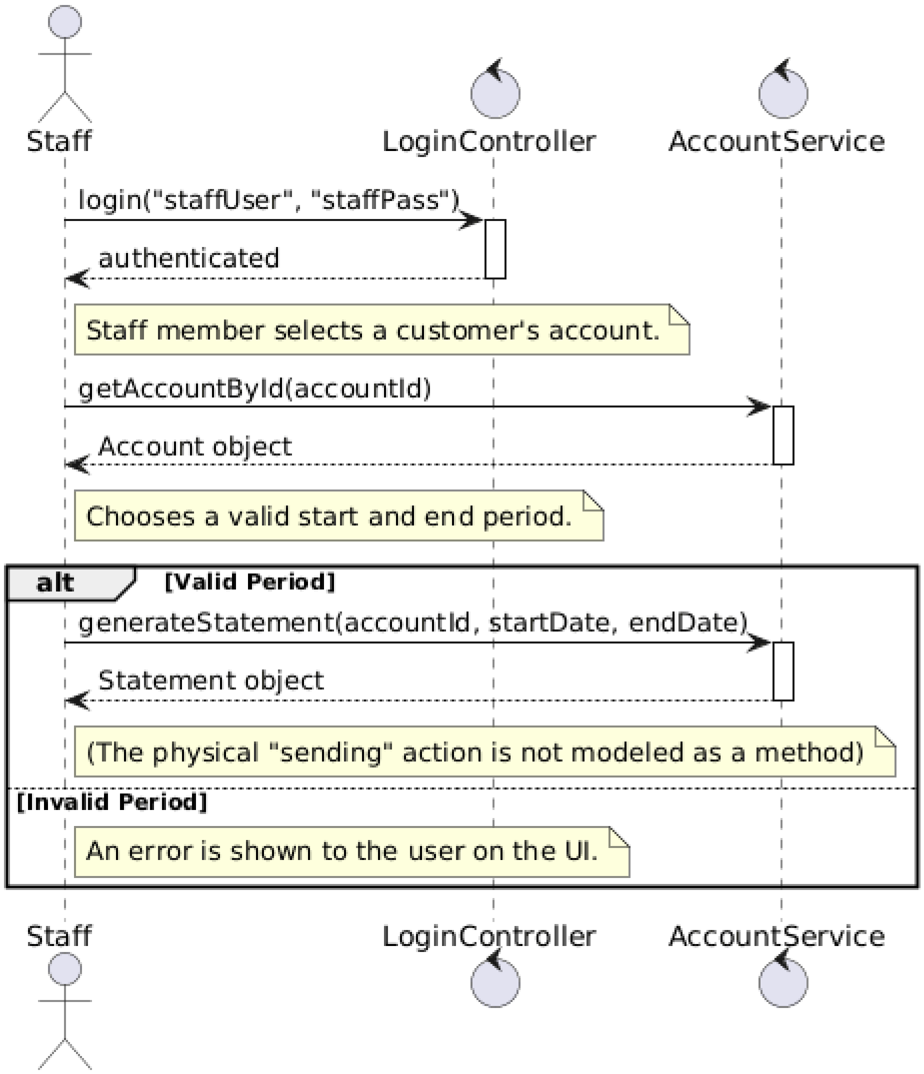
BS_CS_04


Customer
[System Portal]
LoginController
DocumentRequestService
NotificationService
login("user", "pass")
authenticated
Submits document request with selected type
createDocumentRequest(custId, docType)
requestId
The system's backend triggers processing.
processRequest(requestId)
processing_status
alt
[Processed Successfully]
updateRequestStatus(requestId, "Processed")
Delivers document physically/electronically
[Not Processed Successfully]
sendNotification(custId, "Request failed.")
success

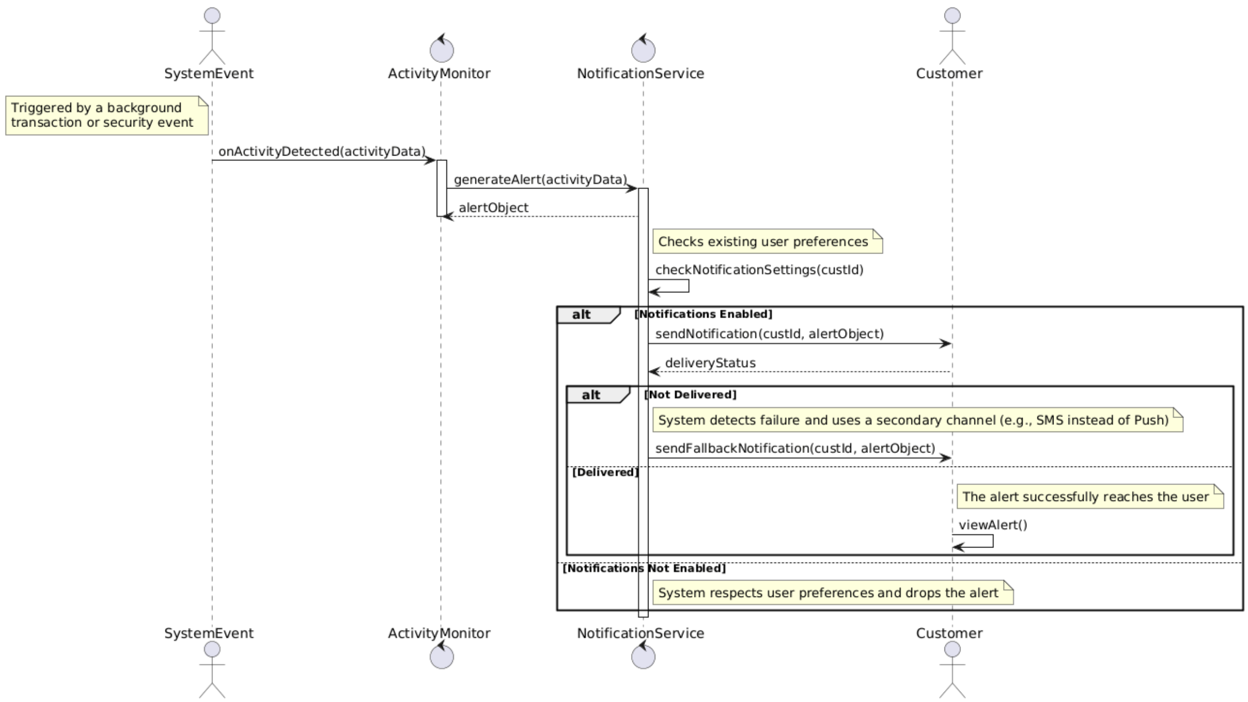
BS_CS_05


Staff
LoginController
ComplaintService
NotificationService
login("staffUser", "staffPass")
authenticated
listAllComplaints()
List<Complaint>
Staff member chooses a complaint to work on.
getComplaintDetails(complaintId)
Complaint object
Investigates issue using provided details
alt
[Not Enough Information]
More information is required from the customer.
requestComplaintInfo(complaintId, staffId, "Need details...")
infoRequestId
The flow for this complaint pauses and loops back.
[Enough Information]
alt
[Escalation Needed]
The issue is complex and requires higher authority.
escalateComplaint(complaintId, staffId, "Complex case")
success_status
[No Escalation Needed]
Staff can resolve the issue directly.
updateComplaintStatus(complaintId, "Resolved")
success_status
The system notifies the customer.
sendNotification(custId, "Your complaint is resolved.")
success_status
Staff
LoginController
ComplaintService
NotificationService

BS_CS_06


Staff
LoginController
AccountService
login("staffUser", "staffPass")
authenticated
Staff member selects a customer's account.
getAccountById(accountId)
Account object
Chooses a valid start and end period.
alt
[Valid Period]
generateStatement(accountId, startDate, endDate)
Statement object
(The physical "sending" action is not modeled as a method)
[Invalid Period]
An error is shown to the user on the UI.
Staff
LoginController
AccountService

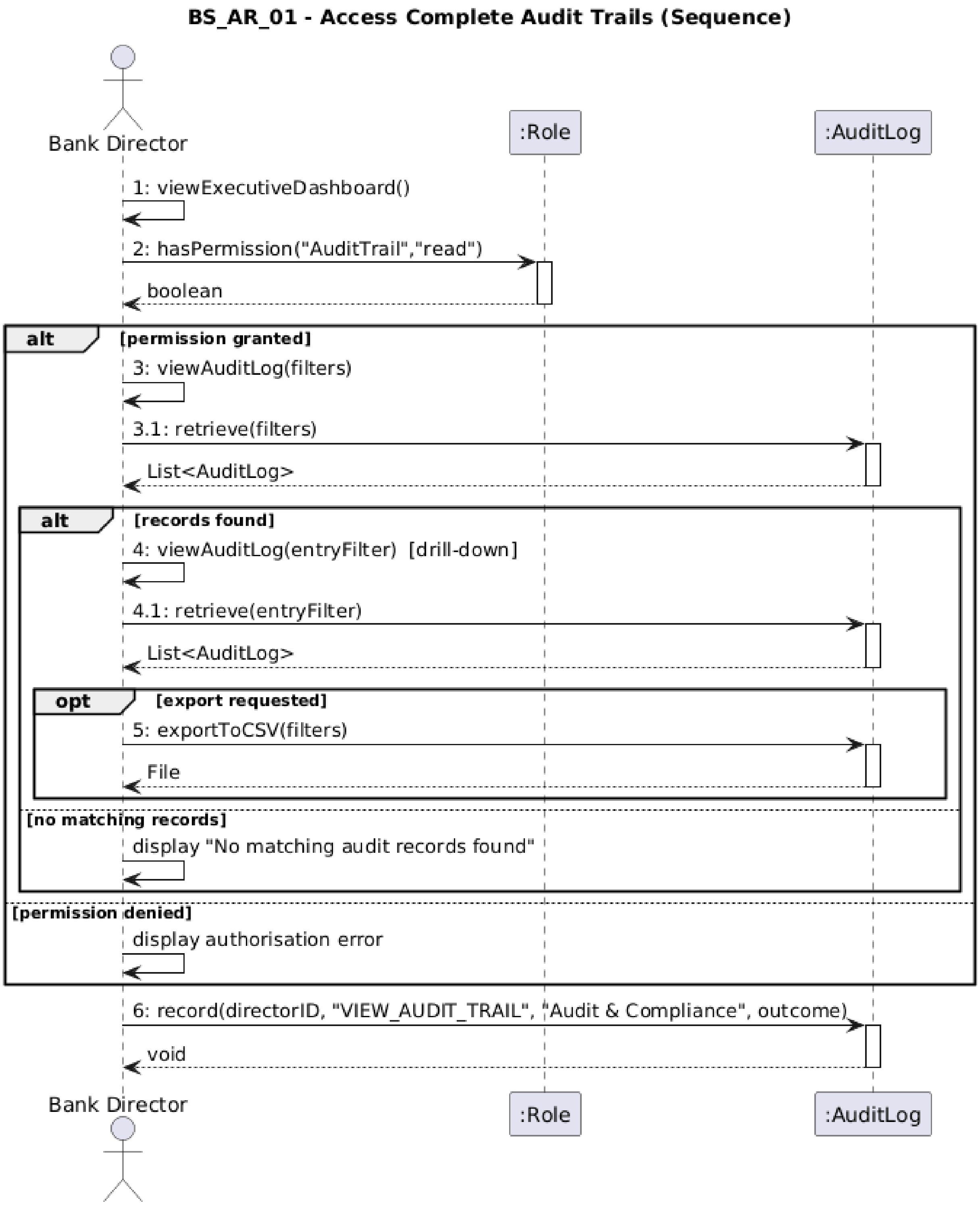
BS_CS_07



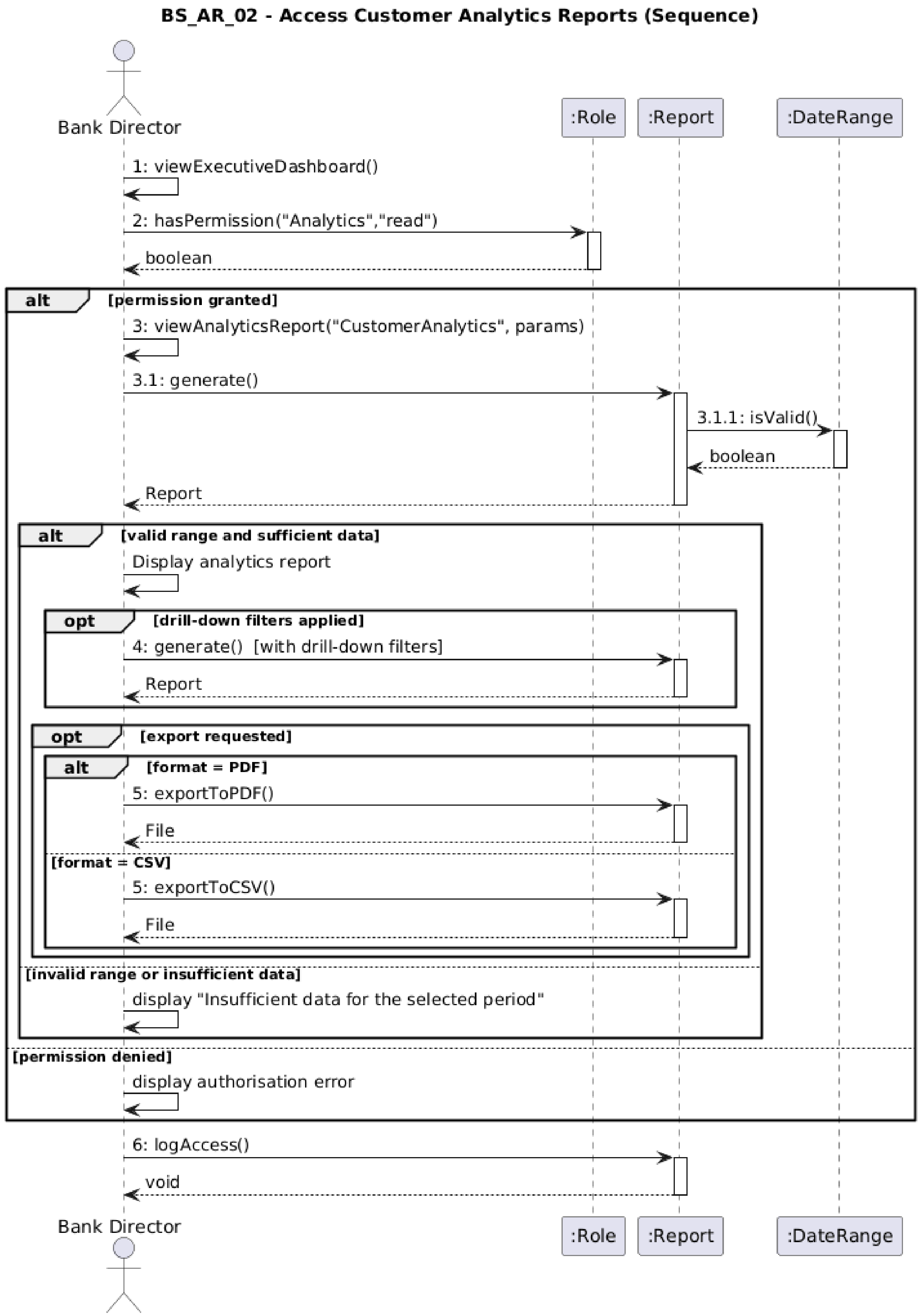
SystemEvent
ActivityMonitor
NotificationService
Customer
Triggered by a background transaction or security event
onActivityDetected(activityData)
generateAlert(activityData)
alertObject
Checks existing user preferences
checkNotificationSettings(custId)
alt
[Notifications Enabled]
sendNotification(custId, alertObject)
deliveryStatus
alt
[Not Delivered]
System detects failure and uses a secondary channel (e.g., SMS instead of Push)
sendFallbackNotification(custId, alertObject)
[Delivered]
The alert successfully reaches the user
viewAlert()
[Notifications Not Enabled]
System respects user preferences and drops the alert
SystemEvent
ActivityMonitor
NotificationService
Customer

Erind Shabani

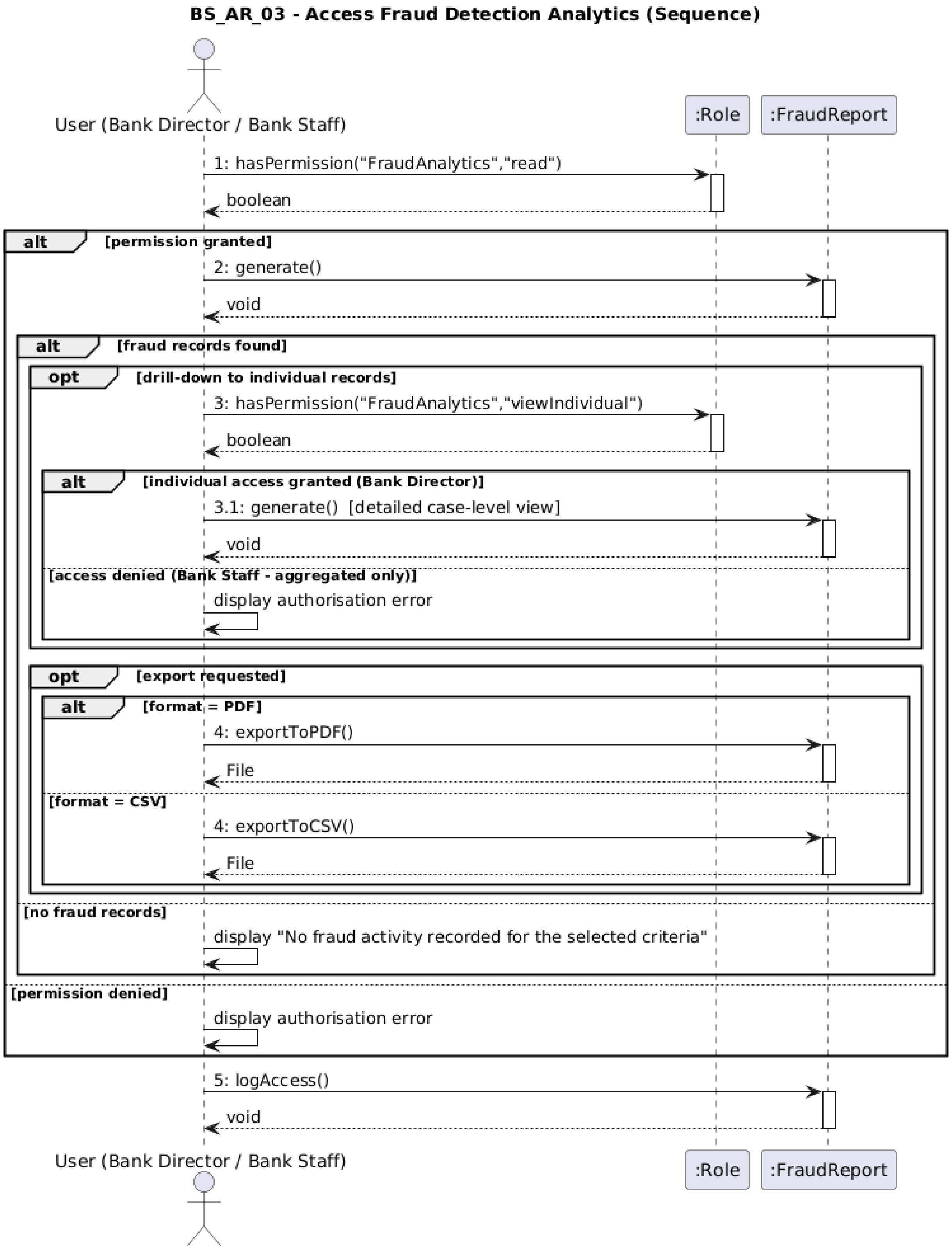
BS_AR_01



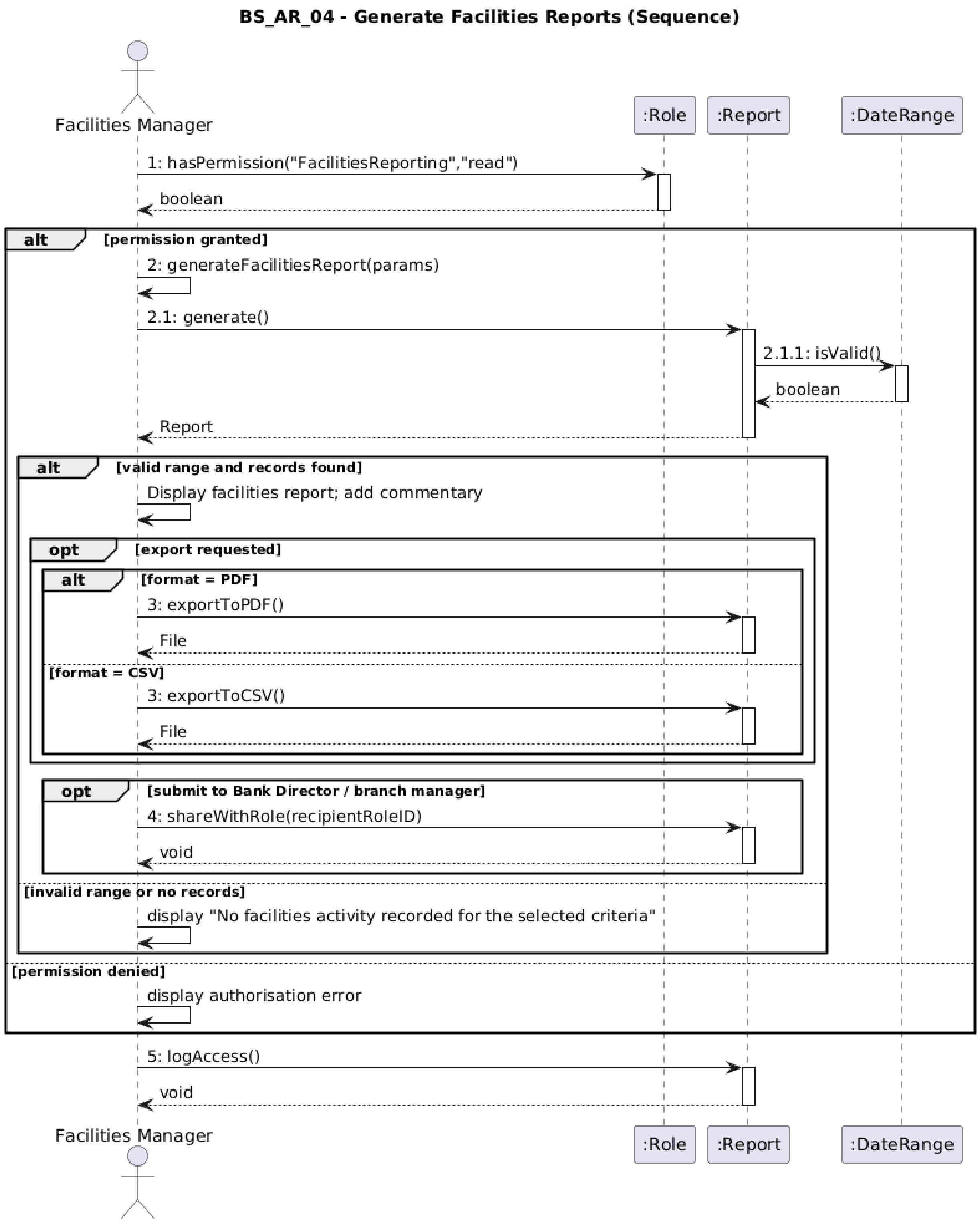
BS_AR_02


BS_AR_02 - Access Customer Analytics Reports (Sequence)
Bank Director
:Role
:Report
:DateRange
1: viewExecutiveDashboard()
2: hasPermission("Analytics","read")
boolean
alt
[permission granted]
3: viewAnalyticsReport("CustomerAnalytics", params)
3.1: generate()
3.1.1: isValid()
boolean
Report
alt
[valid range and sufficient data]
Display analytics report
opt
[drill-down filters applied]
4: generate() [with drill-down filters]
Report
opt
[export requested]
alt
[format = PDF]
5: exportToPDF()
File
[format = CSV]
5: exportToCSV()
File
[invalid range or insufficient data]
display "Insufficient data for the selected period"
[permission denied]
display authorisation error
6: logAccess()
void
Bank Director
:Role
:Report
:DateRange

BS_AR_03


BS_AR_03 - Access Fraud Detection Analytics (Sequence)
User (Bank Director / Bank Staff)
:Role
:FraudReport
1: hasPermission("FraudAnalytics","read")
boolean
alt
[permission granted]
2: generate()
void
alt
[fraud records found]
opt
[drill-down to individual records]
3: hasPermission("FraudAnalytics","viewIndividual")
boolean
alt
[individual access granted (Bank Director)]
3.1: generate() [detailed case-level view]
void
[access denied (Bank Staff - aggregated only)]
display authorisation error
opt
[export requested]
alt
[format = PDF]
4: exportToPDF()
File
[format = CSV]
4: exportToCSV()
File
[no fraud records]
display "No fraud activity recorded for the selected criteria"
[permission denied]
display authorisation error
5: logAccess()
void
User (Bank Director / Bank Staff)
:Role
:FraudReport

BS_AR_04

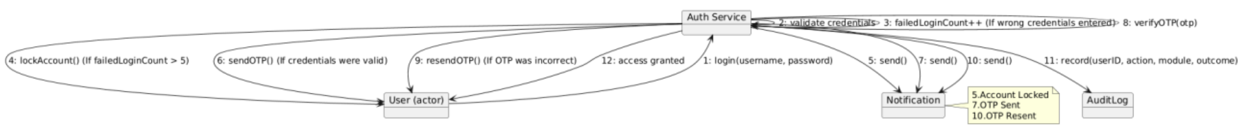

BS_AR_04 - Generate Facilities Reports (Sequence)
Facilities Manager
:Role
:Report
:DateRange
1: hasPermission("FacilitiesReporting","read")
boolean
alt
[permission granted]
2: generateFacilitiesReport(params)
2.1: generate()
2.1.1: isValid()
boolean
Report
alt
[valid range and records found]
Display facilities report; add commentary
opt
[export requested]
alt
[format = PDF]
3: exportToPDF()
File
[format = CSV]
3: exportToCSV()
File
opt
[submit to Bank Director / branch manager]
4: shareWithRole(recipientRoleID)
void
[invalid range or no records]
display "No facilities activity recorded for the selected criteria"
[permission denied]
display authorisation error
5: logAccess()
void
Facilities Manager
:Role
:Report
:DateRange

Collaboration Diagram

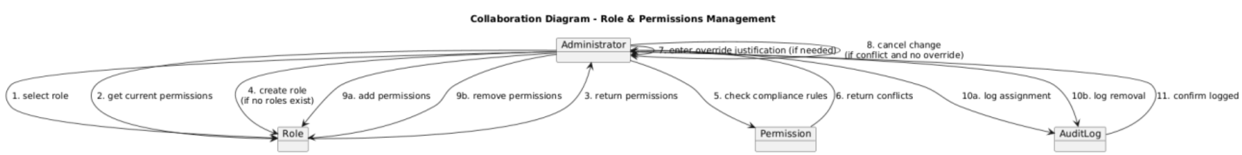
Andrea Ogreni

BS_UM_01

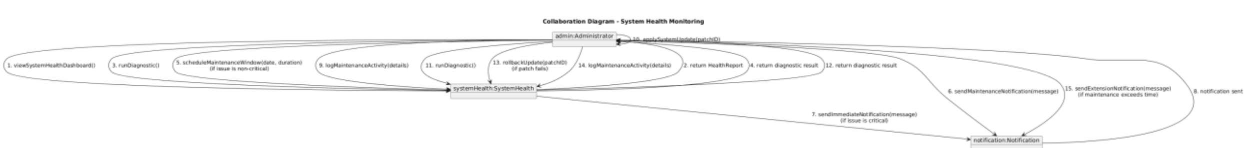
BS_UM_02



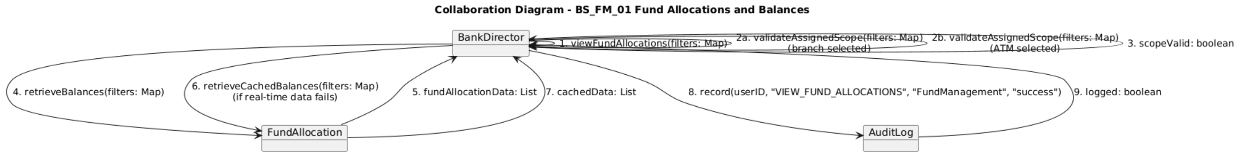
BS_UM_03



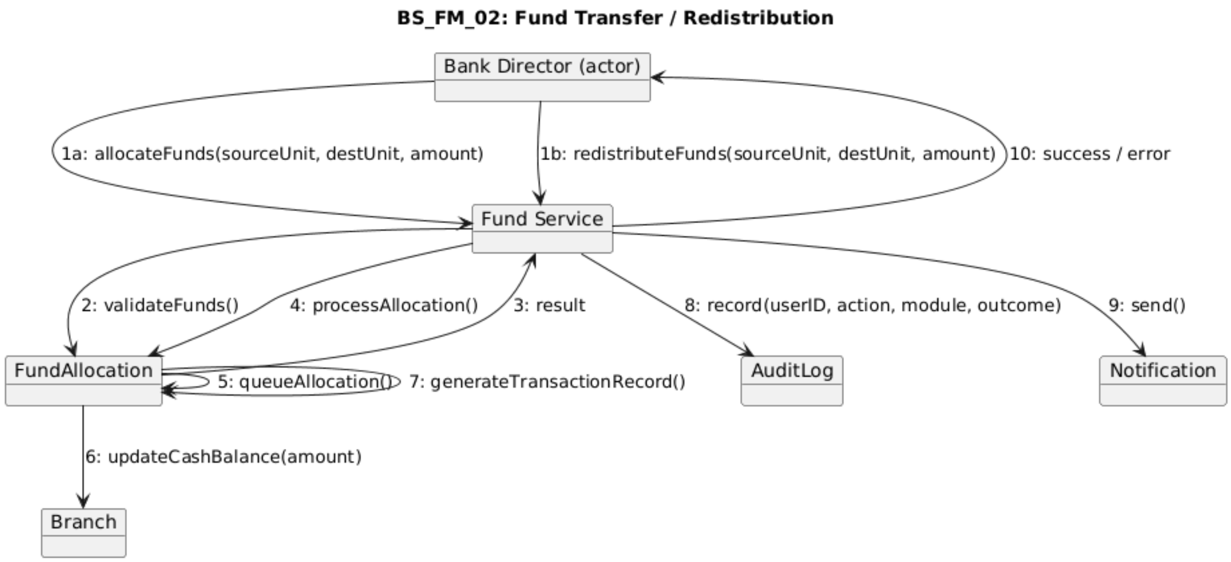
BS_UM_04



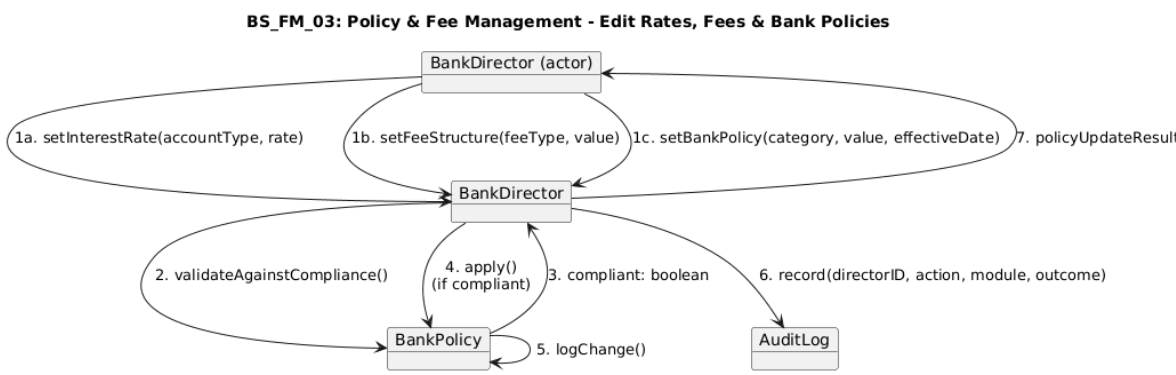
BS_FM_01



BS_FM_02

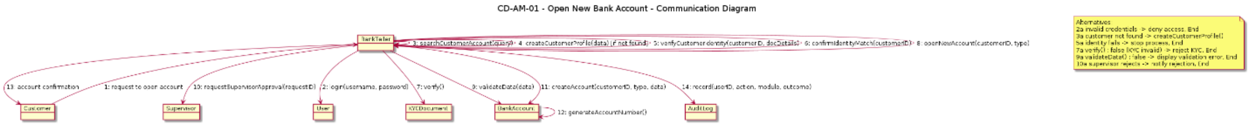


BS_FM_03

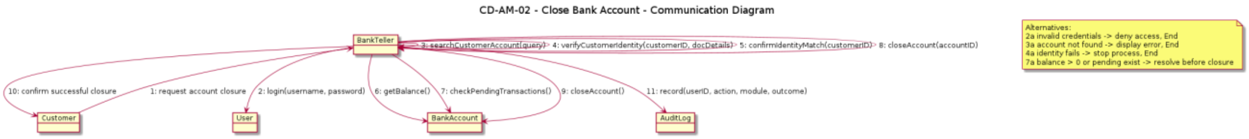


Joel Lluri

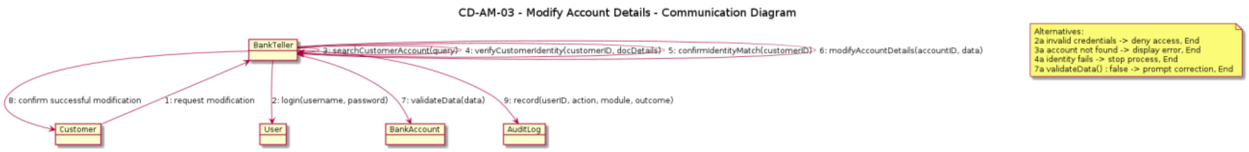
BS_AM_01



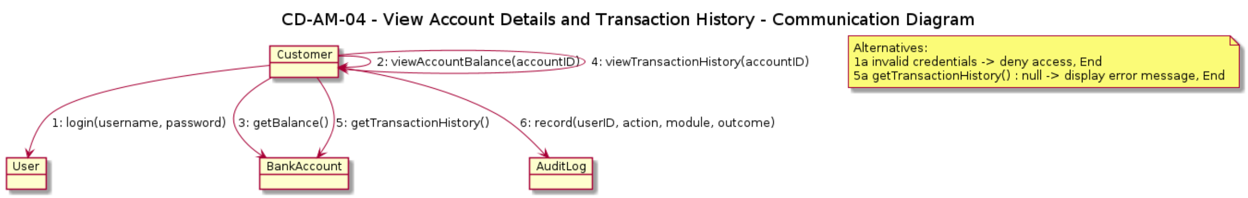
BS_AM_02



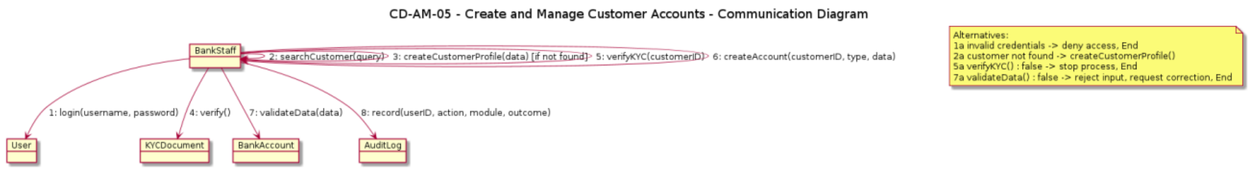
BS_AM_03



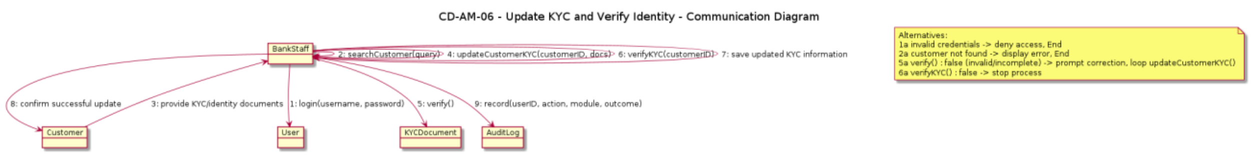
BS_AM_04



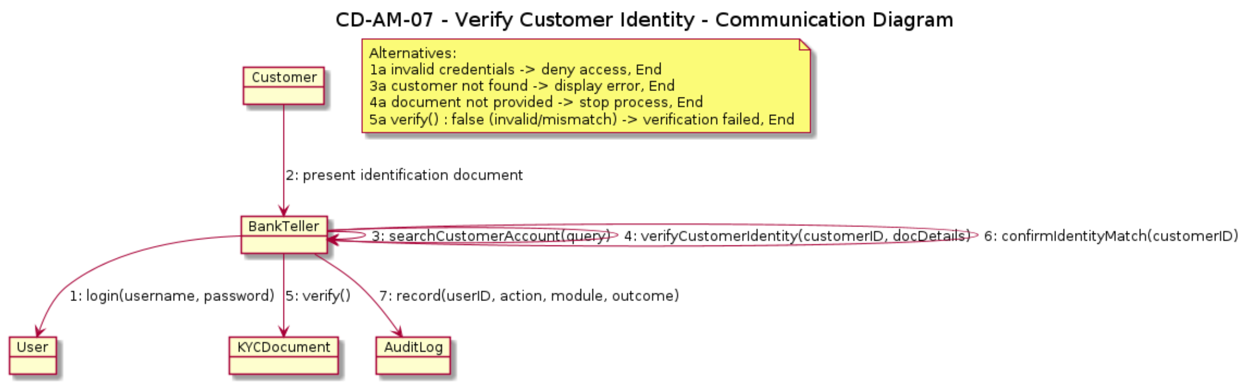
BS_AM_05



BS_AM_06

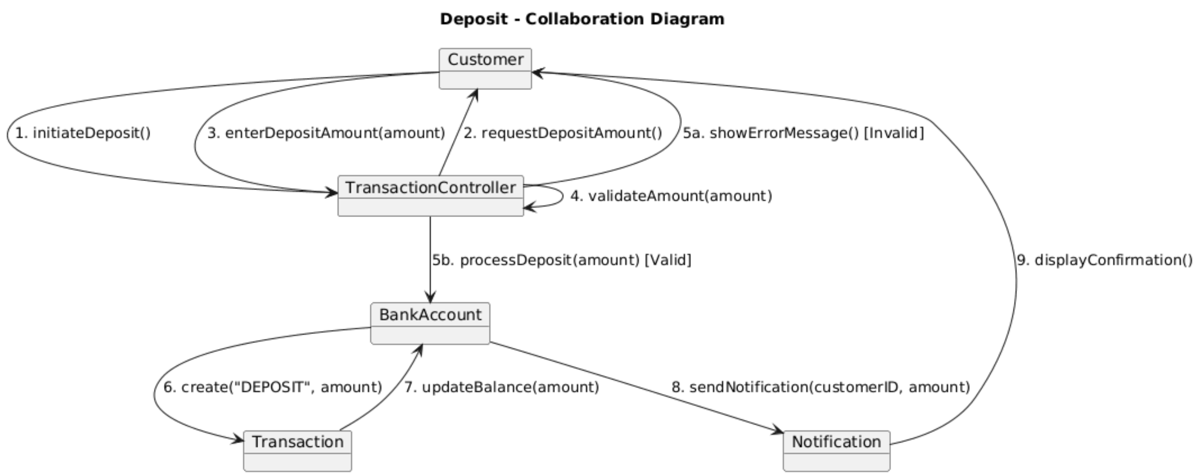


BS_ AM_07

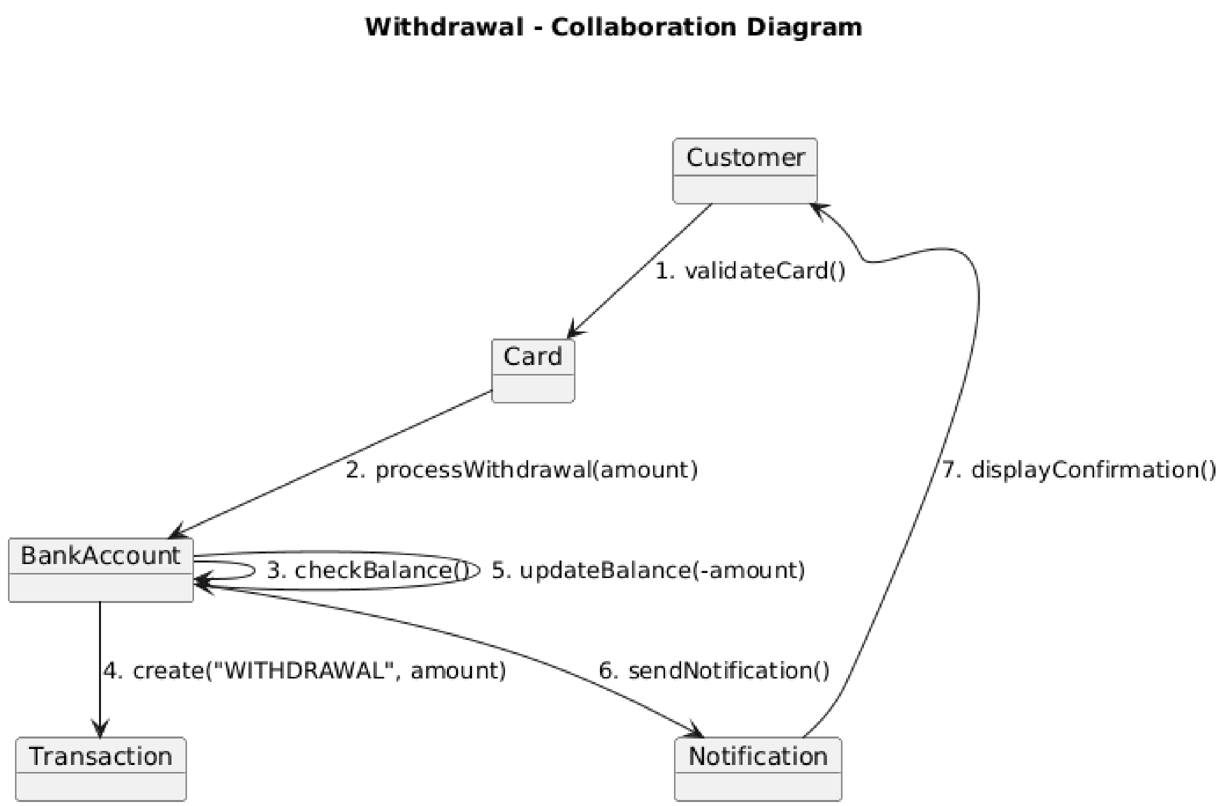


Adriano Demiri

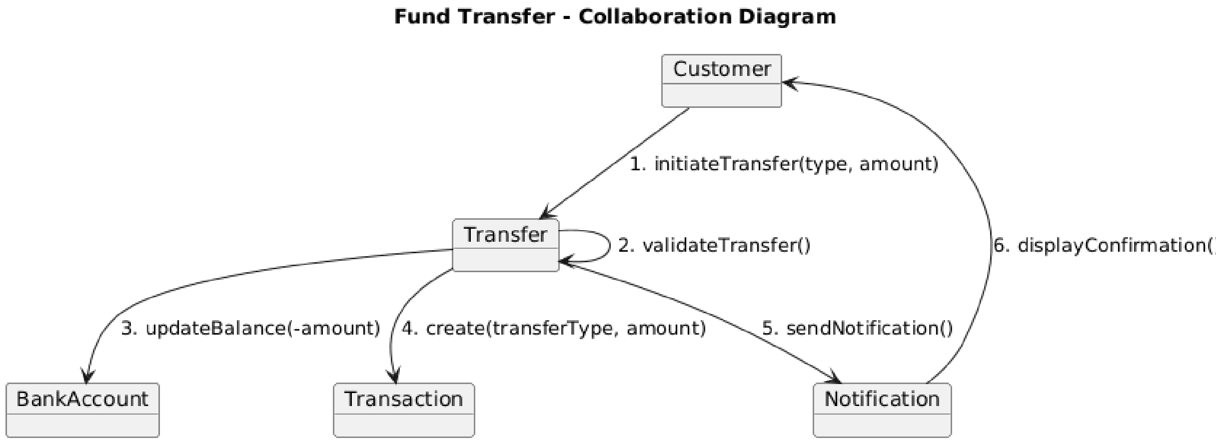
AM_TM_01



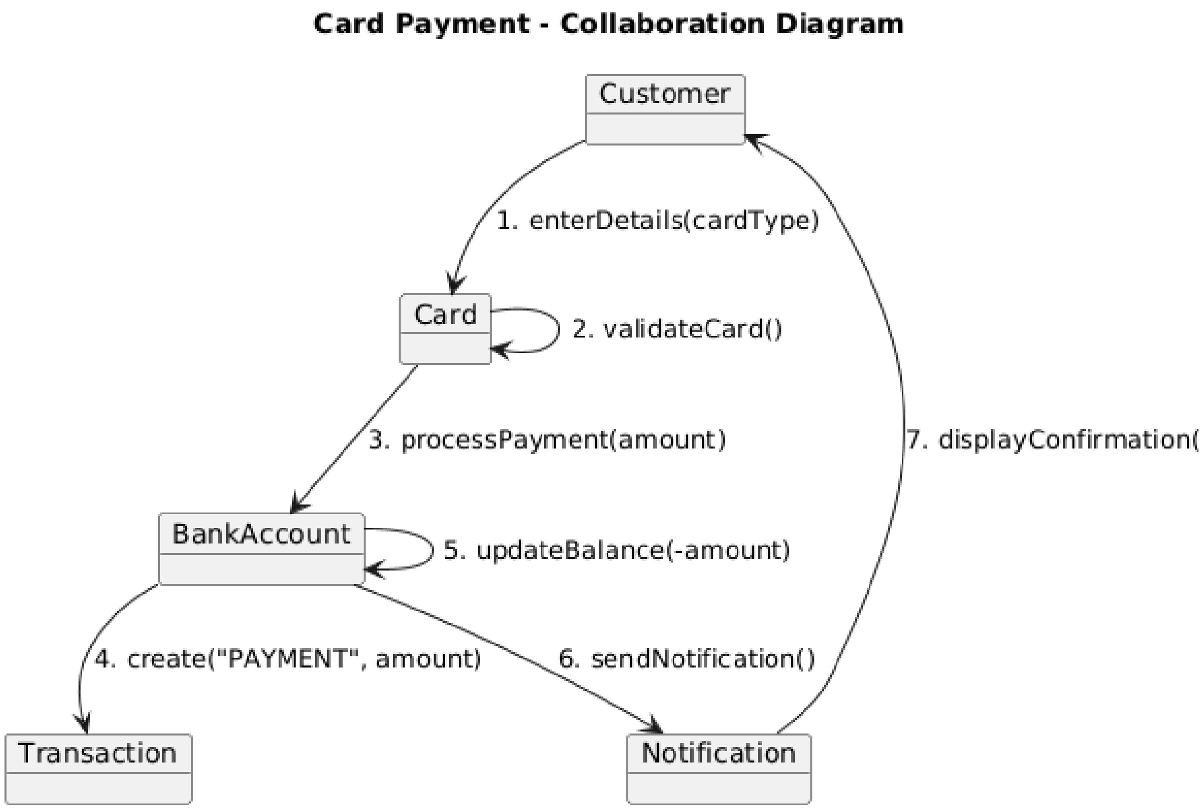
BS_TM_02



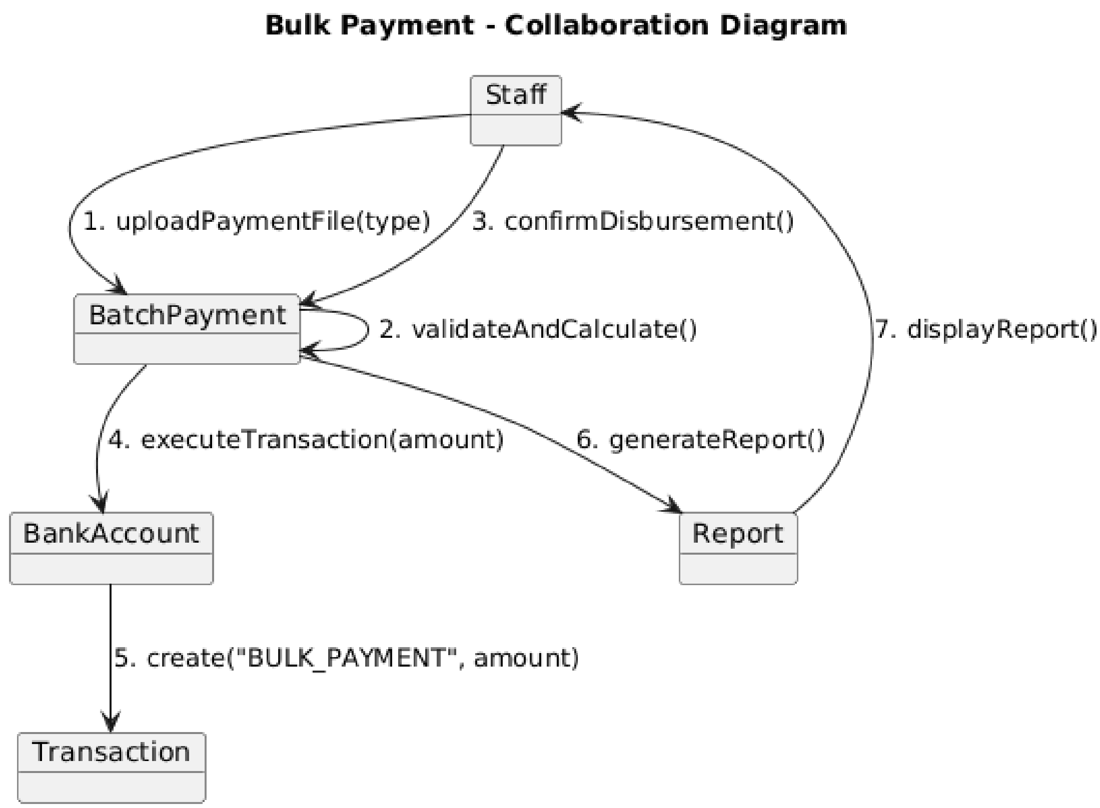
BS_TM_03



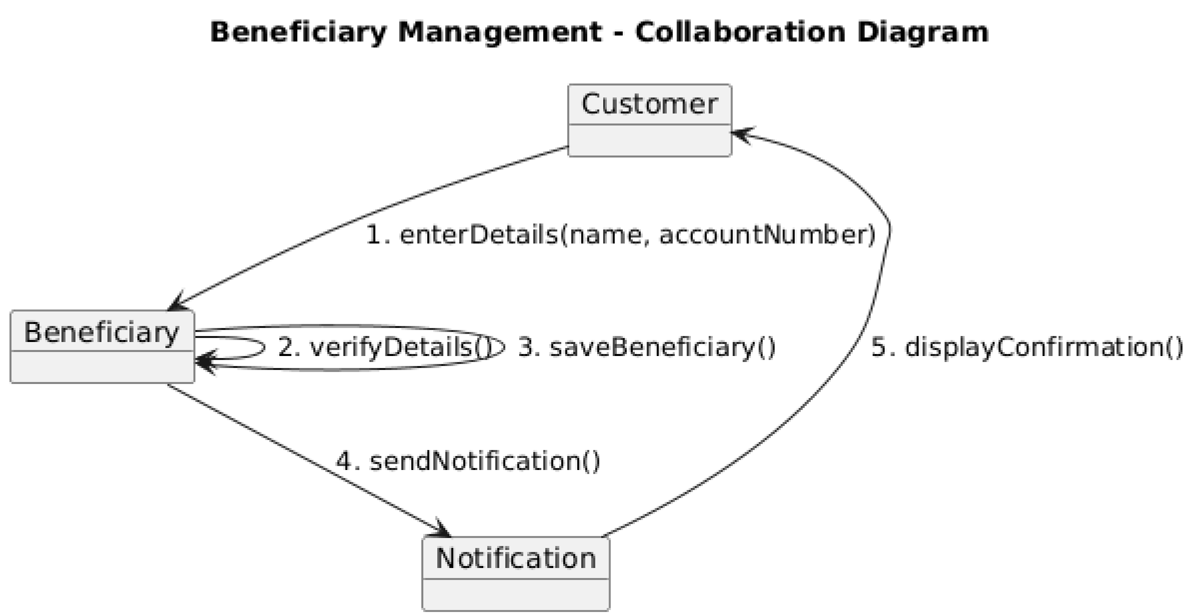
BS_TM_04



BS_TM_05

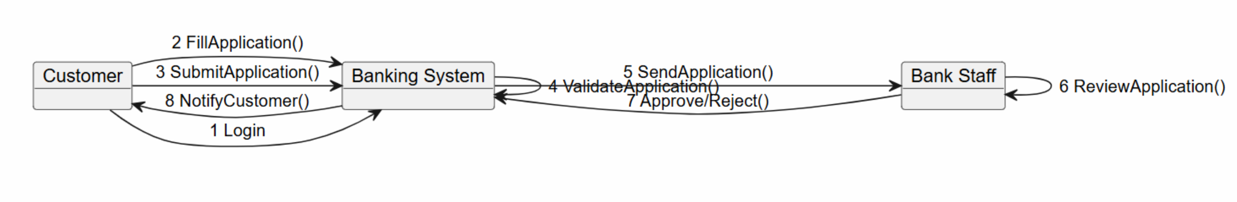


BS_TM_06

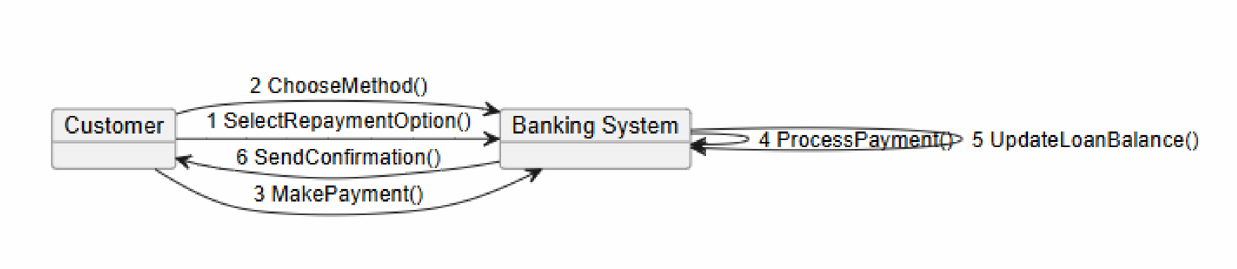


Emanuele Berbiu

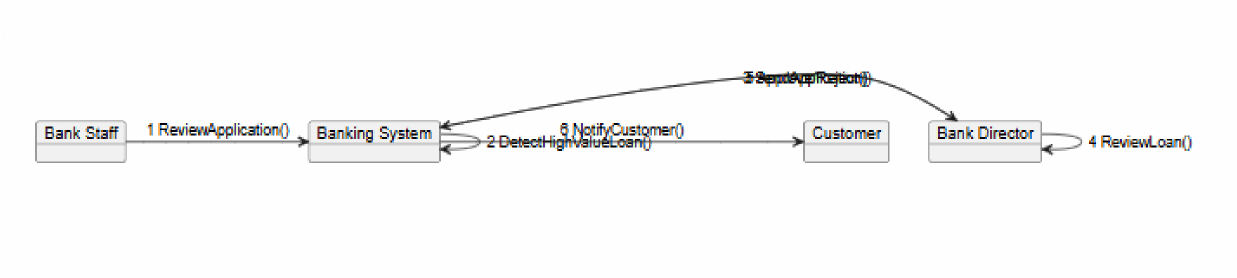
BS_LM_01



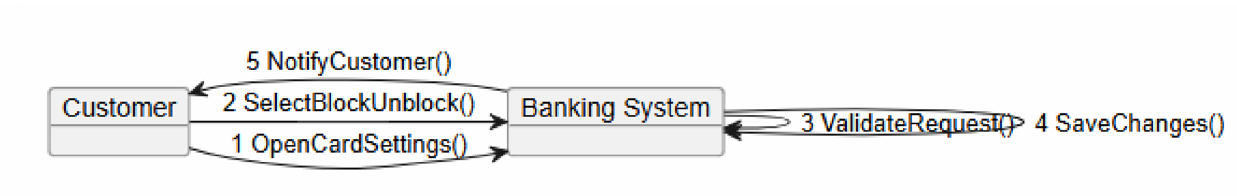
BS_LM_02



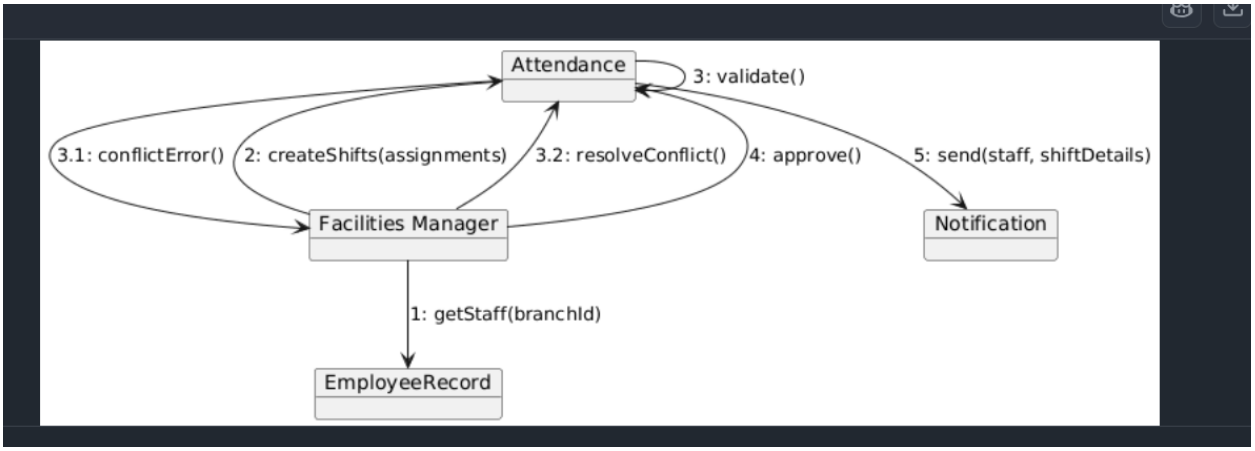
BS_LM_03



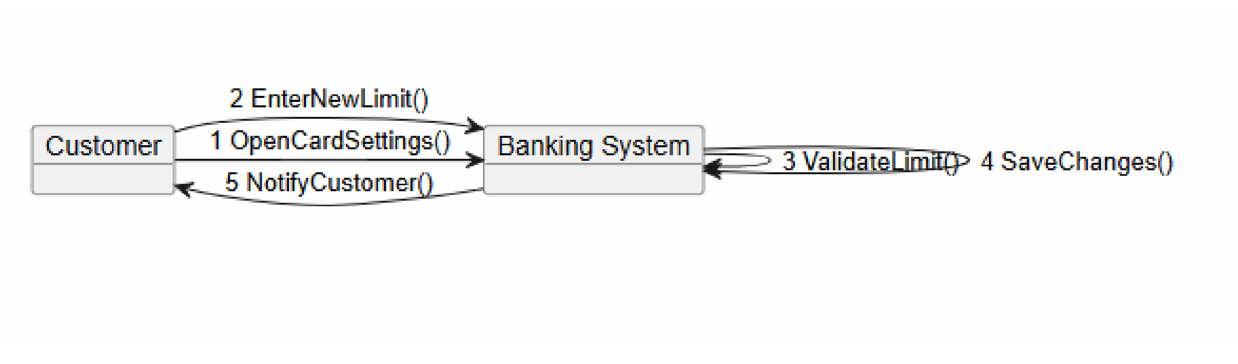
BS_LM_04



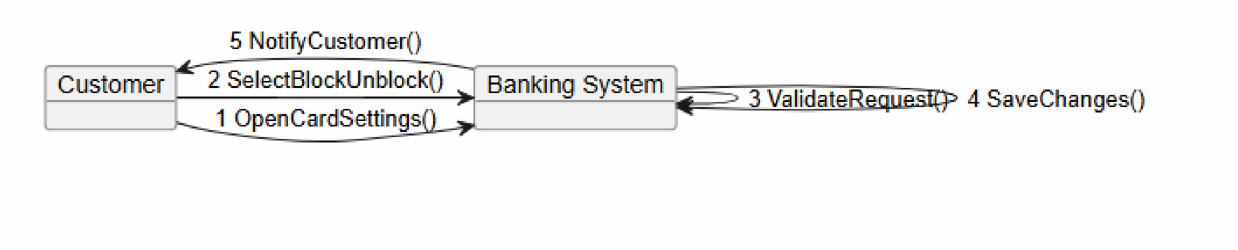
BS_CM_01



BS_CM_02

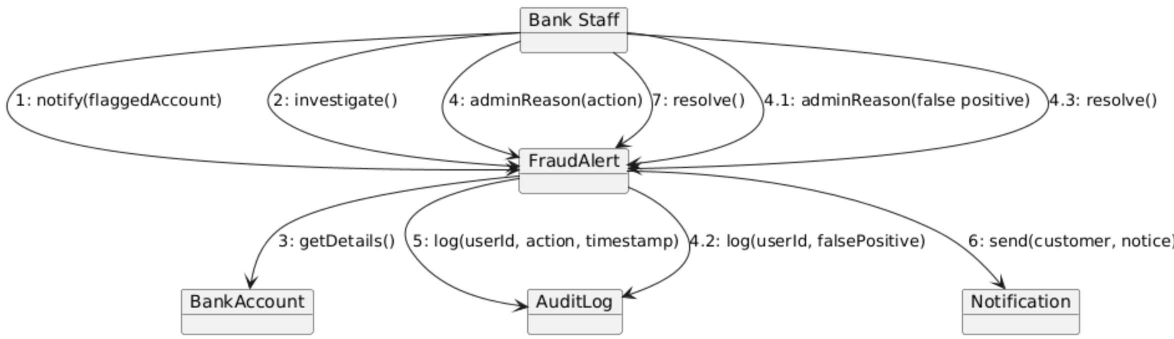


BS_CM_03

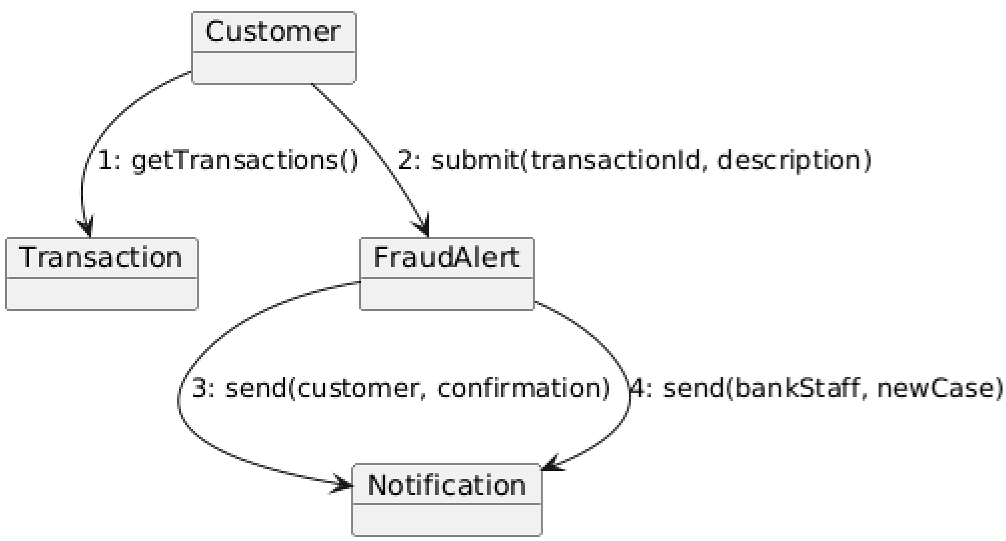


Joandri Domi

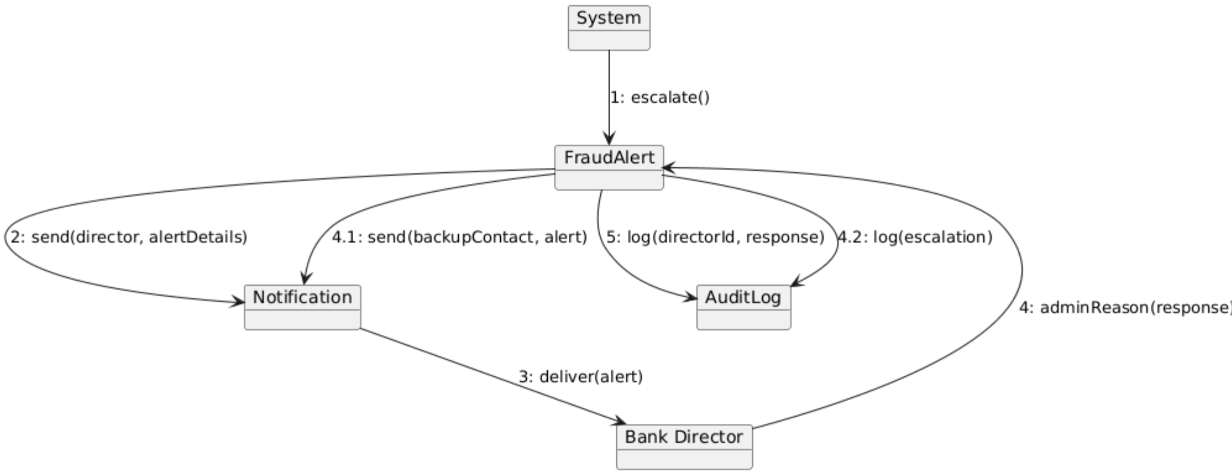
BS_FD_01



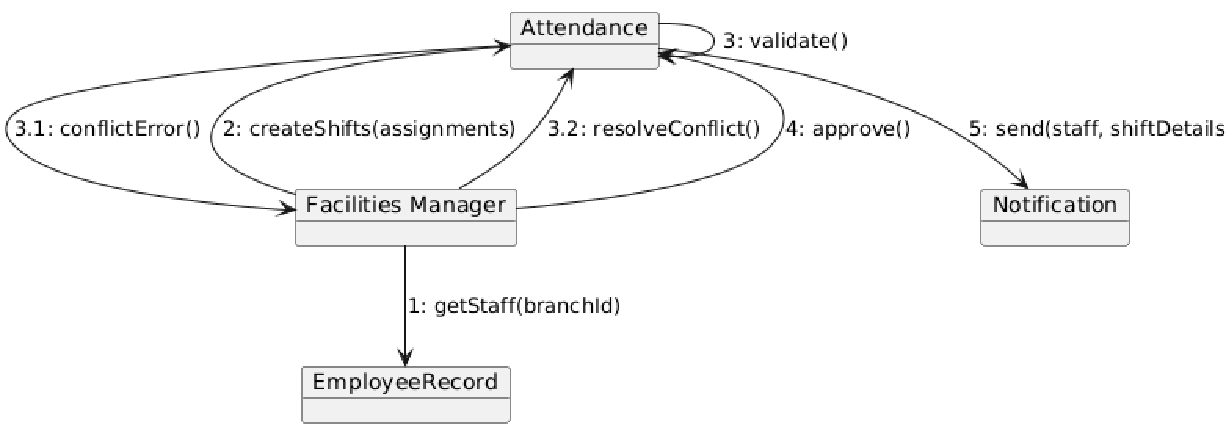
BS_FD_02



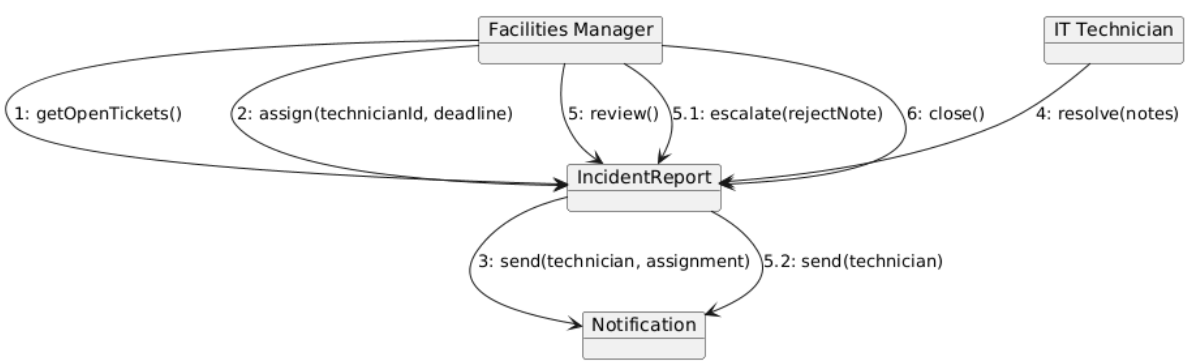
BS_FD_03



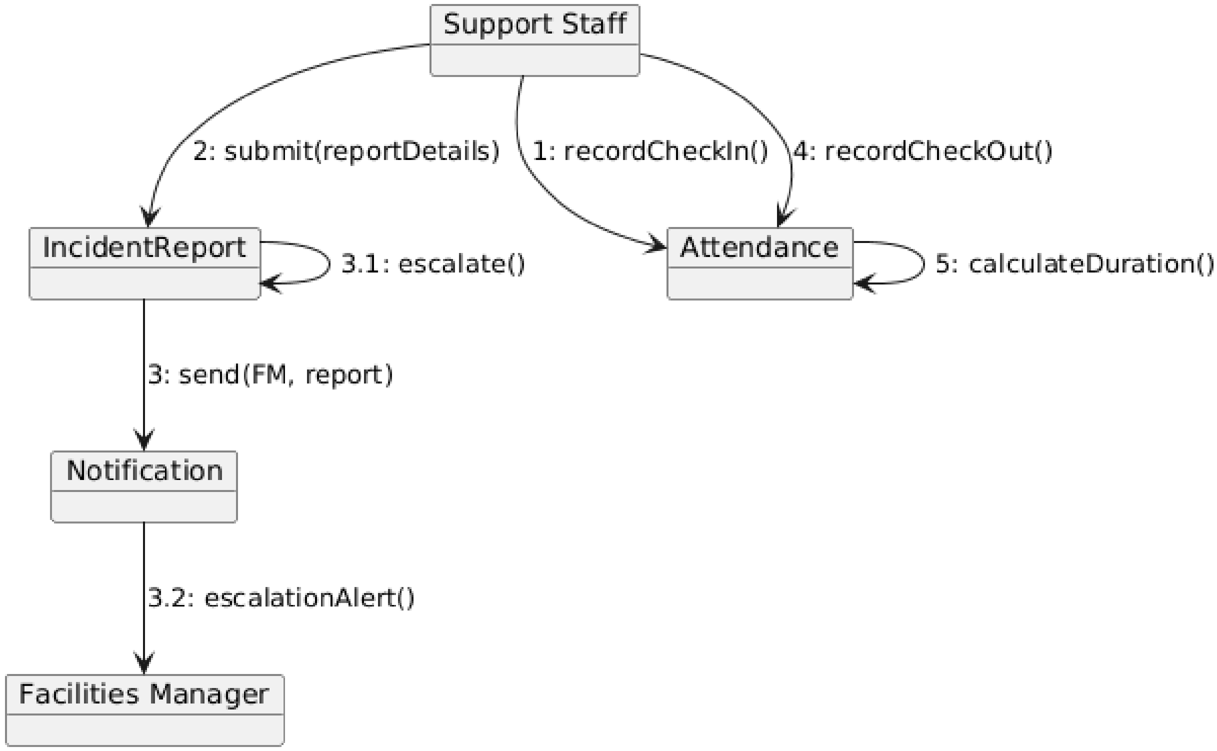
BS_FCM_01



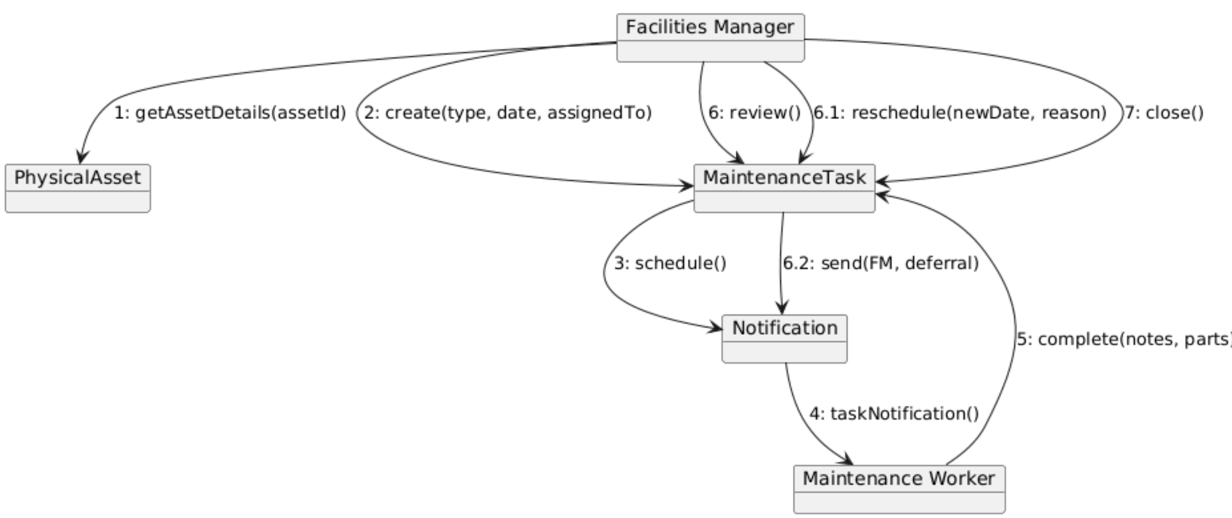
BS_FCM_02



BS_FCM_03

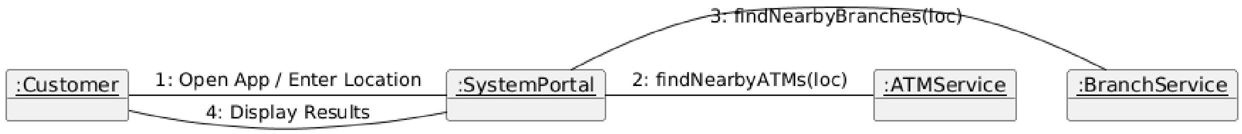


BS_FCM_04

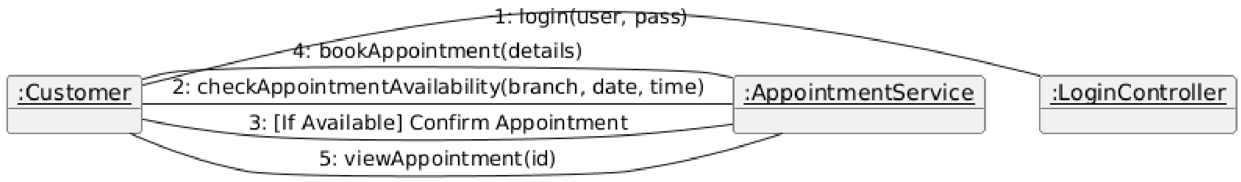


Kleo Miraka

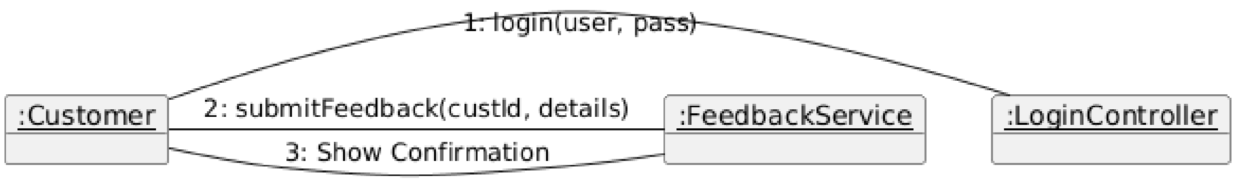
BS_CS_01



BS_CS_02

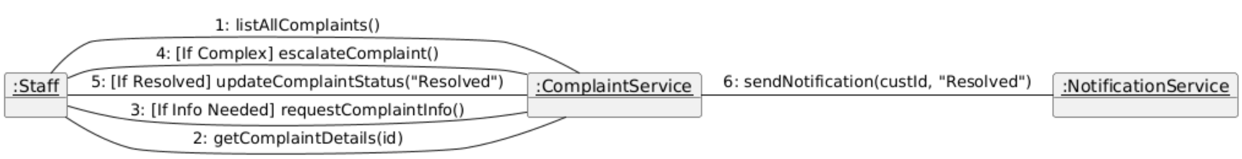


BS_CS_03

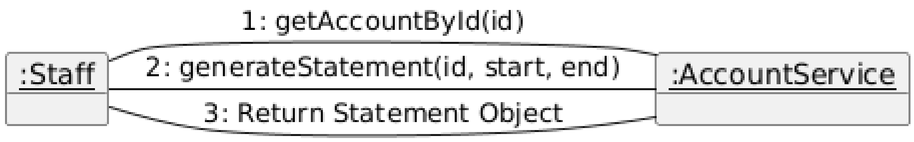


BS_CS_04

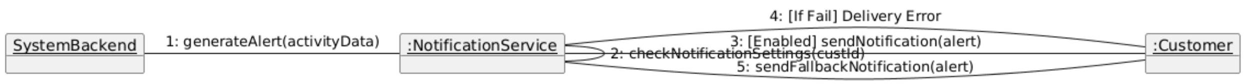
BS_CS_05



BS_CS_06

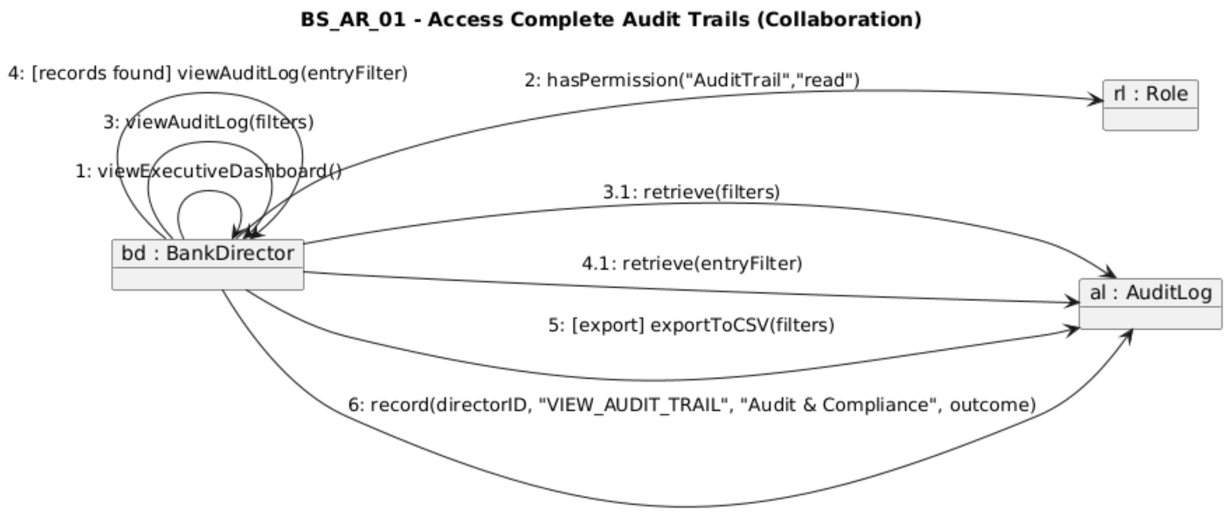


BS_CS_07

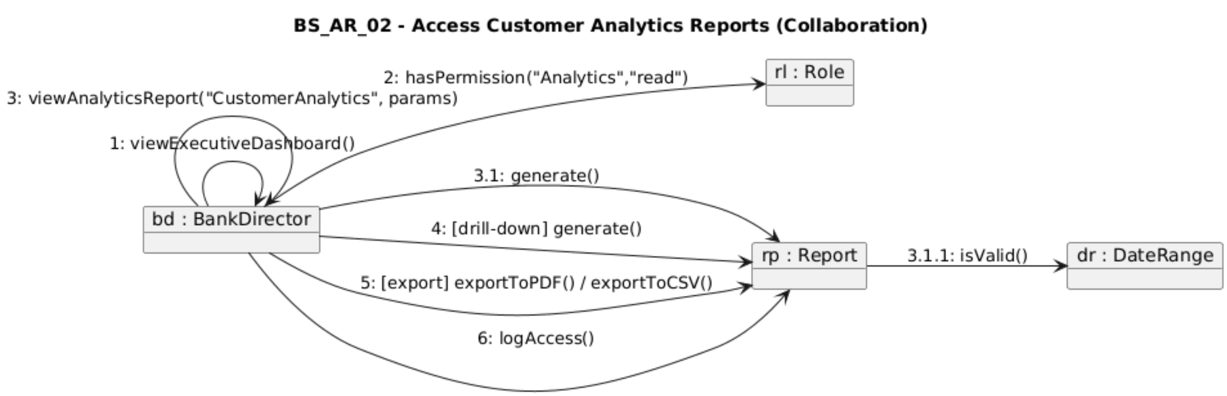


Erind Shabani

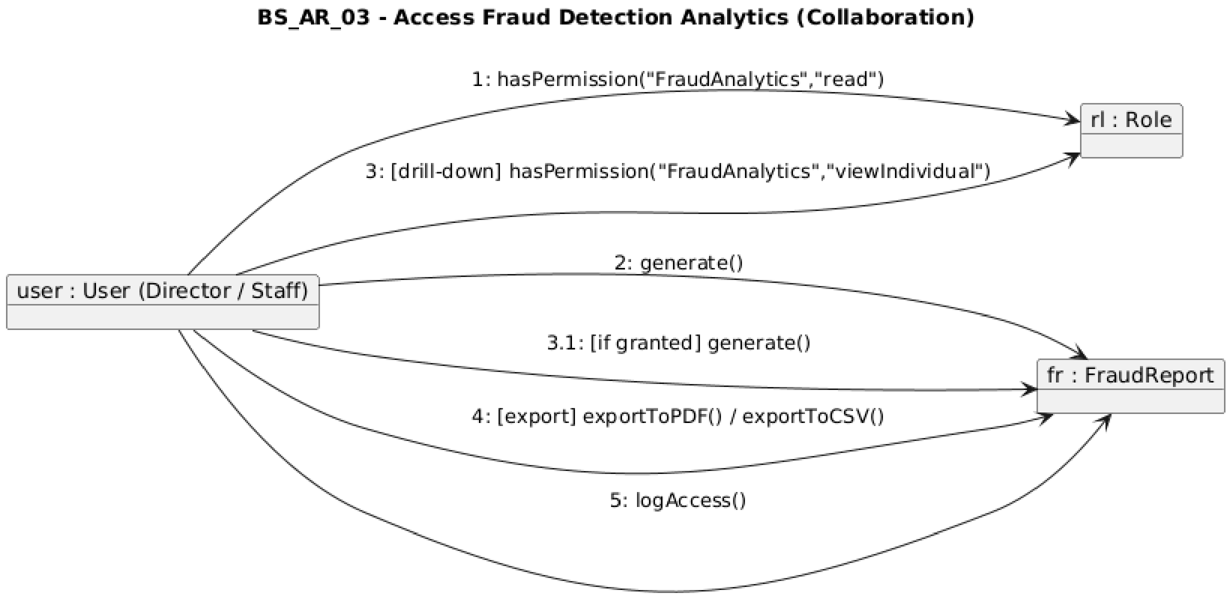
BS_AR_01



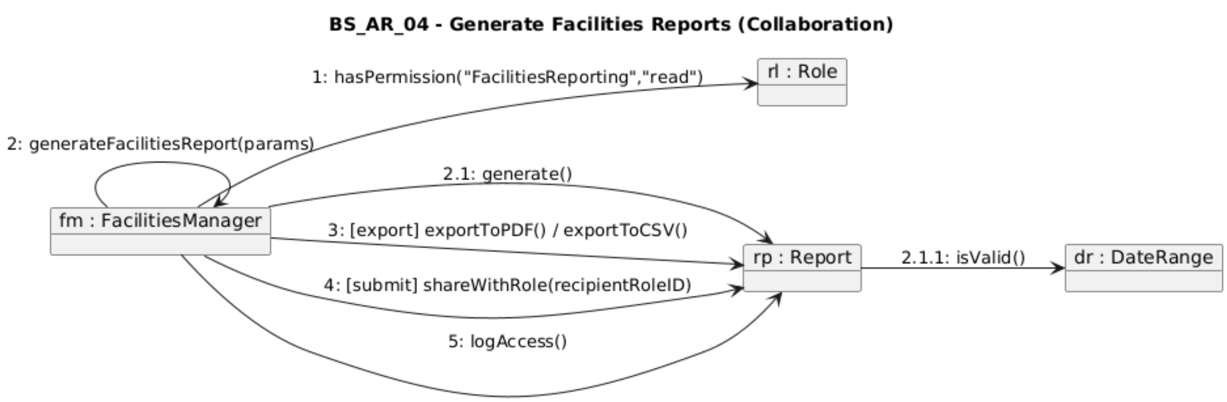
BS_AR_02



BS_AR_03



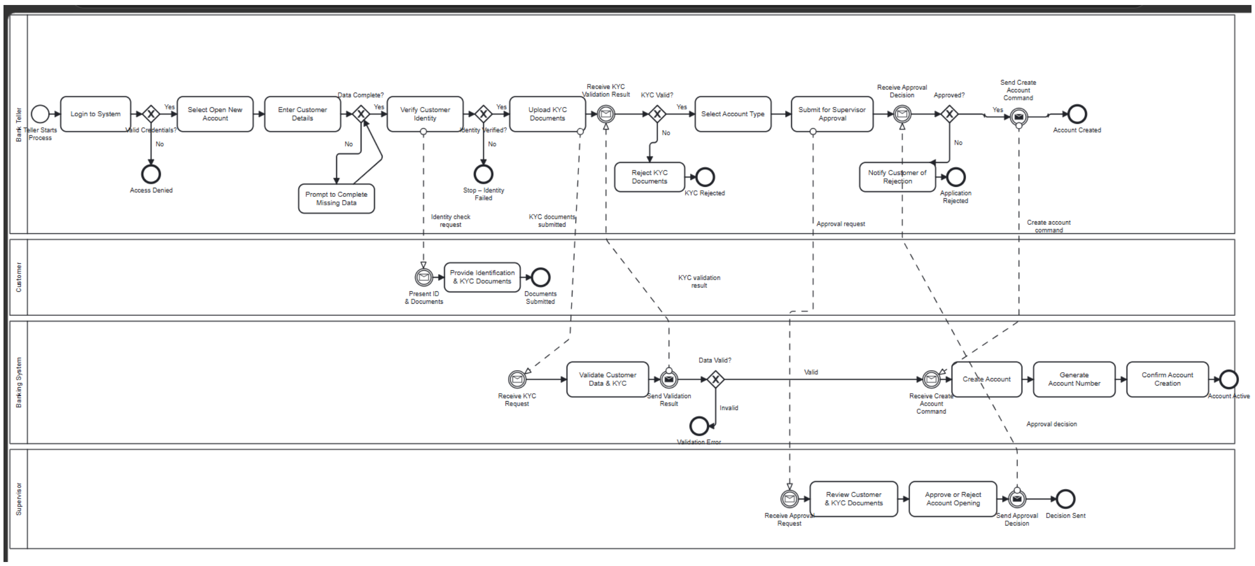
BS_AR_04



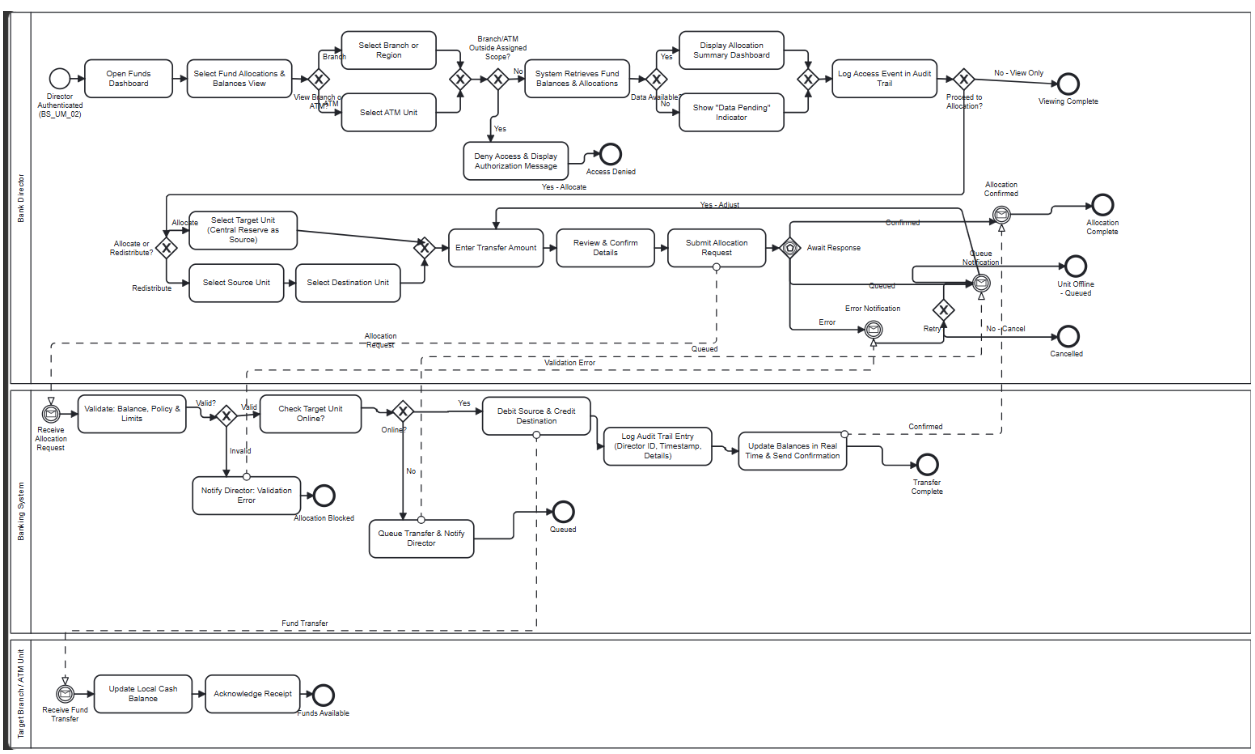
BPMN diagrams

Erind Kotobelli

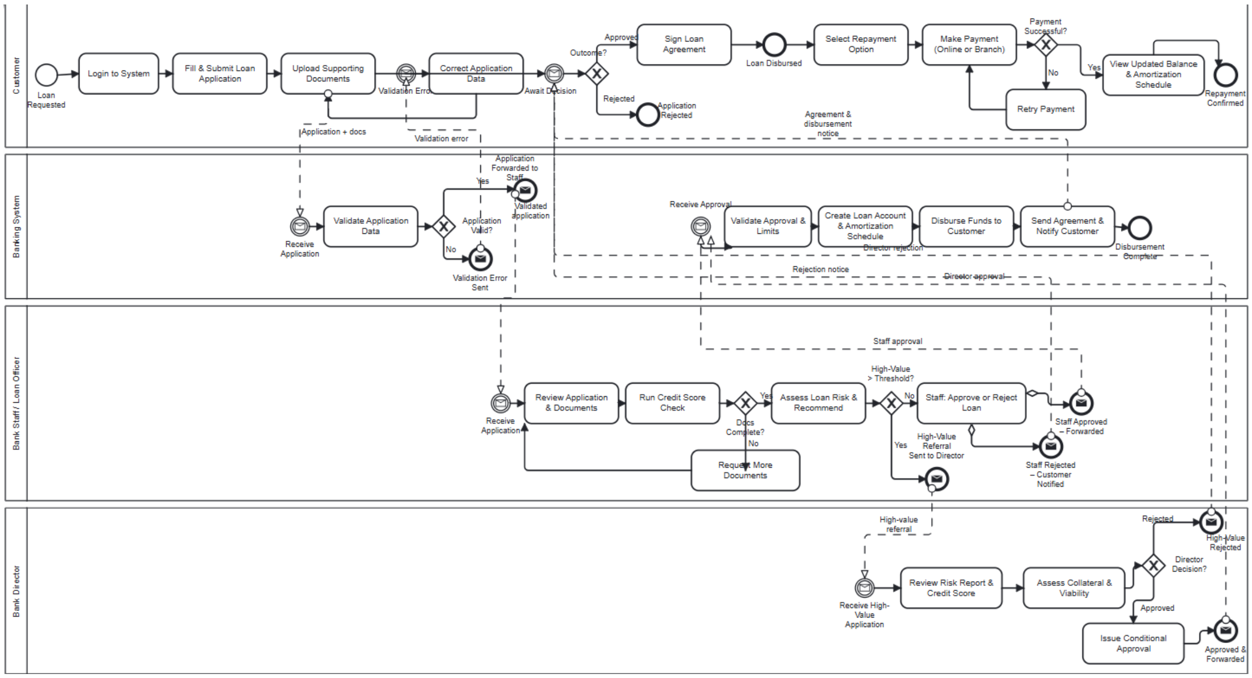
bpmn1_account_oppening



bpmn2_fund_allocation

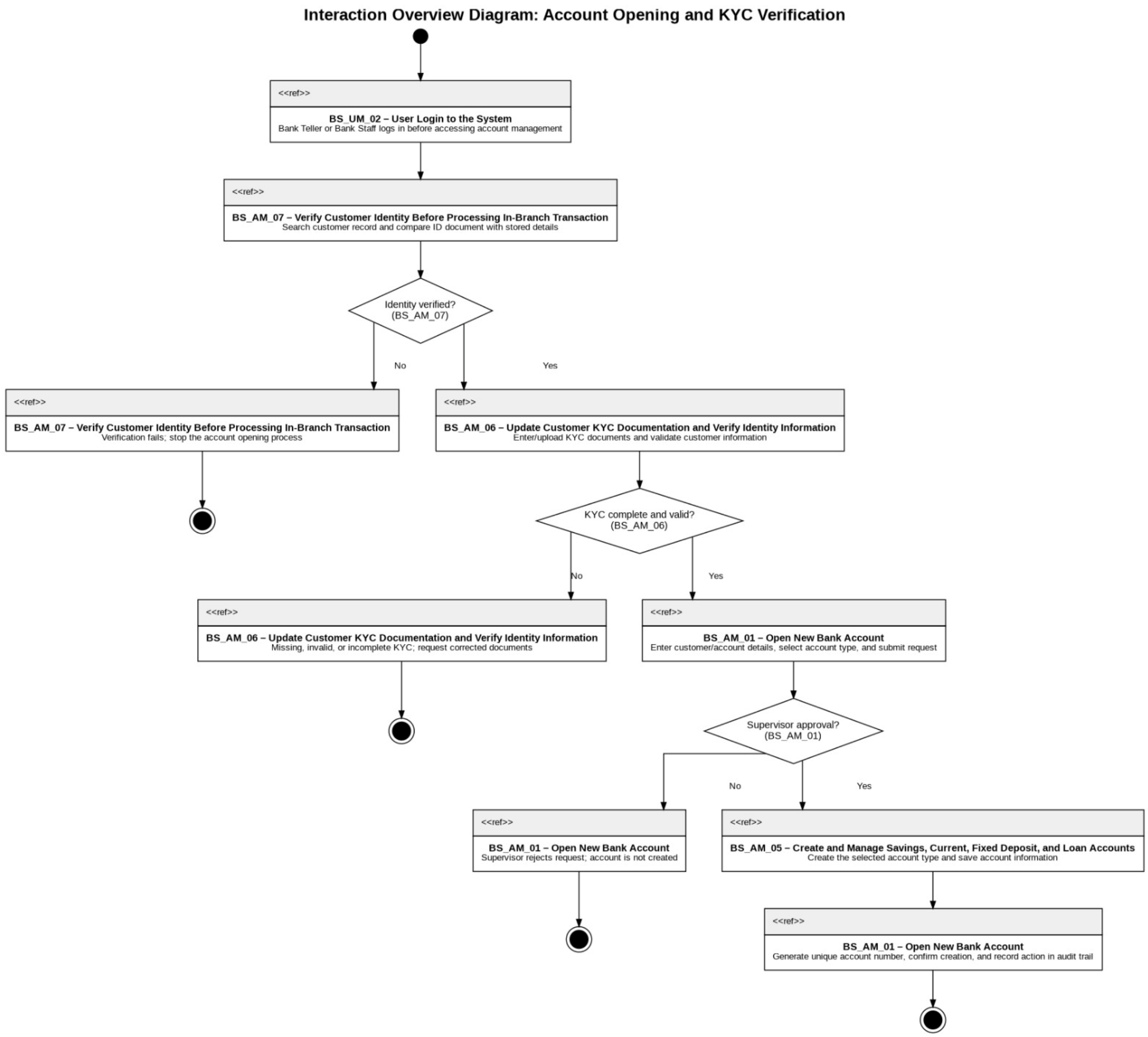


bpmn3_loan_application

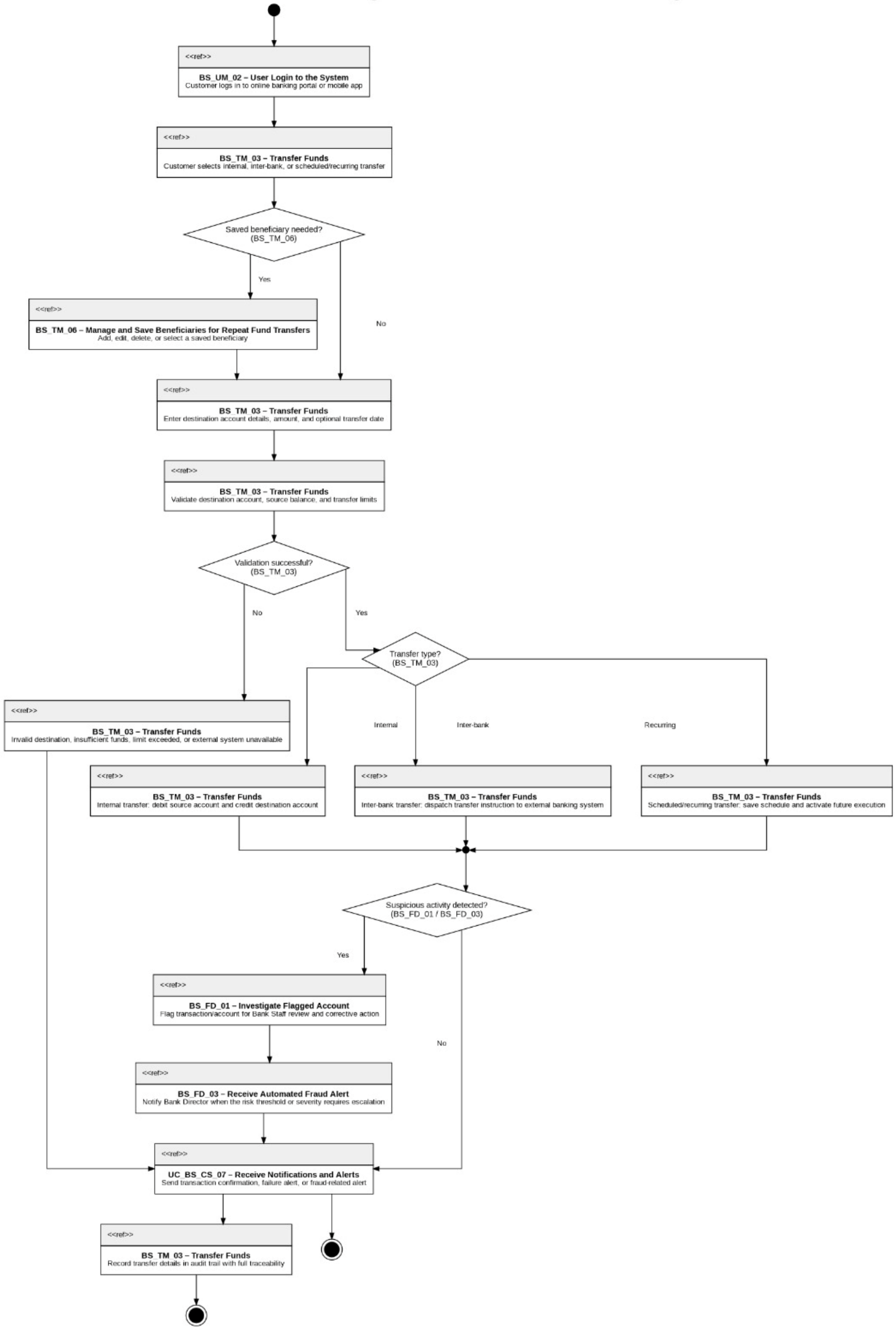


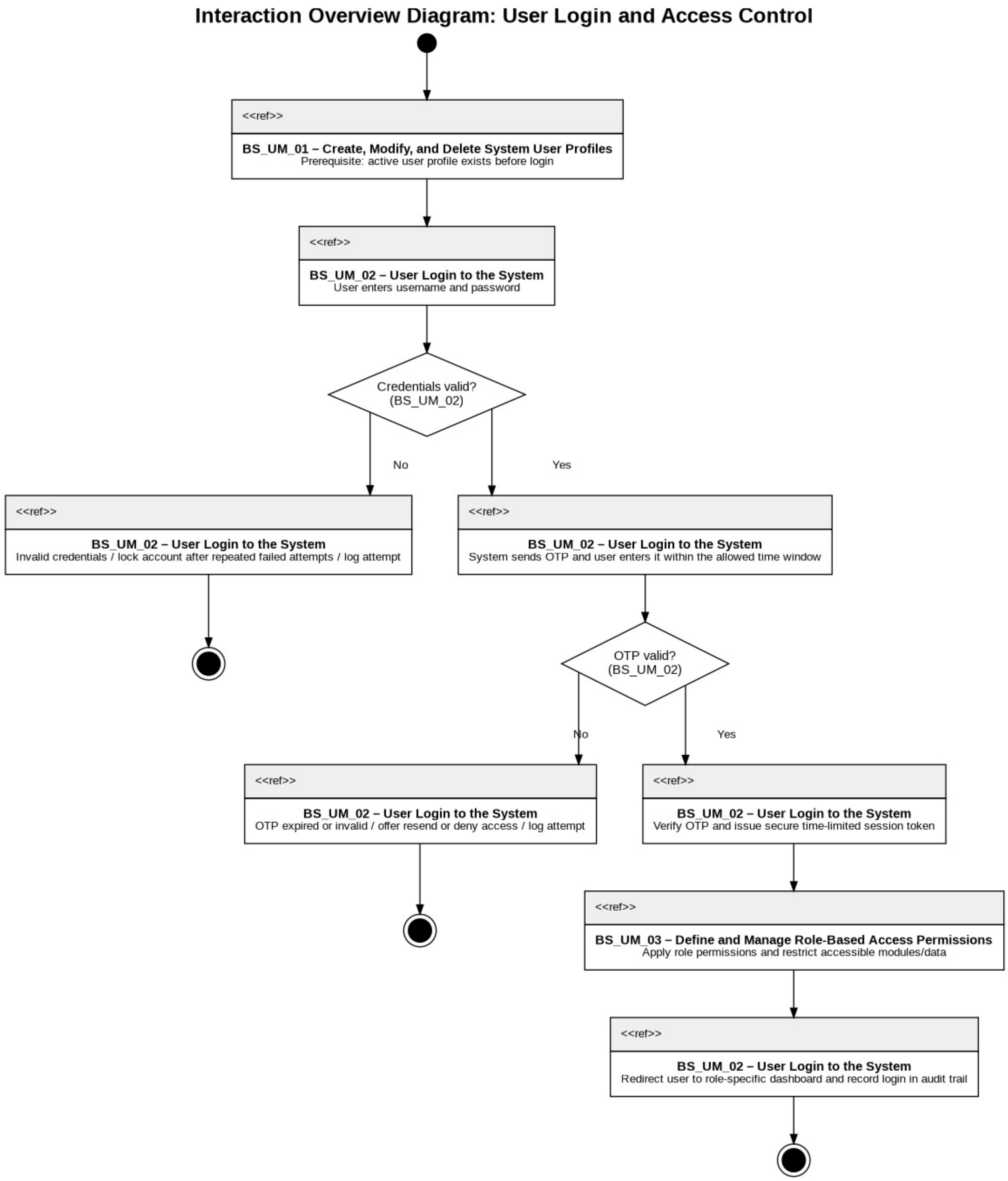
Interaction Overview Diagram

ALL Members



Interaction Overview Diagram: Fund Transfer and Transaction Processing



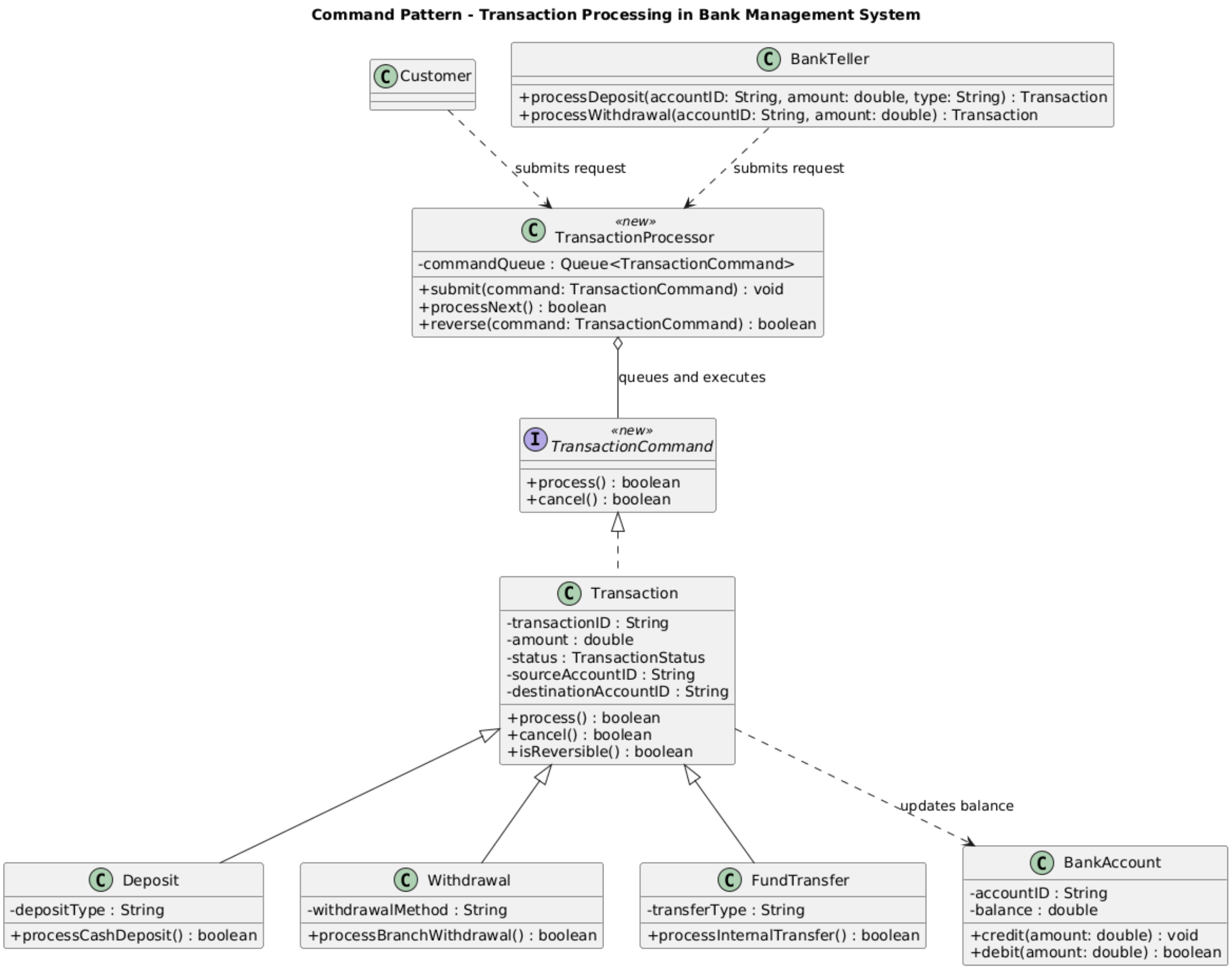

Interaction Overview Diagram: User Login and Access Control
<<ref>>
BS_UM_01 – Create, Modify, and Delete System User Profiles
Prerequisite: active user profile exists before login
<<ref>>
BS_UM_02 – User Login to the System
User enters username and password
Credentials valid? (BS_UM_02)
No
Yes
<<ref>>
BS_UM_02 – User Login to the System
Invalid credentials / lock account after repeated failed attempts / log attempt
<<ref>>
BS_UM_02 – User Login to the System
System sends OTP and user enters it within the allowed time window
OTP valid? (BS_UM_02)
No
Yes
<<ref>>
BS_UM_02 – User Login to the System
OTP expired or invalid / offer resend or deny access / log attempt
<<ref>>
BS_UM_02 – User Login to the System
Verify OTP and issue secure time-limited session token
<<ref>>
BS_UM_03 – Define and Manage Role-Based Access Permissions
Apply role permissions and restrict accessible modules/data
<<ref>>
BS_UM_02 – User Login to the System
Redirect user to role-specific dashboard and record login in audit trail

Design Patterns

All members

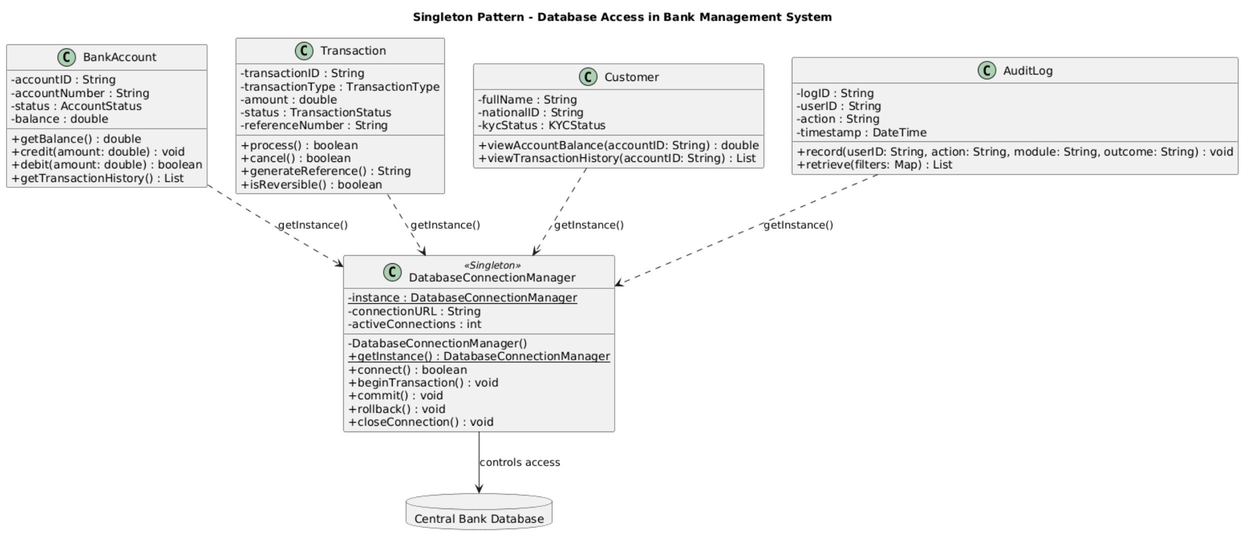
Command Pattern

The Command pattern was chosen for the Bank Management System because the system handles several transaction requests that must be processed consistently, including deposits, withdrawals, fund transfers, bulk payments, and recurring payment executions. The existing Transaction class already contains the shared command data and behavior through attributes such as amount, status, sourceAccountID, destinationAccountID, and methods such as process(), cancel(), and isReversible(), while Deposit, Withdrawal, FundTransfer, and BulkPayment specialize the transaction behavior. This pattern solves the problem of treating different banking operations as executable request objects, so they can be queued, processed, reversed when allowed, and recorded without forcing Customer, BankTeller, or BankStaff to know the internal logic of each transaction type. A new TransactionCommand interface and TransactionProcessor invoker are introduced only to formalize the pattern structure, while the real transaction classes from the system remain the concrete commands. This improves maintainability because transaction execution rules are kept inside transaction classes instead of being scattered across user-role classes. It also improves extensibility because new transaction types can be added later by extending Transaction and implementing the same command interface without modifying the existing invoker logic. The design supports the Single Responsibility Principle by separating request initiation from request execution, and it supports the Open/Closed Principle and Dependency Inversion Principle by allowing the processor to depend on the TransactionCommand abstraction rather than concrete transaction classes.


Command Pattern - Transaction Processing in Bank Management System
Customer
BankTeller
+processDeposit(accountID: String, amount: double, type: String) : Transaction
+processWithdrawal(accountID: String, amount: double) : Transaction
submits request
submits request
«new»
TransactionProcessor
-commandQueue : Queue<TransactionCommand>
+submit(command: TransactionCommand) : void
+processNext() : boolean
+reverse(command: TransactionCommand) : boolean
queues and executes
«new»
TransactionCommand
+process() : boolean
+cancel() : boolean
Transaction
-transactionID : String
-amount : double
-status : TransactionStatus
-sourceAccountID : String
-destinationAccountID : String
+process() : boolean
+cancel() : boolean
+isReversible() : boolean
updates balance
Deposit
-depositType : String
+processCashDeposit() : boolean
Withdrawal
-withdrawalMethod : String
+processBranchWithdrawal() : boolean
FundTransfer
-transferType : String
+processInternalTransfer() : boolean
BankAccount
-accountID : String
-balance : double
+credit(amount: double) : void
+debit(amount: double) : boolean

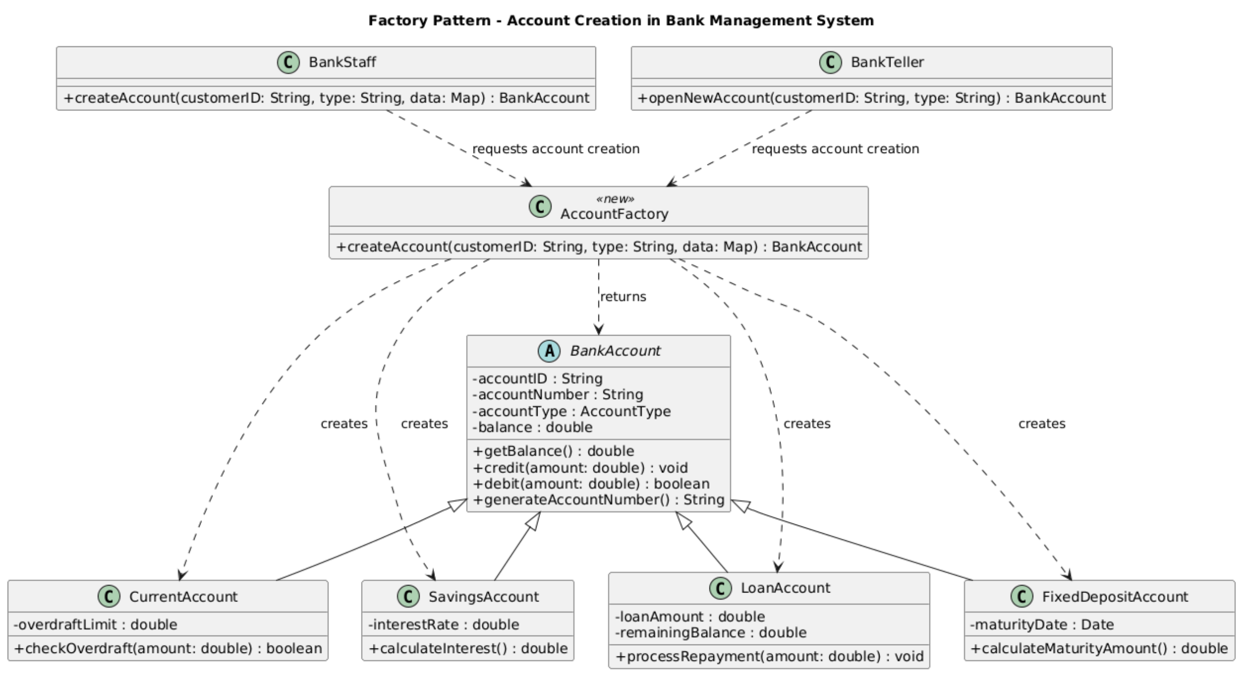
Singleton

The Singleton pattern was chosen for the Bank Management System because all sensitive data operations should pass through one controlled database access point instead of allowing multiple classes to create separate and unmanaged database connections. The system handles customer records, bank accounts, transactions, loans, audit logs, and financial reports, so uncontrolled database access could create security risks, inconsistent updates, duplicate connections, or incomplete transaction handling. By introducing DatabaseConnectionManager as a Singleton, classes such as BankAccount, Transaction, Customer, and AuditLog can use getInstance() to access the same database connection manager whenever they need to read or update persistent data. The pattern improves maintainability because connection setup, transaction control, rollback behavior, and connection closing are centralized in one class rather than duplicated across many banking classes. It also improves security because the database has one clear gateway, making it easier to enforce access rules, logging, and controlled connection usage. This design supports the Single Responsibility Principle by separating database connection management from business classes, the Open/Closed Principle because database access internals can change without modifying client classes, and the Dependency Inversion Principle because system classes depend on a controlled access point rather than directly managing low-level database details.



Factory

The Factory pattern was chosen for the Bank Management System because the system must create different account objects, such as SavingsAccount, CurrentAccount, FixedDepositAccount, and LoanAccount, while keeping account creation logic separate from user-role classes like BankStaff and BankTeller. In the existing system, BankStaff.createAccount(customerID, type, data) and BankTeller.openNewAccount(customerID, type) start the account-opening process, but these classes should not directly decide which concrete account subclass to instantiate. The common superclass BankAccount already defines shared account data and behavior through attributes such as accountID, accountNumber, accountType, status, balance, and currency, while each account subclass adds its own specialized behavior such as calculating interest, checking overdraft, calculating maturity amount, or processing repayment. The Factory pattern solves this by introducing an AccountFactory that centralizes account object creation and returns the result as a BankAccount, allowing the rest of the system to work with the common account abstraction. This improves maintainability because account creation rules are localized in one creation component instead of being scattered across teller and staff workflows. It also improves extensibility because new account types can be added later by creating a new BankAccount subclass and extending the factory creation logic without changing the client workflow. The design supports the Single Responsibility Principle by moving creation responsibility into the factory, the Open/Closed Principle by allowing account variations to be extended through new products, and the Dependency Inversion Principle because clients depend on the BankAccount abstraction rather than concrete account implementations.



Proxy

The Proxy pattern was chosen for the Bank Management System because the riskiest operations occur when a user request enters the banking backend, especially for deposits, withdrawals, fund transfers, and bulk payments. Instead of letting Customer, BankTeller, or BankStaff send transaction requests directly to Transaction, the new SecureTransactionProxy filters the request first and only delegates to the real transaction object when the user session, role permissions, transaction limits, and duplicate-request checks are valid. This solves concrete banking problems such as accidental double submission of the same transfer, unauthorized access to transaction processing, repeated withdrawal attempts, and unlogged failed operations. The existing User, Role, and Permission classes are used to verify whether the current user may perform the requested operation, while AuditLog records both accepted and rejected requests. The existing Transaction hierarchy remains responsible for the real transaction behavior through process(), cancel(), and isReversible(), while the proxy adds protection without changing those transaction classes. This improves maintainability because security filtering is centralized at the backend entry point instead of being repeated inside every transaction type or user-role class. The design supports the Single Responsibility Principle by separating access filtering from transaction execution, the Open/Closed Principle because new transaction types can still be added under Transaction, and the Dependency Inversion Principle because clients depend on the TransactionAccess abstraction rather than directly on concrete transaction classes.

Proxy Pattern - Secure Transaction Access in Bank Management System